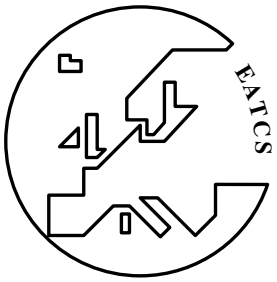
ISSN 0252–9742

Bulletin

of the

European Association for Theoretical Computer Science

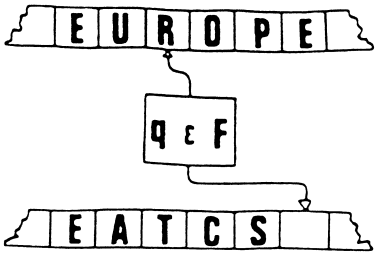
EATCS



Number 150 October 2026

EATCS

General Assembly Members



President:	Giuseppe F. Italiano	Italy
Vice Presidents:	Antoine Amarilli	France
	Inge Li Gørtz	Denmark
Treasurer:	Gabriele Fici	Italy
Bulletin Editor:	Stefan Schmid	Germany

Antoine Amarilli	France	Giuseppe F. Italiano	Italy
Alkida Balliu	Italy	Kasper Green Larsen	Denmark
Ivona Bezakova	USA	Yannic Maus	Austria
Thomas Colcombet	France	Anca Muscholl	France
Anne Driemel	Germany	Yasamin Nazari	Netherlands
Funda Ergün	USA	Charles Paperman	France
Javier Esparza	Germany	Eva Rotenberg	Denmark
Gabriele Fici	Italy	Jukka Suomela	Finland
Pawel Gawrychowski	Poland	Szymon Torunczyk	Poland
Inge Li Goertz	Denmark	Bianca Truthe	Germany

Past Presidents:

Maurice Nivat	(1972–1977)	Mike Paterson	(1977–1979)
Arto Salomaa	(1979–1985)	Grzegorz Rozenberg	(1985–1994)
Wilfred Brauer	(1994–1997)	Josep Díaz	(1997–2002)
Mogens Nielsen	(2002–2006)	Giorgio Ausiello	(2006–2009)
Burkhard Monien	(2009–2012)	Luca Aceto	(2012–2016)
Paul Spirakis	(2016–2020)	Artur Czumaj	(2020–2024)

Secretary Office:	Dmitry Chistikov	UK
	Efi Chita	Greece

EATCS General Assembly Members

email addresses

Antoine Amarilli . . . . . a3nm@a3nm.net
Alkida Balliu . . . . . alkida.balliu@gssi.it
Ivona Bezakova . . . . . ib@cs.rit.edu
Thomas Colcombet . . . . . Thomas.Colcombet@irif.fr
Anne Driemel . . . . . driemel@cs.uni-bonn.de
Funda Ergün . . . . . chair.sigact@sigact.acm.org
Javier Esparza . . . . . esparza@in.tum.de
Gabriele Fici . . . . . gabriele.fici@unipa.it
Pawel Gawrychowski . . . . . awry@cs.uni.wroc.pl
Inge Li Goertz . . . . . inge@dtu.dk
Giuseppe F. Italiano . . . . . gitaliano@luiss.it
Kasper Green Larsen . . . . . larsen@cs.au.dk
Yannic Maus . . . . . yannic.maus@tugraz.at
Anca Muscholl . . . . . anca@labri.fr
Yasamin Nazari . . . . . y.nazari@vu.nl
Charles Paperman . . . . . charles.paperman@univ-lille.fr
Eva Rotenberg . . . . . eva@rotenberg.dk
Stefan Schmid . . . . . stefan.schmid@tu-berlin.de
Jukka Suomela . . . . . jukka.suomela@aalto.fi
Szymon Torunczyk . . . . . szymtor@mimuw.edu.pl
Bianca Truthe . . . . . bianca.truthe@informatik.uni-giessen.de

Bulletin Editor: Stefan Schmid, Berlin, Germany
Cartoons: DADARA, Amsterdam, The Netherlands

The bulletin is entirely typeset by pdfTEX and ConTEXt in TXfonts.

All contributions are to be sent electronically to

bulletin@eatcs.org

and must be prepared in LaTeX 2ε using the class `beatcs.cls` (a version of the standard LaTeX 2ε `article` class). All sources, including figures, and a reference PDF version must be bundled in a ZIP file.

Pictures are accepted in EPS, JPG, PNG, TIFF, MOV or, preferably, in PDF. Photographic reports from conferences must be arranged in ZIP files layed out according to the format described at the Bulletin's web site. Please, consult `http://www.eatcs.org/bulletin/howToSubmit.html`.

We regret we are unfortunately not able to accept submissions in other formats, or indeed submission not *strictly* adhering to the page and font layout set out in `beatcs.cls`. We shall also not be able to include contributions not typeset at camera-ready quality.

The details can be found at `http://www.eatcs.org/bulletin`, including class files, their documentation, and guidelines to deal with things such as pictures and overfull boxes. When in doubt, email `bulletin@eatcs.org`.

Deadlines for submissions of reports are January, May and September 15th, respectively for the February, June and October issues. Editorial decisions about submitted technical contributions will normally be made in 6/8 weeks. Accepted papers will appear in print as soon as possible thereafter.

The Editor welcomes proposals for surveys, tutorials, and thematic issues of the Bulletin dedicated to currently hot topics, as well as suggestions for new regular sections.

Table of Contents

EATCS MATTERS
Letter from the President 3
Letter from the Editor 9

EATCS COLUMNS

The Interview Column, *by C. Avin, S. Schmid*
Know the Person behind the Papers Today:
Peter Winkler, ... 17

The Interview Column, *by C. Avin, S. Schmid*
Know the Person behind the Papers Today: Tom
Leighton, ... 27

TCS on the Web, *by S. Neumann*
Polylog: Widely Accessible Videos About TCS,
by S. Neumann 35

The Computational Complexity Column *by M. Koucký*
Learning, Compression, and Meta-Complexity, *by*
H. Goldberg, M. Nanashima, I. C. Oliveira 41

The Machine Learning Column, *by P. Barcelo, D. Saulpic*
The Computational Power of Transformers:
Interview with Anthony Widjaja Lin, ... 165

The Logic in Computer Science Column, *by A. Dawar*
Linear Sampling Revisited, *by H. R. Lewis* 179

The Concurrency Column, *by M. Miculan, N. Yoshida*
Interviews with the Recipients of the CONCUR
Test-of-Time Award 2026, *by L. Aceto* 203

Conference Reports Column, *by L. Feuilloley*
Conference report on STOC 2026, *by M. Bentert* 217

Viewpoint Column *by A. Muscholl, S. Schmid*

Informatics Is a Very Old Science!, *by M. Raynal* 225

The Education Column *by D. Komm, T. Zeume*
Informatics Education in the Age of AI
Lessons from Austria 's New Upper-Secondary
Curriculum, *by A. Bollin, R. Bloem* 235

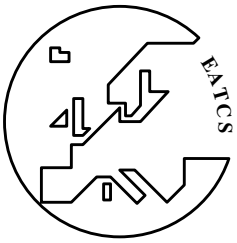
News and Conference Reports

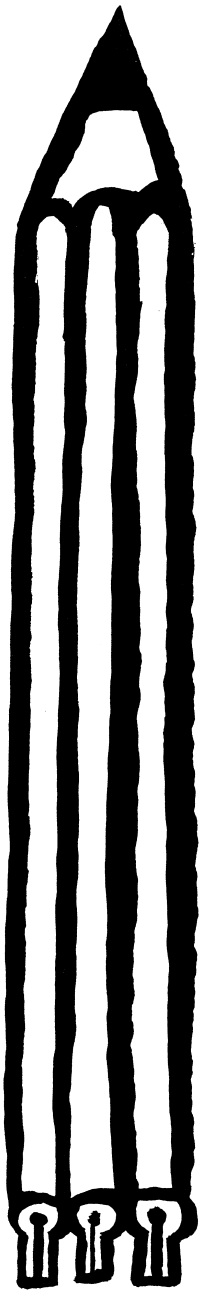
Report from the Italian Chapter, *by L. Geatti, C. Piazza* . 253

Report on CIAA 2026, *by M. Holzer* 255

EATCS LEAFLET 259

EATCS Matters

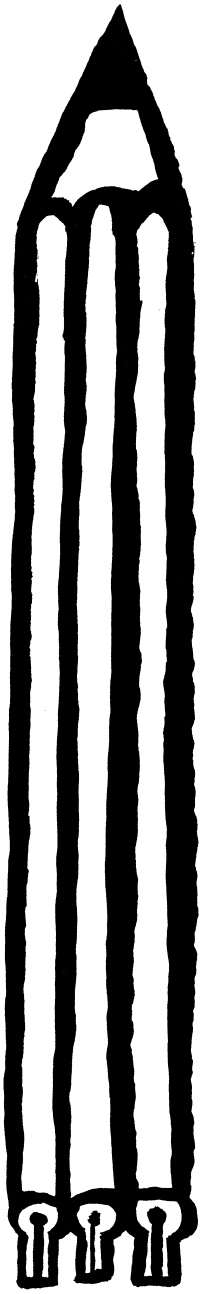




Dear EATCS Members,

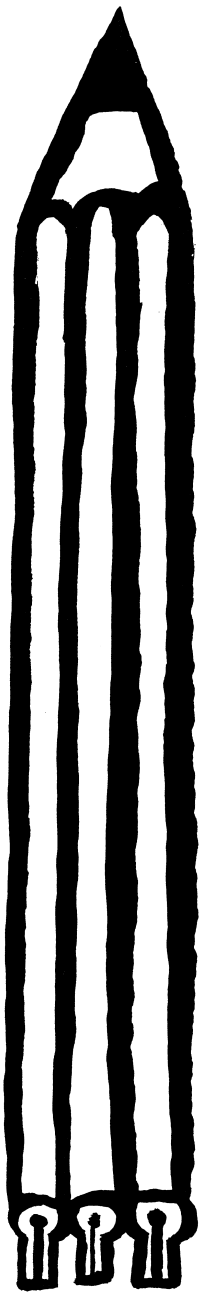
It was a great pleasure to meet so many of you at ICALP 2026 at Royal Holloway, University of London. The 53rd edition of ICALP was a wonderful occasion, both scientifically and socially. I was particularly encouraged by the strong participation of young researchers and students, and very pleased to see such a good attendance at the EATCS General Assembly. This vibrant participation is, to me, a particularly encouraging sign of the strength and vitality of our community. On behalf of EATCS, I would like to warmly thank the Program Committee Chairs, Danupon Nanongkai, Sayan Bhattacharya, and Michael Benedikt, all the members of the Program Committee, the Organizing Committee Chair, Matthew Hague, and the entire local team for their outstanding work. They put together an impressive scientific program and created an environment in which participants could exchange ideas, establish new connections, and enjoy being part of our community.

The energy of our community is particularly important at a time when mathematics and theoretical computer science are undergoing rapid change. One question, in particular, is becoming increasingly relevant to all of us: how will mathematical and theoretical computer science research evolve in the age of AI? We are witnessing a profound change in the way mathematical knowledge can be explored. AI systems are becoming increasingly capable of assisting with formal reasoning, discovering patterns, generating conjectures, suggesting proof



strategies, and even tackling problems that have long been considered highly challenging. This opens up exciting new possibilities, but also raises fundamental questions about the future of our research. For decades, progress in theoretical computer science has often followed a familiar path: formulate a challenging question, develop the right concepts, discover an appropriate technique, and ultimately prove a theorem. AI may increasingly become part of every stage of this process. It may help us identify promising conjectures, explore enormous spaces of possibilities, uncover unexpected connections between seemingly unrelated areas, or suggest approaches that might be difficult for a human researcher to discover alone.

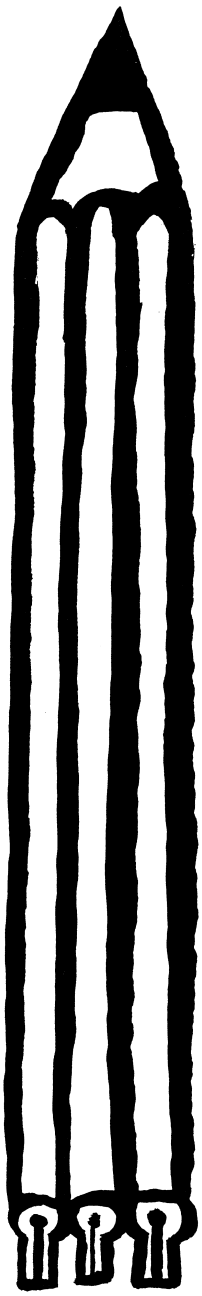
But this does not make the human role less important. I believe it's quite the opposite. The ability to generate a plausible proof or conjecture is not the same as understanding why it is true, why it matters, or where it fits within the conceptual structure of a field. Theoretical computer science is not simply a collection of statements that can be proved. It is a discipline built around concepts, abstractions, explanations, and the discovery of the right questions. This may lead us to a particularly interesting paradigm shift: perhaps the scarce resource of future theoretical research will not only be the ability to solve problems, but also the ability to formulate important problems and recognize meaningful solutions. AI could allow us to attack mathematical and TCS problems at a scale and speed that was previously impossible.



It could also change the relationship between theory and experimentation, making computational exploration an increasingly important component of theoretical discovery. Some problems that have resisted decades of human effort may eventually become more understandable with the assistance of AI. Others may turn out not to be the questions we should have been asking in the first place.

This is an opportunity for our community to think proactively about where research is heading. What kinds of problems should we formulate for an AI-assisted scientific world? What does it mean to understand a proof when its discovery involves AI? How should we evaluate and communicate results generated with substantial machine assistance? And, perhaps most importantly, which new theoretical questions will emerge from AI itself? These are not questions for computer scientists working on AI alone. They are questions for the entire theoretical computer science community. Complexity theory, algorithms, logic, formal methods, combinatorics, cryptography, semantics, verification, and many other areas may all be affected by this transformation. I believe EATCS has an important role to play in fostering this discussion. Our community has always been at its strongest when it has combined technical depth with a willingness to rethink its fundamental questions. The AI era gives us another opportunity to do exactly that.

Another important way in which we shape the future of our discipline is through recognition. New calls for nominations will soon open for several EATCS-sponsored



awards, including the EATCS Award, the Presburger Award, the EATCS Distinguished Dissertation Award, and the EATCS Fellows. EATCS is also proud to co-sponsor major international prizes such as the Gödel Prize, the Alonzo Church Award, and the Dijkstra Prize. I warmly encourage you to take a moment to nominate outstanding colleagues and researchers. Recognition is not only a celebration of individual achievement. It is also one of the ways in which a scientific community tells its own story. The people and ideas we choose to recognize help highlight the contributions that have shaped our field and, perhaps even more importantly, the directions in which we believe theoretical computer science should develop. In a period of significant change, this responsibility becomes particularly meaningful.

Finally, I would like to reiterate something that is particularly important to me as President of EATCS: my inbox is always open. Your comments, suggestions, ideas, and constructive criticism are very welcome and deeply valued. EATCS is a community-driven organization, and its strength ultimately depends on the engagement of its members. Whether you have an idea for a new activity, a suggestion for improving an existing one, or simply a thought about where our field should be heading, I would very much like to hear from you. Every suggestion will receive careful consideration.

The coming years will undoubtedly bring significant changes to theoretical computer science. I am convinced that our community is exceptionally well placed to contribute to shaping them, not simply by adapting to

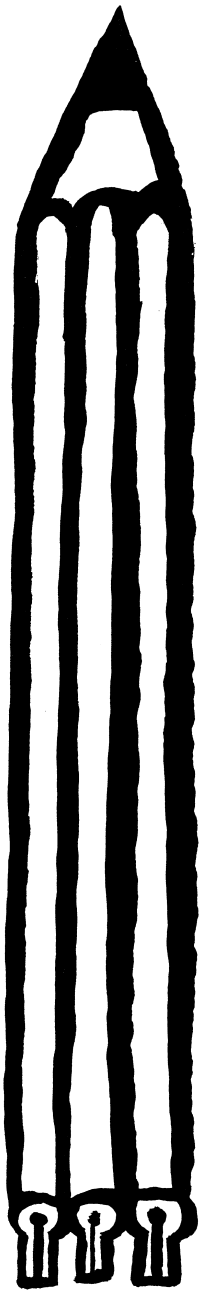
new technologies, but by asking the deeper questions about knowledge, computation, reasoning, and discovery that have always been at the heart of our discipline.

I look forward to continuing this conversation with all of you.

With warm regards,

Giuseppe F. Italiano
Luiss University, Rome
President of EATCS
president@eatcs.org

October 2026



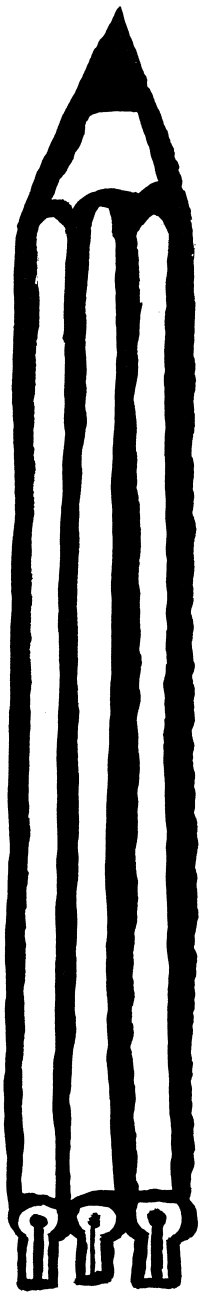
Dear EATCS member!

The Autumn issue of the EATCS Bulletin comes with plenty of reading material. In our Know the Person behind the Papers interviews, we have the pleasure of featuring two fascinating personalities from the theoretical computer science community. Peter Winkler reflects on his journey from mathematical logic to theoretical computer science and his passion for mathematical puzzles. And if you are curious what he used to keep in the blue and brown notebooks he carried in his pocket, you will find the answer in the interview as well!

Tom Leighton looks back on a remarkable career connecting theory and practice, from graph embeddings and parallel computing to the founding of Akamai and the development of Internet infrastructure at global scale. He also shares his thoughts on entrepreneurship and on opportunities for theory in areas such as AI and quantum computing.

In the Viewpoint Column, Michel Raynal takes us much further back in time. Arguing that informatics is fundamentally the science of algorithms and their applications, he illustrates how algorithmic thinking long predates modern computers and invites us to view informatics as a much older science than its technological manifestations might suggest.

In the TCS on the Web Column, Stefan Neumann speaks with Tomáš Sláma and Richard Hladík from Polylog, the successful YouTube

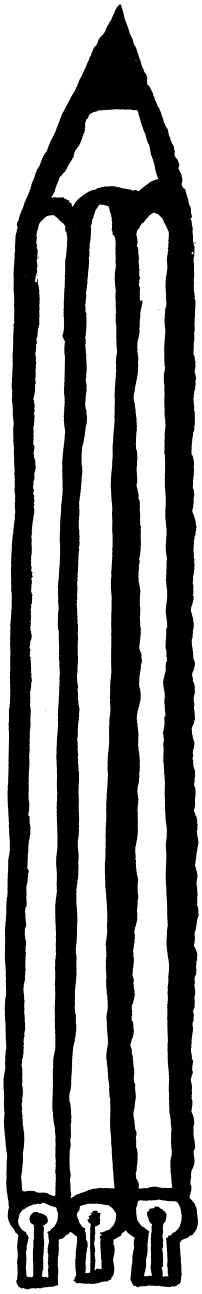


channel devoted to accessible and visually engaging explanations of theoretical computer science. They discuss how they choose topics, combine technical depth with accessibility, produce their videos, and see human intuition play an important role in an age of increasingly capable AI.

The Machine Learning Column features an interview with Anthony Widjaja Lin on the computational power of transformers. The discussion explores how classical tools from logic, automata theory, and circuit complexity can help us understand what transformers can and cannot compute, while also highlighting the still substantial gap between our theoretical understanding and the remarkable capabilities of modern AI systems.

The Computational Complexity Column features a survey by Halley Goldberg, Mikito Nanashima, and Igor C. Oliveira on the connections between learning, compression, and meta-complexity. It explores how compression can serve as a bridge between learning and complexity theory, and how ideas from complexity theory can both inform the design of learning algorithms and, conversely, how learning algorithms can yield new insights into computational hardness and lower bounds.

In the Logic in Computer Science Column, Harry Lewis revisits Linear Sampling, the combinatorial problem he introduced with Stål Aanderaa more than fifty years ago in connection with decision problems for fragments of first-order logic. Dedicated to the memory of Aanderaa, the article gives a new and fuller exposition of the problem and its undecidability proof.



The Education Column turns to the role of informatics in schools in the age of AI. Andreas Bollin and Roderick Bloem discuss Austria's new upper-secondary subject "Informatics and AI" and the broader debate over whether informatics should remain a distinct part of general education. Their central message is particularly timely: the rise of AI does not diminish the importance of understanding informatics-it makes this need even more visible.

The Concurrency Column features Luca Aceto's interviews with the recipients of the 2026 CONCUR Test-of-Time Award. Naoki Kobayashi looks back on his influential work on type systems for deadlock-free processes, while Krishnendu Chatterjee, Thomas Henzinger, and Nir Piterman discuss the origins and subsequent impact of Strategy Logic. Along the way, the interviews offer insights into how research ideas evolve over time and how formal methods may themselves change in the era of AI-assisted software development.

The Conference Reports Column takes us to two major gatherings of our community. Matthias Bentert reports from STOC 2026 in Salt Lake City, including the newly established fifth Busy Beaver value, Charles Bennett's Turing Award lecture, and the lively workshop "Can AI Do Theory?". Dominick Banasik and Nithin Salevemula report from PODC 2026 at Royal Holloway, co-located with SPAA and ICALP, highlighting the new Student Spotlight, discussions around AI policies, and several memorable contributions in distributed computing.

The Bulletin also takes us to Kingston, Canada, for a report by Markus Holzer on

the 30th International Conference on Implementation and Application of Automata (CIAA 2026). The report covers the scientific programme and invited talks, celebrates the conference's 30th edition, and highlights a special session honouring Kai Salomaa and his contributions to automata and formal language theory.

Finally, the Bulletin includes a report from the Italian Chapter of the EATCS on ICTCS 2026 in Udine. This year's conference introduced special tracks on Cyber-Physical Systems, Quantum Computing, and Bioinformatics, and the report also celebrates the recipients of the Italian Chapter's Young Researcher awards.

Please enjoy the new issue of the Bulletin!

Stefan Schmid, Berlin
October 2026

Institutional Sponsors

CTI, Computer Technology Institute & Press "Diophantus"
Patras, Greece

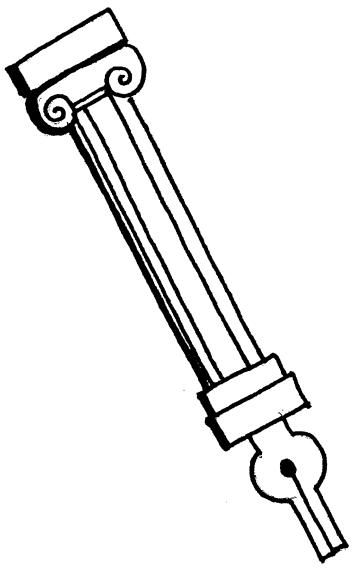
CWI, Centum Wiskunde & Informatica
Amsterdam, The Netherlands

MADALGO, Center for Massive Data Algorithmics
Aarhus, Denmark

Microsoft Research Cambridge
Cambridge, United Kingdom

Springer-Verlag
Heidelberg, Germany

EATCS
Columns



Know the Person behind the Papers

Today: Peter Winkler

Bio: *Peter Winkler is the William Morrill Professor of Mathematics and Computer Science at Dartmouth College. His research is primarily in combinatorics, probability, and the theory of computing, with forays into statistical physics. He is a winner of the Mathematical Association of America's Lester R. Ford and David P. Robbins prizes and a Fellow of the American Association for the Advancement of Science. In 2019–2020, he served as Distinguished Visiting Professor for the Public Dissemination of Mathematics at the National Museum of Mathematics in New York City. He has written three books on mathematical puzzles, a book on cryptology in the game of bridge, and a portfolio of compositions for ragtime piano.*

Figure 1: Peter Winkler in his office. The quilt behind him was made by his wife, Lois Winkler, from a pattern Peter devised using Charles Radin's recursive pinwheel tiling.

Stefan Schmid: Peter, before we talk about research, there is already some mathematics in the photograph you sent us. What is the story behind the quilt?

Peter Winkler: The quilt was made by my wife, Lois Winkler. The pattern I suggested is based on Charles Radin's recursive "pinwheel tiling." Each tile is a right triangle whose legs have lengths 1 and 2.

What I like about the pinwheel tiling is that it has some remarkable geometric properties. In the full planar tiling, tile edges occur at a dense set of angles. And any two points that are very far apart can be connected along tile edges with total length asymptotic to their Euclidean distance. So it seemed like a particularly appropriate pattern for a quilt in a mathematician's office.

Stefan Schmid: You started out as a mathematician, with a PhD in mathematical logic. How did you find your way into the theoretical computer science community?

Peter Winkler: Almost by accident — and through one of my students! I did my PhD at Yale in mathematics, with a thesis in mathematical logic. I was always interested in questions about what can and cannot be computed, but I wasn't moving in theoretical computer science circles.

After Yale and a period at Stanford, I joined Emory University. There I had a student named Sam Buss. Sam went to a conference and came back with this thick paperback proceedings volume. The conference turned out to be FOCS. Looking through it was a revelation for me.

It wasn't only that I had discovered a whole community interested in many of the things I liked. I also liked the style of the conference. In mathematics, speakers are often invited by name and then talk for a relatively long time about whatever they are working on. In theory of computing, things moved much faster: people competed to present new ideas, talks were short and succinct, and there was an enormous amount of fresh material.

I started going to FOCS, STOC, SODA, and so forth, and got to know the community. I still like that culture very much.

Stefan Schmid: Many readers may know you at least as well for your puzzle books as for your research papers. How did the puzzle books happen?

Peter Winkler: That also happened rather unexpectedly. I had always enjoyed puzzles. Starting perhaps in high school, I carried two little notebooks around in my pocket. One was blue, and in it I wrote a few words to remind myself of good puzzles. The other was brown, and in that one I wrote down jokes — but only the punchlines, because I assumed I would be able to reconstruct the whole joke from the punchline.

So in a sense even the joke book was a puzzle book!

Years later, Klaus Peters approached me at a conference. He had left Springer and founded A K Peters with his wife, Alice. He said something like, "I hear you have a great collection of mathematical puzzles. Why don't you email them to me and I'll publish them?" He made it sound very easy. And, in fact, he made the process about as easy as publishing a book can be.

Once the first book appeared, people began emailing me puzzles from all over the world. That became a wonderful source of material for the later books. I always try to credit the person from whom I heard a puzzle, whether they invented it themselves or passed it along.

Stefan Schmid: What makes a puzzle a *good* puzzle for you? Is it mainly about being difficult?

Peter Winkler: No — that's exactly the misconception I would like to fight.

People often think puzzles are challenges or tests: What's the hardest puzzle? Can you solve it? They can even become nervous when somebody tells them a puzzle because they fear they are going to be tested.

That is not how I think about puzzles. A good puzzle asks a question that you might never have thought of asking and gives an answer that you might never have expected. Most importantly, it should be entertaining. It should be neat. It should communicate an idea.

In fact, sometimes the puzzle works even better if you *don't* find the answer yourself. Puzzles for examinations or competitions have a different purpose; they are there to test you, and that's perfectly legitimate. But the puzzles I collect for my books are not there to test people. They are there to amuse them, get them excited, and show them something they didn't know before.

Stefan Schmid: Can you give us an example of such a puzzle?

Peter Winkler: Here's one that reached me indirectly from Japan. The question is wonderfully short:

> Can any ten points in the plane be covered by pairwise disjoint unit disks?

Every word matters. If the disks did not have to be disjoint, the problem would be trivial. If you had a huge number of points packed into a fine grid, you clearly could not cover all of them by disjoint unit disks. But for a small number of points it seems plausible.

The exact largest number of points that can always be covered is not known. But there is a beautiful probabilistic argument showing that ten points can always be covered.

Take the densest familiar packing of unit disks, the hexagonal or honeycomb packing. Its density is

$$\frac{\pi}{\sqrt{12}} > 0.9.$$

Now shift the entire packing uniformly at random. Any fixed point is covered with probability $\pi/\sqrt{12}$. So for ten fixed points, the expected number covered is more than nine.

Because the number covered is an integer, some shift must cover all ten.

That's the kind of proof I love. It's a question I would never have thought of asking, followed by a very short argument that makes you see something differently. Paul Erdős would have liked it.

Stefan Schmid: Do puzzles ever turn into research papers?

Peter Winkler: Absolutely. A very recent example began with a high-school student, Tejo Madhavarapu. I was on Zoom with Tejo and Kyle Petersen from DePaul University, and Kyle mentioned that Tejo composed puzzles. I asked him to send me some favorites.

He sent only five. Two of them were puzzles I had never seen and really liked. That's an extraordinary success rate — when people send me puzzles, perhaps one in a hundred really excites me.

One of Tejo's puzzles was called "Is There an Ideal Color Wheel?" Think of the familiar six-color wheel from elementary school, where, for example, blue and yellow mix to make the green between them. Tejo asked whether one could arrange six colors so that *every* color is obtainable by blending its two neighbors, without all six colors being identical.

What is striking is that the problem can be abstracted away from color almost completely. It connects to the familiar fact that a harmonic function on the vertices of a finite connected graph must be constant. We found that one can formulate and prove a much more general statement on finite graphs without needing the usual machinery of averaging. That puzzle became a joint research paper with Tejo and Kyle.

I like that transition very much: a question posed for fun reveals a mathematical structure, and suddenly you're doing research.

Stefan Schmid: You also composed a probability puzzle about baseball that seems designed to trick one's intuition.

Peter Winkler: Yes. Imagine two equally matched baseball teams, except that each has the same small home-field advantage. They play a best-of-seven series: the first team to win four games wins the series. Suppose Team A hosts the first three games and Team B hosts the remaining four.

The obvious argument says that the series lasts fewer than six games on average, so more games are actually played at A's home. Therefore A should have the advantage.

I used this puzzle in a course called *Probability and Intuition* that I taught while I was Distinguished Visiting Professor at the National Museum of Mathematics in New York. We met over dinner with a group of people from many professions — finance, medicine, law, journalism, film — and discussed probability through puzzles.

Everybody came back with essentially that argument: Team A should have the advantage.

Then one of the participants, a cardiologist, raised her hand and said: Suppose they simply play all seven games, even after somebody has already won four. The extra games cannot affect who wins the series. But if all seven games are played, Team B obviously has the advantage because four games are at B's home and only three at A's.

And that's correct: Team B has the advantage.

The nice paradox is that if the teams play the series every year for a very long time, Team A will win more individual games, while Team B will win more series.

Someone asked how home-field advantage in game seven, which is rarely needed, can be as valuable as home-field advantage in game one, which is always played. My analogy was transportation in Manhattan: having a car available exactly when you need one can be just as useful as owning one all the time. Game seven is there when you need it.

Stefan Schmid: Where and when do you tend to get your best ideas?

Peter Winkler: In the morning, in the shower.

If I'm going to have a good idea during the day, there's a good chance it will happen during those five minutes or so, when my brain is waking up. I realize there is a perfectly good argument for showering in the evening: after a long day, you get clean and then sleep in clean sheets. But if I switched to evening showers, I fear I would have missed a lot of good ideas!

More seriously, ideas are often triggered by something accidental. The important thing is what Erdős used to call keeping your "brain open."

He liked to tell a story about the discovery of X-rays. According to Erdős, Crookes and his students had already noticed that photographic plates behaved strangely near a Crookes tube. Crookes's reaction was essentially to tell his students not to leave photographic plates near the tube. Röntgen, faced with the anomaly, asked what was going on.

I can't vouch for the historical accuracy of the story, but the lesson is excellent: when something unexpected happens, don't automatically treat it as an annoyance. Your brain should be open enough to notice that it might be telling you something.

Stefan Schmid: That sounds like a philosophy for choosing research problems as well. How do you decide what to work on?

Peter Winkler: I'm not sure anyone should take my advice on this, because I have shown a certain absence of self-discipline in choosing research problems.

I get interested in a question. If it then leads into an area that is not one of my mathematical strengths, the disciplined thing would be to give it up and choose a problem that better matches my expertise.

I don't.

That's one reason I started in mathematical logic, then moved into combinatorics, probability, theory of computing, and even some statistical physics. It seems to suit my personality, and somehow I've gotten away with it.

For students, though, my advice is somewhat different: try to get really good at *something* first. Having a strong base gives you more freedom later to wander.

Stefan Schmid: And what do you do when you get stuck?

Peter Winkler: Most of the time you're stuck. That's the steady state for a researcher.

If you really hit a wall and don't see what to do next, put the problem aside and work on another one. Or watch a soccer game. That was actually my plan for later on the day we spoke.

Sometimes the problem you're having can even suggest another research problem. At Bell Labs, I had a wonderful intern, Ioana Dumitriu. We were trying to decide what project to work on, and that led us to ask a question about random walks: suppose several tokens are walking on a graph toward a target, and at every step you are allowed to choose which token makes the next random move. Which token should you move?

If you had to choose one token forever, you would simply choose the one with the smallest expected hitting time. But if you can change your mind after seeing where a token moves, the problem looks much more complicated.

It turned out that there is still a remarkably simple index policy: assign a certain value, which we called a "grade," to each state and always move a token with the best grade. We later realized that this was related to the Gittins index. The resulting paper was called *On Playing Golf with Two Balls*: hit one ball, and if you don't like where it goes, hit the other one.

The funny part is that we forgot to explain the golf analogy anywhere in the paper. Golf and balls occur only in the title.

Stefan Schmid: How do collaborations happen for you? Do you deliberately seek out particular collaborators?

Peter Winkler: Not very deliberately. A lot of it is happenstance. You talk to somebody, you both get interested in the same problem, and something happens.

I've written many papers with Graham Brightwell. We've hardly ever been at the same institution, but somehow the combination works. There's a certain magic to a good collaboration that I don't know how to explain. The same has been true of my work with László Lovász.

On the other hand, there are people I admire enormously whom I've known for decades and with whom I've barely worked. I've known Noga Alon since he was a student. I remember him showing me a beautiful combinatorial proof using exterior algebra. I told him I knew essentially nothing about exterior algebra beyond the definition. He said, "That's all you need," and showed me the proof.

We eventually ended up on a joint paper, but even there we had worked on different ends of the problem and were connected through the other coauthors. Collaboration is often less planned than people imagine.

Stefan Schmid: You mentioned teaching. Do puzzles find their way into your courses?

Peter Winkler: I can't help it.

I tell myself I'm only going to introduce a puzzle when it's exactly relevant to what I'm teaching. In practice, of course, puzzles have a way of appearing.

I teach a course called *Topics in Probability* about once a year, and I can essentially choose the subject. One year it might be percolation, another random graphs and graphons, another game theory or decision theory. Recently I taught permutations and sorting, partly because I've been interested in permutons — limit objects for permutations, analogous to graphons for graphs.

That course gave me an excuse to use one of my favorite probability examples, Pólya's urn. Start with one coin in each of two urns. Add coins one at a time, with the probability of choosing an urn proportional to the number of coins already in it. Now imagine doing this until there are a million coins, and ask students how much they would pay in advance for the contents of the *smaller* urn, at one dollar per coin.

People are extremely cautious. In one class, collectively, they were willing to invest about ten dollars.

But the expected value of the smaller urn is about a quarter of a million dollars.

There is a lovely proof using a shuffled deck of cards. Insert cards one by one into uniformly chosen positions. Track only how many cards lie above and below the first card. The evolution is exactly the urn process. Since the first card ends in a uniformly random position, the final split between the two urns is uniform. Again, the point is not to test the students. It's to give them an intuition they won't forget.

Stefan Schmid: Students now have another tool available: generative AI. Have you experimented with it for mathematics?

Peter Winkler: Yes, and I find it fun and occasionally useful.

It is especially helpful when I ask it to compute something that I can then verify independently. If I ask an AI system to prove something, I don't trust the answer — I have to prove it myself anyway.

I've also had amusing conversations with the systems. Claude tends to be very apologetic when I point out an error and eventually starts asking me for hints. Then I have to say, "This isn't a puzzle. I don't know the answer either. I'm telling you everything I know!"

I'm impressed by some of the recent work where stronger systems have contributed to mathematical problems. But at the moment I mostly regard AI as another interesting tool, one whose output needs to be checked carefully.

Stefan Schmid: You spent a year in Germany as a Humboldt Fellow. What do you remember from that time?

Peter Winkler: I spent 1984–85 at TH Darmstadt, which had a very good combinatorics group. The Humboldt Foundation was a wonderful host. They didn't just bring you to Germany and support your research; they organized trips and social events where Humboldt fellows from around the country could meet.

I also remember being surprised by the Christmas payment. I received what seemed like two months' salary around Christmas, which was completely unheard of to me as an American.

With a name like Winkler — and my middle name is Mann — people naturally expected me to speak German. My family with the Winkler name had come to the United States in the nineteenth century from Bessarabia, in what is now Moldova, so the German language had long since disappeared from the family.

I did learn enough German in Darmstadt that I once gave a talk in German. I can truthfully say that I did it. I can also truthfully say that it was a bad talk in German!

Stefan Schmid: Do you have a message for the theoretical computer science community? Anything we should be doing differently?

Peter Winkler: I'm delighted with the theory community. I think it's a great community, so I'm not going to give it advice — it seems to be doing very well without my help.

Here at Dartmouth we have a Theory Reading Group with colleagues and students from mathematics and computer science. That's something I enjoy very much: the community is broad enough that probability, combinatorics, algorithms, security, and other areas naturally interact.

So I'm happy to leave the community alone and let it keep doing what it's doing.

Stefan Schmid: Before we finish, you said there was one more STOC story you wanted to tell us.

Peter Winkler: Yes. Graham Brightwell and I proved that counting linear extensions of a partial order is #P-complete. This is an interesting counting problem because every finite partial order has a linear extension — the difficulty is not deciding whether one exists, but counting how many there are.

When I arrived at STOC in New Orleans, someone remarked that it was surprising that our reduction went all the way back to #3-SAT instead of starting from a problem that looked more similar.

That reminded me of a story about a theory course at Berkeley. A student was asked on the final exam to prove that Hamiltonian Path is NP-complete. He spent the whole exam trying to reduce directly from 3-SAT, even though he could have obtained the result almost immediately from Hamiltonian Cycle.

The professor — Umesh Vazirani, as I remembered after our interview — asked him why on earth he had done that. The student replied that his undergraduate advisor had been Christos Papadimitriou, and Christos had told him:

> "Real men reduce from SAT."

So for my STOC talk I went to a shop near the hotel and had a T-shirt made saying exactly that.

It got a small laugh. Then I explained that I really should have put "— Christos Papadimitriou" underneath it, but the shop charged 25 cents per letter and I would have had to sell my car.

That got a much bigger laugh.

Stefan Schmid: Peter, many thanks for this wonderful conversation. It seems appropriate that an interview about puzzles ends with a complexity-theory joke.

Peter Winkler: It was fun to talk to you. I hope you find something useful in all of this!

KNOW THE PERSON BEHIND THE PAPERS

Today: Tom Leighton

Bio: *Tom Leighton is a theoretical computer scientist whose career has taken an unusual path from combinatorics, algorithms, graph embeddings, and parallel computing to building Internet infrastructure at Akamai. He continues to teach discrete mathematics at MIT while spending his professional life at Akamai. His trajectory offers a rare perspective on how ideas from theoretical computer science can migrate into systems operating at global scale.*

Figure 1: Tom Leighton during a recent visit to the Singapore Institute of Technology. Photo supplied for the interview.

Stefan Schmid: The photo you shared with us was taken recently in Singapore. What was the occasion?

Tom Leighton: It was at the Singapore Institute of Technology, where I gave a lecture about starting a company and told the story of Akamai and its origins at MIT. The audience was mainly computer science and engineering students. SIT

is very interested in entrepreneurial activities and in inspiring students to think about the possibility of starting a company.

I've done similar things at a few universities over the years. What is interesting is that the questions and the level of interest are actually quite similar around the world. At least in technology, students today tend to be interested in many of the same things wherever you go.

Stefan Schmid: Your career itself is an unusual combination of theoretical computer science and entrepreneurship. Do you still think of yourself as a researcher?

Tom Leighton: These days I spend my time working at Akamai, not in a research capacity. I still teach every other fall at MIT, where I give lectures in the discrete mathematics class taken by computer science students. But today I have a full-time — and then some — job at Akamai.

So the daily routine is very different from when I was doing research. But of course the background in theory never really goes away. It affects how you look at problems and how you think about possible solutions.

Stefan Schmid: Before we get to Akamai, what originally attracted you to theoretical computer science?

Tom Leighton: The early work on the theory of computation and algorithms was very influential for me, and so was combinatorial mathematics, particularly the Hungarian school around Erdős. I remember being fascinated by results such as the Ajtai–Komlós–Szemerédi sorting-network result and, more generally, by the use of probability to prove that combinatorial objects exist. Those were very cool ideas.

At the time, the STOC and FOCS communities were also a wonderful source of interesting problems. I came from more of a combinatorial background, so I naturally liked combinatorial problems, and later problems connected to parallel computing and algorithms.

Stefan Schmid: How did you decide which problems were worth working on?

Tom Leighton: Mostly I worked on problems that I liked and where I felt I might be able to make some headway. It helped if a problem had been open for a while and other people had looked at it — if you solve such a problem, people will recognize the contribution.

I also liked it when a problem had some potential relevance, even if the result itself was still theoretical. If you can explain why the question might matter in the real world, that is useful. It can also help when applying for grants or jobs: even as a theoretician, it is valuable to be able to have conversations with people working in other parts of computer science.

What is interesting in retrospect is that some of the topics I liked for theoretical reasons turned out to be very relevant indeed.

Stefan Schmid: Is that the bridge to Akamai? Which theoretical questions unexpectedly became practical ones?

Tom Leighton: Yes. I had worked on parallel-computing problems and questions such as how to move data efficiently in large networks, how to replicate it, and how to decide what should be stored where and when. Those are combinatorial questions on which you can make theoretical advances.

It turns out that they are also very relevant to the Internet. When we started Akamai, many of the fundamental problems were of exactly this flavor. The context had changed enormously, of course, but the way of thinking about them was familiar. My theoretical background was very helpful in making design decisions and in deciding which approaches were likely to scale.

Stefan Schmid: What surprised you most when moving from proving theorems about algorithms to building systems that actually have to work?

Tom Leighton: There is a big difference between proving theorems about algorithms and writing code for systems that really work. They are pretty different areas, and you quickly gain a lot of respect for what systems people do.

When proving a theorem, you normally have a simplified model, an abstraction that you can get your arms around. The real world is messier and more complicated, and the code has to work in that real world. In practice, you may therefore use heuristic approaches that you would not encounter in the same way in a theorem.

At the same time, theory gives you a good sense of what should scale and what kinds of approaches are promising. That was a big advantage for us. And when we started the company, we had some terrific developers — among the best MIT had to offer. Theoreticians normally do not spend all their time building production systems; that is a different skill set. Combining the two was very powerful.

Stefan Schmid: Can you give us an example where an elegant algorithm had to evolve once it met Internet scale?

Tom Leighton: One recurring problem is assigning a user to a server depending on the content the user is trying to retrieve. Over time we went through a series of algorithms for doing that.

Initially, some of the approaches involved max-flow and min-cost algorithms. But those became too expensive to scale. At one point we used a variation of stable marriage. We knew these algorithms collectively, so we could keep evolving the solution and trying better approaches.

But the criterion in the end is not whether you have proved a theorem about the algorithm. The question is: Does it really work in the real world? What performance do you get? How much time does it take? At Akamai we are constantly measuring how the content-delivery network performs and looking for ways to make it better. That is an ongoing process.

Stefan Schmid: So has your notion of a "good algorithm" changed?

Tom Leighton: The objectives are different. In research, a beautiful theorem can be the result. In a production system, the result has to work, and it has to keep working as the system and the environment change.

But I would not say the two worlds are opposed. Theoretical training can be extremely useful precisely because it gives you a disciplined way of thinking about scalability and about the structure of a problem. You then combine that with measurement, engineering experience, and sometimes heuristics to handle everything that the model left out.

Stefan Schmid: You have now seen both sides. What qualities did you look for in students, and what do you look for in people at a company?

Tom Leighton: For students, I looked for people who were smart, hard-working, and interested in the same kinds of problems I was interested in.

For employees, the task is different. Being smart and hard-working still matters, and so does wanting to solve hard problems. But teamwork becomes especially important. We are a business, so making the customer happy is also very important. You want people who can solve difficult problems together and who fit the culture of the organization.

Of course, you never really know in an interview how someone will work in a team. You usually find that out later. But you can talk to people about what they enjoy doing and sometimes get a sense of whether the environment will be a good fit.

Stefan Schmid: Has your own way of working changed over the years?

Tom Leighton: I used to work late. Then I think having kids ended that! I gradually became more of a morning person, and these days I would say the morning is my most productive time.

But I am also doing a very different kind of work now. I am not sitting down in the morning trying to prove theorems. So it is hard to compare directly with my research years.

Stefan Schmid: Looking at theoretical computer science today from your unusual vantage point, where do you see particularly interesting opportunities?

Tom Leighton: AI is obviously going to change a lot of things in practice. One interesting question is whether theoretical researchers can contribute on the algorithmic side to making AI computations more efficient.

A lot of the action is connected to numerical analysis and linear algebra, including very large matrix computations. AI is unusual from a traditional STOC-and-FOCS perspective because there often is not a proof that something works — empirically, it just seems to work. That makes the problem more challenging for a community that is used to proving theorems.

But perhaps one can isolate simpler problems, understand them rigorously, and find provable algorithmic improvements that turn out to matter in practice. There is an enormous amount of money, computing power, and energy being devoted to AI. If we can make those computations more efficient, that would be very valuable.

Stefan Schmid: Is AI already changing how work is done at Akamai?

Tom Leighton: Yes, all over the place. It is useful for creating and synthesizing documents, and it is useful for coding. Developers can do much more efficiently now. I expect it to be useful on the financial side as well. I would say it is helpful in most roles, and we are still just at the beginning.

Stefan Schmid: What about quantum computing? Is that another area where theoreticians should be looking for the right questions?

Tom Leighton: I think there is an interesting question there: Will quantum computing really be useful at scale, and are there many important things you can do with it besides factoring and the number-theoretic applications we already know about? It is a very interesting computational model, and perhaps someday large-scale quantum computers will be built. What else can we do with them?

At the same time, encryption protocols are being changed precisely so that they are not vulnerable to the known quantum attacks. At Akamai we support post-quantum cryptographic protocols, and some customers with particularly sensitive information are already converting because they do not want data collected today to be decrypted five or ten years from now if breakthroughs occur. Ultimately, I think everything will switch over.

That makes the broader algorithmic question about quantum computing even more interesting: what other applications will justify the model?

Stefan Schmid: You mentioned that you still teach discrete mathematics. If a student asked whether computer science — and theory in particular — is still a good choice today, what would you say?

Tom Leighton: I think theory is important. At the same time, understanding AI to the extent that we can is probably very important now, because it is going to

change a lot of things and consume a lot of resources. There is real value in finding ways to make it more efficient.

In technical positions at Akamai, people come from different backgrounds, but there is a lot of computer science and engineering. So yes, computer science remains very relevant.

Stefan Schmid: What advice would you give a young theoretical computer scientist who is tempted to step outside academia — perhaps even to start a company?

Tom Leighton: Keep an open mind. I remember hearing a Nobel laureate speak when I was probably a teenager. The advice was essentially: as you go through life, keep your eyes and ears open and your brain turned on, because you never know when you are going to encounter a situation or a problem where you can make a real positive difference.

I think that also connects to a growth mindset. Just because you could not solve something yesterday does not mean you will not solve it tomorrow. Keep after it and keep thinking about other ways to get there.

And do not be afraid to get out of your comfort zone and try something different. That is especially true if you are thinking about experimenting with entrepreneurship rather than following only the straight academic path. Research and entrepreneurship are different experiences, and some skills transfer while others do not, but both can be challenging and rewarding ways to make a difference.

Stefan Schmid: Before we finish, let us go much further back. What were the first research problems you tried to solve?

Tom Leighton: In high school I tried to prove Goldbach's conjecture and to prove that there are infinitely many twin primes. I did not succeed with either one!

I did write a very rudimentary computer model that led to predictions about how many twin primes there might be in an interval, but I never proved anything. I cannot really count that as a research discovery. Later I worked at the Bureau of Standards on some zero-one matrix problems, and some papers came out of that. But it was really in graduate school at MIT, when I started working on graph-embedding and parallel-computing problems, that I began proving real theorems.

Stefan Schmid: Is there one of those old problems that you still occasionally think about?

Tom Leighton: Yes. My thesis involved graph embedding. One version of the question is this: take an n-node planar graph of bounded degree, say degree three, and embed it in a grid. Vertices are mapped one-to-one to grid nodes, and graph edges are routed along paths in the grid. Two routed edges may cross, but they cannot share a grid edge. How large does the grid need to be?

Using the Lipton–Tarjan planar separator theorem, you can get an upper bound with an $O(n \log^2 n)$-size grid. In my thesis I showed that some planar graphs require an $\Omega(n \log n)$-size grid. So there is still a logarithmic gap between the bounds, and, as far as I know, nobody knows the right answer.

Those are the kinds of problems I liked to work on. Maybe it is not the most important problem to solve today, but it is still a beautiful question.

Stefan Schmid: Is there something readers of your old theory papers may not know about you today?

Tom Leighton: I like tennis. I play, although certainly not at the professional level! And if I happen to be in a city where there is a tennis tournament, I sometimes go and watch. Federer was always the family favorite. These days, probably Alcaraz.

Please complete the following sentences.

- *Being a researcher or an entrepreneur*
  are both great experiences. They are different, and some skills from one transfer to the other, but both are challenging and rewarding ways to make a difference.
- *My first research discovery*
  is hard to identify. In high school I tried Goldbach's conjecture and the twin-prime conjecture, unsuccessfully. It was really in graduate school at MIT, working on graph embeddings and parallel computing, that I started proving real theorems.
- *The key to being a happy academic or entrepreneur*
  is that you like what you are working on.
- *Computer science in 100 years*
  I have no idea. Just look at the explosion of what has happened in the last few years with AI — and we are only at the beginning.

Stefan Schmid: Tom, many thanks for this conversation. Any final words for the Bulletin's readers?

Tom Leighton: Just say hello to a lot of folks I haven't seen in a long time.

TCS ON THE WEB

BY

STEFAN NEUMANN

TU Wien
Erzherzog-Johann-Platz 1
1040 Vienna, Austria
stefan.neumann@tuwien.ac.at
https://neumannstefan.com

Polylog[1] is a YouTube channel devoted to high-quality, broadly accessible videos about theoretical computer science. The channel is run by Václav Rozhoň[2], Václav Volhejn[3], Richard Hladík[4], and Tomáš Sláma[5]. Its videos cover topics ranging from zero-knowledge proofs and pseudorandomness to radix sort and A^* search.

At the time of our conversation, *Polylog* had published only 19 videos since 2021, yet had more than 119,000 subscribers and roughly seven million views. This makes it an interesting example of a channel that has grown through a small number of videos with very high production values.

Today I am very happy to talk to Tomáš Sláma and Richard Hladík about how the channel started, how they choose and produce their videos, and what advice they have for others who want to start their own YouTube channels.

[1] https://polylog.cc and https://www.youtube.com/@PolylogCS
[2] Assistant Professor at Charles University in Prague, https://vaclavrozhon.github.io
[3] https://vvolhejn.com
[4] PhD student at ETH Zurich, https://rihl.cz
[5] Founder and lead developer of Climbuddy, https://slama.dev

POLYLOG: WIDELY ACCESSIBLE VIDEOS ABOUT TCS

A Conversation with Tomáš Sláma and Richard Hladík

Can you describe what motivated you to start the Polylog[1] YouTube channel?

Richard: We make videos about computer science, and especially theoretical computer science. The channel originally grew out of 3Blue1Brown's Summer of Math Exposition[2], which encourages people to make mathematical explainer videos. Václav Rozhoň and Václav Volhejn decided to give it a try. Their first video did not really take off, but they tried again the next year and that video eventually blew up. I joined later, and Tomáš, who already had his own YouTube channel, joined after that.

Tomáš: For me, joining forces simply made sense. None of us has unlimited time, and different people enjoy different parts of the process. I like making animations; others enjoy writing the script or working out the theory. Instead of each of us making a complete video from start to finish, we can focus on the parts we are best at and enjoy most.

Who is your intended audience?

Richard: It is hard to pinpoint exactly. For me, I often think of myself five years ago, when I was an undergraduate student. We want the videos to be accessible, but we also have to satisfy our own urge to go down technical rabbit holes. Václav Rozhoň, who often writes the scripts, would happily add another long technical section at the end, and then we have to cut things back.

Tomáš: The target audience varies somewhat from video to video, but I agree that an undergraduate is a good mental model. When I was at university, I liked videos that gave me an accessible way into a topic I did not know well yet, but still let me get my teeth into something substantial. There is usually a fair amount we would like to include and eventually cut. The back-and-forth hopefully leaves us with something that is both understandable and fun to watch.

How did the four of you come together, and how do your different backgrounds fit into the channel?

[1] `https://polylog.cc` and `https://www.youtube.com/@PolylogCS`
[2] `https://some.3b1b.co`

Richard: We mostly knew each other through the Czech community around high-school mathematics and programming competitions and the camps associated with them. It is a fairly small community, so people get to know one another quickly. I later lived with one member of the group and did research with another, so those connections became quite close.

Tomáš: I came in somewhat differently. I had my own YouTube channel, and we had already been in contact through a pair of related videos. At some point Václav Rozhoň asked whether I wanted to join. I did not really have enough time to keep producing full videos on my own, so joining *Polylog* meant that I could concentrate on the parts I enjoy most, especially animation. That combination of strong theory on one side and technical or visual skills on the other works very well for us.

You started the channel in 2021 and have made only 19 videos, yet you now have more than 119,000 subscribers and roughly seven million views. What do you think is the recipe for that success?

Richard: The first thing I should say is that I often still feel that I do not really know what I am doing. I am not sure that we have a precise formula. But there is clearly a niche for creators who make one high-effort video every few months, or even one per year, and have an audience that is willing to wait for it. We seem to have ended up in that niche, even if that was not a deliberate strategy.

Tomáš: Over time, you do learn which kinds of topics people find interesting. The Rubik's Cube video[3] is a good example: the starting point is immediately accessible, but the same topic also lets us explain how to break DES ciphers using bidirectional search. We try to find subjects that have not already been covered from exactly the same angle, where we feel that we can add our own perspective. There are many computer-science videos on YouTube, so having an angle that distinguishes you matters.

How important are the high production values and the YouTube recommendation algorithm?

Tomáš: To me, putting more effort into a video is a bit like getting another roll of the die with the YouTube algorithm. If the video is pleasant to watch, has good graphics, and is carefully made, perhaps the chance that it gets picked up is higher. But there is a lot of survivorship bias here: I know channels that make very good videos and still do not get much attention. I do not think there is a magic formula. You make good videos consistently, keep rolling the dice, and sometimes one lands.

[3] `https://www.youtube.com/watch?v=wL3uWO-KLUE`

Richard: I agree that the production quality helps, but I think a lot of the magic is in the topic and the script. The animations are nice, but the topic has to work at two levels: it should be accessible enough that somebody can enter without much background, while still having enough depth later on to keep people who already know the basics interested. That balance is very important for us.

What future projects do you have planned for Polylog?

Richard: We have a beta version of our next video, which is about the Busy Beaver function. Václav Rozhoň met Scott Aaronson at a conference and asked whether he would do an interview. They talked about Busy Beaver and related topics, and we decided to build a video around that conversation.

Tomáš: We have also started to like the format of combining our explanation with an interview with somebody who really knows the topic. Veritasium[4] does this very well: instead of one narrator speaking throughout, an expert can come in for a short segment and add another perspective before the explanation continues. We tried that in the previous video, we are doing it again now, and I think it makes the videos more interesting.

If some of our readers wanted to get started on YouTube, what advice would you give them?

Tomáš: Your first videos are probably going to be bad, and that is fine. Looking back at the first videos on my own channel, there are many things I would do differently now. You improve at the script, the voice-over, the animations, and everything else by actually making videos. It can be discouraging when the first few get little attention, but if you care about it, I think the important thing is to keep going.

Richard: It is useful to have some kind of feedback loop, but do not over-focus on views. Our Rubik's Cube video sat there for months without much happening and then suddenly got picked up by the recommendation algorithm and gained a huge number of views. So "this did not get views immediately" is not necessarily useful feedback. Reflect on what you would improve, ask other people, and keep making things. There is that startup advice that if your first product is not bad, you launched it too late. I think something similar applies here.

People are talking a lot about the possible impact of AI on theoretical computer science, and in particular about the growing importance of exposition and building understanding as a community. Do you think channels like Polylog could play a role there?

[4] `https://www.youtube.com/@veritasium`

Tomáš: I have a somewhat biased perspective because I do not enjoy consuming AI-generated content very much. I use AI for programming, and it can be very useful there, but I do not particularly want to read or watch material that was generated by an LLM. I suspect there are many people who feel similarly. So there is still something valuable about hearing people explain a topic they genuinely find interesting.

Richard: If AI eventually produces a lot of new mathematics, then it depends on what those results look like. If the result is something where a human can still understand the main idea—say, a new construction or counterexample—then a channel like ours could be a very good medium for explaining the intuition. If the mathematics becomes so exotic that people themselves struggle to understand it, then of course exposition becomes much harder. Our advantage is that we try to build intuition visually, and good animations can sometimes make quite difficult ideas much easier to grasp.

What are you currently most excited about, in your own work and for the future of Polylog?

Richard: On the research side, I am currently excited about some work Václav Rozhoň and I are doing on data structures, in particular splay trees. For *Polylog*, I am looking forward to seeing how the interview-based format develops.

Tomáš: I am not in academia, so I do not really have a research answer. For *Polylog*, I am mostly excited about continuing to make fun animations.

Finally, is there anything else you want our readers to know?

Tomáš: I would encourage more people in the research community to make this kind of material. It is rare to find somebody who both knows a technical subject very deeply and also has all the skills needed to make a polished video. But you do not have to do everything yourself. An academic who knows the theory can team up with somebody who enjoys animation, editing, or visual communication. I think that is very fertile ground for high-quality educational content.

Richard: And if somebody likes the idea but does not know where to start, the Summer of Math Exposition community is worth looking at. It has a Discord server[5] where people discuss projects, learn from one another, and can potentially find collaborators. The exposition is also broader than just traditional videos, so there are ways to experiment with other explanatory formats as well.

Thank you for this interview, Tomáš and Richard!

[5] See the Discord server linked on `https://some.3b1b.co/contact`.

The Computational Complexity Column

BY

Michal Koucký

Computer Science Institute, Charles University
Malostranské nám. 25, 118 00 Praha 1, Czech Republic
koucky@iuuk.mff.cuni.cz
https://iuuk.mff.cuni.cz/~koucky/

BEATCS no 150

LEARNING, COMPRESSION, AND META-COMPLEXITY

Halley Goldberg[*]
Department of Computer Science
University of Warwick

Mikito Nanashima[†]
School of Computing
Institute of Science Tokyo

Igor C. Oliveira[‡]
Department of Computer Science
University of Warwick

Abstract

The idea that learning is closely tied to compression has long served as both a powerful principle and, at times, an informal analogy across machine learning and learning theory. Here we focus on settings in which that intuition can be made mathematically precise, surveying formal connections among learning algorithms, data compression, and computational complexity theory, with an emphasis on recent progress and current frontiers. We describe how techniques from complexity theory can guide the design of both supervised and unsupervised learning algorithms with provable guarantees, and conversely, how learning algorithms can yield new insights and results in complexity theory. A central theme is the role of data compression as a bridge between learning and complexity, especially through an area called meta-complexity, which studies the computational difficulty of compression problems and time-bounded variants of Kolmogorov complexity.

[*]Email: halley.goldberg@warwick.ac.uk
[†]Email: nanashima@comp.isct.ac.jp
[‡]Email: igor.oliveira@warwick.ac.uk

1 Introduction

1.1 Overview

We describe connections between learning, compression, and meta-complexity (see Figure 1). We start off with a brief introduction to each of these themes, emphasising some important conceptual points along the way.

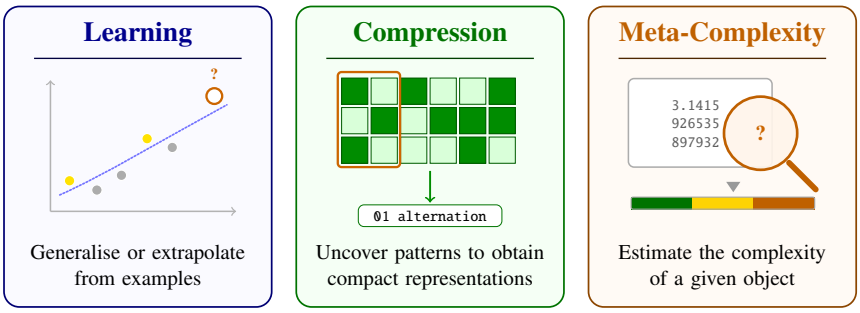


Figure 1: Three perspectives on structure in data: learning, compression, and meta-complexity. We investigate correspondences among them through several lenses, including Shannon's information theory, modern variants of (time-bounded) Kolmogorov complexity, and the theory of computational pseudorandomness. Interestingly, these connections can be used both to design new learning algorithms and to prove that certain compression problems admit no efficient algorithms.

Learning and Compression. At a high level, learning and compression are two ways of asking whether observed data admit a concise explanation:

- A *learning algorithm* uses examples to construct a model that makes useful predictions on new data.
- A *compression algorithm* represents the data using fewer bits, together with a procedure that recovers the original data.

The outputs are different, but both tasks succeed by identifying *structure* or *regularity*: a pattern that permits prediction also makes some outcomes more likely than others, and likely outcomes can be assigned shorter descriptions. Conversely, a short description is evidence that the object is not completely unstructured and can therefore be a useful starting point for prediction.

This connection takes different forms in supervised and unsupervised learning. In *supervised learning*, examples arrive with labels, and the learner seeks a hypothesis that predicts the label of a fresh example. Here compression may

mean inferring from a large labelled sample a short consistent hypothesis, as in Occam's Razor, or retaining a small subsample from which a good hypothesis can be reconstructed. In *unsupervised learning*, there are no labels. The learner may instead produce an approximate distribution that generates the observations, or predict the next part of a sequence from its prefix. Here the relationship with *source coding* is especially direct: if a probabilistic model assigns probability p to an outcome, then the outcome can be encoded using roughly $\log(1/p)$ bits. Thus accurate probabilistic predictions lead to short descriptions of likely data, while valid code lengths can conversely be interpreted as probability assignments. This correspondence connects distribution learning, sequence prediction, and source coding.

One of the main goals of this survey is to determine precisely how far this correspondence extends. We formulate the relevant computational tasks and present reductions showing when a learner can be transformed into a compressor, and when compression can be used to construct a learner.

Two Theories of Information. We use two complementary notions of *information*. In *Shannon's information theory*, a message is drawn from a *probability distribution*, and entropy measures the optimal expected description length when coding repeated outputs of that source [Sha48] (see [CT91]). This is a statistical and distributional theory: the information content of an outcome is determined by its probability under the chosen source. In contrast, in *algorithmic information theory*, the Kolmogorov complexity of an *individual string* is the length of its shortest generating program [Kol65] (see [LV19]). No source distribution needs to be specified, and the theory specifies the information content of each string.

These two theories are related, but not interchangeable. For instance, *source coding*, mentioned above, links probability distributions with description length: a distribution with a short sampler gives short algorithmic descriptions to its typical outputs. Nevertheless, compressing individual strings in this way may be computationally expensive and may require access to the code of the sampling procedure; moreover, a string with a short program need not be well compressed by a particular sampler and the associated source coding procedure.

A major focus of this survey is on connections between learning and compression under *computational efficiency constraints*. Consider for instance Kolmogorov complexity. It describes the shortest possible program, but it places no bound on how long that program may take to produce its output. Given a string, estimating its Kolmogorov complexity is *uncomputable* in general. Moreover, central properties of the theory do not automatically survive once efficiency is im-

posed. For example, *symmetry of information* holds for Kolmogorov complexity up to a small additive loss, whereas its time-bounded analogues do not hold in general (see, e.g., [HIL+23, HMO26]). Classical Kolmogorov complexity, while philosophically relevant, is not by itself an adequate foundation for computational learning. To study *efficient* learning through compression, we need both a theory of *time-bounded* descriptions and a theory of the *computational complexity* of finding or recognizing them.

Meta-Complexity. These considerations lead naturally to *meta-complexity*, the study of the computational complexity of problems that ask about the complexity of input objects. Canonical examples ask whether a string has a short program that generates it within a prescribed time bound, or whether the truth table of a Boolean function can be computed by a small Boolean circuit. Such problems formalise the task of detecting whether an object is compressible or structured; see [Sim23] for further context. The central question is no longer merely whether short descriptions exist, but whether they can be recognised or found efficiently.

The complexity of these compression problems is intertwined with some of the most fundamental unresolved questions in theoretical computer science. In the terminology of Impagliazzo's five worlds [Imp95], we do not yet know which computational world we inhabit: whether worst-case hard problems remain hard on typical inputs, whether such hardness supports secure cryptography, and how broadly efficient algorithms can exploit structure. Meta-complexity, through the study of the complexity of compression problems, turns out to provide a suitable framework in which to investigate this tension between tractability on typical inputs and cryptographic hardness. In turn, this will allow us to compare the feasibility of different learning tasks and their relation to Impagliazzo's computational worlds.

There is, however, an important mismatch between learning and the standard formulation of compression problems in meta-complexity. Most canonical problems in meta-complexity are *decision problems*. For example, given a string x, a time bound t, and a description length s, the Minimum Time-Bounded Kolmogorov Complexity problem (MINKT) asks whether some program of size at most s produces x in at most t steps. The corresponding *search problem* asks for such a program. Learning is inherently a search task: a learner must output a hypothesis, sampler, or predictor, rather than merely certify that a concise representation exists. Thus, in principle, an efficient algorithm that merely recognises compressible data does not automatically yield a corresponding search algorithm. Interestingly, many results surveyed here bridge this gap indirectly: a decision procedure separating structured objects from random ones is sufficient to construct a learning algorithm.

Having introduced the main themes, we elaborate on the scope of the survey. We first revisit classical connections in which compression yields generalisation in supervised learning. After this, we compare learning tasks of different strengths, from universal sequence extrapolation and distribution learning to more traditional forms of PAC learning, and relate this hierarchy to average-case complexity, meta-complexity, and cryptography. Next, we explain how algorithms for detecting objects of low description complexity can produce learning algorithms, both through reconstruction procedures in supervised learning and through time-bounded variants of Solomonoff induction in unsupervised learning. This perspective also reveals novel phenomena such as generic learning speedups and, in certain settings, equivalences between efficient learning and the easiness of an associated meta-complexity problem. Finally, we discuss an unexpected converse direction, showing how sufficiently powerful learning algorithms can yield computational hardness results and new lower bounds in meta-complexity. For a more detailed description of each section of the survey, we refer to Section 1.2 below.

We place particular emphasis on recent developments in these rapidly evolving areas. These include new connections among learning, time-bounded Kolmogorov complexity, meta-complexity, cryptography, average-case complexity, and universal sequence extrapolation. Our aim is not only to state these results, but also to explain the underlying ideas and, where appropriate, provide proofs or detailed proof sketches. The emerging theory remains far from complete, so open problems form an integral part of the survey.

Intended Audience and Required Background. The primary audience includes students and researchers interested in computational complexity, computational learning, and theoretical computer science more broadly. We aim to make the main results and proof ideas accessible without assuming prior expertise in every area involved. The preliminaries review the necessary concepts and definitions, and provide pointers to standard textbooks that cover the required material in more detail.

We also hope that researchers and students in machine learning will find the survey useful as an entry point to the complexity-theoretic foundations of learning and compression. We warn that some arguments might omit standard background or routine intermediate steps. Readers may find it helpful to use their preferred AI assistant as an interactive companion for unpacking definitions, reconstructing such steps, or translating between the terminology of theoretical computer science and intuitions from machine learning.

We stress that the selection of topics necessarily reflects the authors' back-

ground and perspective. In particular, we do not attempt to provide a comprehensive account of learning and compression from the perspective of statistics, information theory, or theoretical machine learning. We refer the reader to Section 8 for some additional pointers and complementary perspectives.

Finally, to present a general theory, much of this survey considers expressive learning problems involving unrestricted polynomial-size Boolean circuits or all efficiently samplable example distributions. As explained later in the text, these learning problems are intractable under cryptographic conjectures. However, many ideas and techniques surveyed here also apply to more restricted learning tasks, and such tasks are more likely to admit efficient algorithms. It is possible, therefore, that some results can inform practical methods or analyses, and it would be interesting to see further work along these lines. In fact, going beyond the formal connections covered here, the compression perspective has found numerous concrete applications in both classical and contemporary machine learning. A comprehensive treatment of the rich empirical literature on learning and compression lies beyond the scope of this survey, although we provide some pointers in Section 8.

1.2 Organisation

The remainder of the survey is organised as follows.

- **Section 2: Preliminaries.** We establish the notation and background used throughout the survey. After reviewing the required probability and information-theoretic notions, we define the supervised and unsupervised learning frameworks considered later, including sequence extrapolation, distribution learning, PAC learning, agnostic learning, and heuristic learning. We also recall the necessary concepts from cryptography, circuit complexity, Kolmogorov complexity, meta-complexity, and natural properties.

- **Section 3: Classical Connections Between Learning and Compression.** We present two fundamental roles of compression in supervised learning. Model compression formalises the Occam principle that a short hypothesis fitting the observed labels is unlikely to generalise poorly, while sample compression retains a small portion of the labelled sample from which a hypothesis can be reconstructed. These results illustrate, without computational efficiency considerations, how the amount of information retained from a sample controls generalisation.

- **Section 4: Complexity-Theoretic Foundations of Learning and Compression.** We compare five increasingly weak "learning paradises", ranging

from universal sequence extrapolation and distribution learning to heuristic PAC learning, and establish reductions among them. We then place these learning tasks within Impagliazzo's worlds, relating their feasibility to average-case complexity, meta-complexity, and cryptographic hardness. Finally, we study white-box learning, where the learner receives the code of a sampler generating its labelled examples, and connect this additional access to the inversion of one-way functions.

- **Section 5: Learning via Meta-Complexity and Algorithmic Information Theory.** We explain how algorithms that distinguish low complexity objects from random ones can be combined with reconstruction procedures of pseudorandom generators to obtain supervised learning algorithms. For unsupervised learning, Section 5.2 revisits Solomonoff induction and asks when prediction based on shortest programs can be implemented efficiently using estimates of time-bounded Kolmogorov complexity. We also present equivalences between learning and meta-complexity in Section 5.3, and a technique to boost the running time of learning algorithms in Section 5.4.

- **Section 6: Learning vs Compression via Shannon's Information Theory.** We develop the correspondence among source coding, universal coding, and distribution learning, first at the information-theoretic level and then under computational efficiency constraints. The discussion covers arithmetic coding, next-bit prediction, source coding theorems for time-bounded Kolmogorov complexity, conditional sampling, and symmetry of information. We conclude by relating this framework to autoregressive language modelling, where next-token probabilities determine predictions and code lengths.

- **Section 7: Meta-Complexity and Computational Hardness from Learning.** We study the converse direction, explaining how nontrivial deterministic, randomized, and quantum learners can imply circuit size lower bounds, a major challenge in computational complexity theory. We then show how this paradigm can establish unconditional lower bounds for meta-complexity problems. In particular, Section 7.3 describes a general template in which an assumed algorithm for measuring compressibility produces a learner whose complexity-theoretic consequences contradict the original easiness assumption.

- **Section 8: Related Work.** We provide pointers to related connections and research directions outside the survey's main scope, including further interactions among learning, compression, algorithmic information theory, complexity theory, and cryptography. We also provide links to the empirical

literature on learning and compression and to related practical aspects of machine learning.

Acknowledgements. We thank Shuichi Hirahara for helpful discussions on deriving meta-complexity lower bounds from deterministic learning, which led to Item (*ii*) of Theorem 7.1. We also thank Valentine Kabanets for helpful feedback on an earlier draft of this survey.

Use of AI Tools. We used AI tools to search for relevant literature, assist with the preparation of diagrams, identify typographical errors, check proofs for correctness, and make textual edits to improve the exposition.

Funding Information. This work was supported in part by the UKRI Frontier Research Guarantee Grant EP/Y007999/1 and the Centre for Discrete Mathematics and its Applications (DIMAP) at the University of Warwick. The authors would also like to thank the Isaac Newton Institute for Mathematical Sciences, Cambridge, for support and hospitality during the programme Logical Foundations of Complexity Theory (LFC), where part of the work on this paper was undertaken. The LFC programme was supported by EPSRC grant EP/Z000580/1.

2 Preliminaries

2.1 Basic Notation

For $m \geq 1$, we let $[m] = \{1, \ldots, m\}$.

We denote the empty string by ϵ, and the length of a string x by $|x|$. For strings $x \in \{0,1\}^n$ and $y \in \{0,1\}^m$, their concatenation is written as $xy \in \{0,1\}^{n+m}$.

Given a string $z \in \{0,1\}^n$ and a set $S \subseteq [n]$, we let $z|_S \in \{0,1\}^{|S|}$ denote the string obtained by restricting z to the coordinates in S.

When considering algorithms, PPT denotes probabilistic polynomial time.

2.2 Probability Theory and Information Theory

If $x \sim D$, then x is sampled from the distribution D. For a set S, $x \sim S$ denotes a uniformly random element from S. For any a in the support of D, $D(a)$ denotes its probability. We write $\mathcal{U}_\ell$ for the uniform distribution over $\{0,1\}^\ell$. We use $\Delta_{\mathsf{TV}}(P, Q)$ to denote the statistical distance of two distributions with the same support. We will also use KL-divergence and its relation to statistical distance, as reviewed below.

Definition 2.1 (KL Divergence)**.** Let P and Q be distributions on the same finite set Ω. The *Kullback–Leibler divergence* from P to Q is defined by

$$\mathsf{KL}(P \,\|\, Q) \triangleq \mathbb{E}_{x \sim P}\left[\log \frac{P(x)}{Q(x)}\right] = \sum_{x \in \Omega} P(x) \log \frac{P(x)}{Q(x)}.$$

Lemma 2.2 (Pinsker's Inequality)**.** *Let P and Q be distributions on the same finite set. Then*

$$\Delta_{\mathsf{TV}}(P, Q)^2 \leq \frac{1}{2} \mathsf{KL}(P \,\|\, Q).$$

Definition 2.3 (Conditional KL Divergence)**.** Let (P_X, P_Y) and (Q_X, Q_Y) be random pairs on the same finite product set $\mathcal{X} \times \mathcal{Y}$, and suppose that $\mathsf{Support}(P_X) \subseteq \mathsf{Support}(Q_X)$. The *conditional Kullback–Leibler divergence* from P_Y conditioned on P_X to Q_Y conditioned on Q_X is defined by

$$\mathsf{KL}(P_Y \mid P_X \,\|\, Q_Y \mid Q_X) \triangleq \mathbb{E}_{x \sim P_X}\left[\mathsf{KL}(P_Y \mid P_X = x \,\|\, Q_Y \mid Q_X = x)\right].$$

Lemma 2.4 (Chain Rule for KL Divergence)**.** *Let $(P_1, \ldots, P_m)$ and $(Q_1, \ldots, Q_m)$ be random vectors on the same finite product set $\mathcal{X}_1 \times \cdots \times \mathcal{X}_m$, and suppose that $\mathsf{Support}(P_1, \ldots, P_m) \subseteq \mathsf{Support}(Q_1, \ldots, Q_m)$. For every $i \in [m]$, write*

$$P_{<i} \triangleq (P_1, \ldots, P_{i-1}) \qquad \text{and} \qquad Q_{<i} \triangleq (Q_1, \ldots, Q_{i-1}),$$

where $P_{<1}$ and $Q_{<1}$ denote the empty tuple. Then

$$\begin{aligned}\mathsf{KL}((P_1,\ldots,P_m)\,\|\,(Q_1,\ldots,Q_m)) &= \sum_{i=1}^{m} \mathsf{KL}(P_i \mid P_{<i} \,\|\, Q_i \mid Q_{<i}) \\ &= m \cdot \mathbb{E}_{i\sim[m]}\left[\mathsf{KL}(P_i \mid P_{<i} \,\|\, Q_i \mid Q_{<i})\right].\end{aligned}$$

We use the following definition of entropy.

Definition 2.5 (Entropy)**.** For a probability distribution $\mathcal{D}$ over a finite set, define the entropy of $\mathcal{D}$ as

$$\mathsf{H}(\mathcal{D}) \triangleq \sum_{x\in\mathsf{Support}(\mathcal{D})} \mathcal{D}(x)\log\frac{1}{\mathcal{D}(x)}.$$

For more information about information theory and its applications, see [CT91, PW25].

2.3 Learning Frameworks

In *supervised learning*, the learner receives examples together with the desired answers, usually labels produced by an unknown target function, and the goal is to predict the correct label on new examples. In *unsupervised learning*, the learner receives samples without such labels and must instead capture some structure of the source that generated them.

In the learning models introduced below, sequence extrapolation and distribution learning are unsupervised, since the input consists only of samples from an unknown process or distribution. On the other hand, PAC learning and heuristic learning are supervised, since the learner is given labelled examples of the form $(x, f(x))$ and aims to predict the label of future examples.

2.3.1 Sequence Extrapolation

A common way to learn from data is to ask what should come next. After observing a prefix of a sequence, a learner tries to reproduce the distribution of the next block without being told the process that generated the sequence. This viewpoint captures settings ranging from sequential prediction to universal induction: the past is useful precisely when it reveals regularity in an unknown source. Here we make that intuition computational by assuming only that the sequence is produced by a short randomized program running within a prescribed time bound. Since the source itself may be random, the learner is not expected to guess the next block exactly. Instead, it should make a probabilistic prediction close to the source's true conditional distribution.

Fix a *sequence length* $m \in \mathbb{N}$, a *block length* $n \in \mathbb{N}$, an *error bound* $\varepsilon > 0$, a *confidence value* $\delta > 0$, a *description length* $s \in \mathbb{N}$, and a time bound $t \in \mathbb{N}$.

We consider *randomized* Turing machines M of description length at most s (according to a fixed universal randomized machine U^r). We will be interested in the random variable $U^r(M, \epsilon, 1^t)$ encoding the output of M on the empty string ϵ when U^r runs over the input pair (M, ϵ) for t steps. If this simulation does not produce an output, we set the output to $0^{n \cdot m}$. For convenience, for a choice of n and m, if the output x of $U^r(M, \epsilon, 1^t)$ is of length less than $n \cdot m$, we add 0's to it and consider instead the string $x' = x0^{nm-|x|}$. Therefore, given n and m, we will view $x \sim U^r(M, \epsilon, 1^t)$ as a *random variable* $(x_1, \ldots, x_m)$ distributed over $\{0,1\}^{n \cdot m}$. For simplicity, we might omit the empty string ϵ.

We consider a task called $(m, n, s, t, \varepsilon, \delta)$-*extrapolation*. In this computational task, we consider a randomized algorithm L such that, for every (unknown) choice of M of length at most s and index i, given the first $i-1$ blocks $(x_1, \ldots, x_{i-1})$ of $x \sim U^r(M, 1^t)$ (denoted by $x_{<i}$), the goal is for $L(x_{<i}) \sim \{0,1\}^n$ to be distributed according to $(\mathcal{X}_i \mid x_{<i})$, where for a given choice of n, m, M, t, the random variable $(\mathcal{X}_i \mid x_{<i})$ denotes the conditional distribution of the i-th block x_i given preceding blocks $x_1, \ldots, x_{i-1} \in \{0,1\}^n$. Formally, we have the following definition.

Definition 2.6 (Sequence Extrapolation)**.** We say that a randomized algorithm L *solves* $(m, n, s, t, \varepsilon, \delta)$-*extrapolation* if for every choice of $M \in \{0,1\}^{\leq s}$ the following holds:

$$\Pr_{i \sim [m],\ x \sim U^r(M,1^t)} \Big[\Delta_{\mathsf{TV}}(L(x_{<i}), (\mathcal{X}_i \mid x_{<i})) \leq \varepsilon \Big] \geq 1 - \delta.$$

We assume that the parameters $(m, n, s, t, \varepsilon^{-1}, \delta^{-1})$ are also given to L in unary when considering uniform algorithms and parameter sequences.

For broader treatments of sequential prediction and algorithmic approaches to extrapolation, see [CL06, HQC24].

2.3.2 Learning Distributions

Sometimes the goal is not merely to predict the next observation, but to construct a model of the source itself. Given independent samples from an unknown distribution, a distribution learner seeks a new sampling procedure whose outputs are statistically indistinguishable from those of the original source, up to a prescribed error. Such a model captures enough of the source's structure to generate fresh representative data, even though it need not reveal how the source was originally implemented.

Formally, we consider the task of learning an unknown discrete distribution D from a fixed class $\mathcal{D}$ of distributions [KMR+94]. The learner has access to

i.i.d. samples from D, and the goal is to output a circuit $C\colon \{0,1\}^T \to \{0,1\}^n$ that samples a distribution that is ε-close to D.

Definition 2.7 (Learning Distributions [KMR+94])**.** Let $\mathcal{D} = \{\mathcal{D}_n\}_{n\geq 1}$ be a class of distributions, where each $\mathcal{D}_n$ is a set of distributions supported over $\{0,1\}^n$. The class $\mathcal{D}$ is said to be learnable in time $T(n,\varepsilon,\delta)$ and with sample complexity $m(n,\varepsilon,\delta)$ if there is a randomized algorithm A such that, for every n, $D \in \mathcal{D}_n$, $\varepsilon > 0$, $\delta > 0$, and $m = m(n,\varepsilon,\delta)$, $A(1^n, 1^{\varepsilon^{-1}}, 1^{\delta^{-1}}, \cdot)$ runs in time at most $T = T(n,\varepsilon,\delta)$ and

$$\Pr_{A,\,\vec{x}\sim D^{(m)}}\left[C \leftarrow A(1^n, 1^{\varepsilon^{-1}} 1^{\delta^{-1}}, \vec{x}) \;\text{ and }\; \Delta_{\mathsf{TV}}(C(\mathcal{U}_T), D) \leq \varepsilon\right] \geq 1-\delta, \tag{1}$$

where $C(\mathcal{U}_T)$ is the distribution sampled by C, and $\vec{x} \sim D^{(m)}$ denotes a tuple of m i.i.d. samples from D.

For convenience, when Equation (1) holds for a distribution D, we say that A ε-learns D with probability at least $1-\delta$.

For functions $s,t\colon \mathbb{N} \to \mathbb{N}$, we can consider the class $\mathcal{D}^{s,t} = \{\mathcal{D}_n^{s(n),t(n)}\}_{n\geq 1}$ of distributions, where each $\mathcal{D}_{n'}^{s',t'}$ consists of the set of all distributions D supported over $\{0,1\}^{n'}$ sampled by a randomized machine M that runs in time at most t' and has description length at most s'.

Similarly, we can consider the distribution learning problem for $\mathcal{D}_n^{s,t}$ when $n,s,t \in \mathbb{N}$ are *input parameters* provided to the learning algorithm $A(1^n, 1^{\varepsilon^{-1}}, 1^{\delta^{-1}}, 1^t, 1^s, \cdot)$.

2.3.3 PAC Learning

PAC (Probably Approximately Correct) learning formalises the familiar supervised task of learning a rule from labelled examples. An unknown target concept assigns a label to each input, and the learner observes examples drawn from a distribution that determines which inputs matter. From sufficiently many such examples, the learner should produce a hypothesis that makes few mistakes on a fresh input from the same distribution. The name highlights the two allowances that make the goal realistic: the hypothesis need only be approximately correct, and the learner need only succeed with high probability. In other words, PAC learning asks for reliable generalisation to the inputs that are likely to be encountered.

Formally, in the PAC learning framework, one considers an unknown function $f\colon \{0,1\}^n \to \{0,1\}$ for some $n \in \mathbb{N}$. Such a function is referred to as a *concept*. A *concept class* C refers to a set of concepts, with C_n denoting $C \cap \{f : \{0,1\}^n \to \{0,1\}\}$ for a fixed $n \in \mathbb{N}$. The PAC learnability of a concept class is defined with respect to a class $\mathcal{D}$ of probability distribution ensembles $D = \{D_n\}_{n\in\mathbb{N}}$, where each D_n is supported over $\{0,1\}^n$.

PAC learners are provided a *labelled example oracle* $\mathsf{EX}(f, D_n)$, with each query returning an independently sampled pair $(x, f(x))$, where $x \sim D_n$.

Definition 2.8 (PAC Learning [Val84]; see [Ser25] for a survey)**.** Let C be a concept class, and let $\mathcal{D}$ be a class of distributions. An algorithm A *PAC learns* C over $\mathcal{D}$ if, for all $D \in \mathcal{D}$, all sufficiently large $n \in \mathbb{N}$, all $\delta, \varepsilon \in (0, 1)$, and all $f \in C_n$,

$$\Pr_{A;\, \mathsf{EX}(f,D_n)}\left[A^{\mathsf{EX}(f,D_n)}(1^n, 1^{\varepsilon^{-1}}, \delta^{-1}) \text{ outputs } h \text{ s. t. } \Pr_{x\sim D_n}[h(x) = f(x)] \geq 1 - \varepsilon \right] \geq 1 - \delta.$$

For $s : \mathbb{N} \to \mathbb{N}$, we can consider the class $C^s = \{C_n^{s(n)}\}_{n\in\mathbb{N}}$ of concepts, where each $C_n^{s(n)}$ is the subset of C_n with representations of size $s(n)$. For example, if $C = \mathsf{P/poly}$, then $C^s = \mathsf{SIZE}[s]$, the set of functions $f : \{0, 1\}^n \to \{0, 1\}$ computable by circuits of size $s(n)$. As in the previous section, we define $\mathcal{D}^{a,t}$ to be the class of distributions samplable by a t-time machine with description length a. To make these parameters explicit, s, t, a may be given as inputs to the learning algorithm.

One may also consider the model of PAC learning with *membership queries*, which is exactly the same as the definition above, except the learner A is additionally provided an f-oracle.

A learner that always outputs a hypothesis h belonging to C is called *proper*. Otherwise, the learner is *improper*.

If ε is not given as an input but the hypothesis output by A satisfies $\Pr_{x\sim D_n}[h(x) = f(x)] \geq 1/2 + \gamma(n)$ with probability $1 - \delta$ for some $\gamma(n) = \mathsf{poly}^{-1}(n)$, then A is referred to as a *weak learner*.

For textbook introductions to computational learning theory and the PAC framework, see [KV94a, SSBD14].

2.3.4 Agnostic Learning

The PAC model assumes that every observed label is generated by some target concept from the class under consideration. In practice, labels may be noisy, internally inconsistent, or governed by a relationship that the chosen class cannot represent. *Agnostic learning* removes this assumption and asks what can still be learned from an arbitrary joint distribution over inputs and labels. Since nearly perfect prediction may then be impossible, success is measured relative to the best concept available in the class. The learner must perform almost as well as that benchmark without knowing either the source of the labels or the benchmark concept in advance. This formulation preserves a meaningful learning objective while allowing both noise and model misspecification.

In technical terms, one cannot assume that the given examples are labelled according to a function f belonging to the relevant concept class C. In fact, one cannot assume that the labels correspond to a fixed function at all. In particular,

we now consider a class $\mathcal{E}$ of joint distribution ensembles $E = \{E_n\}_{n\in\mathbb{N}}$, where each E_n is supported over $\{0,1\}^n \times \{0,1\}$. The example oracle $\mathsf{EX}(E_n)$ simply returns an independent random sample from E_n each time it is queried.

Now, there is clearly no hope in general for an efficient learner to produce a hypothesis with high correctness probability over E (i.e., $\Pr_{(x,b)\sim E_n}[h(x) = b] \approx 1$).[1] Therefore, as alluded to above, we measure the success of an agnostic learner relative to the concept class C in question, asking for a hypothesis that does "almost as well" as the best concept in C. To that end, for $n \in \mathbb{N}$, define

$$\mathsf{opt}_n(C, E) \triangleq \max_{c\in C_n}\left\{\Pr_{(x,b)\sim E_n}[c(x) = b]\right\}.$$

Definition 2.9 (Agnostic Learning [KSS94, Hau92])**.** Let C be a concept class, and let $\mathcal{E}$ be a class of joint distributions. An algorithm A *agnostically learns* C over $\mathcal{E}$ if, for all $E \in \mathcal{E}$, all sufficiently large $n \in \mathbb{N}$, and all $\delta, \varepsilon \in (0,1)$,

$$\Pr_{A;\,\mathsf{EX}(E_n)}\left[A^{\mathsf{EX}(E_n)}(1^n, 1^{\varepsilon^{-1}}, \delta^{-1}) \text{ outputs } h \text{ s. t. } \Pr_{(x,b)\sim E_n}[h(x) = b] \geq \mathsf{opt}_n(C, E) - \varepsilon\right] \geq 1-\delta.$$

In this context, if we say that an algorithm agnostically learns over the distribution class $\mathcal{D}^{a,t} = \{\mathcal{D}_m^{a(m),t(m)}\}_{m\geq 1}$ as defined above, we interpret each $\mathcal{D}_m^{a(m),t(m)}$ as a set of joint example distributions supported over $\{0,1\}^{m-1} \times \{0,1\}$.

For a textbook treatment of agnostic learning, see [SSBD14].

2.3.5 Heuristic Learnability

Standard supervised learning guarantees usually require the learner to succeed for every target concept in the class. In complexity theory and cryptography, however, target functions often arise from a randomized generation procedure, just as the inputs on which they are evaluated arise from their own sampling process. *Heuristic learnability* reflects this situation by measuring prediction success on average over both the chosen target and the chosen input. The learner may therefore fail on exceptional targets or examples, provided that these failures are sufficiently rare in the overall experiment. Even a small advantage over random guessing can be significant in this setting, making the model especially useful for expressing connections among prediction, average-case hardness, and cryptographic security.

Let C be a concept class, with each $f \in C_n$ having a representation σ_f of size at most polynomial in n. Let $P = \{P_n\}_{n\in\mathbb{N}}$ be a distribution ensemble, with each P_n supported over representations σ_f of target functions $f \in C_n$ such that f can be

[1] For example, consider $E_n = (\mathcal{U}_n, \mathcal{U})$. This joint distribution can be sampled efficiently, but no fixed circuit has correctness probability greater than $1/2$.

evaluated in polynomial time given σ_f. Let $D = \{D_n\}_{n\in\mathbb{N}}$ be another distribution ensemble, with each D_n supported over example strings $x \in \{0,1\}^n$.

Definition 2.10 ([BFKL93])**.** For any $\varepsilon : \mathbb{N} \to (0, 1/2)$, we say that C is ε-*predictable on average* over P and D if there exists an algorithm A such that for all sufficiently large $n \in \mathbb{N}$,

$$\Pr_{\substack{A;\ \sigma_f\sim P_n;\\ \mathsf{EX}(f,D_n);\\ x'\sim D_n}}\left[A^{\mathsf{EX}(f,D_n)}(x') = f(x')\right] \geq 1 - \varepsilon(n).$$

We say that C is *strongly predictable on average* over P and D if it is $(1/q(\cdot))$-predictable on average for any polynomial q. We say it is *weakly predictable on average* over P and D if, for some polynomial q', it is $(1/2 - 1/q'(\cdot))$-predictable on average. We mention that weak predictability for the case of $C = \mathsf{P/poly}$ and all $P, D \in \mathsf{PSAMP}$ implies strong predictability for such C, P, and D [BFKL93].

For $t : \mathbb{N} \to \mathbb{N}$, let $\mathsf{SAMP}[t]$ denote the class of distributions samplable uniformly in time at most t. In the case of a heuristic learner A working on some fixed concept class C^s, $P \in \mathsf{SAMP}[t_1]$, and $D \in \mathsf{SAMP}[t_2]$, the parameters s, t_1, t_2 may be given as additional inputs to A.

See [Nan21] for further background on heuristic learnability and related notions.

2.4 Theoretical Computer Science

We will require several notions from cryptography, complexity theory, Kolmogorov complexity, pseudorandomness, and meta-complexity. For general background in computational complexity, including randomized computation, nonuniform computation, cryptography, and circuit complexity, see [AB09].

2.4.1 Cryptography

Infinitely-Often Cryptography. For the purposes of cryptography, it is important for security to hold *almost everywhere*. That is, one would hope for a cryptographic primitive that cannot be broken at any (but perhaps finitely many) input lengths. Likewise, for the purposes of learning and positive algorithmic tasks more broadly, one would hope for algorithms to be successful at all but finitely many input lengths. While it is reasonable to ask for this property in both cases, it poses a problem if one would like to investigate dualities between the possibility of secure cryptography and the hardness of tasks such as learning. In particular, if the hardness of learning at some fixed input length translates into the hardness of breaking a cryptographic primitive at a related input length, then the impossibility

of learning in the sense described above would only yield security at infinitely many input lengths.

For this reason, to aid in a better understanding of connections between algorithmic hardness and cryptography, we consider cryptography that may only be secure *infinitely often*.

A *pseudorandom function family* is a family of keyed polynomial-time computable functions that look like a random oracle to any efficient observer. Here, we state the definition of an infinitely-often variant.

Definition 2.11 (Infinitely-Often Pseudorandom Function)**.** An *infinitely-often pseudorandom function*, denoted ioPRF, is a polynomial-time computable function family $f = \{f : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}\}_{n\in\mathbb{N}}$ such that, for any inverse polynomial $\delta : \mathbb{N} \to [0,1]$ and PPT machine A, there exist infinitely many $n \in \mathbb{N}$ such that

$$\left| \Pr_{A;\, u\sim\{0,1\}^n}\left[A^{f(u,-)}(1^n) = 1\right] - \Pr_{A;\, R_n}\left[A^{R_n(-)}(1^n) = 1\right]\right| < \delta(n),$$

where $R_n : \{0,1\}^n \to \{0,1\}$ is a uniformly random function over the set of all functions with these parameters.

Auxiliary-Input Cryptography. We also consider variants of cryptographic primitives that are allowed to make use of an additional publicly available non-uniform input string. Since this kind of cryptography generalizes the standard setting in which no such auxiliary string is given, its existence is a weaker assumption.

As with infinitely-often cryptography, auxiliary-input cryptography can arise naturally when investigating connections between secure cryptography and the hardness of learning. In particular, a specific hard instance of learning might translate to a related description of a secure cryptographic primitive. In such settings, one may only obtain auxiliary-input cryptography rather than standard (uniform) cryptography.

An *auxiliary-input function family*

$$f = \{f_z : \{0,1\}^{n(|z|)} \to \{0,1\}^{\ell(|z|)}\}_{z\in\{0,1\}^*}$$

is said to be *polynomial-time computable* if there is a polynomial-time machine F such that for all $z \in \{0,1\}^*$ and $x \in \{0,1\}^{n(z)}$, $F(z,x) = f_z(x)$.

A one-way function is a function that may be computed efficiently but cannot be inverted efficiently. This is a basic building block of many other objects in private-key cryptography. Here, we state the definition of the auxiliary-input variant.

Definition 2.12 (Auxiliary-Input One-Way Function [OW93])**.** An *auxiliary-input one-way function*, denoted AIOWF, is a polynomial-time computable auxiliary-input function family f such that, for any inverse polynomial δ and PPT machine A, there exist infinitely many $z \in \{0,1\}^*$ such that

$$\Pr_{A;\, x\sim\mathcal{U}_{n(|z|)}}\left[A(z, f_z(x)) \in f_z^{-1}(f_z(x))\right] < \delta(|z|).$$

Impagliazzo and Luby [IL89] proved that, if all candidate one-way functions can be inverted in the sense of finding *some* pre-image of a given output, then they can be inverted in the stronger sense of finding *nearly uniformly random* pre-images. Thus, the existence of one-way functions is equivalent to the existence of *distributional one-way functions*, which are secure against adversaries finding nearly uniformly random pre-images. More specifically, for a uniformly random input x and a poly-time computable function f, no adversary A on input $f(x)$ returns x' such that $(x', f(x))$ is close in statistical distance to $(x, f(x))$. Below, the auxiliary-input generalization is stated.

Definition 2.13 (Distributional Auxiliary-Input One-Way Function)**.** A *distributional auxiliary-input one-way function*, denoted distAIOWF, is a polynomial-time computable auxiliary-input function family $f = \{f_z : \{0,1\}^{n(|z|)} \to \{0,1\}^{\ell(|z|)}\}_{z\in\{0,1\}^*}$ such that, for some inverse polynomial δ, for every PPT machine A, there exist infinitely many $z \in \{0,1\}^*$ such that

$$\Delta\big(\ (A(z, f_z(x)), f_z(x)),\ (x, f_z(x))\ \big) > \delta(|z|),$$

where $x \sim \mathcal{U}_{n(|z|)}$, and Δ denotes statistical (total variation) distance.

A *pseudorandom generator* is a deterministic machine that maps a short uniformly random *seed* to a longer output string such that the distribution on outputs is indistinguishable from uniformly random to efficient observers.

Definition 2.14 (Auxiliary-Input Pseudorandom Generator)**.** An *auxiliary-input pseudorandom generator*, denoted AIPRG, is a polynomial-time computable auxiliary-input function family $G = \{G_z : \{0,1\}^{n(|z|)} \to \{0,1\}^{\ell(|z|)}\}_{z\in\{0,1\}^*}$ such that $\ell(|z|) > n(|z|)$, and for any inverse polynomial δ and PPT machine A, there exist infinitely many $z \in \{0,1\}^*$ such that

$$\left|\Pr_{A;\, v\sim\mathcal{U}_{n(|z|)}}[A(z, G_z(v)) = 1] - \Pr_{A;\, u\sim\mathcal{U}_{\ell(|z|)}}[A(z, u) = 1]\right| \leq \delta(|z|).$$

We use the statement $\exists$ *AIPRG* to denote that the above exists for any choice of $\ell(|z|) = n(|z|)^{O(1)}$.

Definition 2.15 (Auxiliary-Input Pseudorandom Function)**.** An *auxiliary-input pseudorandom function*, denoted AIPRF, is a polynomial-time computable auxiliary-input function family $f = \{f_z : \{0,1\}^{n(|z|)} \times \{0,1\}^{n(|z|)} \to \{0,1\}\}_{z\in\{0,1\}^*}$ such that, for any inverse polynomial δ and PPT machine A, there exist infinitely many $z \in \{0,1\}^*$ such that

$$\left| \Pr_{A;\, u\sim\{0,1\}^{n(|z|)}} \left[A^{f_z(u,-)}(z) = 1\right] - \Pr_{A;\, R_{|z|}} \left[A^{R_{|z|}(-)}(z) = 1\right] \right| < \delta(|z|),$$

where $R_{|z|} : \{0,1\}^{n(|z|)} \to \{0,1\}$ is a uniformly random function over the set of all functions with these parameters.

The next statement results from auxiliary-input analogues of standard transformations between one-way functions, distributional one-way functions, pseudorandom generators, and pseudorandom functions [IL89, HILL99, GGM86]. See, e.g., [ABX08] for a discussion.

Lemma 2.16. *The following statements are equivalent.*

- $\exists$ *AIOWF;*
- $\exists$ *distAIOWF;*
- $\exists$ *AIPRG;*
- $\exists$ *AIPRF.*

For more information about one-way functions, pseudorandom generators, and pseudorandom functions, see [Gol01, KL07].

2.4.2 Kolmogorov Complexity

Kolmogorov complexity can be viewed as a universal measure of compression: it quantifies how concisely an individual object can be described by an effective procedure. Any particular compression scheme provides a way to encode certain objects succinctly, while a universal programming language can simulate every such scheme with only a fixed additional overhead. Consequently, Kolmogorov complexity captures, up to an additive constant, the best compression achievable by any computable method. Similarly, time-bounded Kolmogorov complexity captures the best compression achievable when decompression is required to run within a prescribed time bound.[2]

Formally, we fix a time-efficient universal machine U, and define time-bounded Kolmogorov complexity with respect to U (see [LV19] for the required

[2] We refer to [Ble13] for an introduction to data compression.

background in Kolmogorov complexity). In the context of Kolmogorov complexity, whenever we say that a program M running on input y outputs x in t steps, formally, this means that $U(M, y, 1^t) = x$.

Definition 2.17 (K^t Complexity)**.** For $x, y \in \{0,1\}^*$ and $t \in \mathbb{N}$, the *t-time-bounded Kolmogorov complexity* of x conditioned on y is defined as

$$\mathsf{K}^t(x \mid y) \triangleq \min_{M \in \{0,1\}^*} \{|M| \;\big|\; U(M, y, 1^t) \text{ outputs } x\}.$$

We define $\mathsf{K}^t(x)$ as $\mathsf{K}^t(x \mid \epsilon)$.

The definition of K^t requires us to fix the time bound in advance. On the other hand, Levin's notion of time-bounded Kolmogorov complexity simultaneously considers description length and running time.

Definition 2.18 (Levin's Kt Complexity [Lev84])**.** For $x, y \in \{0,1\}^*$, the *Levin complexity* of x conditioned on y is defined as

$$\mathsf{Kt}(x \mid y) \triangleq \min_{t \in \mathbb{N},\, M \in \{0,1\}^*} \{|M| + \lceil \log t \rceil \;\big|\; U(M, y, 1^t) \text{ outputs } x\}.$$

We define $\mathsf{Kt}(x)$ as $\mathsf{Kt}(x \mid \epsilon)$.

When no time bound is considered, we simply write $\mathsf{K}(x)$ to denote the (classical time-unbounded) Kolmogorov complexity of the string x.

We also consider *randomized* notions of time-bounded Kolmogorov complexity, in which the universal machine is given an additional random string as input. These complexity measures are particularly suitable in computational contexts where computations can be randomized, as is often the case with learning algorithms. Note that these measures include an additional parameter $\lambda \in [0,1]$ to denote the probability with which the machine outputs the string in question. If λ is omitted, we take $\lambda = 2/3$.

Definition 2.19 (rK^t)**.** For $x, y \in \{0,1\}^*$, $\lambda \in [0,1]$, $t \in \mathbb{N}$, the *randomized t-time-bounded Kolmogorov complexity* of x conditioned on y is defined as

$$\mathsf{rK}^t_\lambda(x \mid y) \triangleq \min_{M \in \{0,1\}^*} \left\{ |M| \;\middle|\; \mathop{\mathbf{Pr}}_{r \sim \{0,1\}^t} [U(M, y, r, 1^t) \text{ outputs } x] \geq \lambda \right\}.$$

We define $\mathsf{rK}^t(x)$ as $\mathsf{rK}^t(x \mid \epsilon)$.

Note that in rK^t, the string still has a deterministic, standard shorter description encoding a program; it is the program's execution that is randomized, so the natural decoding procedure uses randomness to recover the string. Randomization can provide significant savings in description length in some concrete settings

[CLO+23]. Nevertheless, under derandomization assumptions from complexity theory, it follows that $\mathsf{K}^{\mathsf{poly}(t)}(x)$ and $\mathsf{rK}^{\mathsf{poly}(t)}(x)$ are related up to an additive logarithmic term for every large enough string x (see, e.g., [GKLO22, Appendix A.2]).

In recent years, another variant of time-bounded Kolmogorov complexity has been particularly useful due to its coding properties [LOZ22] and connections to pseudorandomness.

Definition 2.20 (pK^t [GKLO22])**.** For $x, y \in \{0,1\}^*$, $\lambda \in [0,1]$, $t \in \mathbb{N}$, the *probabilistic t-time-bounded Kolmogorov complexity* of x conditioned on y is defined as

$$\mathsf{pK}^t_\lambda(x \mid y) \triangleq \min\left\{k \;\middle|\; \Pr_{r\sim\{0,1\}^t}\left[\exists\, M \in \{0,1\}^{\le k} \text{ such that } U(M,y,r,1^t) \text{ outputs } x\right] \ge \lambda\right\}.$$

We define $\mathsf{pK}^t(x)$ as $\mathsf{pK}^t(x \mid \epsilon)$.

Equivalently, $\mathsf{pK}^t(x) \le k$ if $\Pr_r[\mathsf{K}^t(x \mid r) \le k] \ge 2/3$. As a consequence, a string x has bounded pK^t complexity if for most choices of r it admits a small description given access to r. Note that here the description of x depends on r. This captures the setting where both encoder and decoder share access to a random string r. In contrast, in rK^t there is a fixed randomised machine that produces x with high probability over its internal randomness.[3]

We will also consider a randomized variant of Levin's Kt complexity that admits particularly useful coding properties (see [HLO26a] and references therein for additional context).

Definition 2.21 (rKt [Oli19])**.** For $x, y \in \{0,1\}^*$, $\lambda \in [0,1]$, the *randomized Levin complexity* of x conditioned on y is defined as

$$\mathsf{rKt}_\lambda(x \mid y) \triangleq \min_{t\in\mathbb{N},\, M\in\{0,1\}^*}\left\{|M| + \lceil \log t \rceil \;\middle|\; \mathop{\mathbf{Pr}}_{r\sim\{0,1\}^t}[U(M,y,r,1^t) \text{ outputs } x] \ge \lambda\right\}.$$

We define $\mathsf{rKt}(x)$ as $\mathsf{rKt}(x \mid \epsilon)$.

For completeness, we mention that one can also consider a "probabilistic" variant of Kt called pKt [HLO24], which is similar to the extension of K^t to pK^t. While we will not employ pKt in the survey, we refer to [Pas26] for a recent application of a related notion in the context of machine learning.

A choice of universal machine U gives rise to a certain *universal distribution* that may be sampled by returning the output of U on a uniformly random input.

[3] For the reader familiar with interactive proofs in the context of complexity theory, we mention that the distinction between rK^t and pK^t is analogous to the definitions of the complexity classes MA and AM, respectively.

In the case of time-unbounded Kolmogorov complexity, for any string x, $\mathsf{K}(x)$ is approximately equal to the logarithm of the inverse of the probability of x under this distribution.

Analogously, one may define a *time-bounded universal distribution* for a given time bound $t \in \mathbb{N}$. Namely, for a fixed universal machine U, [IL90] define Q^t as the distribution of the output of $U(r, 1^t)$ for a uniformly random string $r \sim \{0,1\}^t$. For any auxiliary string $y \in \{0,1\}^*$, let $Q^{t,y}$ be the distribution with the same definition as Q^t, except the universal machine U is additionally given access to y.

The following is the complexity measure corresponding to this family of distributions.

Definition 2.22 (q^t [HN23])**.** For $x, y \in \{0,1\}^*$ and $t \in \mathbb{N}$, the q^t*-complexity* of x conditioned on y is defined as

$$\mathsf{q}^t(x \mid y) \triangleq \log\left(\frac{1}{\Pr[Q^{t,y} = x]}\right).$$

We define $\mathsf{q}^t(x)$ as $\mathsf{q}^t(x \mid \epsilon)$; that is, $\log(1/\Pr[Q^t = x])$.

We refer to the textbooks [LV19, SUV17] for a comprehensive treatment of Kolmogorov complexity. For a modern perspective on time-bounded Kolmogorov complexity, see [LO22].

2.4.3 Circuit Complexity

A *boolean circuit* is a finite directed acyclic graph with one or more source nodes, one sink node, and each internal node corresponding to some boolean function $g : \{0,1\}^* \to \{0,1\}$ from a fixed set G, called a *basis*. A standard choice of G is the set $\{\mathsf{AND}, \mathsf{OR}, \mathsf{NOT}\}$. Each node of a boolean circuit is called a *gate*, with source nodes being called *input gates* and the sink node being called the *output gate*.[4] A boolean circuit C with n input nodes induces a boolean function $f : \{0,1\}^n \to \{0,1\}$, computed by associating each bit of a given input string $x \in \{0,1\}^n$ with the corresponding input gate of C, performing the computation specified by the structure of C and the functions of the internal gates, and taking $f(x) \in \{0,1\}$ to be the value resulting at the output gate.

Since a single boolean circuit has a fixed number of input gates, it computes a function on inputs of only one length. To capture a boolean function $f : \{0,1\}^* \to \{0,1\}$ defined on all input lengths, we consider a *circuit family* $C = \{C_n\}_{n \in \mathbb{N}}$, where each C_n has n input gates and computes the restriction of f to $\{0,1\}^n$. Two measures of the cost of a circuit are its *size*, i.e. the number of gates

[4] One may also consider circuits with multiple output gates, but we are mainly interested here in circuits representing boolean functions, which have only one bit of output.

it contains, and its *depth*, i.e. the length of the longest path from an input gate to the output gate.

We use the term *fan-in* to refer to the number of inputs to a gate. For unrestricted boolean circuits, we adopt the convention that the fan-in of each gate is at most two. For bounded-depth circuit classes, we allow the fan-in to be unbounded unless stated otherwise. Note that NOT gates always have fan-in equal to 1.

From the perspective of compression, the truth table of a Boolean function f can be seen as a long string of length $N = 2^n$. A small circuit C that computes f can then be interpreted as a compressed representation with a local decoder: given an index $i \in [N]$, $C(i)$ outputs the i-th bit of f.

Next, we define several classes of boolean functions, which may all be specified according to a class of boolean circuit families.

Definition 2.23 ($\mathsf{P/poly}$)**.** $\mathsf{P/poly}$ is the class of boolean functions computable by a circuit family $\{C_n\}$ over the basis $\{\mathsf{AND}, \mathsf{OR}, \mathsf{NOT}\}$, where for some constant $d \in \mathbb{N}$, each C_n has size at most n^d.

Definition 2.24 (AC^0)**.** AC^0 is the class of boolean functions computable by a circuit family $\{C_n\}$ over the basis $\{\mathsf{AND}, \mathsf{OR}, \mathsf{NOT}\}$, where for some constants $c, d \in \mathbb{N}$, each C_n has size at most n^c and depth at most d.

Definition 2.25 (TC^0)**.** The class TC^0 has the same definition as AC^0, except circuits are over the larger basis $\{\mathsf{AND}, \mathsf{OR}, \mathsf{NOT}, \mathsf{THR}\}$, where THR is a *threshold* gate (also known as a weighted majority), outputting 1 iff $\sum_i w_i \cdot z_i \geq t$, where $w_i, t \in \mathbb{R}$.[5]

TC^0 circuits, also known as threshold circuits, can be interpreted as a discrete analogue of neural networks. We recall that a current frontier in complexity theory is to establish super-polynomial circuit size lower bounds against depth-2 threshold circuits for an explicit function (see [KW16, CTW26] for partial progress and related discussion). We refer to Figure 2 for a diagram representation of this circuit class.

Definition 2.26 (AC^0 with MOD_m gates)**.** For a modulus $m \geq 2$, let MOD_m output 1 iff the number of its inputs equal to 1 is divisible by m. $\mathsf{AC}^0[m]$ has the same definition as AC^0, except circuits are over the basis $\{\mathsf{AND}, \mathsf{OR}, \mathsf{NOT}, \mathsf{MOD}_m\}$. The case of $m = 2$ gives the parity gate, sometimes written $\mathsf{AC}^0[\oplus]$. ACC^0 is the class defined as $\bigcup_{m \geq 2} \mathsf{AC}^0[m]$.

The following inclusion of circuit classes hold:

$$\mathsf{AC}^0 \subseteq \mathsf{AC}^0[\oplus] \subseteq \mathsf{ACC}^0 \subseteq \mathsf{TC}^0 \subseteq \mathsf{P/poly}.$$

[5] Since each AND, OR, and NOT gate can be simulated by a THR gate with appropriate weights w_i and threshold t, we typically assume that TC^0 circuits contain THR gates only.

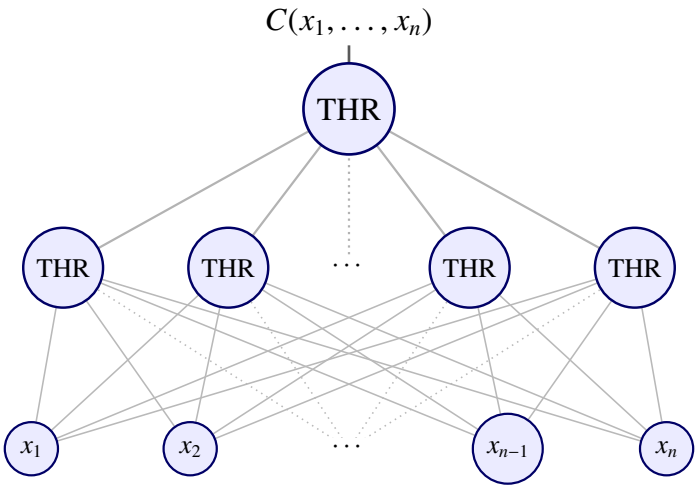


Figure 2: A depth-2 threshold circuit C. The top threshold gate receives the outputs taking values in $\{0, 1\}$ from the threshold gates in the bottom layer, each of which is connected to the input variables $x_1, \ldots, x_n$.

For standard background on Boolean circuits, nonuniform computation, and circuit classes, see [AB09, Juk12].

2.4.4 Meta-Complexity

Broadly speaking, problems in *meta-complexity* ask about the complexity of a given object. One may restrict these problems to different kinds of object (for example, the input may simply be a string, or it may be understood to represent a partly or fully specified truth-table), and one may ask about different measures of complexity (for example, one may ask about the circuit complexity or the Kolmogorov complexity of the input).

We begin with the Minimum Circuit Size Problem [KC00], a prototypical example of a problem in meta-complexity.

Definition 2.27 (MCSP)**.** The *Minimum Circuit Size Problem*, denoted MCSP, is the problem defined as follows. The input consists of a truth-table T, represented as a string, along with a size threshold $s \in \mathbb{N}$. (T, s) is a yes-instance of MCSP if and only if there exists a boolean circuit of size at most s computing the function with truth-table T.

Definition 2.28 (Partial-MCSP)**.** Partial-MCSP is the problem defined as follows. For some $n, m \in \mathbb{N}$, the input consists of a collection of input/output pairs

$((x_i, b_i))_{i\in[m]}$, where $x_i \in \{0,1\}^n$ and $b_i \in \{0,1\}$ for each $i \in [m]$, and a threshold parameter $s \in \mathbb{N}$. $((x_i, b_i)_{i\in[m]}, s)$ is a yes-instance of Partial-MCSP iff there exists a boolean circuit C of size at most s such that $C(x_i) = b_i$ for all $i \in [m]$.

Next, we will consider computational problems related to the task of estimating the time-bounded Kolmogorov complexity of an input string (see [Sim23] for further context and resources).

Definition 2.29 (MINKT)**.** Inputs to the problem MINKT consist of a string x along with a size threshold $s \in \mathbb{N}$ and time parameter $t \in \mathbb{N}$. $(x, 1^s, 1^t)$ is a yes-instance of MINKT if and only if $\mathsf{K}^t(x) \leq s$.

Observe that MINKT is in NP. It is not in P under the existence of cryptographically secure one-way functions [KC00]. Interestingly, while believed to be intractable, MINKT is not known to be NP-complete (see [HLO26b] and references therein).

For the connections to learning and cryptography discussed in Section 4.3, the complexity of certain robust variants of MINKT is more relevant. We will informally refer to Gap-MINKT as the task of estimating the K^t complexity of an input string x of length n up to an additive term $O(\log n)$.[6] We say that "Gap-MINKT $\in$ P" if there is a polynomial-time algorithm that solves this problem on every input. We refer to [Hir21] for the relevant formal definition. We say that "Gap-MINKT $\in$ HeurP" if there is a polynomial-time algorithm that is heuristically correct, i.e., it solves the problem on most inputs (but might output an incorrect estimate on a small fraction of inputs). We refer to [LP21] for the relevant formal definition.

It is known that the task of estimating the K^t complexity of an input string is essentially equivalent (under polynomial-time reductions) to the following problem [Hir21].

Definition 2.30 (The GapK^t-vs-K Problem)**.** We say that GapK^t-vs-K $\in$ pr-BPP if there is a randomized polynomial-time algorithm A such that for every input $x \in \{0,1\}^*$ of length n and every time bound $t \in \mathbb{N}$,

$$\Pr_A\left[\, \mathsf{K}(x) \leq A(x, 1^t) \leq \mathsf{K}^t(x) + O(\log n) \,\right] \geq 2/3.$$

Similarly, we consider the complexity of computing Levin's Kt complexity. In the MKtP problem, we are given as input a tuple $(x, 1^s)$, and the task is to decide if $\mathsf{Kt}(x) \leq s$. This problem is in the complexity class EXP, i.e., deterministic exponential time.

[6] A bit more formally, in this promise problem we are interested in distinguishing $\mathsf{K}^t(x) \leq s$ from $\mathsf{K}^{\mathsf{poly}(t)}(x) \geq s + O(\log n)$.

While the statement $\mathsf{MINKT} \notin \mathsf{P}$ immediately implies that $\mathsf{NP} \not\subseteq \mathsf{P}$, a proof that $\mathsf{MKtP} \notin \mathsf{P}$ merely implies that $\mathsf{EXP} \not\subseteq \mathsf{P}$, which is already known by the time hierarchy theorem. Still, it remains a longstanding open problem to establish this lower bound for MKtP [ABK+06]. We will revisit this question in Section 7.3.

We refer to some general surveys of meta-complexity for broader context [All17, All21, Hir22a, MP25].

2.4.5 Natural Properties

In an influential work, [RR97] showed that many combinatorial circuit size lower bound arguments implicitly provide an efficient algorithm that distinguishes "structured" functions from "random" functions. As explained later in the survey, this provides a fruitful connection between complexity lower bounds and algorithmic upper bounds.

Let C be a Boolean circuit class, and for a size parameter $s\colon \mathbb{N} \to \mathbb{N}$, let $C[s]$ denote the set of Boolean functions $f\colon \{0,1\}^n \to \{0,1\}$ computed by a C-circuit of size at most $s(n)$. We write $C_n[s(n)]$ when we restrict the class to Boolean functions on n input bits.

A function $f\colon \{0,1\}^n \to \{0,1\}$ can be represented by a string $\mathsf{tt}(f) \in \{0,1\}^N$ of length $N = 2^n$ that describes the truthtable of f. Similarly, given a string $w \in \{0,1\}^N$, we write $\mathsf{fnc}^w\colon \{0,1\}^n \to \{0,1\}$ to denote the corresponding Boolean function.

In the discussion below, we will consider algorithms that receive as input the description of a Boolean function as an N-bit string. Roughly speaking, a natural property is an efficient algorithm that rejects every function in $C[s]$ but accepts a large set of functions. In other words, it provides an efficient test to reject functions of low complexity while at the same time accepting a large collection of "random" functions.

Definition 2.31 (Natural Properties [RR97])**.** We say that there is a prBPP-*natural property* against a circuit class $C[s]$ if there is a probabilistic polynomial-time algorithm A operating over input strings of length $N = 2^n$ for which the following conditions hold:

- *Constructive*: A runs in time polynomial in its input length N.
- *Largeness*: For every n and $N = 2^n$, there is a set $\mathcal{F}_n \subseteq \{0,1\}^N$ with $\Pr_{w\sim\mathcal{U}_N}[w \in \mathcal{F}_n] \geq 1/2$ such that for every $w \in \mathcal{F}_n$, we have $\Pr_A[A(w) = 1] \geq 2/3$.
- $C[s]$-*usefulness*: For every n and $f\colon \{0,1\}^n \to \{0,1\}$ such that $f \in C_n[s(n)]$, we have $\Pr_A[A(\mathsf{tt}(f)) = 1] \leq 1/3$.

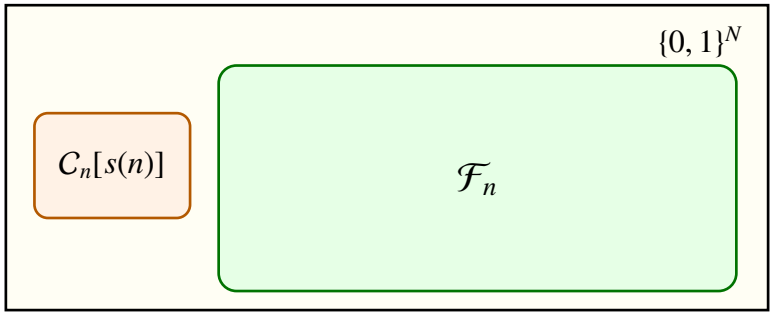


Figure 3: A diagram representation of a natural property (Definition 2.31). The outer rectangle is the ground set of all N-bit strings. The small region represents the "structured" strings corresponding to functions in the low complexity class $C_n[s(n)]$. The large region represents the "random" strings in $\mathcal{F}_n \subseteq \{0,1\}^N$. Its relative size illustrates the *largeness* condition, while $\mathcal{F}_n \cap C_n[s(n)] = \varnothing$ illustrates *usefulness* against $C_n[s(n)]$. A natural property is *constructive*: it postulates the existence of an efficient algorithm that distinguishes these two sets, thereby separating the structured strings in $C_n[s(n)]$ from the random strings in $\mathcal{F}_n$.

In the definition above, we use prBPP instead of BPP because the randomized procedure A might accept some strings with probability close to $1/2$. In the terminology of complexity theory, it might define a promise problem as opposed to a language. We refer to Figure 3 for a pictorial description of a natural property.

For broader background on circuit lower bounds and the natural proofs barrier, see [AB09, Juk12].

2.4.6 Pseudorandomness

A pseudorandom generator with *seed length* n and *output length* $m > n$ is a deterministic map

$$G\colon \{0,1\}^n \to \{0,1\}^m.$$

For a uniform seed $s \sim \mathcal{U}_n$, we write $G(\mathcal{U}_n)$ for the distribution of $G(s)$. The quantity $m - n$ is called the *stretch* of G. More generally, one considers families $\{G_n\}_{n\in\mathbb{N}}$ whose output length $m = m(n)$ grows with the seed length.

Definition 2.32 (Distinguishers and Pseudorandomness)**.** A *distinguisher* is a Boolean function, or more generally a randomized procedure, $D\colon \{0,1\}^m \to \{0,1\}$. Its distinguishing advantage against G is

$$\mathsf{Adv}_G(D) \triangleq \left| \Pr_{s\sim\mathcal{U}_n}[D(G(s)) = 1] - \Pr_{y\sim\mathcal{U}_m}[D(y) = 1] \right|,$$

where the probabilities also include the internal randomness of D, if any. We say that D *ε-distinguishes* the output of G from the uniform distribution if $\mathsf{Adv}_G(D) > \varepsilon$. Given a class C of distinguishers, we say that G *fools C with error ε* if $\mathsf{Adv}_G(D) \leq \varepsilon$ for every $D \in C$. Equivalently, G is *ε-pseudorandom* against C.

Pseudorandom generators commonly appear in two regimes. In the *cryptographic regime*, G is efficient and must fool distinguishers whose running time may be substantially larger than the time needed for one evaluation of G. The existence of cryptographic pseudorandom generators is equivalent, under polynomial-time reductions, to the existence of one-way functions and pseudorandom functions. These primitives are often used to establish conditional hardness results for learning.

In the *complexity-theoretic regime*, the generator may use more time than the distinguishers it fools. Here a generator is often accompanied by a *reconstruction procedure* that converts a distinguisher into useful computational structure. Such generators and reconstruction procedures will later be used to construct learning algorithms.

The survey will use several variants of these notions. We record the basic definitions here as a guide for the reader and refer to Vadhan's monograph [Vad12] for a comprehensive treatment.

3 Classical Connections Between Learning and Compression

We start off with two classical connections between learning and compression. They allow us to link learnability, a statistical notion, with compressibility, a combinatorial one. In many contexts, this offers a particularly fruitful perspective for the design of learning algorithms.

3.1 Model Compression

We start with one of the most basic connections between learning and compression. In the (realizable) PAC learning setting, a labelled sample may contain many examples, but a learner can summarize the information they exhibit by a comparatively short hypothesis ("model") that agrees with every observed label. We refer to this operation as *model compression*.[7]

Why should a compressed model generalize well, i.e., remain accurate on fresh examples that are not part of the training data? The answer is the principle commonly known as *Occam's Razor*: among explanations that fit the observations, a simple explanation is less likely to fit merely by accident [BEHW87]. The informal justification is easy to understand: any fixed substantially incorrect hypothesis is unlikely to agree with many independent examples, and a short model description leaves few candidate hypotheses over which this unlikely event must be union bounded. Thus compression of the model controls generalization.

We now formalize this intuition. Let C be a concept class and $\mathcal{H}$ a hypothesis class, where $\mathcal{H}$ need not be equal to C. For a distribution D_n over $\{0,1\}^n$, define

$$\mathsf{err}_{D_n}(h, c) \triangleq \Pr_{x \sim D_n}[h(x) \neq c(x)].$$

Definition 3.1 (Consistent Hypothesis Finder)**.** A *consistent hypothesis finder* for C using $\mathcal{H}$ is a procedure A with the following property. For every $n, m \geq 1$, every $c \in C_n$, and every labelled sample

$$S = \{(x_1, c(x_1)), \ldots, (x_m, c(x_m))\},$$

$A(S)$ outputs a hypothesis $h \in \mathcal{H}_n$ such that $h(x_i) = c(x_i)$ for every $i \in [m]$.

Thus a consistent hypothesis finder solves a search problem with no explicit reference to a distribution or to the PAC parameters ε and δ. Note that the hypothesis class $\mathcal{H}$ records the set of possible models, and it may differ from the target

[7] This differs from the *sample compression* schemes considered in the next subsection, where the learner retains a small part of the sample from which a hypothesis can later be reconstructed.

class C. The next result shows that consistency implies generalization whenever the range of possible output models is sufficiently small and the number of samples is large enough.

Theorem 3.2 (PAC Learning by Finding a Consistent Hypothesis [BEHW87])**.** *Let C be a concept class, $\mathcal{H}$ be a hypothesis class, and suppose that A is a consistent hypothesis finder for C using $\mathcal{H}$. Fix $n \geq 1$ and assume that $\mathcal{H}_n$ is finite. Let D_n be any distribution over $\{0,1\}^n$, let $c \in C_n$, and let $\varepsilon, \delta \in (0,1)$. If A is run on m independent examples from $\mathsf{EX}(c, D_n)$, where*

$$m \geq \frac{1}{\varepsilon}\left(\ln|\mathcal{H}_n| + \ln\frac{1}{\delta}\right),$$

then, with probability at least $1-\delta$ over the sample, its output hypothesis h satisfies $\mathsf{err}_{D_n}(h,c) \leq \varepsilon$.

Proof. Let c be the unknown target concept, and let $\mathcal{B} \subseteq \mathcal{H}_n$ be the set of *bad* hypotheses h satisfying $\mathsf{err}_{D_n}(h,c) > \varepsilon$. Fix $h \in \mathcal{B}$. A single example $(x, c(x))$ drawn from $\mathsf{EX}(c, D_n)$ is consistent with h with probability $1 - \mathsf{err}_{D_n}(h,c) < 1-\varepsilon$. Since the m examples are independent, the probability that h is consistent with the entire sample is at most

$$(1-\varepsilon)^m \leq e^{-\varepsilon m}.$$

If the consistent hypothesis finder outputs a bad hypothesis, then the sample must be consistent with at least one member of $\mathcal{B}$. Consequently, a union bound gives

$$\begin{aligned}
\Pr[\mathsf{err}_{D_n}(A(S), c) > \varepsilon] &\leq \Pr[\exists h \in \mathcal{B} \text{ consistent with } S] \\
&\leq \sum_{h \in \mathcal{B}} \Pr[h \text{ is consistent with } S] \\
&\leq |\mathcal{H}_n|\, e^{-\varepsilon m} \\
&\leq \delta,
\end{aligned}$$

where the final inequality follows from the assumed lower bound on m. □

The role of model compression can now be made explicit. Suppose every $h \in \mathcal{H}_n$ can be encoded using $\ell(n)$ bits. Then $\ln|\mathcal{H}_n| \leq \ell(n)\ln 2$ and it is therefore enough to take

$$m \geq \frac{1}{\varepsilon}\left(\ell(n)\ln 2 + \ln\frac{1}{\delta}\right).$$

Thus the number of examples required for generalization is controlled by the description length of the output model. The training sample is described by $m(n+1)$ bits, while the explanation extracted from it uses only $\ell(n)$ bits. If the consistent

hypothesis finder runs in polynomial time, $\ell(n) = \mathsf{poly}(n)$, and its output hypotheses are efficiently evaluatable under the encoding, then this construction gives an efficient PAC learner.

To sum up, compressing the observations into a short model prevents overfitting because there are too few short models for a substantially incorrect one to agree with all the data, except with small probability.

A Classical Example: Learning Through Polynomial Threshold Representations. An important example of the preceding model compression perspective is the class of *linear threshold functions* (LTFs), also known as halfspaces or perceptrons. Such a function has the form $h(x) = \mathrm{sign}(w_0 + \langle w, x\rangle)$. Given a labelled sample realizable by a linear threshold function, consistency can be expressed as a linear program. After encoding the labels as elements of $\{\pm 1\}$, one seeks w_0, w satisfying $y_i(w_0 + \langle w, x_i\rangle) \geq 1$ for every sample point (x_i, y_i). Realizability guarantees feasibility after a suitable rescaling, and linear programming finds rational weights of polynomial description length in polynomial time. Thus linear programming provides an efficient consistent hypothesis finder. Moreover, there are only $2^{O(n^2)}$ distinct linear threshold functions on $\{0,1\}^n$, so the preceding Theorem 3.2 shows that the resulting hypothesis generalizes from polynomially many examples. This yields a distribution-independent PAC learner for linear threshold functions.

Feature expansion extends this observation to *polynomial threshold functions* (PTFs), the natural extension of LTFs to higher degree. Let $\Phi_d(x)$ be the vector whose coordinates are all multilinear monomials $\prod_{i\in T} x_i$ with $|T| \leq d$. Its dimension is $N = \sum_{j=0}^{d} \binom{n}{j} \leq (en/d)^d$, and every degree-$d$ polynomial threshold function becomes a linear threshold function over the expanded representation $\Phi_d(x)$. Mapping each example through Φ_d and applying the preceding linear programming algorithm therefore gives a PAC learner with running time $\mathsf{poly}(n^d, 1/\varepsilon, \log(1/\delta))$. Consequently, proving that a concept class has polynomial threshold degree at most d immediately gives a learning algorithm with this running time. The learner need not know a representation of the target, since the linear program searches for any consistent polynomial threshold function.

This approach yields learning algorithms with provable guarantees for rich concept classes such as decision trees, DNF formulas, bounded-depth Boolean formulas, and more. We refer to the survey [HS07] for a comprehensive discussion of this technique and its applications.

The Occam principle and closely related ideas appear in the literature under several names. It admits several generalizations. For instance, infinite hypothesis classes can be treated using the theory of VC dimension (e.g., LTFs over $\mathbb{R}^n$),

while exact consistency can be relaxed by allowing the hypothesis finder to output an h that agrees with most examples in the sample. For further background, we refer to standard textbooks such as [KV94a, SSBD14].

3.2 Sample Compression

The preceding subsection compressed a labelled sample into a short model. Sample compression takes a different approach: it retains only a few of the observed examples, together with a small amount of side information, and reconstructs a hypothesis from this summary. The reconstructed hypothesis need not itself have a short standalone description. What matters is that, relative to the original sample, it is determined by very few examples.

Fix an input length n. A *sample compression scheme* for a concept class C_n with parameters (k, ℓ) consists of a compression map Comp and a reconstruction map Rec with the following guarantee. Given any labelled sample sequence

$$S = ((x_1, c(x_1)), \ldots, (x_m, c(x_m)))$$

consistent with some unknown $c \in C_n$, the map Comp outputs a pair $(S|_I, \sigma)$, where $I \subseteq [m]$ has size at most k and $\sigma \in \{0,1\}^\ell$. Here $S|_I$ denotes the subsequence of S indexed by I, listed in increasing order of indices. The reconstruction map uses only this pair to produce a hypothesis

$$h = \mathsf{Rec}(S|_I, \sigma)$$

that is consistent with the entire original sample S. As usual, strings shorter than ℓ can be padded, and the reconstructed hypothesis may be improper. We call $k + \ell$ the *size* of the scheme.

A Classical Example: Rectangles in the Plane. Although we stated the definition over the Boolean cube, sample compression applies without change to arbitrary (possibly infinite) instance domains. Consider the class of axis-aligned rectangles in $\mathbb{R}^2$, where an example is labelled positive precisely when it belongs to an unknown target rectangle $R^\star$. Given a realizable sample containing at least one positive example, Comp retains a leftmost, a rightmost, a lowest, and a highest positive example, breaking ties according to their order in the sample and retaining only distinct examples. The reconstruction map Rec outputs the smallest axis-aligned rectangle R_S containing these points. (If there are no positive examples, the compression is empty and Rec returns the empty hypothesis.) Every positive example lies in R_S by the choice of the four extreme points. On the other hand, $R_S \subseteq R^\star$, since it is the bounding rectangle of points belonging to

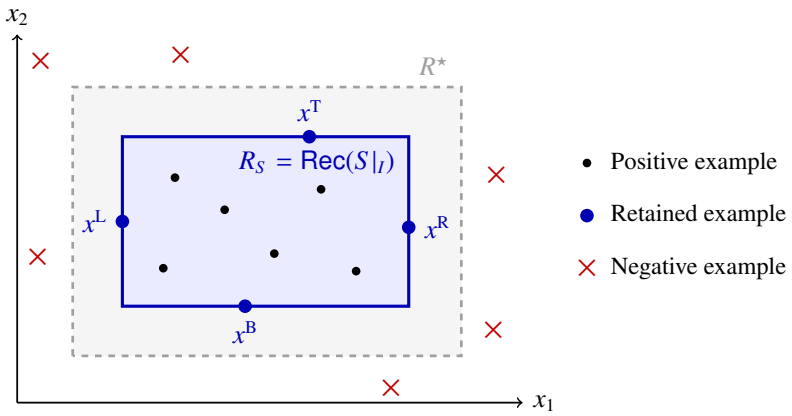


Figure 4: A sample compression scheme for axis-aligned rectangles in the plane. The four extreme positive examples determine the reconstructed rectangle R_S. Since $R_S \subseteq R^\star$, where $R^\star$ denotes the unknown target rectangle, R_S contains every positive sample point and no negative sample point. All labels in the original sample can be reconstructed from the four extreme positive points.

the axis-aligned rectangle $R^\star$. Therefore every negative example lies outside R_S, and the reconstructed rectangle is consistent with the entire sample. This gives a sample compression scheme with parameters $(k = 4, \ell = 0)$. We refer to Figure 4 for a pictorial description.

Next, we explain why any such compression scheme yields a PAC learner.

Theorem 3.3 (PAC Learning from Sample Compression [LW86])**.** *Suppose that C_n has a sample compression scheme with parameters (k, ℓ). Let D_n be any distribution over $\{0,1\}^n$, let $c \in C_n$, and let $\varepsilon, \delta \in (0,1)$. Draw m independent examples from $\mathsf{EX}(c, D_n)$, apply the compression map, and output the reconstructed hypothesis h. Then*

$$\Pr[\mathsf{err}_{D_n}(h,c) > \varepsilon] \le 2^\ell \sum_{i=0}^{k} \binom{m}{i} e^{-\varepsilon(m-i)}.$$

In particular, it suffices to take

$$m = O\left(\frac{k\log(2/\varepsilon) + \ell + \log(1/\delta)}{\varepsilon}\right)$$

to obtain error at most ε with probability at least $1-\delta$.

Proof Sketch. The subtlety compared with the model compression argument is that the reconstructed hypothesis depends on the retained examples, which are

themselves random. We therefore cannot fix the hypothesis before drawing the sample and charge it for agreeing with all m examples.

Instead, fix a set of indices $I \subseteq [m]$ of size at most k and a message $\sigma \in \{0,1\}^{\ell}$, and condition on the examples indexed by I. The reconstructed hypothesis $h_{I,\sigma} = \mathsf{Rec}(S|_I, \sigma)$ is now fixed, while the remaining $m - |I|$ examples are still independent of it. If $\mathsf{err}_{D_n}(h_{I,\sigma}, c) > \varepsilon$, the probability that it agrees with all these remaining examples is therefore at most

$$(1-\varepsilon)^{m-|I|} \leq e^{-\varepsilon(m-|I|)}.$$

Although the compression map may choose (I, σ) adaptively after seeing the entire sample, if the scheme outputs a bad hypothesis then this event occurs for its selected pair. A union bound over the at most $2^{\ell}\binom{m}{i}$ choices with $|I| = i$, for every $0 \leq i \leq k$, therefore gives the claimed probability bound. The standard estimate $\sum_{i=0}^{k}\binom{m}{i} \leq (em/k)^k$, together with the stated choice of m, makes this bound at most δ. □

Thus a bounded sample compression scheme yields a PAC learner. If compression, reconstruction, and evaluation of the reconstructed hypothesis are efficient, then the resulting learner is efficient as well. Without these requirements, the conclusion is information-theoretic only, providing an upper bound on the sample complexity of learning C_n.

Remarkably, the converse also holds at the qualitative level in the information-theoretic setting. A (possibly infinite) binary concept class is PAC learnable exactly when it has finite VC dimension, and Moran and Yehudayoff [MY16] proved that every class of VC dimension d has a sample compression scheme of size $2^{O(d)}$. Consequently, for binary concept classes, the existence of a bounded sample compression scheme characterizes PAC learnability, even though the known quantitative relationship is far from tight.

Problem 3.4 (Sample Compression Conjecture)**.** Does every binary concept class of VC dimension d admit a sample compression scheme of size $O(d)$?

Closing the existing exponential gap would show that the amount of information one needs to retain from a sample is governed, up to constant factors, by the same parameter that controls PAC sample complexity.

4 Complexity-Theoretic Foundations of Learning and Compression

4.1 Learning Paradises

We consider five possible learning scenarios, ordered from a strong unsupervised setting to a far weaker supervised setting in which only weak heuristic learnability is assumed. Still, even the weakest of these assumptions asks for learning power over *unrestricted* Boolean circuits, a capability well beyond current theoretical results and practical algorithms (see Section 4.3 for more details). We therefore call each scenario a learning paradise.

- **P.I: Efficient Extrapolation.**
  There are polynomials $p_1(\cdot), p_2(\cdot)$ and a randomized algorithm L running in time $p_1(n, m, s, t, 1/\varepsilon, 1/\delta)$ that solves the $(m, n, s, t, \varepsilon, \delta)$-extrapolation task whenever $m \geq p_2(s, 1/\varepsilon, 1/\delta)$.

- **P.II: Efficient Distribution Learning.**
  There is a polynomial $p(\cdot)$ and a randomized algorithm A running in time $p(n, s, t, 1/\varepsilon, 1/\delta)$ that ε-learns every distribution $D \in \mathcal{D}_n^{s,t}$ with probability at least $1 - \delta$.

- **P.III: Efficient Agnostic Learning (Under Feasible Example Distributions).** There is a polynomial $p(\cdot)$ and a randomized algorithm A running in time $p(n, 1/\varepsilon, \log(1/\delta), s, t, a)$ that agnostically learns $\mathsf{SIZE}[s]$ over $\mathcal{D}^{a,t}$.

- **P.IV: Efficient PAC Learning (with MQs and Under Feasible Example Distributions).** There is a polynomial $p(\cdot)$ and a randomized algorithm A running in time $p(n, 1/\varepsilon, \log(1/\delta), s, t, a)$ that PAC learns $\mathsf{SIZE}[s]$ over $\mathcal{D}^{a,t}$ with membership queries.

- **P.V: Efficient Heuristic Learning (without MQs).** For any polynomials $s(\cdot), t_1(\cdot), t_2(\cdot)$ and distribution families $P \in \mathsf{SAMP}[t_1]$ and $D \in \mathsf{SAMP}[t_2]$, there is a polynomial $p(\cdot)$ and a randomized algorithm A running in time $p(n, s, t_1, t_2)$ that weakly predicts $\mathsf{SIZE}[s]$ on average over P and D.

There are many natural variants of assumption P.IV. For example, one can vary whether the learner is allowed to make membership queries, whether the examples come from the uniform distribution or from arbitrary distributions, and whether the learner is proper. To keep the presentation simple and to avoid the overly pessimistic distribution-free model, we adopt the membership query PAC model over feasible distributions as a representative supervised learning scenario.

Learning under feasible distributions is a natural restriction in complexity theoretic and cryptographic applications. There are two distributions to keep track of. In PAC Learning (P.IV), the feasible distribution is over input examples x, while the target concept f is an arbitrary function from the class. In Heuristic Learning (P.V), there is also a feasible distribution over the concepts themselves. Cryptographic reductions often have both kinds of efficient sampling: the target concept is generated by an efficient setup or key generation experiment, while the inputs are generated by an efficient challenge or encryption experiment. In particular, attacks on public-key encryption can often be phrased as learning a polynomial-size predicate, with the concept sampled by key generation and the examples sampled by encryption. We refer to Section 8 for additional discussion.

4.2 Basic Relations

We now explain the basic ordering among the five learning paradises:

$$\text{P.I} \implies \text{P.II} \implies \text{P.III} \implies \text{P.IV} \implies \text{P.V}.$$

Throughout the proof sketches below, parameters are rescaled by fixed polynomials. We suppress these routine changes in notation.

Lemma 4.1 (P.I $\implies$ P.II)**.** *If Efficient Extrapolation is possible, then Efficient Distribution Learning is possible.*

Proof Sketch. Fix a distribution $D \in \mathcal{D}_n^{s,t}$, and let M be a randomized machine of description length at most s and running time at most t that samples D. We reduce learning D from samples to extrapolating a sequence whose coordinates are independent samples from D.

Choose a sequence length m large enough for the extrapolation guarantee, with accuracy and confidence parameters η and γ to be fixed below. Let $M^{\otimes m}$ be the randomized machine that runs M independently m times and outputs the resulting sequence $(X_1, \ldots, X_m)$. Its description length is $s' = s + O(\log m)$ and its running time is $t' = O(m(t + n))$. Since m only needs to dominate a polynomial in the relevant parameters, we may choose it polynomially large so that the assumption P.I applies to $(m, n, s', t', \eta, \gamma)$.

For this product source, the conditional distribution of the next block is always the original distribution:

$$(X_i \mid X_{<i} = x_{<i}) \equiv D$$

for every i and every prefix $x_{<i}$ in the support. Given sample access to D, the distribution learner draws samples $x_1, \ldots, x_m$, chooses $I \sim [m]$, and forms the prefix $y = x_{<I}$. It outputs the sampler C_y that has y hardwired and, on fresh random coins, runs the extrapolation algorithm L on input y.

The distribution of (I, y) in this procedure is exactly the distribution of $(i, X_{<i})$ in the extrapolation experiment for $M^{\otimes m}$. Therefore, by assumption P.I,

$$\Pr_{I,y}\left[\Delta_{\mathsf{TV}}(C_y, D) \leq \eta\right] \geq 1 - \gamma.$$

Taking $\eta = \varepsilon$ and $\gamma = \delta$ gives a distribution learner for D. The returned sampler is efficient because it only simulates the polynomial-time extrapolator with a polynomial-length prefix hardwired. □

Lemma 4.2 (P.II $\implies$ P.III; [Xia10])**.** *If Efficient Distribution Learning is possible, then Efficient Agnostic Learning is possible.*

Proof Sketch. Fix an agnostic learning instance consisting of a joint example distribution $E_n \in \mathcal{D}^{a,t}_{n+1}$. Since E_n is feasibly samplable, it belongs to the distribution class handled by P.II, with polynomially related parameters.

Simulating the distribution learner on samples obtained from $\mathsf{EX}(E_n)$ gives, with high probability, an explicit sampler $S = (S^{(0)}, S^{(1)})$ for a distribution $\widehat{E}$ satisfying $\Delta_{\mathsf{TV}}(\widehat{E}, E_n) \leq \eta$. If we could sample from the conditional distribution of $S^{(1)}$ given $S^{(0)} = x$, then we would obtain labels that are close to the true labels according to E_n. The only obstacle is that S is an explicit (i.e., white-box) sampler for pairs, not an explicit conditional sampler.

Interestingly, distribution learning also supplies the ingredient needed to remove this obstacle. By Lemma 4.8, P.II rules out AIPRFs; by Lemma 2.16, this rules out AIOWFs; and by Lemma 4.11, the nonexistence of AIOWFs allows one to produce a randomised hypothesis h such that $(x, h(x))$, with x drawn from the marginal $S^{(0)}$, is close to $\widehat{E}$.

By the triangle inequality, the joint distribution $(x, h(x))$ is close to E_n. As in Lemma 4.12, by sampling many choices of internal randomness for h and taking the majority hypothesis, one obtains, with high probability, a deterministic hypothesis H such that

$$\Pr_{(x,b)\sim E_n}[H(x) = b] \geq \operatorname*{opt}_n(\mathcal{F}_{all}, E_n) - \varepsilon' \geq \operatorname*{opt}_n(C, E_n) - \varepsilon',$$

where $\mathcal{F}_{all}$ denotes the set of all boolean functions $f : \{0,1\}^n \to \{0,1\}$, $C \subseteq \mathcal{F}_{all}$ is any desired concept class, and ε' depends on the closeness of E_n and $(x, h(x))$. Choosing η and all failure probabilities as small enough polynomial fractions of ε and δ yields the desired agnostic learning guarantee. More details about this approach can be found in Section 4.5.

□

Lemma 4.3 (P.III $\implies$ P.IV)**.** *If Efficient Agnostic Learning is possible, then Efficient PAC Learning is possible.*

Proof. This implication is immediate from the definitions. More specifically, observe that PAC learning a target function $f \in \mathsf{SIZE}[s]$ over a distribution $D_n \in \mathcal{D}_n^{a,t}$ is equivalent to agnostically learning $\mathsf{SIZE}[s]$ over the distribution $E_n \in \mathcal{D}_{n+1}^{\mathsf{poly}(a,s),\mathsf{poly}(t,s)}$ that samples $x \sim D_n$ and outputs $(x, f(x))$. Since

$$\operatorname*{opt}_n(\mathsf{SIZE}[s], E) = 1,$$

the definition of an agnostic learner A guarantees that, with probability at least $1 - \delta$, $A(1^n, 1^{\varepsilon^{-1}}, \delta^{-1})$ with oracle access to $\mathsf{EX}(f, D_n) = \mathsf{EX}(E_n)$ outputs h such that $\Pr_{x \sim D_n}[h(x) = f(x)] \geq 1 - \varepsilon$. This satisfies the requirement of a PAC learner. □

Lemma 4.4 (P.IV $\implies$ P.V)**.** *If Efficient PAC Learning is possible, then Efficient Heuristic Learning is possible.*

Proof Sketch. Note that, under our definitions, the PAC learning algorithm (P.IV) is allowed to make membership queries, while the heuristic learner (P.V) is not. This means in particular that we cannot simply simulate the PAC learner when designing the heuristic learner from random examples.

The first step is to show that P.IV rules out auxiliary-input pseudorandom functions. Given a candidate AIPRF F_z, fix an auxiliary string z and a random key u, and view $g(x) = F_z(u, x)$ as a polynomial size circuit. The PAC learner can be run under the uniform distribution, with both its random examples and its membership queries answered by the oracle being tested. If the oracle is $F_z(u, \cdot)$, the learner outputs a hypothesis with high agreement with the oracle on a fresh uniform input. If the oracle is a truly random function, then after only polynomially many queries its value on a fresh input is independent of the learner's hypothesis except with negligible collision probability. Testing agreement on fresh inputs therefore distinguishes the AIPRF from a random function, contradicting security. (We refer the reader to Lemma 4.6 for a more detailed argument.)

In turn, by Lemma 2.16, the failure of AIPRFs gives the failure of AIOWFs, and in particular rules out infinitely-often one-way functions and pseudorandom generators. We can then apply the framework of Blum, Furst, Kearns, and Lipton [BFKL93], discussed in Section 5.1. Their argument starts from the fact that every candidate generator can be distinguished at every large enough input length, applies a hybrid argument to obtain a next-bit predictor for the direct-product generator associated with a random target function, and interprets that predictor as a learner from random examples. We refer to Section 5.1 for more details.

Consequently, for every feasible distribution P over polynomial-size target circuits and every feasible example distribution D, there is an efficient learner that predicts $f(y)$ with advantage $1/\mathsf{poly}(n)$, where $f \sim P$, the learner receives examples from $\mathsf{EX}(f, D)$, and $y \sim D$. This is exactly weak heuristic learning as in P.V. □

4.3 Impagliazzo's Worlds and Meta-Complexity

In an influential paper, Impagliazzo [Imp95] introduced five potential computational worlds connected to the complexity of NP problems and the possibility of secure cryptography. The point was not only to ask whether P = NP, but to distinguish several possible realities according to whether hard problems remain hard on typical inputs and whether such hardness can support secure cryptography. The two relevant worlds in the context of our discussion are Heuristica and Pessiland.

In Heuristica, NP is hard in the worst case, but this hardness disappears on average: every NP problem has an efficient algorithm that is correct on almost all inputs. In the notation of the diagram in Figure 5, this corresponds to a world where $\mathsf{P} \neq \mathsf{NP}$ but $\mathsf{DistNP} \subseteq \mathsf{AvgP}$.

In Pessiland, hardness exists on average, but the hard problems are not structured enough to yield secure cryptography. The relevant feature of Pessiland is the failure of one-way functions, equivalently the existence of efficient inverters for every candidate one-way function.

We refer to [BT06] for a detailed discussion of Heuristica, Pessiland, and their connections to cryptography and average-case complexity theory.

The learning paradises from Section 4.1 sit between these two worlds, as summarized in Figure 5. The left side of the diagram starts from the average-case easiness of NP promised by Heuristica and passes to meta-complexity via "$\mathsf{Gap\text{-}MINKT} \in \mathsf{P}$", i.e., the worst-case easiness of estimating time-bounded Kolmogorov complexity.[8] This leads to extrapolation and then, by the reductions in Section 4.1, to progressively weaker learning paradises. The right side records that the weakest of these paradises is already tied to the nonexistence of one-way functions and to average-case algorithms for estimating time-bounded Kolmogorov complexity. Thus the feasibility of different learning tasks is closely tied to meta-complexity, i.e., the complexity of estimating time-bounded Kolmogorov complexity.

Another consequence is that secure cryptography excludes all five learning paradises. Indeed, P.V rules out one-way functions by Lemma 4.6. Consequently, in a cryptographic world, unrestricted Boolean circuits cannot be provably learned efficiently even with the weak guarantees of P.V: any such theorem would also refute standard cryptographic assumptions.[9]

[8] Note that this is a nontrivial result: from the assumption that every problem in NP is easy on average, we derive that a specific NP problem from meta-complexity is easy in the worst case. This worst-case easiness seems particularly useful when applying the assumption to construct learning algorithms that must work on every input concept.

[9] This is not in apparent contradiction to the astonishing success of machine learning algorithms in practice. Recall that the learning guarantees here concern unrestricted Boolean circuits. Learn-

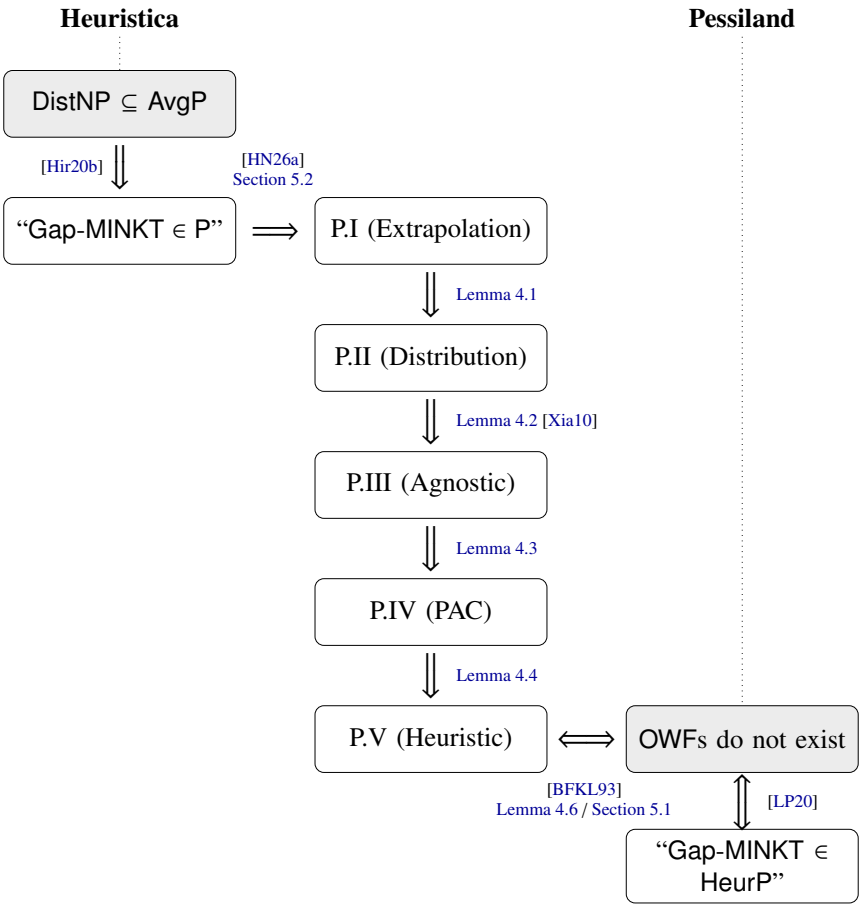


Figure 5: Learning paradises P.I – P.V (Section 4.1) between Impagliazzo's worlds [Imp95] Heuristica and Pessiland, including their relationship to compression and meta-complexity via the computational task of estimating time-bounded Kolmogorov complexity.

A precise characterization of the learning power associated with both Heuristica and Pessiland is possible, although in a different learning setting [HN26b]. It would be very interesting to obtain an equally precise understanding of how the learning paradises P.I through P.V relate to standard complexity and cryptographic

ing algorithms deployed in practice are not known to satisfy such guarantees, and they are not known to break standard cryptographic assumptions.

statements.

While the preceding discussion has focused on the relationship between learning and compression, and on its connections to cryptography and average-case complexity, the broader role of compression problems in complexity theory remains far from fully understood. A central open question is whether efficient algorithms for estimating time-bounded Kolmogorov complexity would yield efficient algorithms for NP-complete problems. This is open even in an average-case sense: we do not know whether Gap-MINKT $\in$ P implies that NP is easy on average. Informally, such a result would say that the ability to perform compression, or to recognize strings admitting short efficient descriptions, is powerful enough to solve broad classes of computational search and optimization tasks. From the perspective of learning, this would elevate the role of compression beyond efficient prediction: it would show in a certain sense that compression also underlies efficient planning and decision-making. For recent papers on the computational complexity of compression, see [Hir22b, HLO26b] and references therein.

Problem 4.5 (Computational Complexity of Compression)**.** What is the hardness of the task of computing/estimating the time-bounded Kolmogorov complexity of an input string? Formally, the following central problems from meta-complexity remain open:

- Does Gap-MINKT $\in$ P imply that DistNP $\subseteq$ AvgP? In other words, does the easiness of estimating complexity imply that NP is easy on average?
- Does MINKT $\in$ P imply that NP $\subseteq$ P? In other words, does the easiness of (exactly) computing complexity imply that NP is easy in the worst case?

4.4 Auxiliary Results

We collect here some lemmas needed in results mentioned in the preceding sections.

Lemma 4.6. *If Efficient Heuristic Learning is possible, then ioPRFs do not exist. It follows that (infinitely-often) one-way functions and pseudorandom generators do not exist.*

Proof. Let $f = \{f : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}\}_{n\in\mathbb{N}}$ be an ioPRF, and let $n \in \mathbb{N}$ be a sufficiently large input length at which it is secure. Define a distribution P_n over target functions as follows: sample $u \sim \{0,1\}^n$ uniformly at random and output a description of the circuit $f_u\colon \{0,1\}^n \to \{0,1\}$ defined by $f_u(x) = f(u,x)$ (with u hardcoded). Since u is sampled uniformly and f is polynomial-time computable, $P_n \in$ PSAMP. Let $D_n = \mathcal{U}_n \in$ PSAMP.

By assumption, Boolean circuits are heuristically learnable over PSAMP. Hence there exists an efficient machine A and a polynomial q' such that for all sufficiently large n,

$$\Pr_{\substack{u\sim\mathcal{U}_n;\\ \mathsf{EX}(f_u,\mathcal{U}_n);\\ x'\sim\mathcal{U}_n}}\left[A^{\mathsf{EX}(f_u,\mathcal{U}_n)}(x') = f_u(x')\right] \;\geq\; \frac{1}{2} + \frac{1}{q'(n)}. \tag{2}$$

We now construct an efficient distinguisher D for f as follows.

> Given oracle access to some function O, which is either $f(u,\cdot)$ for a random $u \sim \{0,1\}^n$ or a truly random function $R_n\colon \{0,1\}^n \to \{0,1\}$, D draws $x' \sim \mathcal{U}_n$, queries O to obtain b, and outputs 1 if and only if $A^{\mathsf{EX}(O,\mathcal{U}_n)}(x') = b$, where the oracle $\mathsf{EX}(O,\mathcal{U}_n)$ may be simulated by sampling $x \sim \mathcal{U}_n$ and querying $O(x)$ to get the label.

When the oracle O is $f(u,\cdot)$ for $u \sim \{0,1\}^n$, the learner A is run in a setting exactly matching the setting of Equation (2), so D outputs 1 with probability at least $(1/2) + (1/q'(n))$.

When the oracle O is R_n, $b = R_n(x')$ is uniformly random and independent of the output of A on any input not previously inspected by A. Since A inspects at most polynomially many inputs, D outputs 1 with probability exponentially close to $1/2$.

Therefore,

$$\left|\Pr_{u\sim\{0,1\}^n}\left[D^{f(u,\cdot)}(1^n) = 1\right] - \Pr_{R_n}\left[D^{R_n(\cdot)}(1^n) = 1\right]\right| \;=\; \Omega\left(\frac{1}{q'(n)}\right),$$

contradicting the assumption that f is an ioPRF. Thus, ioPRFs do not exist. This implies that standard PRFs likewise do not exist. The non-existence of both infinitely-often and standard OWFs and PRGs follows by well-known reductions among these primitives [HILL99, GGM86]. □

Using a very similar argument, we can rule out *auxiliary input* PRFs assuming easiness of PAC learning, even when membership queries are allowed.

Lemma 4.7. *If Efficient PAC Learning is possible, then AIPRFs do not exist.*

Proof Sketch. The proof is very similar to that of Lemma 4.6. Let $f_z : \{0,1\}^{n(|z|)} \times \{0,1\}^{n(|z|)} \to \{0,1\}$ be a purported AIPRF. The key idea is that, for *every* choice of $z \in \{0,1\}^*$ and $u \in \{0,1\}^{n(|z|)}$, $f_z(u,-) \in \mathsf{SIZE}[p]$ for some fixed polynomial p. We are guaranteed that the learner A will succeed (with high probability) on every such target function. Another difference is that A is allowed to make membership queries to the target function, which the distinguisher D may simulate by querying O. □

Finally, the same consequence holds assuming easiness of distributional learning. This lemma is a key ingredient in the proof that distribution learning is at least as strong as agnostic learning.

Lemma 4.8 ([KMR+94, Xia10])**.** *If Efficient Distribution Learning is possible, then AIPRFs do not exist.*

Proof Sketch. Let $f = \{f_z\}$ be a purported AIPRF, and let A be the distribution learner promised by P.II. Fix $z \in \{0,1\}^*$, write $n = n(|z|)$, and for an oracle $O\colon \{0,1\}^n \to \{0,1\}$ define the graph distribution

$$G_O \equiv (X, O(X)) \quad \text{where } X \sim \mathcal{U}_n.$$

If $O = f_z(u,\cdot)$, then G_O is sampled by the efficient sampler that has (z,u) hardwired, chooses $X \sim \mathcal{U}_n$, and outputs $(X, f_z(u,X))$. Thus, for every z and u, P.II applies to $G_{f_z(u,\cdot)}$ with polynomially related size and time parameters.

We construct an oracle distinguisher D. Given z and oracle access to O, it simulates sample access to G_O by drawing $X \sim \mathcal{U}_n$ and querying $O(X)$. It runs A with a small constant accuracy parameter η and confidence parameter η, obtaining a sampler C over pairs $(x,b) \in \{0,1\}^n \times \{0,1\}$. It then validates C as follows. Letting Q be the set of inputs $x \in \{0,1\}^n$ queried to O during the training phase, take one fresh sample $(x',b') \sim C$, and accept if and only if $x' \notin Q$ and $b' = O(x')$.

On the pseudorandom side, A is being run on samples from a distribution in the class handled by P.II. So, with probability at least $1-\eta$, the output sampler C satisfies $\Delta_{\mathsf{TV}}(C(\mathcal{U}), G_{f_z(u,\cdot)}) \le \eta$. Total variation closeness implies that C is supported almost entirely on the graph of $f_z(u,\cdot)$ (so $b' = O(x')$ with high probability) and that its first marginal is close to $\mathcal{U}_n$ (so $x' \notin Q$ with high probability). Therefore the validation test accepts with probability $1 - O(\eta)$.

On the random side, let $O = R$ be a uniformly random function. After conditioning on the polynomially many graph points revealed while simulating the samples for A, the values of R at fresh points are still independent fair bits. The "$x' \notin Q$" test rules out the only simple way for the sampler C to pass the graph consistency test, namely putting noticeable mass on the already revealed inputs. Since C depends only on the coins of A and $R|_Q$, $R(x')$ is uniform and independent of C for $x' \notin Q$. Consequently, the output of any polynomial-time learner passes the "$b' = O(x')$" test with probability bounded away from the pseudorandom case, after fixing η sufficiently small.

Thus D distinguishes $f_z(u,\cdot)$ for $u \sim \mathcal{U}_n$ from a truly random function with inverse polynomial advantage for all sufficiently large relevant input lengths. This contradicts the auxiliary input pseudorandomness condition. Therefore AIPRFs do not exist. □

4.5 White-Box Learning and One-Way Functions

In this section, we explore connections between *white-box learning*, in which a labelled example distribution is given explicitly as a sampling circuit, and the existence of (auxiliary-input) one-way functions. These connections are useful as tools to establish the ordering among the learning paradises of Section 4.1, particularly that P.II (Efficient Distribution Learning) implies P.III (Efficient Agnostic Learning), and may also be of interest in their own right.

4.5.1 White-Box Learning Frameworks

The model of white-box learning we consider is the same as Agnostic Learning (P.III), except that the learner is additionally provided an explicit description of a sampler for the example distribution in question. For this reason, we restrict ourselves here to distribution families $E = \{E_n\}_{n\in\mathbb{N}}$ samplable by polynomial-size circuits.

More specifically, consider a joint labelled example distribution $E_n \in \mathsf{PSAMP}/\mathsf{poly}$ supported over $\{0,1\}^n \times \{0,1\}$, where for $(x,b) \sim E_n$, $x \in \{0,1\}^n$ is called the "example string" and $b \in \{0,1\}$ is called the "label". We let $E_n^{(0)}$ denote the marginal distribution over example strings and $E_n^{(1)}$ that over the labels.

As mentioned above, the learner is provided a sampling circuit S_n for E_n. Of course, this could potentially make the task of learning much easier. However, note that the learning task is still non-trivial: even though we can easily sample input/output pairs for the function in question, it may still be hard to compute the function on a desired input.

As in the setting of agnostic learning, we consider optimality with respect to a fixed concept class, C. That is, define

$$\mathsf{opt}_n(C,E) \triangleq \max_{c\in C_n}\left\{\Pr_{(x,b)\sim E_n}[c(x)=b]\right\}.$$

Since a white-box learner has the sampler S_n available, it can simulate this circuit on random inputs to obtain labelled examples distributed according to E_n. For this reason, it does not require oracle access to $\mathsf{EX}(E_n)$.

White-box learning is defined as follows.

Definition 4.9 (White-Box Learning [ABX08, Xia09])**.** Let $\mathcal{E} \subseteq \mathsf{PSAMP}/\mathsf{poly}$ be a class of joint distributions. An algorithm A *white-box learns* C over $\mathcal{E}$ if, for every $E = \{E_n\}_{n\in\mathbb{N}} \in \mathcal{E}$ and sequence $S = \{S_n\}_{n\in\mathbb{N}}$ with S_n being a sampling circuit for E_n, for all sufficiently large $n \in \mathbb{N}$, for all $\delta, \varepsilon \in (0,1)$,

$$\Pr_A\left[A(1^n, 1^{\varepsilon^{-1}}, \delta^{-1}, S_n) \text{ outputs } h \text{ s. t. } \Pr_{(x,b)\sim E_n}[h(x)=b] \geq \mathsf{opt}_n(C,E) - \varepsilon\right] \geq 1-\delta.$$

We will primarily be interested in White-Box Learning for the following choices of concept class C: $C = \mathsf{SIZE}[s]$ for a fixed function $s : \mathbb{N} \to \mathbb{N}$, and $C = \mathcal{F}_{all}$, where $(\mathcal{F}_{all})_n$ denotes the set of all boolean functions $f : \{0,1\}^n \to \{0,1\}$. In particular, in the latter case, the learner aims to approximate the Bayes-optimal predictor for $\mathcal{E}$.

4.5.2 White-Box Learning and Breaking Auxiliary-Input One-Way Functions

In this section, we establish the following:

$$\neg\exists\text{AIOWF} \iff \text{White-Box Learning } \mathcal{F}_{all}.$$

The implication from $\neg\exists$AIOWF to the easiness of White-Box Learning $\mathcal{F}_{all}$ was proved in [Xia09] and implicit in [ABX08].

Also note that, since White-Box Learning $\mathcal{F}_{all}$ implies White-Box Learning C for any desired concept class $C \subseteq \mathcal{F}_{all}$, the above implies that

$$\neg\exists\text{AIOWF} \implies \text{White-Box Learning } \mathsf{SIZE}[s]$$

for any $s : \mathbb{N} \to \mathbb{N}$. The latter may be used in a proof that P.II (Efficient Distribution Learning) $\implies$ P.III (Efficient Agnostic Learning), as in Lemma 4.2.

[Xia09] leaves as an open question whether White-Box Learning $\mathsf{SIZE}[s]$, for a fixed polynomial s, implies that AIOWFs can be broken. Note that a positive answer would show the equivalence of White-Box Learning with respect to $\mathsf{SIZE}[s]$ and $\mathcal{F}_{all}$ over distributions samplable in $\mathsf{PSAMP}/\mathsf{poly}$.

Problem 4.10 (White-Box Learning and AIOWF)**.** Does White-Box Learning $\mathsf{SIZE}[s]$ imply the non-existence of AIOWF? Alternatively, is there evidence to the contrary?

We begin with the following auxiliary lemma.

Lemma 4.11. *Assume $\neg\exists$ AIOWF. Let E be a family of labelled example distributions with sampling circuits S, as in Definition 4.9. There exists a polynomial-time algorithm that, for all sufficiently large $n \in \mathbb{N}$, on input $(1^n, 1^{(\varepsilon')^{-1}}, S_n)$, outputs a* randomized *hypothesis h such that $\Delta((x, h(x)), E_n) \leq \varepsilon'(n)$, where $x \sim E_n^{(0)}$.*

Proof. Let $\{E_n = (E_n^{(0)}, E_n^{(1)})\}_{n\in\mathbb{N}}$, with each E_n supported over $\{0,1\}^n \times \{0,1\}^{m(n)}$, be a labelled example distribution samplable by poly-size circuits $\{S_n = (S_n^{(0)}, S_n^{(1)})\}_{n\in\mathbb{N}}$, each taking randomness of length $\rho(n) = \mathsf{poly}(n)$ as input. Define a poly-time computable auxiliary input function f as follows.

$$f_{S_n^{(0)}}(r) = S_n^{(0)}(r).$$

That is, f has the sampler $S_n^{(0)}$ hard-coded as an auxiliary input. Its input is a choice of randomness r, and its output is the output of $S_n^{(0)}$ run on r.

Assuming $\neg\exists$ AIOWF, by Lemma 2.16, $\neg\exists$ distAIOWF. For any desired inverse polynomial ε', let I be a PPT ε'-distributional inverter for f. That is, for any sufficiently large $n \in \mathbb{N}$,

$$\Delta\Big((f_{S_n^{(0)}}(r), I(S_n^{(0)}, f_{S_n^{(0)}}(r))),\ (f_{S_n^{(0)}}(r), r) \Big) \leq \varepsilon'(n), \tag{3}$$

where $r \sim \mathcal{U}_{\rho(n)}$.

The desired algorithm will output the following randomized hypothesis h:

> On input $x \in \{0,1\}^n$, h invokes I on x with auxiliary input $S_n^{(0)}$ to produce a pre-image r; then, h outputs $S_n^{(1)}(r)$.

Observe that for $r \sim \mathcal{U}_{\rho(n)}$ and $x = S_n^{(0)}(r)$ (that is, $x \sim E_n^{(0)}$),

$$\begin{aligned}
\Delta((x,h(x)),E_n) &= \Delta((x, S_n^{(1)}(I(S_n^{(0)},x))),(x,S_n^{(1)}(r))) && \text{(definition of } h\text{)}\\
&\leq \Delta((x, I(S_n^{(0)},x)),(x,r))\\
&= \Delta((f_{S_n^{(0)}}(r), I(S_n^{(0)},x)),(f_{S_n^{(0)}}(r),r)) && \text{(definition of } f\text{)}\\
&\leq \varepsilon'(n). && \text{(Eq. (3))}
\end{aligned}$$

This completes the proof.

□

The next two lemmas establish an equivalence between $\neg\exists$ AIOWF and the easiness of White-Box Learning $\mathcal{F}_{all}$.

Lemma 4.12 ([ABX08, Xia09])**.** *If $\neg\exists$ AIOWF, then White-Box Learning $\mathcal{F}_{all}$ over* PSAMP/poly *is easy.*

Proof. The desired white-box learner, given a sampler S_n for E_n and parameters ε and δ, will construct h from S_n as in Lemma 4.11 with parameter $\varepsilon' = \varepsilon^4$. It will then sample $m \triangleq 8\varepsilon^{-4} \cdot n \cdot \log(1/\delta)$ choices of randomness $r_1, .., r_m$ for h, and output the deterministic hypothesis H that, on input x, outputs $\mathsf{maj}_{i\in[m]}\{h(x; r_i)\}$.

We now argue for the correctness of this white-box learner. Recall that h is such that $\Delta((x,h(x)),E_n) \leq \varepsilon'$, where $x \sim E_n^{(0)}$. Let $f_{opt} : \{0,1\}^n \to \{0,1\}$ be the function that maximises $\Pr_{(x,b)\sim E_n}[f_{opt}(x) = b]$, and, for $x \in \{0,1\}^n$, let B_x denote the conditional distribution of $E_n^{(1)}$ given $E_n^{(0)} = x$. Define

$$\alpha(x) = \max_{b\in\{0,1\}} \{\Pr[B_x = b]\}.$$

Note that $f_{opt}(x) = \arg\max_{b\in\{0,1\}}\{\Pr[B_x = b]\}$, so $\Pr[f_{opt}(x) = B_x] = \alpha(x)$.

By Markov's inequality, the set $\mathcal{X}_{bad} = \{x : \Delta(h(x), B_x) > \varepsilon^2\}$ satisfies $\Pr_{x\sim E_n^{(0)}}[x \in \mathcal{X}_{bad}] \leq \varepsilon^2$. For all $x \notin \mathcal{X}_{bad}$, we have $|\Pr[h(x) = b] - \Pr[B_x = b]| \leq \varepsilon^2$ for every $b \in \{0,1\}$.

For $x \notin \mathcal{X}_{bad}$, we consider two cases.

Case I: $\alpha(x) > 1/2 + 2\varepsilon^2$**.** Note that

$$\begin{aligned}\Pr_{r_i}[h(x;r_i) = f_{opt}(x)] &\geq \alpha(x) - \varepsilon^2 \\ &> \frac{1}{2} + \varepsilon^2.\end{aligned}$$

Then, by a Chernoff bound,

$$\Pr_{r_1,\ldots,r_m}\left[H(x) = f_{opt}(x)\right] \geq 1 - \delta \cdot 2^{-n}.$$

◇

Case II: $\alpha(x) \leq 1/2 + 2\varepsilon^2$**.** Note that $\alpha(x) \geq 1/2$ also holds in this case, by definition of $\alpha(x)$. Consider any fixed H. If $H(x) = f_{opt}(x)$, then $\Pr[H(x) = B_x] \geq 1/2$. On the other hand, if $H(x) = 1 - f_{opt}(x)$, then $\Pr[H(x) = B_x] \geq 1 - \Pr[f_{opt}(x) = B_x] \geq 1/2 - 2\varepsilon^2$. It follows that

$$\Pr[f_{opt} = B_x] - \Pr[H(x) = B_x] \leq \frac{1}{2} + 2\varepsilon^2 - \left(\frac{1}{2} - 2\varepsilon^2\right) = 4\varepsilon^2.$$

◇

By a union bound, with probability $1 - \delta$ over the choice of $r_1, \ldots, r_m$, all x satisfying Case I satisfy $H(x) = f_{opt}(x)$. Then, with probability $1 - \delta$, for all x satisfying either case,

$$\Pr[f_{opt} = B_x] - \Pr[H(x) = B_x] \leq 4\varepsilon^2.$$

Overall, with probability at least $1 - \delta$ over the randomness of the white-box learner, it outputs a hypothesis H such that

$$\Pr_{(x,b)\sim E_n}[H(x) = b] \geq \mathsf{opt}_n(\mathcal{F}_{all}, E) - 4\varepsilon^2 - \varepsilon^2 \geq \mathsf{opt}_n(\mathcal{F}_{all}, E) - \varepsilon,$$

assuming $\varepsilon < 1/5$ without loss of generality.

This completes the proof.

□

Lemma 4.13. *If White-Box Learning $\mathcal{F}_{all}$ over* PSAMP/poly *is easy, then* $\neg\exists$ *AIOWF.*

Proof. Toward a contradiction, suppose there exists AIOWF. By Lemma 2.16, there likewise exists an AIPRG, $G_z : \{0,1\}^n \to \{0,1\}^{2n}$. By an exhaustive search running in time exponential in n, it is possible to distinguish outputs of G_z from

truly random strings. Let D be this inefficient distinguisher. In particular, for any given z,

$$\left| \Pr_{v \sim \mathcal{U}_n} [D(z, G_z(v)) = 1] - \Pr_{u \sim \mathcal{U}_{2n}} [D(z, u) = 1] \right| \geq 1 - o(1).$$

By Yao's hybrid argument (see Section 5.1.1), there exists an inefficient next-bit predictor P^D such that, for any z,

$$\Pr_{P^D;\, i \sim [2n];\, v \sim \mathcal{U}_n} \left[P^D(z, G_z(v)_{<i}) = G_z(v)_i \right] > \frac{1}{2} + \frac{1}{8n},$$

where y_j denotes the j^{th} bit of a string y, and $y_{<j}$ denotes the prefix of y up to and including position $j - 1$.

Now, consider the distribution E_z, which samples a random index $i \sim [2n]$ and a random seed $v \sim \mathcal{U}_n$, and then outputs

$$(z, i, G_z(v)_{<i}, 0^{2n-i+1}; G_z(v)_i).$$

It is easy to check that E_z is samplable by a polynomial-size circuit, which can be constructed in polynomial time given z. Moreover, by the existence of P^D, we have that

$$\operatorname*{opt}_N(\mathcal{F}_{all}, E_z) \geq \frac{1}{2} + \frac{1}{8n},$$

where $N = |z| + 2n + O(\log n)$ is the length of an example string from E_z. Going forward, we omit the trailing 0s used for padding.

We claim that a white-box learner for E_z may be used to break G_z. Let A' be a white-box learner with respect to $\mathcal{F}_{all}$. Consider the following adversary A:

> On input (z, y), construct a sampler S_z for E_z, and simulate $A'(S_z)$ with accuracy parameter $\varepsilon = n^{-2}$ and confidence $\delta = 2^{-n}$ to obtain a hypothesis h. Sample a random $j \sim [2n]$, and accept iff $h(z, j, y_{<j}) = y_j$.

First consider the case that $y \sim G_z(\mathcal{U}_n)$. In this case,

$$\begin{aligned}
\Pr_{A,y}[A(z, y) = 1] &= \Pr_{A',y,j}\left[h(z, y_{<j}) = y_j\right] \\
&= \Pr_{A',(x,b) \sim E_z} [h(x) = b] \\
&\geq \frac{1}{2} + \frac{1}{8n} - \frac{1}{n^2} - \frac{1}{2^n} \\
&> \frac{1}{2} + \frac{1}{10n}.
\end{aligned}$$

Next, in the case that $y \sim \mathcal{U}_{2n}$, y_j is a uniformly random bit independent of $(z, y_{<j})$, so

$$\Pr_{A,y}[A(z, y) = 1] = \frac{1}{2}.$$

We conclude that A breaks G_z, as desired. □

5 Learning via Meta-Complexity and Algorithmic Information Theory

5.1 Detecting Structure: Supervised Learning via Distinguishers

Valiant introduced his framework of *PAC learning* in 1984 [Val84]; in the same paper, he already recognized a tension between the possibility of efficient learning and the possibility of cryptography. He points out in particular that if PAC learning is easy, then it would be easy to break a pseudorandom function. Though the converse of this specific statement remains unknown, the connection also goes in the other direction, where algorithms for "detecting structure from randomness" may be used to carry out learning tasks.

In this section, we describe a general framework for establishing relationships in the latter direction. In this framework, algorithms for *meta-complexity* (namely, deciding whether a given object is of "high" or "low" complexity) are used to break cryptographic and complexity-theoretic pseudorandom generators, yielding learning algorithms by applying reconstruction properties of those generators. We consider two different styles of generator, which we refer to as the Direct Product Generator and the Nisan-Wigderson Generator, respectively.

The Direct Product Generator takes the form

$$G^f_{DP}(\overline{x}) = (x_1, ..., x_m; b_1, ..., b_m),$$

where $\overline{x} = (x_1, ..., x_m) \in (\{0,1\}^n)^m$ are input examples, and $(b_1, ..., b_m) \in \{0,1\}^m$ are some labels associated with a target function $f\colon \{0,1\}^n \to \{0,1\}$.

The Nisan-Wigderson Generator takes the form

$$G^f_{NW}(z) = (f(z|_{S_1}), ..., f(z|_{S_m})),$$

where $z \in \{0,1\}^{O(n^2)}$ is an input specifying some examples, $f\colon \{0,1\}^n \to \{0,1\}$ is a target function, and $S_1, ..., S_m$ are *NW-designs*. An NW-design is a collection of m subsets (where m is the desired output length) chosen from a small ground set of size ℓ. They are constructed to have bounded intersections: for instance, any two distinct subsets S_i, S_j might overlap in at most $\log m$ elements. Roughly speaking, because the subsets intersect very little, the inputs fed into a "hard" function f producing the output bits remain mostly independent.

In the results we survey below, we will see various instantiations of these generators, which may be broken under various assumptions about the easiness of meta-complexity. We mention in advance that learners obtained by breaking the Direct Product Generator can work from *random examples*, whereas learners obtained by breaking the Nisan-Wigderson Generator require *membership queries*.

In cases where the corresponding assumption from meta-complexity is known to hold, the approach has led to unconditional learning algorithms.

5.1.1 From Distinguishers to Predictors

The central technique driving all the results in this section is Yao's hybrid argument, which converts a *distinguisher* into a *next-bit predictor* [Yao82]. First, we explain what it means for an algorithm to be a distinguisher. Consider an arbitrary joint distribution (X, Y), where X is supported over $\{0,1\}^m$ and Y over $\{0,1\}^\ell$. Let A be a function (which we will typically identify with an algorithm) mapping $m + \ell$ input bits to one output bit. We consider the case that A *distinguishes* (X, Y) from $(X, \mathcal{U}_\ell)$ with advantage $\varepsilon > 0$, where $\mathcal{U}_\ell$ denotes the uniform distribution over $\{0,1\}^\ell$. Namely,

$$\left| \Pr_A[A(X,Y) = 1] - \Pr_A[A(X,\mathcal{U}_\ell) = 1] \right| \geq \varepsilon.$$

Yao showed that, from any such A, one can construct another algorithm $P_A : \{0,1\}^m \times \{0,1\}^{<\ell} \to \{0,1\}$ taking an output of X together with some prefix of an output of Y and producing the next unseen bit of Y with better-than-trivial success probability. In particular,

$$\Pr_{P_A, i\sim[\ell]}[P_A(X, Y_1, ..., Y_{i-1}) = Y_i] \geq \frac{1}{2} + \frac{\varepsilon}{2\ell}.$$

We refer to such an algorithm P_A as a *next-bit predictor*.

To prove this statement, Yao employs a hybrid argument. Roughly, one defines "hybrid" distributions

$$H_j = (X, Y_1, ..., Y_j, \mathcal{U}_{\ell-j})$$

for $j \in \{0, ..., \ell\}$, so $H_0 = (X, \mathcal{U}_\ell)$ and $H_\ell = (X, Y)$. Observe that

$$\Pr[A(H_0) = 1] - \Pr[A(H_\ell) = 1] = \sum_{j\in\{0,...\ell-1\}} \Pr[A(H_j) = 1] - \Pr[A(H_{j+1}) = 1].$$

Supposing that $\Pr[A(X, \mathcal{U}_\ell) = 1] - \Pr[A(X, Y) = 1] \geq \varepsilon$, so the above is at least ε, it holds that the difference $\Pr[A(H_j) = 1] - \Pr[A(H_{j+1}) = 1]$ is at least ε/ℓ in expectation over the choice of j.[10] This expected advantage of A at the j^{th} bit of Y can then be leveraged to obtain a next-bit predictor at the corresponding position: when given a random bit $u \sim \mathcal{U}$ at position $j + 1$, A is slightly less likely to output 1 when $u = Y_{j+1}$ than when $u \neq Y_{j+1}$. We refer the reader to [Vad12] for further detail.

[10] The case that $\Pr[A(X, Y) = 1] - \Pr[A(X, \mathcal{U}_\ell) = 1] \geq \varepsilon$ can be handled similarly.

5.1.2 Direct Product and Random Examples

From PRG Distinguishers to Learning Random Target Functions. An early instantiation of the framework described above can be seen in the work of Blum et al. [BFKL93], where the authors show that breaking a cryptographic pseudorandom generator yields a learner working in a certain average-case setting. In more detail, from a successful distinguisher against a candidate PRG construction, one obtains a learner succeeding with high probability over randomly sampled target functions (see Definition 2.10).

The starting point is the assumption that for every candidate PRG $G : \{0,1\}^n \to \{0,1\}^{\ell(n)}$ there exists an efficient algorithm $A : \{0,1\}^{\ell(n)} \to \{0,1\}$ distinguishing outputs of G from uniformly random strings. That is, for some polynomial p, for all sufficiently large n,

$$\left|\Pr[A(G(\mathcal{U}_n)) = 1] - \Pr[A(\mathcal{U}_{\ell(n)}) = 1]\right| \geq \frac{1}{p(n)}.$$

Let $P = \{P_n\}_{n\geq 1}$ be a collection of distributions over target functions. For the sake of this exposition, we will discuss the case of weak prediction over example strings drawn from the uniform distribution.[11] We define a candidate PRG G as follows, where $m = m(n)$ is a sufficiently large parameter.

> On input $(r_P, x_1, ..., x_m)$, with $r_P \in \{0,1\}^{\mathsf{poly}(n)}$ and each $x_i \in \{0,1\}^n$, G samples $\sigma_f \sim P_n$ using randomness r_P, where σ_f is the representation of a function f. For each $i \in [m]$, it then computes $b_i \triangleq f(x_i)$. G outputs $(x_1, ..., x_m, b_1, ..., b_m)$.

The goal is then to transform a distinguisher A for G into a learner for P_n over $\mathcal{U}_n$ in the sense of Definition 2.10. By definitions of G and A, we have

$$\left|\Pr_{\substack{\sigma_f \sim P_n; \\ x_1,...,x_m \sim \mathcal{U}_n}}[A(x_1, ..., x_m, f(x_1), ..., f(x_m)) = 1] - \Pr[A(\mathcal{U}_{m\cdot n+m}) = 1]\right| \geq \frac{1}{p(n)}.$$

By Yao's hybrid argument, for some PPT machine A' built from A,

$$\Pr_{\substack{\sigma_f \sim P_n;\, A';\, i\sim[m]; \\ x_1,...,x_m \sim \mathcal{U}_n}}[A'(x_1, ..., x_m, f(x_1), ..., f(x_{i-1})) = f(x_i)] \geq \frac{1}{2} + \frac{1}{2m \cdot p(n)}.$$

We may then define a learner A'' as follows.

[11] [BFKL93] use a different argument, inverting an appropriate one-way function, to yield strong prediction over examples drawn from other distributions in PSAMP. See Theorem 11 in [BFKL93].

On input $((x_1, f(x_1)), ..., (x_m, f(x_m)); x')$, sample $i \sim [m]$, and output $A'(x_1, ..., x_{i-1}, x', x_{i+1}, ..., x_m, f(x_1), ..., f(x_{i-1}))$.

By the definitions of A' and A'', we have that

$$\Pr_{\substack{\sigma_f \sim P_n; \\ (x_i, f(x_i))_{i \in [m(n)]} \sim \mathsf{EX}(f, \mathcal{U}_n)^{m(n)}; \\ x' \sim \mathcal{U}_n}} [A''((x_1, f(x_1)), ..., (x_{m(n)}, f(x_{m(n)})), x') = f(x')] \geq \frac{1}{2} + \frac{1}{2m \cdot p(n)}.$$

Thus, A'' satisfies the requirement of weak prediction.

From RRHS Refutation to PAC Learning over Arbitrary Distributions. Since the insight of [BFKL93], several papers continued to employ distinguishers to obtain learners. In an elegant example of such work, Vadhan [Vad17] formalises a distinguishing task to capture *worst-case* PAC learning over *arbitrary* example distributions. Specifically, for a concept class P and its dual class $P^* = Q$ (where for an evaluation function $\mathsf{Eval} : \{0,1\}^s \times \{0,1\}^t \to \{0,1\}$, $p_y(x) = q_x(y) = \mathsf{Eval}(y, x)$), Vadhan defines the *Random Right-Hand Side (RRHS) Refutation* problem for Q, where the task is to distinguish

- $(x_1, b_1), ..., (x_m, b_m)$, for independently random bits $b_i \sim \mathcal{U}$, from
- $(x_1, b_1), ..., (x_m, b_m)$, where $b_i = q_{x_i}(y) = p_y(x_i)$ for a fixed concept $f = p_y \in P$.

Observe that this is precisely the task of distinguishing the output of the Direct Product Generator

$$G^f_{DP}(x_1, \ldots, x_m) = (x_1, \ldots, x_m,\ f(x_1), \ldots, f(x_m))$$

from

$$(x_1, \ldots, x_m, \mathcal{U}_m),$$

for arbitrary strings $x_1, ..., x_m$.

We are interested here in the reduction from PAC learning to RRHS Refutation. Let A be an RRHS Refuter for some concept class C: that is, for any $f \in C$ and any choice of $x_1, ..., x_m$, A distinguishes the two distributions above. As in the work of [BFKL93], Yao's hybrid argument yields the existence of a machine A' such that

$$\Pr_{r \sim \mathcal{U}_{\mathsf{poly}(n)};\ i \sim [m]} [\ A'(x_1, ..., x_m, f(x_1), ..., f(x_{i-1}); r) = f(x_i)\ ] \geq \frac{1}{2} + \frac{1}{\mathsf{poly}(n)}.$$

Letting D_n be an arbitrary example distribution, an averaging argument implies that for some polynomial p, with probability at least $1/p(n)$ over the choice of r, i, and $x_1, ..., x_{i-1}, x_{i+1}, ..., x_m \sim D_n^{m-1}$,

$$\Pr_{x_i \sim D_n} [\ A'(x_1, ..., x_m, f(x_1), ..., f(x_{i-1}); r) = f(x_i)\ ] \geq \frac{1}{2} + \frac{1}{p(n)}.$$

Now, let B be the algorithm that takes m examples $(x_1, f(x_1))..., (x_m, f(x_m))$ from its oracle $\mathsf{EX}(f, D_n)$, chooses a random $i \sim [m]$ and randomness r for A', and then outputs the hypothesis

$$h(x) \triangleq A'(x_1, ..., x_{i-1}, x, x_{i+1}, ..., x_m, f(x_1), ..., f(x_{i-1}); r).$$

By the above,

$$\Pr_{B;\, \mathsf{EX}(f,D_n)} \left[B^{\mathsf{EX}(f,D_n)} \text{ outputs } h \text{ such that } \Pr_{x \sim D_n} [h(x) = f(x)] \geq \frac{1}{2} + \frac{1}{p(n)} \right] \geq \frac{1}{p(n)}.$$

Note that the confidence parameter $\delta = 1 - p(n)^{-1}$ can be boosted by parallel repetition, using empirical estimation to determine the best hypothesis produced.

In this way, the easiness of RRHS Refutation yields a weak PAC learner. A classic result of Schapire shows that such a learner can be *boosted* to obtain a (strong) PAC learner [Sch90].

Correlative RRHS Refutation and Agnostic Learning. In work independent of [Vad17], Kothari and Livni [KL18] give a related refutation problem, denoted *correlative RRHS Refutation*, to characterise the harder task of *agnostic learning*. Recall that in the setting of agnostic learning, examples from the oracle EX are not guaranteed to correspond exactly to any fixed target function from the relevant concept class; the learner is only asked to produce a hypothesis that is "almost" as accurate as the most accurate function from the concept class [KSS94, Hau92].

One difference compared to the RRHS Refutation task of [Vad17] is that correlative RRHS Refutation is defined in a *distribution-specific* setting. Then, the task of correlative RRHS Refutation over some particular example distribution characterises that of agnostic learning over the *same* distribution, thanks to the existence of distribution-specific boosting algorithms for agnostic learners [KK09, Fel10]. This is unlike the setting of [Vad17], which requires an RRHS refuter working on *every* distribution to obtain a learner working on every distribution.

As in the works discussed above, a weak learner in [KL18] arises from a hybrid argument. More specifically, suppose we have a *correlative RRHS refuter*

that is capable of detecting samples with correlation δ with circuits in $\mathsf{SIZE}[s]$. Namely, let $\mathcal{E}$ be a joint example distribution satisfying

$$\max_{c \in \mathsf{SIZE}[s(n)]} \left\{ \Pr_{(x,b)\sim\mathcal{E}_n} [c(x) = b] \right\} \geq \frac{1}{2} + \delta,$$

and suppose we have an algorithm A that $\Omega(1)$-distinguishes

- $(x_1, b_1), ..., (x_m, b_m)$, for independently random $(x_i, -) \sim \mathcal{E}_n$ and $b_i \sim \mathcal{U}$, from
- $(x_1, b_1), ..., (x_m, b_m)$, where $(x_i, b_i) \sim \mathcal{E}_n$ independently.

Then, by a hybrid argument, given m random examples from $\mathcal{E}_n$, A yields a construction of a hypothesis h such that

$$\Pr_{(x,b)\sim\mathcal{E}_n} [h(x) = b] \geq \frac{1}{2} + \frac{1}{O(m)}.$$

This is sufficient to apply known boosting algorithms [KK09, Fel10] to obtain a (strong) agnostic learner over $\mathcal{D}$.

From Average Case Easiness of NP to Agnostic Learning. Hirahara and Nanashima [HN21] build on [KL18] to prove that efficient agnostic learning of $\mathsf{P/poly}$ over distributions in $\mathsf{PSAMP/poly}$ is possible under the assumption that $\mathsf{DistNP} \subseteq \mathsf{AvgP}$. The paper also builds on an earlier line of work due to Hirahara, which proved that several promise problems in meta-complexity exhibit *worst-case to average-case reductions* (e.g. [Hir18, Hir20a]). In particular, if an appropriate 'gap' version of the time-bounded Kolmogorov complexity problem, denoted $\mathsf{Gap\text{-}MINKT}$, can be solved in deterministic poly-time on average over the uniform distribution, then it is likewise easy in the worst case.

The key contribution of [HN21] consists in reducing correlative RRHS Refutation to $\mathsf{Gap\text{-}MINKT}$. The intuition is that samples drawn according to the 'random' case are of high complexity with high probability, whereas the K^t complexity of samples from the 'correlative' case can be bounded. For simplicity, consider the setting of realizable PAC learning, so the examples in the latter case are actually consistent with some function in $\mathsf{SIZE}[s]$ for a polynomial s.

In the 'random' case, by Symmetry of Information for Kolmogorov complexity and a counting argument, one can show that for any time-bound t, with high probability over $x_i \sim \mathcal{D}_n$ and $b_i \sim \mathcal{U}$,

$$\mathsf{K}^t(x_1, ..., x_m, b_1, ..., b_m) \gtrsim \mathsf{K}(x_1, ..., x_m) + m.$$

On the other hand, if the labels b_i are equal to $f(x_i)$ for some function $f \in \mathsf{SIZE}[s]$, for a sufficiently large polynomial t', we have

$$\begin{aligned}\mathsf{K}^{\mathsf{poly}(t')}(x_1, ..., x_m, b_1, ..., b_m) &\leq \mathsf{K}^{t'}(x_1, ..., x_m) + O(s(n)) + O(\log n) \\ &\ll \mathsf{K}^{t'}(x_1, ..., x_m) + m,\end{aligned}$$

provided that m is sufficiently larger than s. This follows by encoding a description of an s-size circuit for f, and computing each b_i by running this circuit on input x_i. The idea is then to run an algorithm for $\mathsf{Gap\text{-}MINKT}$ on the given samples to distinguish "high" complexity from "low" complexity.

At a technical level, several difficulties arise from the differing (finite and infinite) time-bounds involved, and from the fact that one cannot exactly compute K^t complexity for a desired $t \in \mathbb{N}$ under the assumption $\mathsf{DistNP} \subseteq \mathsf{AvgP}$. In fact, the definition of $\mathsf{Gap\text{-}MINKT}$ is such that one only gets a poly-time algorithm A such that $A(1^t, x)$ outputs $k \in \mathbb{N}$ such that

$$\mathsf{K}^{p(t)}(x) - \log p(t) \leq k \leq \mathsf{K}^t(x)$$

for some polynomial p. Roughly speaking, the authors overcome this by proving that, for a sufficiently large time-bound t (depending on the running time to sample from a given distribution $\mathcal{D}_n \in \mathsf{PSAMP}/a(n)$),

$$\mathsf{K}^t(\overline{x}) \approx \mathsf{K}(\overline{x}) \pm a(n)$$

with high probability over strings $\overline{x} = (x_1, ..., x_m)$ sampled from $(\mathcal{D}_n)^m$. Thus, the differences between complexity measures with different time-bounds can be made immaterial by taking the number of samples m sufficiently larger than $a(n)$.

The approximate equality above follows from a *coding theorem* for time-bounded Kolmogorov complexity, which holds under the assumption $\mathsf{DistNP} \subseteq \mathsf{AvgP}$. In particular, for any distribution $\mathcal{D}_n \in \mathsf{PSAMP}/a(n)$ and any string x in the support of $\mathcal{D}_n$, for sufficiently large $t \in \mathbb{N}$ (depending on the exact complexity of sampling $\mathcal{D}$),

$$\mathsf{K}^t(x) \leq \log(1/\mathcal{D}_n(x)) + a(n) + O(\log n).$$

Since $\mathcal{D}_n \in \mathsf{PSAMP}/a(n)$ implies $(\mathcal{D}_n)^m \in \mathsf{PSAMP}/a(n)$ when $m = \mathsf{poly}(n)$, one obtains a similar statement for $\overline{x} \in \mathsf{Support}((\mathcal{D}_n)^m)$. Given this statement, it holds that

$$\begin{aligned}\mathop{\mathbb{E}}_{\overline{x}\sim\mathcal{D}_n^m}[\mathsf{K}^t(\overline{x}) - \mathsf{K}(\overline{x})] &\leq \mathop{\mathbb{E}}_{\overline{x}\sim\mathcal{D}_n^m}[\mathsf{K}^t(\overline{x})] - \mathsf{H}(\mathcal{D}_n^m) + O(\log n) \\ &\leq \mathop{\mathbb{E}}_{\overline{x}\sim\mathcal{D}_n^m}[\log(1/\mathcal{D}_n^m(\overline{x}))] + a(n) + O(\log n) - \mathsf{H}(\mathcal{D}_n^m) \\ &\leq a(n) + O(\log n),\end{aligned}$$

yielding the desired closeness between time-bounded and -unbounded Kolmogorov complexity in expectation over $\mathcal{D}_n^m$.

A subsequent work of Goldberg, Kabanets, Lu, and Oliveira [GKLO22] extends the result from [HN21], obtaining the same conclusion of efficient agnostic learning from the weaker assumption that $\mathsf{DistNP} \subseteq \mathsf{AvgBPP}$: that is, average-case easiness of NP for *randomised* algorithms. In this setting, one does not have derandomisation (which is available to [HN21] via [BFP05]), and thus does not have a worst-case to average-case reduction for $\mathsf{Gap\text{-}MINKT}$. Instead, the authors obtain worst-case easiness of a notion of time-bounded *probabilistic* Kolmogorov complexity, pK^t, which generalizes K^t and rK^t. Given this, the rest of the proof proceeds similarly to [HN21].

Agnostic Learning from Average Case Kolmogorov Complexity. Goldberg and Kabanets [GK23] further weaken the prerequisite assumptions for agnostic learning, exhibiting a direct connection from the average-case easiness of time-bounded (deterministic) Kolmogorov complexity to the existence of agnostic learners. The authors consider average-case easiness of K^t complexity as well as another complexity measure denoted KT [ABK+06], showing that agnostic learners follow from either assumption. KT complexity is known to be polynomially related to boolean circuit size, bringing the assumption close to a 'natural property' in the sense of Razborov and Rudich [RR97], and thus drawing a connection to a line of work obtaining learners from natural properties, discussed further in Section 5.1.3 below. For both K^t and KT, [GK23] assume average-case easiness for randomised algorithms over the uniform distribution.

Compared to the settings of [HN21] and [GKLO22], a few additional challenges arise. One difficulty has to do with the task of showing, as in [HN21], that $\mathsf{K}^t(\overline{x}) \approx \mathsf{K}(\overline{x})$ with high probability over strings $\overline{x} \sim (\mathcal{D}_n)^m$. As mentioned above, in the prior works, this was proved via coding theorems for time-bounded Kolmogorov complexity. In the case of [HN21], a coding theorem holds for K^t under the stronger assumption of average-case easiness of NP. In the case of [GKLO22], due to [LOZ22], a coding theorem holds unconditionally for pK^t. In the setting of [GK23], though a coding theorem is not available, a similar statement can be shown to hold with high probability in the presence of an additional random string r. Namely, for sufficiently large t,

$$\Pr_{r\sim\mathcal{U}_t}\left[\mathsf{K}^{\mathsf{poly}(t)}(x,r) \leq \log(1/\mathcal{D}_n(x)) + a(n) + |r| + O(\log n)\right] \geq 1 - o(1).$$

By incorporating this additional randomness r into the algorithm for correlative RRHS refutation, one may obtain an agnostic learner as desired.

Consideration of KT rather than K^t leads to additional challenges, primarily stemming from the fact that KT complexity does not admit a worst-case to

average-case reduction in the regime of a small approximation gap. On the other hand, the assumed average-case easiness of KT is still sufficient to learn *over the uniform distribution*. Hence, one method to obtain a learner is to reduce the task of learning over arbitrary distributions in $\mathsf{PSAMP}/\mathsf{poly}$ to the task of learning over the uniform distribution. This can be accomplished by using an average-case algorithm for KT as an inverter for a distributional one-way function; more details about this general approach can be found in the discussion of [BCK+22] below. However, in the case of non-uniformly sampled target distributions, the learner resulting from this argument needs to know the distribution explicitly.[12] [GK23] show that this caveat can be removed by assuming worst-case easiness of computing KT complexity, employing a more complicated amortization argument to bound the KT complexity of many concatenated samples from $\mathcal{D}_n$ in lieu of a coding theorem (see also [AGvM+18]). It is left open whether the caveat can be removed assuming only average-case easiness of KT.

5.1.3 Nisan-Wigderson and Membership Queries

All the results we have covered so far have employed distinguishers for variants of the Direct Product Generator, G^f_{DP}, to construct efficient learners. We now shift attention to learners arising from distinguishers for the Nisan-Wigderson Generator, G^f_{NW}.

The NW PRG is very well known for its applications in hardness/randomness tradeoffs [NW94, BFNW93, IW97]. In that context, one instantiates the PRG with a function f known to be hard. The NW PRG then takes the form

$$G^f_{NW}(z) = \left(f(z|_{S_1}),\ f(z|_{S_2}),\ \ldots,\ f(z|_{S_m}) \right),$$

where $f\colon \{0,1\}^n \to \{0,1\}$, $|z| = d = O(n^2)$, $m < 2^n$, and $S_1, \ldots, S_m \subseteq [d]$ are "NW designs", with $|S_i| = n$ and $|S_i \cap S_j| \leq \log m$ for all $i \neq j \in [m]$. A key idea is that the generator admits a *reconstruction procedure*, R. Namely, if D is an efficient algorithm $(1/\mathsf{poly}(m))$-distinguishing G_f from uniform, then $R^f(D)$ outputs a $\mathsf{poly}(m)$-size circuit approximating f with advantage $1/\mathsf{poly}(m)$ with high probability. For the purposes of the hardness/randomness paradigm, if we assume in advance that $\mathsf{poly}(m)$-size circuits cannot $(1/2+\mathsf{poly}^{-1}(m))$-approximate f, then we reach a contradiction, and we conclude that the NW PRG is indeed pseudorandom.

Learning from Natural Properties via NW Reconstruction. Carmosino et al. [CIKK16] provide the first and most influential example of this approach,

[12] Unlike the setting of white-box learning discussed in Section 4.5, here, the learner only needs the code of a sampler for the marginal distribution over example strings without labels.

showing that a distinguisher for G_{NW}^f implies the easiness of PAC learning with membership queries. As in the previous section, for the purposes of learning, [CIKK16] apply this reconstruction framework in reverse. The starting point is an assumption of the existence of a *natural property*. Razborov and Rudich introduced the notion of a natural property to pinpoint limitations of existing techniques for proving circuit lower bounds [RR97]. In particular, known proofs of circuit lower bounds were based on natural properties for those circuit classes (including lower bounds for AC^0 [FSS84] and $\mathsf{AC}^0[p]$ for prime p [Raz87, Smo87]). On the other hand, there is no (say, $\mathsf{P/poly}$-constructive) natural property for general boolean circuits unless cryptographic PRGs do not exist. Very roughly, a natural property for a given circuit class C is equivalent to average-case easiness of the Minimum Circuit Size Problem for C: the existence of a poly-time algorithm accepting all truth-tables of low C-complexity and rejecting a large enough fraction of truth-tables with high C-complexity (see Definition 2.31 for the formal details). The assumption of [CIKK16] is in this sense an assumption of the easiness of meta-complexity. The assumed natural property algorithm becomes a distinguisher for G_{NW}^f, and one applies a hybrid argument to obtain a next-bit predictor as specified by the reconstruction procedure.

There are several differences here compared to the setting of G_{DP}, which arise from how the generator makes use of its input. In the case of G_{DP}, several "input" strings $x_1, ..., x_m$ are given explicitly and independently, and each x_i leads to a distinct label b_i depending on f. In the case of G_{NW}, the input strings to f are given implicitly by a single seed z; "extracting" the strings x_i from z requires computing some restrictions based on the designs S_i. Due to this more "economical" use of the input, G_{NW} can be made to yield a much longer output from a short seed. For this reason, a natural property suffices to distinguish outputs of G_{NW} from uniformly random strings. On the other hand, in the case of G_{DP}, inputs of length $n \cdot m$ lead to outputs of length only $n \cdot m + m$, and it is not clear whether outputs of G_{DP} have circuit complexity less than this.

A drawback of G_{NW} compared to G_{DP} is that the reconstruction procedure requires oracle access to the function f. This is due to the fact that a given seed $z \sim \mathcal{U}_d$ gives rise to *dependent* input strings $x_1, ..., x_m = z|_{S_1}, ..., z|_{S_m}$. Due to the overlap between sets in the NW design, if one wants to know the value of f on a particular choice of x, this imposes restrictions on the values of the other strings x_j appearing in the hybrid. The NW reconstruction procedure handles this with a pre-processing stage, making oracle queries to f to build a table containing $f(x_j)$ for all possible completions of x_j depending on x. Ultimately, this results in a learner requiring membership queries.

Regardless of the limitations of learners resulting from natural properties, the connection to the literature on circuit lower bounds makes their construction worthwhile: by the known natural property for $\mathsf{AC}^0[p]$, Carmosino et al. uncon-

ditionally obtain a quasi-polynomial time learning algorithm for this circuit class (see Figure 6).

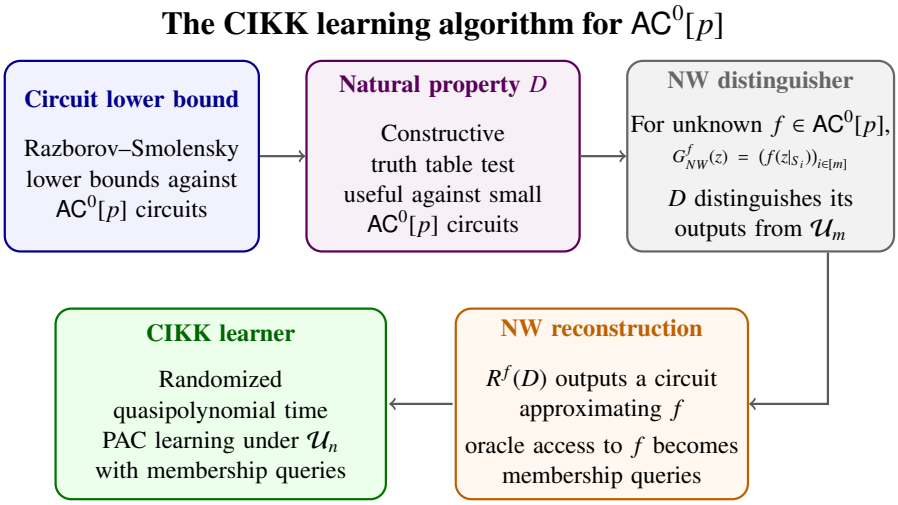


Figure 6: The diagram presents the key steps of the $\mathsf{AC}^0[p]$ learning algorithm of Carmosino, Impagliazzo, Kabanets, and Kolokolova [CIKK16], a breakthrough result in computational learning theory. The Razborov and Smolensky circuit lower bounds [Raz87, Smo87] yield a natural property [RR97] against $\mathsf{AC}^0[p]$. For an unknown target $f \in \mathsf{AC}^0[p]$, this property distinguishes the output of the Nisan-Wigderson generator G^f_{NW} [NW94] from a random truth table. The NW reconstruction procedure converts the distinguisher into a hypothesis using membership query access to f. To date, this complexity-theoretic approach remains the only known learning technique for this circuit class.

In a 2017 follow-up, the authors further show that, from so-called *tolerant* natural properties, one obtains *agnostic* learners [CIKK17]. A tolerant natural property is essentially equivalent to easiness of the Minimum Approximate Circuit Size Problem, MACSP, which asks, for a given truth-table T and parameters s and δ, whether there exists a circuit of size s computing T on at least a $1-\delta$ fraction of inputs. Under this framework, the authors generalize the learning algorithm for $\mathsf{AC}^0[p]$ to target functions with distance $1/\mathsf{poly}(n)$ from $\mathsf{AC}^0[p]$.

Problem 5.1 (Learning $\mathsf{AC}^0[p]$ from random examples)**.** Is there a subexponential time algorithm that learns $\mathsf{AC}^0[p]$ circuits under the uniform distribution using random examples instead of membership queries?

For recent work related to Problem 5.1, we refer to [GJK+24, CJ26].

From Uniform Learning to Samplable Distributions via Distributional Inversion. The approach of [CIKK16] as described above only yields a PAC learner working on the uniform distribution. A subsequent work due to Binnendyk et al. [BCK+22], following the *indirect learning* approach of [FJS91], shows that the result of [CIKK16] can be extended to obtain learners working over any efficiently samplable distribution, provided (1) that the code of the sampler is known to the learner, and (2) that the target function *composed* with this sampler belongs to the learner's concept class. Below, we restrict our attention to the case that the concept class as well as the sampler complexity of the target distribution are unrestricted polynomial-size circuits.

It is well-known that a natural property would preclude the existence of a PRG with polynomial stretch: outputs of the generator have bounded circuit complexity, while uniformly random strings almost always have maximum circuit complexity [RR97]. Then, by a sequence of results from cryptography [GGM86, HILL99, Yao82, IL89], a natural property implies the existence of efficient distributional inverters for every poly-time function. Roughly, when given the output y of some function $g : \{0,1\}^n \to \{0,1\}^{m(n)}$, such an inverter outputs a *nearly uniformly distributed* member of $\{x \mid g(x) = y\}$.

Let A be the PAC learner of [CIKK16] working over the uniform distribution with membership queries. Suppose we would like to learn some target function f over a distribution $D \in \mathsf{PSAMP}$. The definition of PSAMP means that D can be sampled by a polynomial-time machine; i.e. there is a poly-time algorithm S such that $S(r)$ is distributed according to D when the string r is uniformly distributed. Let I be a distributional inverter for S.

The key idea is to simulate A on the target function $f(S(-))$. Clearly, knowing S and given membership-query access to f, we can simulate membership queries to $f(S(-))$. Then by [CIKK16], with high probability, A outputs a hypothesis h such that

$$\Pr_{r \sim \mathcal{U}_{\mathsf{poly}(n)}}[h(r) = f(S(r))] \geq 1 - 1/\mathsf{poly}(n).$$

We output the hypothesis $h'(-) \triangleq h(I(-))$.

Intuitively, on input $x \sim D$, we have $h'(x) = h(I(S(r))) \approx h(r)$, where $r \sim \mathcal{U}_{\mathsf{poly}(n)}$. Since $h(r) = f(S(r)) = f(x)$ with high probability, one can show that the hypothesis h' successfully approximates f over D, as desired.

We refer to [Pic25] for additional settings where one can derive learning consequences from complexity lower bound assumptions.

5.2 Solomonoff Induction: Unsupervised Learning via Kolmogorov Complexity

In this section, we explain how to efficiently achieve sequence prediction under a meta-complexity assumption about the feasibility of estimating time-bounded Kolmogorov complexity. Formally, we have the following result.

Theorem 5.2 ([HN26a, Theorem 2.1])**.** *If* $\mathsf{GapK}^t\text{-vs-K} \in \mathsf{pr\text{-}BPP}$*, then* $(m, n, s, t, \varepsilon, \delta)$*-extrapolation can be solved in randomized time* $\mathsf{poly}(n, m, s, t, 1/\varepsilon, 1/\delta)$ *for* $m \geq \widetilde{O}(n^2 \cdot s \cdot \varepsilon^{-6} \cdot \delta^{-5})$.

First, we review Solomonoff induction, which considers this extrapolation task in the time-unbounded setting.

5.2.1 The Classical Theory

In Solomonoff induction [Sol64a, Sol64b] (see [HQC24] for a modern introduction), given a prefix $x_{<i} = (x_1, \ldots, x_{i-1})$, the extrapolation algorithm predicts the next block $x_i \in \{0,1\}^n$ by *conditioning* a certain "universal" distribution on the event that the generated sequence begins with $x_{<i}$. Equivalently, one gives higher probability weight to continuations that make the full string $(x_{<i}, x_i)$ have small Kolmogorov complexity.[13] Roughly, we predict the next block x_i given the prefix $x_{<i}$ according to the Solomonoff (semi)measure using

$$S(x_i \mid x_{<i}) \propto \sum_y 2^{-\mathsf{K}(x_{<i} x_i y)},$$

where the symbol $\propto$ means "proportional to". This is a form of *Occam's Razor*, i.e., short explanations are preferred. It is then possible to show that, for sequences generated by a randomized program of description length s, the resulting predictor is accurate on most $i \in [m]$ once the number of observations $m \gg s/\varepsilon^2$.

To show that this predictor works as claimed above, we combine the following ingredients:

1. *Universality of the Solomonoff measure.* If the full sequence $x = (x_1, \ldots, x_m)$ is generated by a randomized program M of description length

[13] In this section, we take K to denote *prefix complexity*, which is the same as our stated definition of Kolmogorov complexity, except it requires that the set of inputs on which the chosen universal machine halts is *prefix-free*. By Kraft's inequality, $\sum_y 2^{-\mathsf{K}(wy)} \leq 1$ in the case of prefix complexity, whereas the same sum over all finite strings y diverges in the case of plain complexity. Though the ideas in this section can be reformulated to accommodate plain Kolmogorov complexity, we adopt this convention to match the classical literature.

s, then the measure S dominates the true source μ_M up to a multiplicative factor $2^{-O(s)}$, i.e.,

$$S(x_1, \ldots, x_m) \;\geq\; 2^{-O(s)} \cdot \mu_M(x_1, \ldots, x_m)$$

for every sequence $(x_1, \ldots, x_m)$.

2. *Log-loss bound and average KL-divergence.* Taking logarithms and averaging over $x = (x_1, \ldots, x_m) \sim \mu_M$, one obtains the log-loss bound

$$\mathbb{E}_{x\sim\mu_M}\left[\log \frac{\mu_M(x_1, \ldots, x_m)}{S(x_1, \ldots, x_m)}\right] \;\leq\; O(s).$$

The key point is that this global bound can be decomposed into one-step prediction errors. In more detail, for every $x = (x_1, \ldots, x_m)$ with $\mu_M(x) > 0$, we can factor both distributions sequentially:

$$\mu_M(x) = \prod_{i=1}^{m} \mu_M(x_i \mid x_{<i}), \qquad S(x) = \prod_{i=1}^{m} S(x_i \mid x_{<i}).$$

Consequently,

$$\log \frac{\mu_M(x)}{S(x)} \;=\; \sum_{i=1}^{m} \log \frac{\mu_M(x_i \mid x_{<i})}{S(x_i \mid x_{<i})}.$$

Now taking expectation over $x \sim \mu_M$ and applying linearity of expectation,

$$\begin{aligned}\mathbb{E}_{x\sim\mu_M}\left[\log \frac{\mu_M(x)}{S(x)}\right] &= \sum_{i=1}^{m} \mathbb{E}_{x_{\leq i}\sim\mu_M}\left[\log \frac{\mu_M(x_i \mid x_{<i})}{S(x_i \mid x_{<i})}\right] \\ &= \sum_{i=1}^{m} \mathbb{E}_{x_{<i}\sim\mu_M}\left[\sum_{y\in\{0,1\}^n} \mu_M(y \mid x_{<i}) \log \frac{\mu_M(y \mid x_{<i})}{S(y \mid x_{<i})}\right].\end{aligned}$$

But the inner sum is exactly the KL-divergence from the true conditional distribution of the next block to the Solomonoff posterior. Therefore, combining the equations above,

$$\sum_{i=1}^{m} \mathbb{E}_{x_{<i}\sim\mu_M}\Big[\mathsf{KL}(\mu_M(\cdot \mid x_{<i}) \,\|\, S(\cdot \mid x_{<i}))\Big] \;\leq\; O(s).$$

3. *Passing to statistical distance.* Finally, for any large enough $m = O(s \cdot 1/\varepsilon^2 \cdot 1/\delta)$, using a standard Markov argument and applying Pinsker's inequality, one obtains that the predicted next-block distribution is ε-close in total variation distance to the true conditional distribution produced by M except for at most a δ-fraction of $i \in [m]$.

Thus, in the time-unbounded setting, Solomonoff induction solves the extrapolation problem once the sequence is sufficiently longer than the description of the underlying generator program.

5.2.2 The Time-Bounded Setting

We consider a simpler setting where $\delta > 0$ is a small constant, and describe a randomized algorithm that runs in time $\mathsf{poly}(n, m, t, 1/\varepsilon)$ and succeeds as long as $m = m(n, s, \varepsilon)$ is sufficiently large. The proof of Theorem 5.2, which provides a polynomial-time procedure when $\delta = o(1)$, is significantly more involved. We refer to [HN26a] for the details.

To move to the time-bounded world, we replace the universal distribution by the *time-bounded* universal distribution $Q_{t,z}$, where z is an auxiliary string.

Definition 5.3 (Time-Bounded Universal Distribution with Advice)**.** Fix $t \in \mathbb{N}$ and an advice string $z \in \{0,1\}^*$. The t-time-bounded universal distribution $\mathrm{Q}_{t,z}$ is the distribution induced by the random variable $U^r(w, z, 1^t)$, where $w \sim \mathcal{U}_t$ is chosen uniformly from $\{0,1\}^t$. Equivalently, for every string $y \in \{0,1\}^*$,

$$\mathrm{Q}_{t,z}(y) \triangleq \Pr_{w \sim \mathcal{U}_t,\, U^r}[\, U^r(w, z, 1^t) = y \,].$$

When $z = \epsilon$, we omit the advice string and write Q_t instead of $\mathrm{Q}_{t,z}$. For convenience, we also define

$$q^t(y \mid z) \triangleq -\log Q_{t,z}(y).$$

We also need the following definition.

Definition 5.4 (Universal Extrapolation Distribution)**.** The *universal extrapolation distribution*

$$\mathsf{Next}_n(y; Q_{t,z})$$

is the distribution of the next n bits of a $Q_{t,z}$-sample, conditioned on extending the prefix y.

It is important to distinguish the two roles here: z is an advice string in the Kolmogorov complexity sense, while y is the observed prefix in the probabilistic conditioning sense. This distinction is a crucial idea from [HN26a] to implement Solomonoff's theory in the time-bounded setting.[14] Jumping ahead, what is special in the time-bounded setting is that one splits the given prefix into advice and context strings for efficiency reasons and then proves that this restricted-context extrapolation is still correct.

In more detail, prior work already gave a *universal sampling procedure* for approximately sampling from $\mathsf{Next}_n(y; Q_{t,z})$, but its running time is exponential in

[14] In the classical theory, since computational efficiency is not relevant, there is no need to consider an advice string, and the entire observed prefix is used for conditioning. We stress that in both settings – time bounded and time unbounded – moving the entire prefix to the advice portion does not guarantee extrapolation.

the *computational depth* of y given z [IL90, HN23].[15] In other words, consider the computational depth quantity

$$\mathsf{cd}^t(y \mid z) \;\triangleq\; q^t(y \mid z) - \mathsf{K}(y \mid z),$$

where we set $\mathsf{cd}_t(y) \triangleq \mathsf{cd}_t(y \mid \epsilon)$. If one tries to extrapolate directly from the full prefix by setting $y = x_{<i}$ and $z = \epsilon$, the runtime can be exponential in $\mathsf{cd}_t(x_{<i})$. For some sequences generated by an s-bit randomized program, this depth can be as large as $\Omega(s)$.

As alluded to above, a key idea from [HN26a] is to reorganize the observed prefix into an *advice part* and a *context part*. For appropriate parameters $w, b \in \mathbb{N}$ with $m = b \cdot w$, the sequence $(x_1, \ldots, x_m)$ is partitioned into b larger blocks, each consisting of w of the original n-bit symbols. When predicting position $j \in [w]$ inside block $i \in [b]$, the learner uses all previous blocks as advice, but keeps only the prefix of the current block up to position $j-1$ as context. If we write $\bar{x}_i$ for the i-th large block, $\bar{x}_{<i}$ for the concatenation of the previous blocks, and $\bar{x}_i^{<j}$ for the prefix of the current block before position j, then the predictor outputs according to

$$\mathsf{Next}_n(\bar{x}_i^{<j}; Q_{t,\bar{x}_{<i}}).$$

In other words, the long-range history is moved into *advice*, while a short local prefix is kept as *context*.

Next, we explain in more detail how this approach reduces the running time while maintaining correctness of the corresponding predictor. For a formal presentation, see [HN26a].

I. Bounding the running time. For the runtime analysis, we need two ingredients: the *slow growth law* and the *chain rule for computational depth*.

The slow growth law states that if $x \sim \mu_M = U^{\mathrm{r}}(M, \epsilon, 1^t)$, where M is a t-time randomized machine of description length s, then with high probability

$$\mathsf{cd}^{\mathsf{poly}(t)}(x) \;\le\; O(s).$$

The chain rule states that, for a string $x = (x_1, \ldots, x_{m'})$ with each $|x_i| \le n'$, and any large enough t', we have

$$\sum_{i=1}^{m'} \mathsf{cd}^{\mathsf{poly}(t')}(x_i \mid x_{<i}) \;\le\; \mathsf{cd}^{t'}(x) + m' \cdot O(\log t').$$

These two results are known to hold under the assumption of Theorem 5.2, as established by [HN26a].

[15] This algorithm assumes that auxiliary-input one-way functions do not exist, which follows from the hypothesis of Theorem 5.2.

Putting the results above together, we get that the expected value of $\mathsf{cd}^{\mathsf{poly}(t)}(x_i \mid x_{<i})$ over a random position $i \sim [m]$ is $O(s/m + \log t) = O(\log t)$. By Markov's inequality,

$$\Pr_{i\sim[m]}\left[\mathsf{cd}^{\mathsf{poly}(t)}(x_i \mid x_{<i}) \;\geq\; \delta^{-1}\cdot O(\log t)\right] \;\leq\; \delta.$$

In other words, for most choices of i and x, the time-bounded universal sampling procedure (approximately) samples from $\mathsf{Next}_n(x_i; Q_{t,x_{<i}})$ in time $2^{O(\delta^{-1}\cdot\log t)} = \mathsf{poly}(t)$, for $\delta = \Omega(1)$.[16]

One can perform a similar runtime analysis once we break the sequence $x = (x_1, \ldots, x_m)$ into b larger blocks, each consisting of w of the original n-bit symbols x_i, and use the universal extrapolation procedure to sample instead from $\mathsf{Next}_n(\bar{x}_i^{<j}; Q_{t,\bar{x}_{<i}})$. (In order to guarantee correctness, the proof employs $w = O(\varepsilon^{-2}\cdot\delta^{-1}\cdot\log t)$, as explained below.)

II. Correctness of the time-bounded predictor. Recall that our predictor partitions $x = (x_1, \ldots, x_m)$ into b length-w blocks. Given an index $j \in [w]$ inside block $i \in [b]$, it uses the universal sampling procedure in order to predict according to $\mathsf{Next}_n(\bar{x}_i^{<j}; Q_{t,\bar{x}_{<i}})$. By the analysis explained above, for most blocks the sampler is correct and we (approximately) sample from $\mathsf{Next}_n(\bar{x}_i^{<j}; Q_{t,\bar{x}_{<i}})$. It remains to show that this is close to the true distribution, i.e.,

$$\Pr_{i,\,j,\,(\bar{x}_{<i},\bar{x}_i^{<j})\sim\mu_M}\left[\Delta_{\mathsf{TV}}(\bar{X}_i^j \mid \bar{x}_{<i}\bar{x}_i^{<j}, \mathsf{Next}_n(\bar{x}_i^{<j}; Q_{t,\bar{x}_{<i}})) \;\leq\; \varepsilon\right] \;\geq\; 1-\delta.$$

To prove this, let $\bar{X}_i$ denote the random i-th block under μ_M, and let $\bar{X}_{<i}$ denote the concatenation of the previous blocks. Up to replacing t by $\mathsf{poly}(t)$ (which we suppress in the notation), similarly to the chain rule for computational depth described above, it is possible to show that for every $(\bar{x}_1, \ldots, \bar{x}_b)$,

$$\sum_{i=1}^{b} q^t(\bar{x}_i \mid \bar{x}_{<i}) \;\leq\; q^t(\bar{x}_1, \ldots, \bar{x}_b) + b\cdot O(\log t).$$

On the other hand, the time-bounded universal distribution Q_t dominates every s-bit source μ_M up to a factor $2^{-O(s)}$. Equivalently,

$$q^t(\bar{x}_1, \ldots, \bar{x}_b) \;\leq\; O(s) - \log\mu_M(\bar{x}_1, \ldots, \bar{x}_b)$$

for every sequence in the support of μ_M. Combining the two inequalities and expanding the source probability block-by-block,

$$\sum_{i=1}^{b}\Big(q^t(\bar{x}_i \mid \bar{x}_{<i}) + \log\mu_M(\bar{x}_i \mid \bar{x}_{<i})\Big) \;\leq\; O(s) + b\cdot O(\log t).$$

[16] With low probability over i and x, we can abort the algorithm and output "fail" if its running time reaches a fixed threshold.

Next, we average over $(\bar{x}_1, \ldots, \bar{x}_b) \sim \mu_M$. For each $i \in [b]$, we claim that the expectation of the i-th summand is exactly

$$\mathbb{E}_{\bar{x}_{<i} \sim \mu_M}\Big[\mathsf{KL}((\bar{X}_i \mid \bar{x}_{<i}) \parallel Q_{t,\bar{x}_{<i}})\Big].$$

Indeed, by definition, $q^t(y \mid \bar{x}_{<i}) = -\log Q_{t,\bar{x}_{<i}}(y)$, and so

$$q^t(y \mid \bar{x}_{<i}) + \log \mu_M(y \mid \bar{x}_{<i}) \ = \log \frac{\mu_M(y \mid \bar{x}_{<i})}{Q_{t,\bar{x}_{<i}}(y)}.$$

Taking expectation over $y \sim (\bar{X}_i \mid \bar{x}_{<i})$ yields the desired statement. As a consequence,

$$\mathbb{E}_{i \sim [b]}\, \mathbb{E}_{\bar{x}_{<i} \sim \mu_M}\Big[\mathsf{KL}((\bar{X}_i \mid \bar{x}_{<i}) \parallel Q_{t,\bar{x}_{<i}})\Big] \ \leq \ \frac{O(s)}{b} + O(\log t).$$

In other words, the amortized KL-divergence over the b blocks is $O(\log t)$ for $b = \Omega(s/\log t)$.

The remaining steps follow from standard arguments. First, we apply the chain rule for KL-divergence to distribute the bound of $O(\log t)$ across the w positions in a block, yielding an average KL-divergence of $O(\log t)/w$ over positions $j \in [w]$. For $w = \Omega(\varepsilon^{-2} \cdot \delta^{-1} \cdot \log t)$, we obtain a bound of $O(\log t)/w = \varepsilon^2 \cdot \delta$ per symbol. Then, from Markov's inequality, the per-symbol KL-divergence is at most ε^2 for a $(1-\delta)$-fraction of positions inside a block. Finally, the desired bound on statistical distance then follows from Pinsker's inequality.

To sum up, the predictor satisfies the desired guarantees provided that $m = b \cdot w = \Omega((s/\log t) \cdot (\varepsilon^{-2}\delta^{-1} \log t)) = \Omega(s \cdot \varepsilon^{-2} \cdot \delta^{-1})$.

As mentioned above, in order to obtain an efficient predictor when $\delta = o(1)$, a more involved argument seems required, and the known lower bound on m as a function of ε and δ is sub-optimal.

Problem 5.5 (Optimal Parameters in Extrapolation from Meta-Complexity)**.** Prove that if $\mathsf{GapK}^t\text{-vs-}\mathsf{K} \in \mathsf{pr\text{-}BPP}$, then $(m, n, s, t, \varepsilon, \delta)$-extrapolation can be solved in randomized time $\mathsf{poly}(n, m, s, t, 1/\varepsilon, 1/\delta)$ for $m \geq \widetilde{O}(s \cdot \varepsilon^{-2} \cdot \delta^{-1})$.

5.3 Equivalences Between Learning and Meta-Complexity

In Section 5.1 we described several settings where one can get efficient learning from suitable distinguishers. In turn, the necessary distinguishers follow from the existence of efficient algorithms for meta-computational problems. The latter allow one to distinguish randomness from structure, or equivalently, input strings that can be compressed from (a large fraction of) incompressible ones.

We now discuss two settings where appropriate relaxations of the meta-computational problems are actually *computationally equivalent* to the learning task. Let C be a class of functions (e.g., polynomial-size Boolean circuits), and let Γ be a learning model (e.g., PAC learning under the uniform distribution from random examples). We present equivalences of the following form:

$$C \text{ admits efficient learning algorithms in model } \Gamma \iff \text{The meta-computational problem } L_{C,\Gamma} \text{ is easy}$$

In the equivalence, $L_{C,\Gamma}$ is a suitable computational problem that we associate to the learning model and concept class.

The idea that the design of learning algorithms is equivalent to the search for efficient procedures that detect patterns in data offers a fundamentally new perspective: while *learning is inherently a sublinear task*, where an algorithm must inspect a small portion of a large object (the unknown function) and produce an approximate representation of it (the output hypothesis), a typical problem from meta-complexity, such as MCSP and its variants, involves deciding if an input string is "random" or "structured", a standard algorithmic problem where *the algorithm has access to the entire input*.

Concretely, as explained below, [ILO20] shows that a circuit class C can be PAC learned under the uniform distribution from random examples in polynomial time if and only if a variant of MCSP for partial functions and with respect to C-circuit complexity is easy on average. Similarly, [CIKK16] can be phrased (see [OS17]) as an equivalence between the efficient learnability of C with membership queries under the uniform distribution and the existence of a promise-BPP natural property against C (see Section 2.4.5).

Next, we explain in more detail the aforementioned equivalences. For simplicity, we mostly focus on the case where C corresponds to general Boolean circuits, though the equivalences hold more generally under mild assumptions on C related to their closure properties and basic expressive power.

Learning from Random Examples. Ilango, Loff, and Oliveira [ILO20] rephrase the ideas developed in [Vad17] (see Section 5.1.2), making more explicit a connection between the easiness of meta-complexity and the possibility of learning. More specifically, the authors show that if a certain problem from meta-complexity, denoted Partial-MCSP, is easy on average, then PAC learning general Boolean circuits on the uniform distribution from random examples is easy. Conversely, a learning algorithm that uses random examples implies the easiness on average of Partial-MCSP.

In [ILO20], Partial-MCSP refers to the following formulation of a circuit minimisation problem on partially defined functions.

> An instance of the language Partial-MCSP consists of an integer $n \in \mathbb{N}$ represented in unary, a collection of input/output pairs $((x_i, b_i))_{i\in[m]}$ for some $m \in \mathbb{N}$, where $x_i \in \{0,1\}^n$ and $b_i \in \{0,1\}$ for each $i \in [m]$, and a threshold parameter $s \in \mathbb{N}$.
>
> $(1^n, ((x_i, b_i))_{i\in[m]}, s)$ is a yes-instance of Partial-MCSP iff there exists a Boolean circuit C of size at most s such that $C(x_i) = b_i$ for all $i \in [m]$.

The authors further consider an *average-case* notion of Partial-MCSP, in which an algorithm A is said to solve Partial-MCSP on average over the uniform distribution with m samples, advantage γ, and parameter s if for every boolean function $f \in \mathsf{SIZE}[s]$,

$$\left| \Pr_{\substack{A,\\ x_1,\ldots,x_m\sim\mathcal{U}_n^m}} [A(x_1, ..., x_m; f(x_1), ..., f(x_m)) = 1] - \Pr_{\substack{A,\\ x_1,\ldots,x_m\sim\mathcal{U}_n^m\\ b_1,\ldots,b_m\sim\mathcal{U}_1^m}} [A(x_1, ..., x_m; b_1, ..., b_m) = 1] \right| \geq \gamma(n).$$

For large enough $m(n) = \mathsf{poly}(s(n))$ and small enough $\gamma(n) = \Omega(1/\mathsf{poly}(n))$, we remark that this is a rather lenient notion of average-case easiness over the uniform distribution: it is enough to distinguish labels consistent with a function from the class from random labels.

Now observe that the task above is precisely the problem of breaking the Direct Product Generator G^f_{DP} with the input $\overline{x}$ sampled uniformly at random and the target function f belonging to $\mathsf{SIZE}[s]$. Applying the techniques discussed in the context of [Vad17], such an algorithm may be transformed into a weak learner for $\mathsf{SIZE}[s]$ working on examples drawn from the uniform distribution. To obtain a strong PAC learner for $\mathsf{SIZE}[s]$ over the uniform distribution, the authors apply a result of [BL93] to boost accuracy.

Conversely, it is not difficult to show that an algorithm that learns $\mathsf{SIZE}[s]$ under the uniform distribution from random examples can be used to solve the average-case version of Partial-MCSP introduced above. Given a sequence $(x_1, \ldots, x_m; \ell_1, \ldots, \ell_m)$, where we assume $m(n)$ is large enough, our task is to decide whether the labels $\ell_1, \ldots, \ell_m$ are *consistent* with a fixed but unknown $f \in \mathsf{SIZE}[s]$ or *random*. We simulate the learner over the sample set $\{(x_1, \ell_1), \ldots, (x_{m-1}, \ell_{m-1})\}$. Let h be its output hypothesis. If $h(x_m) = \ell_m$, we output 1 (meaning "consistent"); otherwise we output 0 (meaning "random"). On the one hand, if the labels are consistent with some function $f \in \mathsf{SIZE}[s]$, h is likely to correctly label x_m by ℓ_m, in which case we output 1. On the other hand, in the random case, no matter the output hypothesis h, since the label ℓ_m is an independent random bit we get that $h(x_m) = \ell_m$ with probability exactly $1/2$.

Therefore, we distinguish the two cases with noticeable probability, i.e., we solve Partial-MCSP on average.

Learning from Membership Queries. The preceding discussion used the term natural property informally. The precise randomized formulation relevant here is the notion of a pr-BPP-natural property introduced in Section 2.4.5. Such a property is decided by a randomized algorithm running in time polynomial in the truth-table length $N = 2^n$. It accepts with probability at least $2/3$ every truth table in some dense set, while accepting the truth table of each function in $C[s]$ with probability at most $1/3$. No guarantee is imposed on the remaining truth tables: on these inputs the acceptance probability may lie between $1/3$ and $2/3$, which explains the promise in pr-BPP. Informally, the algorithm efficiently recognizes many unstructured functions while rejecting every function of low C-complexity. This can be seen as a weak average-case procedure for the MCSP problem for C-circuits.

As discussed above, Carmosino, Impagliazzo, Kabanets, and Kolokolova [CIKK16] turn such a property into a learner for C under the uniform distribution: the property distinguishes truth tables produced by the Nisan–Wigderson generator from random truth tables, and the NW reconstruction procedure converts this distinguisher into a hypothesis for the target. The reconstruction uses oracle access to the target, which accounts for the membership queries.

Oliveira and Santhanam [OS17] noted that the reverse implication from learning to natural properties also holds, even if one starts with a barely non-trivial learning algorithm. Concretely, a non-trivial learner is a randomized oracle algorithm A^f that, for some constant $c \geq 1$ and every $f \in C[s]$, outputs with probability at least $1 - 1/n$ a hypothesis h satisfying

$$\Pr_{x \sim \mathcal{U}_n}[h(x) = f(x)] \geq \frac{1}{2} + \gamma(n), \qquad \gamma(n) = \frac{1}{n^c},$$

and runs in time $T(n) = 2^n/n^{\omega(1)}$. The super-polynomial saving in the denominator is a convenient way to make the definition independent of c; a fixed sufficiently large polynomial saving also works.

The idea works as follows. Given $w \in \{0,1\}^N$, simulate A^f for $f = \mathsf{fnc}^w$, answering each membership query directly from w. Estimate the agreement of the resulting h with f to additive error $\gamma/4$ using polynomially many fresh uniform inputs. Reject if the estimate exceeds $1/2 + \gamma/2$, and accept otherwise. If $f \in C[s]$, the learner succeeds and the test rejects with probability at least $2/3$.

To see why the test accepts most functions, fix a size bound $S \leq T(n)$. By a simple counting argument, there are at most $2^{O(S \log S)}$ Boolean circuits of size S. For any fixed circuit h and a uniformly random function f, the values $\mathbf{1}[h(x) =$

$f(x)]$ are independent unbiased bits. A Chernoff bound therefore gives

$$\Pr_f\left[\Pr_x[h(x) = f(x)] \geq \frac{1}{2} + \frac{\gamma}{4}\right] \leq \exp(-\Omega(\gamma^2 2^n)).$$

Thus the fraction of functions that can be approximated by a fixed hypothesis h is at most $\exp(-\Omega(\gamma^2 2^n))$. A union bound over all possible h (circuits of size at most S) shows that the fraction of functions approximated by any such circuit is at most $\exp(O(S \log S) - \Omega(\gamma^2 2^n)) = o(1)$ under the non-trivial time bound. Since every hypothesis output by A has size at most $T(n)$, the test accepts almost every truth table with high probability. It runs in time $T(n) \cdot \mathsf{poly}(n) = \mathsf{poly}(N)$, so it satisfies constructivity, density, and $C[s]$-hardness from Section 2.4.5. Truth tables outside the two guaranteed cases may have intermediate acceptance probability, which is precisely why the promise class pr-BPP is appropriate. Together with the learning-from-natural-properties direction, this gives the desired equivalence, up to standard parameter translations.

It would be interesting to further develop the dictionary between meta-complexity and learning.

Problem 5.6 (Equivalences Between Learning and Meta-Complexity)**.** Can we characterize efficient learnability in other learning frameworks via meta-complexity?

Next, we discuss a surprising application of the equivalence between learning and meta-complexity.

5.4 Learning Speedups: Reducing the Running Time via Meta-Complexity

A PAC learning algorithm for a class C of functions comes with three parameters of interest: the *confidence* that it succeeds, the *error* of its output hypothesis, and its *running time*. While it is not difficult to amplify the confidence of a learning algorithm [KV94a, Chapter 4], reducing its error is much more subtle. Originally developed to address a question in learning theory, *boosting* (see [SF14]) is a key technique in machine learning that allows one to reduce the error of the output hypothesis of a PAC learner. Thus both the confidence and error parameters can be boosted. Could it be the case that the *running time* of a learning algorithm can also be improved in a generic fashion?

Surprisingly, [OS17] relies on complexity-theoretic techniques [CIKK16] to show that, in some cases, one can make learning algorithms run much faster! They termed this phenomenon *learning speedups:* a generic approach to improve the

running time of a learning algorithm. How is this possible? Key to the learning speedup compiler is to explore a fourth dimension of learning algorithms: they tend to solve more difficult tasks than the ones we typically apply them to.

To achieve a learning speedup, we explore the *complexity* of the unknown function $f\colon \{0,1\}^n \to \{0,1\}$ in C that we are challenged to learn, by *reducing this task* to the learnability of a *more complex* function $g\colon \{0,1\}^m \to \{0,1\}$ over a *smaller* number m of input bits. The rationale is simple: on an instance of the learning problem of length n, if we run the original (slow) learning algorithm on a problem of much smaller dimension $m \ll n$, we obtain a faster learning algorithm as a function of n. For this to succeed, we need to guarantee that g, while more complex than f, is still in C (since the original learner is only promised to learn functions in C), and we must discover a way to implement the reduction between learning problems. To achieve the latter, we *implicitly* simulate the original learning algorithm on g (while only having access to f), and show that a good output hypothesis for g can be transformed into a good hypothesis for f.[17]

Here we explain the proof of the following speedup result.

Theorem 5.7 (From Nontrivial Learning to Subexponential Time Learning [OS17])**.** *Let C be a circuit class at least as powerful as bounded-depth circuits with parity gates. Suppose that, for every constant $k \geq 1$, C-circuits of size n^k can be PAC learned under the uniform distribution with membership queries in time $\leq 2^n/n^{\omega(1)}$. Then, for every $k \geq 1$ and $\varepsilon > 0$, C-circuits of size n^k can be PAC learned under the uniform distribution with membership queries in time $O(2^{n^\varepsilon})$.*

Proof Sketch. Fix $k \geq 1$ and $\varepsilon > 0$. The argument is the composition of the two directions described in Section 5.3. First, the assumed non-trivial learner yields a pr-BPP-natural property (Definition 2.31). Indeed, on the truth table of an m-variate function g, we simulate the learner using the truth table to answer its membership queries, and then test whether its output hypothesis has noticeable agreement with g. The learner succeeds when g is computed by a small C-circuit, whereas the counting argument from the preceding section shows that almost every function cannot be noticeably approximated by a circuit of size $2^m/m^{\omega(1)}$. Reversing the output of this test therefore gives, for every constant d, a pr-BPP-natural property against $C[m^d]$. Its running time is polynomial in the truth-table length 2^m, and hence is $2^{O(m)}$.

We now apply the learning-from-natural-properties construction discussed in Section 5.1. Let $f \in C[n^k]$ be the target function, and choose a parameter $m = n^\gamma$, where $\gamma > 0$ will be fixed below. The black-box Nisan–Wigderson generator

[17] At the technical level, this idea is implemented in a more sophisticated fashion, by reducing the learnability of f to the task of learning a collection $\{g_i\}_{i \in S}$ of functions, in a way that the smaller functions g_i collectively encode enough information that allows us to approximate f using approximators to randomly sampled functions g_i that are implicitly constructed from f.

associates with f a family of auxiliary functions $g_z\colon \{0,1\}^m \to \{0,1\}$. Since C contains bounded-depth circuits with parity gates and has the standard closure properties, each g_z is computed by a C-circuit of size m^d, for some constant $d = d(k,\gamma)$. This is the reason the theorem assumes non-trivial learners for $C[n^d]$ for every constant d, rather than only for one fixed size bound.

The natural property against $C[m^d]$ rejects every truth table $\mathsf{tt}(g_z)$ while accepting a dense set of random truth tables. It therefore distinguishes the NW outputs associated with f from a uniformly random function on m bits. The NW reconstruction procedure converts this distinguisher into a hypothesis that approximates f under the uniform distribution. As explained earlier, the reconstruction can form the required truth-table entries of the functions g_z using oracle access to f; these become the membership queries of the resulting learner.

The natural property and the reconstruction run in time polynomial in 2^m, so the total running time is $2^{O(m)} \cdot \mathsf{poly}(n) = 2^{O(n^\gamma)}$. Choose $\gamma < \varepsilon$ small enough to absorb the constant hidden in the exponent. Standard accuracy and confidence amplification then gives a PAC learner running in time $O(2^{n^\varepsilon})$, as desired. □

As explained above, the key conceptual insight behind the learning speedup compiler is that to learn a function $f \in C[n^k]$ in sub-exponential time, we invoke the nontrivial learner on concepts of size m^ℓ, where ℓ is larger than k but the input length m is smaller than n. Crucially, we save time by using that the original learner is able to succeed on relatively harder learning tasks of smaller instance length. The tradeoff is considerably more general. In particular, a nontrivial learner that learns exponential size concepts can be transformed into a polynomial-time learner for polynomial-size concepts [OS17].

Despite its obvious relevance and potential for applications, this intriguing phenomenon is currently restricted to PAC learning algorithms that work under the uniform distribution and can make membership queries. It also requires the class C to be reasonably expressive, since this is needed to implement the techniques from [CIKK16].

Problem 5.8 (Learning Speedups in Different Learning Frameworks)**.** Systematically investigate learning speedups in different learning models. In particular, is there a generic speedup for learning under the uniform distribution from random examples?

6 Learning vs Compression via Shannon's Information Theory

We have seen several connections between learning and compression through the lens of meta-complexity and algorithmic information theory. In this section, we explore connections from the perspective of Shannon's information theory, taking the theory of source coding as our point of departure. As we will see, the relationship between computational learning and meta-complexity can be understood as a complexity-theoretic counterpart of the classical relationship between learning and universal coding. Our goal is to review several tools for establishing such connections and to explain how notions from meta-complexity fit into this framework. We also briefly discuss how this perspective relates to language modeling, one of the central paradigms of modern machine learning.

6.1 Starting from Shannon's Information Theory

Shannon's information theory [Sha48] provided a precise mathematical framework for quantifying "information" and characterized what can and cannot be achieved in data compression and communication. Originally developed to address problems in communication engineering, the theory laid the foundations for the mathematical study of information and has profoundly influenced theoretical computer science.

One of the central results of Shannon's theory, and the one most relevant to our discussion, is the noiseless source coding theorem. In the setting of this theorem, a message x is drawn from a fixed distribution $\mathcal{D}$ over a finite message space $\mathcal{X}$, and $\mathcal{D}$ is known to both the encoder and the decoder. The goal is to encode x as a short binary string from which x can be uniquely recovered. By fixing a canonical binary representation, we may assume without loss of generality that $\mathcal{X} = \{0,1\}^n$ for some n.

Shannon's construction begins by ordering the messages in $\mathsf{Support}(\mathcal{D})$ as $x_1, \ldots, x_m$ so that $p_1 \geq p_2 \geq \cdots \geq p_m$, where $p_i \triangleq \mathcal{D}(x_i)$. For each i, let

$$\ell_i \triangleq \left\lceil \log \frac{1}{p_i} \right\rceil \qquad \text{and} \qquad P_{<i} \triangleq \sum_{j<i} p_j.$$

The encoder Enc maps x_i to the first ℓ_i bits after the binary point in the binary expansion of $P_{<i}$. One can verify that the original message is uniquely determined by its encoding; we omit the argument.[18]

[18] For an introduction to data compression, we refer to [Ble13].

By construction, the expected codeword length satisfies

$$\begin{aligned}\mathop{\mathbf{E}}_{x\sim\mathcal{D}}[|\mathsf{Enc}(x)|] &= \sum_{x\in\mathsf{Support}(\mathcal{D})} \mathcal{D}(x)\left\lceil \log\frac{1}{\mathcal{D}(x)}\right\rceil \\ &< \sum_{x\in\mathsf{Support}(\mathcal{D})} \mathcal{D}(x)\left(\log\frac{1}{\mathcal{D}(x)}+1\right) \\ &= \mathsf{H}(\mathcal{D})+1.\end{aligned}$$

Notably, Shannon also established the converse, showing that entropy gives a lower bound on the expected codeword length of any reasonable coding. In particular, every uniquely decodable[19] binary encoding $C\colon \mathsf{Support}(\mathcal{D}) \to \{0,1\}^*$ satisfies

$$\mathop{\mathbf{E}}_{x\sim\mathcal{D}}[|C(x)|] \geq \mathsf{H}(\mathcal{D}).$$

In this sense, Shannon's information theory *solves* the compression problem, provided that the source distribution is known to both the encoder and the decoder.

For data arising in the real world, however, the source distribution is rarely given explicitly. For instance, we encounter many different kinds of data in everyday life, including text, music, and images. To apply Shannon's construction to all such data using a single encoder, we would first have to specify a distribution over the possible inputs. But can we identify a natural distribution that captures the diverse forms of data generated by human activity? Even in a much narrower setting, such as compressing an email written by a professor, the resulting code may differ substantially depending on whether we model the message as drawn from a distribution over all human-written text or from one restricted to academic writing. How, then, should we specify the underlying source distribution? Should a description of the distribution itself be included as part of the encoding?

6.1.1 Universal Coding and Kolmogorov Complexity as Its Ultimate Goal

The problem of compressing strings without knowing the underlying source distribution is known as *universal coding* and dates back to the early works [Kol65, Fit66, Dav73]. For instance, Lempel–Ziv algorithms are prominent examples [ZL77, ZL78]. For every stationary ergodic source (roughly, a source whose statistical behaviour is stable over time and can be inferred from a single sufficiently long sample sequence), their amortized code length converges to the entropy rate without requiring an explicit description of the source. Such guarantees are source-universal but do not capture every simple individual sequence.

[19] An encoding is uniquely decodable if every finite concatenation of codewords admits a unique decomposition into codewords. Shannon did not state his result using this notion; the corresponding formulation for uniquely decodable codes was later established by McMillan [McM56].

For example, consider the infinite sequence obtained by concatenating the binary representations

$$0, 1, 10, 11, 100, \ldots$$

of the nonnegative integers. The sequence has a short generating program, but LZ78 does not asymptotically compress it.

Kolmogorov complexity and its time-bounded variants, which are central notions in meta-complexity, can be seen as the ultimate goal of universal coding. Recall that the Kolmogorov complexity $\mathsf{K}(x)$ of a string x is the length of the shortest program that outputs x and thus measures the length of its shortest algorithmic description without presupposing any source distribution. In this sense, $\mathsf{K}(x)$ represents the ideal compression length of x in the absence of computational constraints. Its time-bounded variant $\mathsf{K}^t(x)$ additionally requires the program to output x within t steps and thus provides the corresponding ideal when the decoding time is also constrained.

6.1.2 Universal Coding and Learning

We now relate universal coding to distribution learning. Both problems concern an unknown source distribution: universal coding seeks to compress its outputs without knowing the distribution in advance, whereas distribution learning uses samples to approximate it.

Learning ⇒ Universal Coding. In one direction, suppose that the unknown source distribution is $\mathcal{D}$ over $\{0,1\}^n$, and that we have a distribution learner for a class containing $\mathcal{D}$.

Let us first consider the case in which the learning guarantee is measured in KL divergence. Given samples from $\mathcal{D}$, suppose that the learner outputs a distribution $\mathcal{E}$ satisfying $\mathsf{KL}(\mathcal{D} \,\|\, \mathcal{E}) \le \epsilon$. We can then use $\mathcal{E}$ as the coding distribution in place of $\mathcal{D}$. A Shannon code based on $\mathcal{E}$ assigns each string x a codeword of length at most $\log \frac{1}{\mathcal{E}(x)} + 1$. Ignoring this standard one-bit rounding overhead, the excess expected code length over the entropy of $\mathcal{D}$ is exactly the KL divergence:

$$\mathop{\mathbf{E}}_{x \sim \mathcal{D}} \left[\log \frac{1}{\mathcal{E}(x)} \right] - \mathsf{H}(\mathcal{D}) = \mathsf{KL}(\mathcal{D} \,\|\, \mathcal{E}) \le \epsilon.$$

Thus, distribution learning in KL divergence immediately yields a code whose expected length is close to the optimal expected length for $\mathcal{D}$.

Now consider the case in which the learning guarantee is measured in total variation distance, as in Definition 2.7. Suppose that, given samples from $\mathcal{D}$, the learner outputs a distribution $\mathcal{E}$ satisfying $\Delta_{\mathsf{TV}}(\mathcal{D}, \mathcal{E}) \le \epsilon$. Again, we would like to use $\mathcal{E}$ in place of $\mathcal{D}$ when constructing a Shannon code. However, closeness

in total variation distance alone does not imply any bound on $\mathsf{KL}(\mathcal{D}\,\|\,\mathcal{E})$, which is precisely the expected excess ideal code length incurred by using $\mathcal{E}$ instead of $\mathcal{D}$. Indeed, the KL divergence can be infinite if $\mathcal{E}(x) = 0$ for some x satisfying $\mathcal{D}(x) > 0$.

The following proposition provides a substitute for such a KL-divergence bound. Intuitively, it shows that, with high probability over $x \sim \mathcal{D}$, the ideal code length assigned by $\mathcal{E}$ is not much larger than that under the true distribution $\mathcal{D}$. We will use it to derive two consequences: a bound on the expected length of a Shannon code and a comparison with Kolmogorov complexity.

Proposition 6.1. *Let $\mathcal{D}$ and $\mathcal{E}$ be distributions over a finite set satisfying $\Delta_{\mathsf{TV}}(\mathcal{D}, \mathcal{E}) \le \epsilon$. Then, for every $\delta \in (0, 1]$,*

$$\Pr_{x\sim\mathcal{D}}\left[\log\frac{1}{\mathcal{E}(x)} \le \log\frac{1}{\mathcal{D}(x)} + \log\delta^{-1}\right] \ge 1 - \delta - \epsilon.$$

Proof. Let

$$B \triangleq \{x : \mathcal{E}(x) < \delta\mathcal{D}(x)\}.$$

By the definition of B, we have $\mathcal{E}(B) < \delta\mathcal{D}(B) \le \delta$. Since total variation distance bounds the difference in probability assigned to any event,

$$\mathcal{D}(B) \le \mathcal{E}(B) + \Delta_{\mathsf{TV}}(\mathcal{D}, \mathcal{E}) \le \delta + \epsilon.$$

For every $x \in \bar{B} \cap \mathsf{Support}(\mathcal{D})$, we have $\mathcal{E}(x) \ge \delta\mathcal{D}(x)(> 0)$, which is equivalent to

$$\log\frac{1}{\mathcal{E}(x)} \le \log\frac{1}{\mathcal{D}(x)} + \log\delta^{-1}.$$

This completes the proof of the proposition. □

As a first consequence, we show that the expected length of a code based on $\mathcal{E}$ is close to the entropy of $\mathcal{D}$. In constructing the code, we mix $\mathcal{E}$ with the uniform distribution to avoid zero probabilities, which could otherwise make the KL divergence infinite.

Corollary 6.2. *Let $\mathcal{D}$ and $\mathcal{E}$ be distributions over $\{0,1\}^n$ satisfying $\Delta_{\mathsf{TV}}(\mathcal{D}, \mathcal{E}) \le \epsilon$. Define*

$$\bar{\mathcal{E}} \triangleq \frac{1}{2}\mathcal{E} + \frac{1}{2}\mathcal{U}_n.$$

Then,

$$\mathop{\mathbf{E}}_{x\sim\mathcal{D}}\left[\log\frac{1}{\bar{\mathcal{E}}(x)}\right] \le \mathsf{H}(\mathcal{D}) + \epsilon n + O(1),$$

where $O(1)$ is a universal constant.

Proof. For each $x \in \mathsf{Support}(\mathcal{D})$, let $Z(x) \triangleq \log(1/\bar{\mathcal{E}}(x)) - \log(1/\mathcal{D}(x))$ and $Z_+(x) \triangleq \max\{Z(x), 0\}$. Since $\bar{\mathcal{E}}(x) \geq 2^{-n-1}$, we have $Z(x) \leq n+1$. Moreover, since $\bar{\mathcal{E}}(x) \geq \mathcal{E}(x)/2$, for every integer $k \geq 0$,

$$\Pr_{x\sim\mathcal{D}}[Z(x) > k+1] \leq \Pr_{x\sim\mathcal{D}}\left[\log\frac{1}{\mathcal{E}(x)} > \log\frac{1}{\mathcal{D}(x)} + k\right] \leq 2^{-k} + \epsilon,$$

where the last inequality follows from Proposition 6.1 with $\delta = 2^{-k}$. Summing these tail bounds gives

$$\mathop{\mathbf{E}}_{x\sim\mathcal{D}}[Z_+(x)] \leq 1 + \sum_{k=0}^{n}(2^{-k}+\epsilon) \leq \epsilon n + O(1).$$

Since $\mathbf{E}_{x\sim\mathcal{D}}[\log(1/\bar{\mathcal{E}}(x))] = \mathsf{H}(\mathcal{D}) + \mathbf{E}_{x\sim\mathcal{D}}[Z(x)]$, the result follows. □

As a second consequence, we show that, with high probability, the ideal code length assigned by $\mathcal{E}$ is close to the Kolmogorov complexity, i.e., its idealized description length.

Corollary 6.3. *Let $\mathcal{D}$ and $\mathcal{E}$ be distributions over $\{0,1\}^n$ satisfying $\Delta_{\mathsf{TV}}(\mathcal{D},\mathcal{E}) \leq \epsilon$. Then, for every $\delta \in (0,1]$,*

$$\Pr_{x\sim\mathcal{D}}\left[\log\frac{1}{\mathcal{E}(x)} \leq \mathsf{K}(x) + O\Big(\log n + \log\delta^{-1}\Big)\right] \geq 1 - \delta - \epsilon.$$

Proof. For every $s > 0$,

$$\Pr_{x\sim\mathcal{D}}\left[\log\frac{1}{\mathcal{D}(x)} > \mathsf{K}(x) + s\right] = \sum_{\substack{x\in\{0,1\}^n:\\ \mathcal{D}(x)<2^{-\mathsf{K}(x)-s}}} \mathcal{D}(x) \leq 2^{-s}\sum_{x\in\{0,1\}^n} 2^{-\mathsf{K}(x)}.$$

For each $k \geq 0$, by counting the number of programs of length k, there are at most 2^k strings of Kolmogorov complexity k. Moreover, for any string x of length n, $\mathsf{K}(x) \leq n + O(\log n)$. Therefore, the expression above is upper-bounded by

$$2^{-s}\cdot\sum_{k=0}^{n+O(\log n)}\sum_{\substack{x\in\{0,1\}^n:\\ \mathsf{K}(x)=k}} 2^{-k} = 2^{-s}\cdot\sum_{k=0}^{n+O(\log n)} O(1) = 2^{-s}\cdot O(n).$$

Thus, by choosing $s = O(\log n + \log\delta^{-1})$, we obtain $\log(1/\mathcal{D}(x)) \leq \mathsf{K}(x) + O(\log n + \log\delta^{-1})$ except with probability at most $\delta/2$.

By Proposition 6.1 with parameter $\delta/2$, we also have

$$\log\frac{1}{\mathcal{E}(x)} \leq \log\frac{1}{\mathcal{D}(x)} + \log\frac{2}{\delta}$$

except with probability at most $\delta/2+\epsilon$. Combining the two inequalities and taking a union bound proves the claim. □

In particular, if $\mathcal{E}$ admits a short description, then incorporating this description into the code yields, for most $x \sim \mathcal{D}$, an encoding of x whose length is close to optimal.

Universal Coding $\Rightarrow$ Learning. Conversely, any code $C\colon \mathsf{Support}(\mathcal{D}) \to \{0,1\}^*$ for an unknown source distribution $\mathcal{D}$, with code-length function $c(x) \triangleq |C(x)|$, naturally induces a distribution $\mathcal{D}_c$ by normalizing $2^{-c(x)}$:

$$\mathcal{D}_c(x) \triangleq \frac{2^{-c(x)}}{Z_c}, \qquad Z_c \triangleq \sum_{y \in \mathsf{Support}(\mathcal{D})} 2^{-c(y)}. \tag{4}$$

This construction is natural since the ideal code length $c(x) = -\log \mathcal{D}(x)$ yields $\mathcal{D}_c \equiv \mathcal{D}$. It is therefore natural to ask how close $\mathcal{D}_c$ is to the true distribution $\mathcal{D}$ when $c(x)$ only approximates the ideal code length.

The following proposition answers this question. It shows that the KL divergence from $\mathcal{D}$ to $\mathcal{D}_c$ is determined by the excess expected code length over the entropy of $\mathcal{D}$, together with the normalization term $\log Z_c$.

Proposition 6.4. *Let $\mathcal{D}$ be a distribution over $\mathcal{X}$, and let $C\colon \mathsf{Support}(\mathcal{D}) \to \{0,1\}^*$ be a code with code-length function $c(x) \triangleq |C(x)|$. Then*

$$\mathsf{KL}(\mathcal{D} \,\|\, \mathcal{D}_c) = \Big(\mathop{\mathbf{E}}_{x \sim \mathcal{D}}[c(x)] - \mathsf{H}(\mathcal{D}) \Big) + \log Z_c.$$

Proof. By the definition of $\mathcal{D}_c$, for every $x \in \mathsf{Support}(\mathcal{D})$,

$$\log \frac{1}{\mathcal{D}_c(x)} = \log \frac{Z_c}{2^{-c(x)}} = c(x) + \log Z_c.$$

Therefore,

$$\begin{aligned} \mathsf{KL}(\mathcal{D} \,\|\, \mathcal{D}_c) &= \mathop{\mathbf{E}}_{x \sim \mathcal{D}} \left[\log \frac{\mathcal{D}(x)}{\mathcal{D}_c(x)} \right] \\ &= \mathop{\mathbf{E}}_{x \sim \mathcal{D}} \left[\log \frac{1}{\mathcal{D}_c(x)} \right] - \mathsf{H}(\mathcal{D}) \\ &= \mathop{\mathbf{E}}_{x \sim \mathcal{D}}[c(x)] - \mathsf{H}(\mathcal{D}) + \log Z_c. \end{aligned}$$

□

Consequently, if $\mathbf{E}_{x \sim \mathcal{D}}[|C(x)|] \le \mathsf{H}(\mathcal{D}) + \epsilon$, then

$$\mathsf{KL}(\mathcal{D} \,\|\, \mathcal{D}_c) \le \epsilon + \log Z_c.$$

Moreover, if C is a prefix-free code, meaning that no codeword is a prefix of another, then Kraft's inequality gives

$$Z_c = \sum_{y \in \mathsf{Support}(\mathcal{D})} 2^{-|C(y)|} \le 1.$$

Thus, $\log Z_c \le 0$, and hence

$$\mathsf{KL}(\mathcal{D} \,\|\, \mathcal{D}_c) \le \epsilon.$$

Applying Pinsker's inequality, we further obtain the total variation bound

$$\Delta_{\mathsf{TV}}(\mathcal{D}, \mathcal{D}_c) \le O(\sqrt{\epsilon}).$$

Although the preceding proposition gives a direct transformation from universal coding to distribution learning, its hypothesis may appear somewhat stringent. The entropy $\mathsf{H}(\mathcal{D})$ is the minimum expected code length attainable when the source distribution $\mathcal{D}$ is known, whereas a universal code must be constructed without knowledge of $\mathcal{D}$. It is therefore natural to ask what can be concluded when $c(\cdot)$ instead approximates the Kolmogorov complexity $\mathsf{K}(\cdot)$, viewed as an idealized universal code length.

The following proposition answers this question. It gives a similar bound at the cost of an additive term $|\mathcal{D}|$, where $|\mathcal{D}|$ denotes the length of the shortest description of a probabilistic program that samples exactly from $\mathcal{D}$ (with no restriction on its running time).

Proposition 6.5. *Let $\mathcal{D}$ be a distribution over a finite set $\mathcal{X}$, and let $|\mathcal{D}|$ denote the length of the shortest description of a probabilistic program that samples exactly from $\mathcal{D}$. Let $C\colon \mathsf{Support}(\mathcal{D}) \to \{0,1\}^*$ be a code, and let $c(x) \triangleq |C(x)|$. Then*

$$\mathsf{KL}(\mathcal{D} \,\|\, \mathcal{D}_c) \le \mathop{\mathbf{E}}_{x\sim\mathcal{D}}[c(x) - \mathsf{K}(x)] + |\mathcal{D}| + \log Z_c + O(1).$$

Proof. By the coding theorem,

$$\mathsf{K}(x) \le \left\lceil \log \frac{1}{\mathcal{D}(x)} \right\rceil + |\mathcal{D}| + O(1)$$

for every $x \in \mathsf{Support}(\mathcal{D})$. Therefore,

$$\mathop{\mathbf{E}}_{x\sim\mathcal{D}}[\mathsf{K}(x)] \le \mathsf{H}(\mathcal{D}) + |\mathcal{D}| + O(1).$$

From Proposition 6.4, we obtain

$$\begin{aligned}
\mathsf{KL}(\mathcal{D} \,\|\, \mathcal{D}_c) &= \mathop{\mathbf{E}}_{x\sim\mathcal{D}}[c(x)] - \mathsf{H}(\mathcal{D}) + \log Z_c \\
&= \mathop{\mathbf{E}}_{x\sim\mathcal{D}}[c(x) - \mathsf{K}(x)] + \mathop{\mathbf{E}}_{x\sim\mathcal{D}}[\mathsf{K}(x)] - \mathsf{H}(\mathcal{D}) + \log Z_c \\
&\le \mathop{\mathbf{E}}_{x\sim\mathcal{D}}[c(x) - \mathsf{K}(x)] + |\mathcal{D}| + \log Z_c + O(1).
\end{aligned}$$

□

Again, if C is prefix-free and $\mathbf{E}_{x\sim\mathcal{D}}[c(x) - \mathsf{K}(x)] \le \epsilon$, then

$$\mathsf{KL}(\mathcal{D} \,\|\, \mathcal{D}_c) \le |\mathcal{D}| + \epsilon + O(1).$$

However, since $|\mathcal{D}| \ge 1$ this bound does not yield a small total variation distance via Pinsker's inequality.

One might hope to make the contribution of $|\mathcal{D}|$ negligible so that Pinsker's inequality yields a small total variation distance. This can be achieved by compressing a batch of independent samples and thereby amortizing $|\mathcal{D}|$ over the batch. Suppose that a prefix-free code $C_m \colon \mathcal{X}^m \to \{0,1\}^*$ satisfies

$$\mathop{\mathbf{E}}_{\bar{x}\sim\mathcal{D}^m} [|C_m(\bar{x})| - \mathsf{K}(\bar{x})] \le \epsilon_m,$$

and let $\mathcal{D}_{c_m}$ be the distribution induced by the length function $c_m(\bar{x}) \triangleq |C_m(\bar{x})|$.

An exact sampler for $\mathcal{D}^m$ simply runs an exact sampler for $\mathcal{D}$ independently m times. It therefore has description length

$$|\mathcal{D}^m| \le |\mathcal{D}| + O(\log m).$$

Consequently, Proposition 6.5 gives

$$\mathsf{KL}(\mathcal{D}^m \,\|\, \mathcal{D}_{c_m}) \le |\mathcal{D}| + \epsilon_m + O(\log m).$$

To turn this approximation of $\mathcal{D}^m$ into an approximation of $\mathcal{D}$, choose I uniformly at random from $[m]$, draw $X_{<I} \sim \mathcal{D}^{I-1}$, and let $\widehat{\mathcal{D}}$ be the conditional distribution of the I-th coordinate under $\mathcal{D}_{C_m}$, conditioned on $I-1$ independent samples from $\mathcal{D}$.

By the chain rule for KL divergence (Lemma 2.4),

$$\mathop{\mathbf{E}}_{I,X_{<I}} \left[\mathsf{KL}(\mathcal{D} \,\|\, \widehat{\mathcal{D}})\right] \le \frac{|\mathcal{D}| + \epsilon_m + O(\log m)}{m}.$$

Here, the expectation is taken over both the uniformly random index I and the prefix $X_{<I} \sim \mathcal{D}^{I-1}$. Thus, provided that $\epsilon_m = o(m)$, the expected KL divergence tends to zero as $m \to \infty$. Pinsker's inequality and Jensen's inequality then imply

$$\begin{aligned}\mathop{\mathbf{E}}_{I,X_{<I}} \left[\Delta_{\mathsf{TV}}(\mathcal{D}, \widehat{\mathcal{D}})\right] &\le \sqrt{\frac{1}{2} \mathop{\mathbf{E}}_{I,X_{<I}} \left[\mathsf{KL}(\mathcal{D} \,\|\, \widehat{\mathcal{D}})\right]} \\ &\le \sqrt{\frac{1}{2} \cdot \frac{|\mathcal{D}| + \epsilon_m + O(\log m)}{m}} \xrightarrow[m\to\infty]{} 0.\end{aligned}$$

Conclusion. Taken together, these elementary arguments give information-theoretic transformations in both directions. A distribution learner yields a code whose expected length is close to the source entropy, while a code approximating Kolmogorov complexity can be converted into a learner for distributions having short sampler descriptions. These transformations do not, however, address computational efficiency. Kolmogorov complexity is uncomputable, and evaluating or sampling from the conditional distribution $\widehat{\mathcal{D}}$ need not be efficient. This leads to the corresponding complexity-theoretic question: for efficiently samplable distributions, how is efficient learning related to efficient compression near time-bounded Kolmogorov complexity? Efficient learning belongs to computational learning theory, whereas efficiently approximating time-bounded Kolmogorov complexity is a central problem in meta-complexity.

6.2 Computational Learning and Meta-Complexity

As we saw in Section 6.1, distribution learning is closely related to compression, particularly universal coding, at the information-theoretic level. The relationship between efficient distribution learning and meta-complexity can be viewed as a complexity-theoretic counterpart of this connection. The natural question is how much of this connection survives when the transformations between learning and compression must be efficient and code lengths are measured against *polynomial-time-bounded* Kolmogorov complexity.

More specifically, we consider both directions of this question. Can an efficient learner be converted into an efficient encoder that produces short time-bounded descriptions of samples from the target distribution? Conversely, can the ability to find short time-bounded descriptions be converted into an efficient learner? The first direction requires turning an efficiently samplable distribution into a code, whereas the reverse direction requires turning description lengths into a probability distribution that can be efficiently sampled from and, in some settings, conditioned. The purpose of this section is to survey how these two transformations are connected to meta-complexity.

For simplicity, we informally use $\mathsf{K}^{\mathrm{poly}}$ to denote K^t for an unspecified polynomial time bound t, and use $\mathsf{rK}^{\mathrm{poly}}$ and $\mathrm{q}^{\mathrm{poly}}$ analogously. We leave the particular polynomials implicit and do not track polynomial overheads in the time bounds.

6.2.1 Learning to Compression via Arithmetic Coding

In the learning-to-compression transformation from Section 6.1.2, the learner produces a hypothesis distribution $\mathcal{E}$. Applying Shannon coding to the smoothed distribution $\bar{\mathcal{E}}$ from Corollary 6.2 yields an expected description length close to the entropy of the source. This argument, however, is information-theoretic. The

Shannon code described in Section 6.1 first orders all messages and then encodes each message using the cumulative probabilities of the preceding messages. For the message space $\{0,1\}^n$, carrying out this construction directly may require examining exponentially many strings. Moreover, a distribution learner may output only a sampler for $\mathcal{E}$, rather than an algorithm that evaluates $\mathcal{E}(x)$. Given a sampling circuit for $\mathcal{E}$, computing $\mathcal{E}(x)$ amounts to counting the random seeds on which the circuit outputs x, and a brute-force computation may take exponential time. Thus, the information-theoretic argument alone does not yield a polynomial-time encoder and decoder. Indeed, this computational gap is inherent for general efficiently samplable sources: under the plausible assumption that $\mathsf{SZK} \neq \mathsf{BPP}$, some such sources cannot be compressed close to their entropy by any polynomial-time encoder [TVZ05].

One way to avoid this global enumeration is to require the learner to output a *next-bit predictor*, rather than merely a sampler, and then apply *arithmetic coding*. For a distribution $\mathcal{D}$ over $\{0,1\}^n$, write

$$\mathcal{D}(b \mid x_{<i}) \triangleq \Pr_{X\sim\mathcal{D}}[X_i = b \mid X_{<i} = x_{<i}].$$

A next-bit predictor approximates these conditional probabilities to inverse-polynomial additive accuracy. This requirement is a quantitative strengthening of Yao's next-bit prediction criterion, which asks only for a non-negligible advantage over random guessing in predicting the next bit at a uniformly random position from the preceding prefix (see Section 5.1.1). Given a next-bit predictor satisfying this stronger quantitative guarantee, arithmetic coding accesses the learned distribution one bit at a time and constructs the desired encoder and decoder without enumerating its support.

We briefly review how arithmetic coding works. At a high level, it reconstructs the cumulative-probability interval associated with a message using only next-bit conditional probabilities. The procedure can be viewed as repeatedly zooming in on the probability interval $[0,1)$. Initially, the encoder sets $s = 0$ and $t = 1$, so that the current interval is $[s,t) = [0,1)$. After reading a prefix $x_{<i}$, it computes the splitting point

$$u \triangleq s + (t-s)\mathcal{D}(0 \mid x_{<i}).$$

The encoder then reads the next bit x_i. If $x_i = 0$, it updates $t \leftarrow u$ and retains the interval $[s,u)$. If $x_i = 1$, it updates $s \leftarrow u$ and retains the interval $[u,t)$. In either case, the length of the current interval is multiplied by $\mathcal{D}(x_i \mid x_{<i})$. Consequently, after reading the last bit, the retained interval has length

$$\prod_{i=1}^{n} \mathcal{D}(x_i \mid x_{<i}) = \mathcal{D}(x).$$

The encoder then chooses a binary prefix whose corresponding dyadic interval is contained in the final interval. Such a prefix can be chosen to have length

$$\log \frac{1}{\mathcal{D}(x)} + O(1).$$

The decoder reverses this procedure. Given the codeword and access to the same next-bit predictor, it interprets the codeword as specifying a dyadic interval. At each step, it partitions the current interval according to the predicted conditional probability and determines which of the two resulting subintervals contains the dyadic interval, thereby recovering the next bit.

Thus, encoding and decoding require only the n conditional probabilities along the path determined by x, rather than an enumeration of the entire support. They can therefore be performed in polynomial time whenever the next-bit predictor is efficient. Moreover, decoding remains correct as long as the encoder and decoder obtain the same conditional probabilities on every query; the accuracy of these probabilities affects only the resulting code length.

If the next-bit predictor is randomized, its random coins must be fixed in advance and shared by the encoder and decoder so that they obtain identical answers to the same queries. Thus, if the learner outputs the hypothesis distribution $\mathcal{E}$ in the form of a randomized next-bit predictor, then, given shared access to this predictor, the construction yields a $\mathsf{pK}^{\mathsf{poly}}$-type compressor (namely, the resulting program may depend on the shared random string) whose description length is close to $\log(1/\mathcal{E}(x))$, up to the loss incurred by approximating the conditional probabilities.

One may ask how much stronger it is to require a next-bit predictor rather than merely a sampler. In general, there is a gap if AIOWFs exist: if G_z is an AIPRG constructed from an AIOWF and $\mathcal{U}$ is uniform, then $(G_z(\mathcal{U}), \mathcal{U})$ is efficiently samplable but admits no efficient next-bit predictor. Conversely, if AIOWFs do not exist (as is implied by efficient distribution learning for $\mathsf{P/poly}$-samplable distributions; see Lemma 4.8), the distributional-inversion argument of Impagliazzo and Luby [IL89] can be used to construct an efficient next-bit predictor from an explicit description of any efficient sampler. Essentially the same argument appears in our discussion of white-box learning in Section 4.5.2.

Application: A Coding Theorem for $\mathsf{rK}^{\mathsf{poly}}$. The arithmetic-coding construction above has an application in meta-complexity: a coding theorem for randomized time-bounded Kolmogorov complexity. Recall that $\mathsf{rK}^{\mathsf{poly}}(x)$ is the minimum length of a randomized program that outputs x in polynomial time with probability at least $2/3$. Although this is a natural way to incorporate randomness into time-bounded Kolmogorov complexity, no optimal coding theorem is known for

$\mathsf{rK}^{\mathsf{poly}}$ over all efficiently samplable distributions, in contrast to $\mathsf{pK}^{\mathsf{poly}}$. Hirahara, Lu, and Nanashima [HLN24] showed that every next-bits-predictable distribution (i.e., every distribution with a randomized next-bit predictor) admits an efficient coding theorem for $\mathsf{rK}^{\mathsf{poly}}$, with arithmetic coding as a central ingredient.

The main obstacle to obtaining $\mathsf{rK}^{\mathsf{poly}}$-type compression is ensuring that the encoder and decoder reconstruct exactly the same arithmetic-coding intervals. In the rK setting, however, they cannot share random coins, because the encoder must fix the description before the decoder's random coins are drawn. Independent executions of the randomized next-bit predictor may produce slightly different estimates of the same conditional probability, and even a small discrepancy can lead the encoder and decoder to use different interval boundaries, causing decoding to fail. To prevent this, the construction uses the "add noise and round" technique of Goldberg and Sipser [GS91] and Trevisan, Vadhan, and Zuckerman [TVZ05], which adds a small random shift to each estimate and rounds the result to a sufficiently coarse grid. With high probability over the shift, independent accurate executions of the predictor produce the same rounded estimates. Thus, only a logarithmic-length description of the shift, rather than the entire randomness of the predictor, needs to be included in the code. Consequently, if $\mathcal{D} = \{\mathcal{D}_n\}$ is next-bits predictable, then, for every constant $\varepsilon > 0$, there is a polynomial time bound for which

$$\mathsf{rK}^{\mathsf{poly}}(x) \le (1+\varepsilon)\log\frac{1}{\mathcal{D}_n(x)} + O(\log n)$$

for every $x \in \mathsf{Support}(\mathcal{D}_n)$.

Here, the factor $1+\varepsilon$ arises from next bits having very small conditional probabilities. For such a bit, the rounding error may be too large relative to its true probability, so the construction records the bit and its position explicitly rather than encoding it through the estimated probability. The cost of these explicit records accounts for the multiplicative loss in the worst-case bound. For a sample $x \sim \mathcal{D}_n$, however, such low-probability next bits occur only with correspondingly small probability. Their contribution can therefore be absorbed into the additive overhead on average, removing the multiplicative $1+\varepsilon$ loss. Consequently, for every polynomial q, there exists a polynomial p such that

$$\Pr_{x\sim\mathcal{D}_n}\left[\mathsf{rK}^{p(n)}(x) \le \log\frac{1}{\mathcal{D}_n(x)} + \log p(n)\right] \ge 1 - \frac{1}{q(n)}.$$

The construction also gives efficient encoding and decoding algorithms whose expected encoding length is at most $\mathsf{H}(\mathcal{D}_n) + O(\log n)$.

Combining this average-case theorem with the distributional-inversion argument mentioned above yields an average-case coding theorem for $\mathsf{rK}^{\mathsf{poly}}$ over arbi-

trary efficiently samplable distributions under the nonexistence of one-way functions. Such coding theorems relate $\mathsf{rK}^{\mathsf{poly}}(x)$ to both the statistical information content $\log(1/\mathcal{D}_n(x))$ and the unbounded Kolmogorov complexity $\mathsf{K}(x)$. This connection allows characterizations of one-way functions previously formulated using $\mathsf{pK}^{\mathsf{poly}}$ to be extended to $\mathsf{rK}^{\mathsf{poly}}$.

6.2.2 Compression to Learning over Polynomially Many Messages

We now turn to the reverse direction: transforming an *efficient* compressor, ideally one whose code lengths approximate time-bounded Kolmogorov complexity, into an *efficient* learner. Recall that in Section 6.1.2, any code-length function c was converted into a distribution $\mathcal{D}_c$ whose probability mass at each string x is proportional to $2^{-c(x)}$.

Observe that the KL-divergence bound in Proposition 6.5 uses no particular property of $\mathsf{K}(\cdot)$ beyond its coding theorem: for a source distribution $\mathcal{D}$,

$$\mathsf{K}(x) \lesssim \log \frac{1}{\mathcal{D}(x)}.$$

Consequently, the same argument applies when $c(\cdot)$ approximates a complexity measure other than time-unbounded Kolmogorov complexity, provided that the measure satisfies an analogous coding theorem for the source distributions under consideration. In particular, when $\mathcal{D}$ is efficiently samplable and $c(x) \approx \mathsf{pK}^{\mathsf{poly}}(x)$, the coding theorem for $\mathsf{pK}^{\mathsf{poly}}$ yields the same KL-divergence bound for the induced distribution $\mathcal{D}_c$. Thus, if we disregard the efficiency of constructing and sampling from $\mathcal{D}_c$, a compressor approximating $\mathsf{pK}^{\mathsf{poly}}$ already gives an information-theoretic transformation into a distribution learner for efficiently samplable distributions. The remaining question is whether this transformation itself can be implemented efficiently.

From a complexity-theoretic perspective, the immediate obstacle is efficient sampling. Even if each value $c(u)$ can be evaluated efficiently, computing the normalization constant and sampling from $\mathcal{D}_c$ may require enumerating exponentially many possible values of u. Indeed, overcoming this sampling problem is one of the central obstacles to deriving efficient learners from meta-complexity assumptions.

One simple way to set aside this obstacle is to consider the case in which each outcome has logarithmic length, so that the sample space has polynomial size. In this case, both the normalization constant and a sampler for $\mathcal{D}_c$ can be computed by direct enumeration. We therefore begin with this setting. Taken by itself, however, this setting is not particularly interesting from the perspective of distribution learning, since an arbitrary distribution over a polynomial-size domain

can already be learned efficiently by estimating the probability of every possible outcome from samples.

Nevertheless, generating an output from a polynomial-size alphabet becomes a nontrivial and well-motivated learning problem in the conditional setting. Suppose that the learner receives i.i.d. samples from a joint distribution $\mathcal{D}$ over pairs (X, Y), where X is an observed input or context and Y is the output to be predicted. We allow X to have length n, whereas Y has length $O(\log n)$, so its range $\mathcal{Y}$ has polynomial size. This setting is well motivated. Rather than generating an entire pair consisting of an image and its label, we may wish to estimate the probability of each possible label for a given image. Similarly, we may wish to generate an answer conditioned on a given question, rather than generating a pair consisting of a question and an answer. In addition, an output of arbitrary length can be divided into logarithmic-length blocks and generated sequentially. We return to this idea in Section 6.2.3.

Accordingly, in the remainder of this subsection, we focus on the following conditional learning task. Given i.i.d. samples from $\mathcal{D}$ and a new input x, the learner aims to construct a hypothesis distribution approximating $\mathcal{D}(\cdot \mid x)$, where

$$\mathcal{D}(y \mid x) \triangleq \Pr_{(X,Y)\sim\mathcal{D}}[Y = y \mid X = x].$$

Since the range of X may be exponentially large, this task does not reduce to estimating the probability of every pair (x, y). By contrast, since $\mathcal{Y}$ has polynomial size, for any fixed x, once a weight can be evaluated efficiently for every $y \in \mathcal{Y}$, the learner can normalize these weights and sample from them by direct enumeration. The remaining question is which description length should be used to define these conditional weights. Below, we discuss two natural candidates.

The first is the conditional description length $c(y \mid x)$, where x is treated as auxiliary input available to both the encoder and the decoder. This gives the hypothesis distribution

$$\widehat{\mathcal{D}}_c^{\mathrm{cond}}(y \mid x) \triangleq \frac{2^{-c(y|x)}}{\sum_{z\in\mathcal{Y}} 2^{-c(z|x)}}.$$

This construction requires access to a good conditional compressor, which can be substantially more demanding than unconditional compression. Even in the idealized setting, it is currently unclear whether efficient computation of $\mathsf{K}^{\mathsf{poly}}(\cdot)$ implies efficient computation of $\mathsf{K}^{\mathsf{poly}}(\cdot \mid \cdot)$, whereas the converse is immediate by taking the auxiliary input to be the empty string. Thus, even given an efficient unconditional compressor that closely approximates $\mathsf{K}^{\mathsf{poly}}(\cdot)$, this construction itself does not yield an efficiently computable conditional hypothesis distribution.

The second candidate requires only an unconditional description-length mea-

sure. It uses the increase in description length caused by appending y to x:

$$\widehat{\mathcal{D}}_c^{\mathrm{inc}}(y \mid x) \triangleq \frac{2^{-(c(xy)-c(x))}}{\sum_{z\in\mathcal{Y}} 2^{-(c(xz)-c(x))}} = \frac{2^{-c(xy)}}{\sum_{z\in\mathcal{Y}} 2^{-c(xz)}}.$$

This construction has a direct interpretation in terms of the distribution $\mathcal{D}_c$ defined in Equation (4):

$$\widehat{\mathcal{D}}_c^{\mathrm{inc}}(y \mid x) = \mathcal{D}_c(y \mid x).$$

Thus, whenever $\mathcal{D}_c$ is close to $\mathcal{D}$ in KL divergence, the chain rule for KL divergence (Lemma 2.4) implies that $\widehat{\mathcal{D}}_c^{\mathrm{inc}}(\cdot \mid x)$ is close to $\mathcal{D}(\cdot \mid x)$ in KL divergence on average over $x \sim \mathcal{D}_X$:

$$\begin{aligned}\mathop{\mathbf{E}}_{x\sim\mathcal{D}_X}\left[\mathsf{KL}\left(\mathcal{D}(\cdot \mid x) \,\middle\|\, \widehat{\mathcal{D}}_c^{\mathrm{inc}}(\cdot \mid x)\right)\right] &= \mathop{\mathbf{E}}_{x\sim\mathcal{D}_X}\left[\mathsf{KL}\left(\mathcal{D}(\cdot \mid x) \,\|\, \mathcal{D}_c(\cdot \mid x)\right)\right] \\ &= \mathsf{KL}(\mathcal{D} \,\|\, \mathcal{D}_c) - \mathsf{KL}\left(\mathcal{D}_X \,\|\, (\mathcal{D}_c)_X\right) \\ &\le \mathsf{KL}(\mathcal{D} \,\|\, \mathcal{D}_c),\end{aligned}$$

where $\mathcal{D}_X$ and $(\mathcal{D}_c)_X$ denote the respective marginals of $\mathcal{D}$ and $\mathcal{D}_c$ on $\mathcal{X}$. Since $\mathcal{Y}$ has polynomial size, if $c(\cdot)$ is efficiently computable and polynomially bounded, then $\widehat{\mathcal{D}}_c^{\mathrm{inc}}(\cdot \mid x)$ can be efficiently computed and sampled from by enumerating all $y \in \mathcal{Y}$.

The preceding argument is sufficient when the induced joint distribution $\mathcal{D}_c$ is close to $\mathcal{D}$. However, requiring a good approximation to the entire joint distribution can be stronger than necessary for conditional learning, which aims only to approximate $\mathcal{D}(\cdot \mid x)$ and need not model the marginal distribution of X. It is therefore natural to ask whether the desired conditional guarantee can be obtained directly, without first approximating the entire joint distribution. Deriving such a conditional learner from meta-complexity assumptions without implicitly learning the marginal distribution of the conditioning variable remains a challenging problem. In meta-complexity, this question is closely connected to two notions, *conditional coding* and *symmetry of information*, which we discuss at a conceptual level below.

Conditional coding is the conditional counterpart of the coding theorem discussed above. Informally, it asks whether a sample y drawn from $\mathcal{D}(\cdot \mid x)$ admits a $\mathsf{K}^{\mathrm{poly}}$-description of length close to the conditional information content $\log(1/\mathcal{D}(y \mid x))$, with x available as an auxiliary input. More generally, the same question can be formulated for any conditional description-length measure. For the learning application, an average-case guarantee is sufficient, since the learner is evaluated on a fresh pair $(x, y) \sim \mathcal{D}$. If such a guarantee is available for an efficiently computable conditional description-length measure, the same argument as Proposition 6.5 can be applied conditionally to analyze $\widehat{\mathcal{D}}_c^{\mathrm{cond}}$. For

time-unbounded Kolmogorov complexity K, conditional coding follows immediately from the ordinary coding theorem, since the conditional distribution of a samplable joint distribution is itself samplable when no running-time bound is imposed. By contrast, the validity of its time-bounded analogues is closely connected to complexity-theoretic and cryptographic assumptions. Consequently, conditional coding and the complexity of time-bounded conditional descriptions have received considerable attention in meta-complexity [HIL+23, LS24, HLN24, HLO26b, HMO26].

Symmetry of information connects the second construction, which uses only unconditional description lengths, to the first, which describes y with x available as auxiliary input. Roughly speaking, it asks whether $\mathsf{K}^{\mathsf{poly}}(xy)$ can be decomposed as $\mathsf{K}^{\mathsf{poly}}(x) + \mathsf{K}^{\mathsf{poly}}(y \mid x)$ up to a small additive loss. The analogous question can also be posed for other description-length measures. If such a decomposition holds, the incremental description length $\mathsf{K}^{\mathsf{poly}}(xy) - \mathsf{K}^{\mathsf{poly}}(x)$ behaves like the conditional description length $\mathsf{K}^{\mathsf{poly}}(y \mid x)$. For time-unbounded Kolmogorov complexity K, symmetry of information holds unconditionally, up to the logarithmic additive loss [ZL70]. By contrast, whether its time-bounded analogues hold is closely connected to central questions in complexity theory and cryptography, including the P versus NP problem and the existence of one-way functions [e.g., Lev03, LM93, LW95]. Accordingly, time-bounded forms of symmetry of information, their connections to one-way functions and average-case complexity, and unconditional failures in certain formulations have been studied extensively in the meta-complexity literature [LR05, Hir21, LP22, GKLO22, GK22, Hir22c, HIL+23, HLO24, HLN24, HLO26b, GHL+25, HMO26].

We have seen why conditional coding and symmetry of information are natural tools for analyzing the two constructions above. To the best of our knowledge, however, existing results do not directly convert an unconditional compression guarantee over the target distribution into a conditional KL guarantee for the distribution obtained by normalizing the incremental weights $2^{-(c(xy)-c(x))}$, except in settings where the assumptions under consideration already imply that the context x is also compressed nearly optimally on average under the corresponding marginal distribution. Namely, it remains open whether conditional coding and symmetry of information can instead yield direct conditional-learning guarantees beyond those obtained by approximating the entire joint distribution. In particular, it would be of great interest to establish such a transformation for a restricted class of distributions without relying on implausible assumptions that rule out standard cryptography.

Another closely related result is the KL-sampling characterization of conditional pseudoentropy due to Vadhan and Zheng [VZ12]. For polynomial-size output alphabets, their result characterizes conditional pseudo-Shannon entropy in terms of the minimum average conditional KL divergence achievable by an

efficient sampler. This provides another complexity-theoretic link between conditional information and efficient conditional sampling, although it is formulated in terms of pseudoentropy rather than description-length scores. Recent work has revisited and generalized this characterization using multicalibration and weight-restricted calibration [CDV24, HV25].

6.2.3 Compression to Learning over General Messages

For polynomial-length messages, the output space may be exponentially large, making it infeasible to evaluate the score of every possible y and normalize the resulting weights as in Section 6.2.2. We discuss two approaches to this setting. It is worth keeping in mind, however, that the connection between distribution learning and meta-complexity in this time-bounded regime remains largely open.

Universal Extrapolation. The first approach is to use universal extrapolation. Recall that $\mathrm{q}^t(x) \triangleq -\log \mathrm{Q}^t(x)$, where Q^t is the time-bounded universal distribution. Thus, for the code-length function $c(x) = \mathrm{q}^t(x)$, the distribution $\mathcal{D}_c$ defined in Equation (4) coincides exactly with Q^t. The universal distribution satisfies the following domination property. For every distribution $\mathcal{D}$ samplable in time t by a sampler of description length $|\mathcal{D}|$, and every $x \in \mathsf{Support}(\mathcal{D})$,

$$\mathrm{Q}^{\mathsf{poly}(t)}(x) \geq 2^{-O(|\mathcal{D}|)} \cdot \mathcal{D}(x)$$

Indeed, the universal sampler can use an initial portion of its random input to specify the sampler for $\mathcal{D}$ and use the remaining bits as its random seed. The probability of selecting this sampler is at least $2^{-O(|\mathcal{D}|)}$, and, conditioned on doing so, the output distribution is exactly $\mathcal{D}$. This bounds the KL divergence from an unknown samplable distribution $\mathcal{D}$ to $\mathrm{Q}^t = 2^{-\mathrm{q}^t}$ in terms of the description length of a sampler for $\mathcal{D}$, since the domination property immediately implies the following coding-theorem type bound:

$$\mathrm{q}^t(x) = -\log \mathrm{Q}^t(x) \leq \log \frac{1}{\mathcal{D}(x)} + O(|\mathcal{D}|).$$

The claimed KL-divergence bound then follows by the same argument as in Proposition 6.5. To obtain the total variation bound, recall the batch-amortization argument following Proposition 6.5, in which the description-length term is amortized by conditioning a uniformly random coordinate on the preceding independent samples. Implementing this argument here requires sampling from the time-bounded universal distribution conditioned on an observed prefix of the batch. This conditional-sampling task is exactly what is called universal extrapolation in [HN23], which is equivalent to the task described by Definition 5.4

together with the procedure that follows it, specialized here to the case where no additional advice is given, as required for distribution learning.

Under the nonexistence of infinitely-often one-way functions, universal extrapolation (with no advice) can be performed in time polynomial in the number of bits to be extrapolated, the time bound, the inverse accuracy, and $2^{q^t(x)-\mathsf{K}(x)}$ [IL90, HN23]. Its running time is therefore exponential in the computational depth $q^t(x) - \mathsf{K}(x)$ of the conditioning prefix x. The slow-growth law for computational depth (see Section 5.2.2) states that efficient randomized computation increases computational depth by at most a logarithmic amount with high probability. Consequently, if the description of a sampler has low computational depth, then the conditioning prefixes generated by that sampler also have low computational depth with high probability. One concrete example is a sampler that is secretly selected by a uniform efficient randomized algorithm. Although the selected sampler may itself be a nonuniform object of polynomial length, its computational depth is logarithmic with high probability. For samplers of logarithmic computational depth, universal extrapolation therefore runs in polynomial time and, combined with the batch-amortization argument following Proposition 6.5, yields the following result without any restriction on the size of the message space.

Theorem 6.6 ([HN23])**.** *If infinitely-often one-way functions do not exist, then every efficiently samplable distribution represented by a sampler of logarithmic computational depth is distribution learnable in polynomial time (with respect to total variation distance, as defined in Definition 2.7).*

A limitation of this approach is that it requires ruling out one-way functions. Known constructions of universal extrapolation rely on consequences of the nonexistence of one-way functions, such as distributional inversion, and this dependence is unavoidable for general distribution classes because efficient universal extrapolation itself implies the nonexistence of one-way functions. Consequently, even for a restricted class of distributions that might be efficiently learnable unconditionally, this approach still invokes the seemingly implausible assumption that one-way functions do not exist. Thus, universal extrapolation reveals strong connections among learning, cryptography, and meta-complexity, but it remains unclear whether it can yield unconditional learning results for restricted distribution classes that exclude cryptographically hard sources.

Another major open question concerns the complexity of universal extrapolation itself. Existing techniques based on distributional inversion require time exponential in the computational depth of the conditioning prefix. They therefore do not give efficient conditional generation when the prefix has large computational depth.

Problem 6.7. What is the weakest assumption under which universal extrapolation can be performed efficiently for every conditioning prefix? In particular, is there a meta-complexity assumption that characterizes this worst-case form of universal extrapolation?

Sequential Generation. The second approach is to divide a polynomial-length message into polynomially many short blocks and generate them autoregressively. Write a message as $x = x_1 \cdots x_m$, where each block x_i has length $O(\log n)$. After generating the preceding blocks $x_{<i}$, the learner constructs a conditional distribution over the possible values of x_i and samples the next block from this distribution. Since each block has only polynomially many possible values, the required normalization can be performed by direct enumeration, as in Section 6.2.2.

Which compression guarantee is sufficient to construct such a sequential generator? To formulate this question, first observe the easy converse direction: the same conditional distributions can be used to compress the message block by block. Once $x_{<i}$ has been decoded, the conditional distribution associated with this prefix determines a code for x_i through the arithmetic-coding construction reviewed in Section 6.2.1. This form of sequential blockwise coding is known as *prequential compression*. A prequential compressor processes $x_1, \ldots, x_m$ online, encoding each block x_i with the preceding blocks $x_{<i}$ available as auxiliary input. Ideally, its total conditional description length should be comparable to a one-shot time-bounded description of the entire message x, such as $\mathsf{K}^{\mathsf{poly}}(x)$ or one of its variants that admits optimal coding, up to an additive $m \cdot O(\log n)$ overhead for encoding the blocks separately.

This objective is formalized by an *algorithmic chain rule*, as studied in [KK25, HN26a]. For $\mathrm{q}^{\mathsf{poly}}$, suppressing polynomial changes in the time bounds, such a chain rule takes the form

$$\sum_{i=1}^{m} \mathrm{q}^{\mathsf{poly}}(x_i \mid x_{<i}) \leq \mathrm{q}^{\mathsf{poly}}(x_1 \cdots x_m) + m \cdot O(\log n).$$

For a source distribution $\mathcal{D}$, we define the corresponding average-case notion by requiring the expected sum of conditional description lengths to be close to the source entropy, up to the same accumulated blockwise overhead[20]. Namely,

[20]This is equivalent, in the constant-parameter regime, to the formulation in [HN26a], which allows an additive ηN term, where N denotes the total message length. Indeed, if each block has length b, then $m = \Theta(N/b)$ and hence $m \cdot O(\log n) = O(N \log n/b)$. Choosing $b = \Theta(\eta^{-1} \log n)$ makes this overhead $O(\eta N)$. Conversely, when $b = \Theta(\log n)$ and η is constant, an additive ηN term is of the form $m \cdot O(\log n)$. Thus, the two formulations are interchangeable by adjusting the constant in the block length.

average-case prequential $\mathsf{q}^{\mathsf{poly}}$-compression requires the following bound

$$\mathop{\mathbf{E}}_{X\sim\mathcal{D}}\left[\sum_{i=1}^{m}\mathsf{q}^{\mathsf{poly}}(X_i \mid X_{<i})\right] \le \mathsf{H}(\mathcal{D}) + |\mathcal{D}| + m\cdot O(\log n),$$

where the constant in the "$O(\log n)$" term is fixed independently of the block length. Here, $|\mathcal{D}|$ denotes the description length of a sampler for $\mathcal{D}$ and is absorbed into the final term whenever $m \ge |\mathcal{D}|$. We emphasize that this is purely a statement about the source distribution $\mathcal{D}$; we do not assume the existence of an explicit efficient encoder that realizes the conditional description lengths $\mathsf{q}^{\mathsf{poly}}(\cdot \mid \cdot)$. Replacing $\mathsf{q}^{\mathsf{poly}}$ with $\mathsf{rK}^{\mathsf{poly}}$ gives the corresponding average-case notion of prequential $\mathsf{rK}^{\mathsf{poly}}$-compression.

Hirahara and Nanashima [HN26a] use this viewpoint to give an unconditional, distribution-by-distribution characterization. Here, it is important to distinguish the elementary symbols to be predicted from the longer blocks used for compression. Let $Z = (Z_1, \dots, Z_N) \sim \mathcal{D}$, where each Z_i is a bit or, more generally, a symbol of constant length. We group consecutive symbols into blocks $X_1, \dots, X_m$, each consisting of $b = O(\log n)$ symbols. This grouping is used for prequential compression, while the sequential generator still predicts one elementary symbol Z_i at a time.

Let $\mathcal{A}(Z_{<i})$ denote the output distribution of a sequential generator after observing $Z_{<i}$. For accuracy $\epsilon > 0$ and confidence error $\delta > 0$, the desired guarantee is

$$\Pr_{\substack{Z\sim\mathcal{D}\\ I\sim[N]}}\left[\Delta_{\mathsf{TV}}(\mathcal{A}(Z_{<I}), (Z_I \mid Z_{<I})) \le \epsilon\right] \ge 1-\delta,$$

where I is uniform over $[N]$. The authors show that, in the constant-accuracy and constant-confidence regime, the existence of an efficient sequential generator satisfying this guarantee is equivalent to average-case prequential $\mathsf{q}^{\mathsf{poly}}$-compression of the block sequence $X_1, \dots, X_m$ and, equivalently, to average-case prequential $\mathsf{rK}^{\mathsf{poly}}$-compression.

Although this does not yield the total variation guarantee required in Definition 2.7, it does yield the following Yao-style next-bit learning guarantee, with no restriction on the support size of the target distribution.

Theorem 6.8. *For every $\epsilon, \delta \in (0, 1]$, there exist a constant C and a polynomial-time learner L such that the following holds. Let $\mathcal{D}$ be any P/poly-samplable distribution over $\{0,1\}^n$. Suppose that, for every batch size m, the sequence obtained by concatenating m independent samples from $\mathcal{D}$ and partitioning the resulting string into consecutive blocks of length $C \log n$ admits average-case prequential $\mathsf{q}^{\mathsf{poly}}$-compression or average-case prequential $\mathsf{rK}^{\mathsf{poly}}$-compression.*

Then, given sample access to $\mathcal{D}$, the learner L outputs a polynomial-time randomized next-bit predictor h satisfying

$$\Pr_{h \leftarrow L^{\mathcal{D}}} \left[\mathop{\mathbf{E}}_{\substack{x \sim \mathcal{D} \\ i \sim [n]}} [\Delta_{\mathsf{TV}} (h(x_{<i}), (\mathcal{D}_i \mid \mathcal{D}_{<i} = x_{<i}))] \leq \epsilon \right] \geq 1 - \delta.$$

The transformation from prequential $\mathsf{q}^{\mathsf{poly}}$-compression to sequential generation applies the advised universal extrapolation procedure discussed in Section 5.2.2. As reviewed there, the completed blocks $X_{<j}$ are treated as advice. The algorithmic chain rule for prequential compression incurs an additional $O(\log n)$ KL loss per block. To obtain the total variation bound stated above, this loss must once again be amortized through conditional sampling. Rejection sampling provides the required conditioning. When generating Z_i in the current block X_j, rejection sampling is used only to condition on the previously generated symbols in X_j, since the completed blocks $X_{<j}$ are already available as advice. Since the current block contains only $b = O(\log n)$ symbols, every such conditioning prefix has length $O(\log n)$. The corresponding conditioning event therefore has inverse-polynomial probability, and rejection sampling runs in polynomial time. Unlike the universal-extrapolation approach discussed above, this transformation does not require distributional inversion or any assumption ruling out one-way functions.

Accordingly, the result does not claim that every efficiently samplable distribution can be learned, which would be incompatible with cryptographic hardness. Instead, it gives a distribution-by-distribution characterization: a distribution family admits efficient sequential generation precisely when it satisfies the corresponding prequential-compression condition.

The short-block construction balances coding overhead against the running time of rejection sampling. To reduce an additive $O(\log n)$ overhead per block to an ϵ fraction of the block length, the block length must be $\Omega(\epsilon^{-1} \log n)$. Rejection sampling on such a block takes time $2^{O(\epsilon^{-1} \log n)} = n^{O(1/\epsilon)}$. This is polynomial in n for every constant ϵ, but it is not polynomial jointly in n and ϵ^{-1}. The current characterization is therefore limited to constant accuracy and confidence, and extending it to general parameters remains open.

Problem 6.9. Does the equivalence between efficient sequential generation, prequential $\mathsf{q}^{\mathsf{poly}}$-compression, and prequential $\mathsf{rK}^{\mathsf{poly}}$-compression extend to arbitrary polynomial-length blocks and arbitrary accuracy and confidence parameters, with running time polynomial in the message length, the block length, ϵ^{-1}, and δ^{-1}?

In particular, it is important that ϵ can be made inverse-polynomially small in n, as this allows the next-bit prediction guarantee in Theorem 6.8 to be converted into the standard total variation guarantee via the triangle inequality.

6.2.4 Language Modeling

The framework developed above appears directly in modern language modeling. A *token* is a symbol from a finite vocabulary, such as a byte or a fragment of a word. An *autoregressive language model* $\mathcal{E}$ assigns, to every prefix $x_{<i}$, a distribution $\mathcal{E}(\cdot \mid x_{<i})$ over the next token. It generates a message by repeatedly sampling the next token from this distribution. Thus, with each token viewed as a short block, language modeling is precisely the sequential conditional-generation setting considered in Section 6.2.3.

The standard objective for training an autoregressive language model is the next-token log loss [e.g., RWC+19, BMR+20]:

$$L_{\mathcal{E}}(x) \triangleq \sum_{i=1}^{N} \log \frac{1}{\mathcal{E}(x_i \mid x_{<i})} = \log \frac{1}{\mathcal{E}(x)}.$$

As reviewed in Section 6.2.1, arithmetic coding uses the same conditional probabilities to encode x in $L_{\mathcal{E}}(x) + O(1)$ bits. Hence, minimizing log loss is equivalent to minimizing the length of the corresponding lossless encoding. In this sense, a language model is also a compressor, provided that the model is available to both the encoder and the decoder.

The reverse connection uses the normalized incremental weights discussed in Section 6.2.2, assigning each possible continuation y the weight $2^{-(c(xy)-c(x))}$. For each prefix x, normalizing these weights defines a distribution over the next token, and iterating this construction turns a compressor into an autoregressive generator. A good overall compression rate, however, does not in general guarantee that the resulting conditional distributions are accurate.

An early neural implementation of this connection predates modern language models. Schmidhuber and Heil [SH96] trained a feedforward network to estimate conditional next character probabilities and coupled its outputs with Huffman or arithmetic coding for lossless text compression; in their offline scheme, the trained predictor was shared by the encoder and decoder.

Delétang et al. [DRD+24] investigate both directions of this correspondence experimentally. They combine the next-token probabilities produced by large language models with arithmetic coding to obtain lossless compressors for text, images, and audio. Conversely, they normalize incremental code lengths produced by compressors such as gzip and use the resulting conditional distributions for autoregressive generation. Related experiments use prequential code length to evaluate neural predictors trained sequentially on previously revealed examples [e.g., BO18, BLH23]. Together, these recent developments place the relationship between sequential prediction and compression directly within modern machine learning.

The complexity-theoretic results discussed above provide an idealized time-bounded formulation of the same prediction–compression principle, with $\mathsf{K}^{\mathsf{poly}}$ and related measures such as $\mathsf{rK}^{\mathsf{poly}}$ and $\mathsf{q}^{\mathsf{poly}}$ serving as ideal universal benchmarks for the shortest descriptions that can be decoded in polynomial time. The existing correspondences thus describe the relationship between prediction and compression at this idealized time-bounded level, but do not yet yield guarantees for concrete large language models. This raises a natural question: can meta-complexity shed light on concrete language models, for instance, by providing new methods that transfer compression guarantees to sequential-generation guarantees without quantitative loss while preserving computational efficiency? Developing a connection between the idealized time-bounded theory and modern language models remains an important direction for future research.

7 Meta-Complexity and Computational Hardness from Learning

Ideas from complexity have long informed the design of new learning algorithms (see, e.g., [LMN93]); conversely, ideas from learning have yielded insights in complexity (see, e.g., [IW01]). More recently, there have been indications of deep and far-reaching connections between the two fields that go well beyond these results. They are part of an emerging theory that shows useful links between *proving complexity lower bounds* and *designing new algorithms*. The learning algorithm of [CIKK16] and the complexity lower bound of [Wil14] are examples of breakthroughs achieved in this way.

Learning algorithms in different models are known for several important classes of Boolean functions, including DNFs [Jac97], constant-depth circuits with AND, OR, and NOT gates [LMN93], and constant-depth circuits extended with MOD_p gates, where p is prime [CIKK16]. However, despite major efforts, a learning algorithm for the class TC^0_2, i.e., two layers of polynomially many threshold gates, is unknown and corresponds to a central problem in computational learning theory. Note that every reasonable concept class can be learned in time $2^n \cdot \mathsf{poly}(n)$, simply by querying the unknown function $f\colon \{0,1\}^n \to \{0,1\}$ on all input strings. In contrast, even if we allow the learning algorithm to run in *quantum time* slightly below 2^n and make *quantum membership queries*, there is currently no known algorithm for learning TC^0_2.

In a seminal work, [FK06] demonstrated that the existence of polynomial-time learning algorithms implies complexity lower bounds. Subsequent results strengthened this connection by showing that even learning in time slightly below 2^n suffices to derive lower bounds. Put differently, any "non-trivial" learning algorithm can be transformed into a source of hard functions. Hardness can be extracted from non-trivial deterministic learning algorithms [KKO13], randomised PAC learning algorithms [OS17], and quantum PAC learning algorithms [AGG+21]. In general, the existence of any non-trivial learning algorithm for a class C of functions (such as TC^0_2) implies that there are explicit problems that cannot be computed in C.[21] These results shed some light on the difficulty of designing provably correct learning algorithms for classes such as TC^0_2, for which super-polynomial size complexity lower bounds remain unknown.

In the next sections, we survey results of this kind and present a concrete example illustrating how learning algorithms can be used to prove lower bounds in meta-complexity.

[21] Interestingly, previous work [ST17, Kar24] has shown that non-trivial learning algorithms of the required form can be designed for a large number of circuit classes for which lower bounds are already known.

7.1 Deterministic Learners

Below we consider *typical* circuit classes C that are closed under restrictions and complementation,[22] and contain the identically constant functions.[23]

As discussed above, any class C can be learned in time $2^n \cdot \mathsf{poly}(n)$. For this reason, we say that a learning algorithm is *nontrivial* if its running time is of the form $2^n/n^{\omega(1)}$, i.e., if it admits an arbitrarily large polynomial speed-up over the trivial time bound. Our next result shows that the existence of nontrivial deterministic learning algorithms has significant complexity-theoretic consequences.

Recall that in MKtP (Minimum Kt Problem), we are given a string $x \in \{0,1\}^n$ and a parameter $s \in \mathbb{N}$, and the goal is to decide if $\mathsf{Kt}(x) \leq s$. Here $\mathsf{Kt}(x)$ denotes the Levin time-bounded Kolmogorov complexity of x [Lev84], which is defined as

$$\mathsf{Kt}(x) \triangleq \min_{\substack{\Pi \in \{0,1\}^* \\ t \in \mathbb{N}}} \{|\Pi| + \lceil \log t \rceil \;\big|\; \text{program } \Pi \text{ outputs } x \text{ within } t \text{ steps}\}.$$

We can also think of MKtP as a language contained in $\{0,1\}^*$ by fixing the threshold parameter s to a large bound such as $s(n) = n - \log n$. Showing that $\mathsf{MKtP} \notin \mathsf{P}$ is a longstanding open problem in meta-complexity [ABK+06].

Theorem 7.1 (MKtP Lower Bounds from Deterministic Learning)**.** *The following results hold:*

(*i*) Non-Uniform Hardness for MKtP from Subexponential Time Deterministic Learning:
Suppose that a typical circuit class $C = \{C_n\}_{n \geq 1}$ can be PAC-learned to error $\varepsilon \leq 0.499$ under the uniform distribution using membership queries in deterministic time $2^{n/2}/n^{\omega(1)}$. Then $\mathsf{MKtP} \notin C$.

(*ii*) Uniform Hardness for MKtP from Nontrivial Deterministic Learning:
Suppose that, for every $k \geq 1$, Boolean circuits of size n^k can be PAC-learned to error $\varepsilon \leq 0.499$ under the uniform distribution using membership queries in deterministic time $2^n/n^{\omega(1)}$. Then $\mathsf{MKtP} \notin \mathsf{P}$.

Proof Sketch. To prove the first item, let $A^{\mathsf{MQ}(\cdot)}(1^n)$ be a deterministic algorithm running in time $T(n)$ that learns every $f \in C_n$ to error ε. Let $\bar{0}\colon \{0,1\}^n \to \{0,1\}$ be the constant zero function. Suppose towards a contradiction that $\mathsf{MKtP} \in C$. Consider the Boolean function $f_n\colon \{0,1\}^n \to \{0,1\}$ such that $f_n(x) = 1$ if and only

[22] Recall that a class is closed under restrictions if any function g obtained by setting a subset of input bits of $f \in C$ to 0 and 1 is also in C. Closure under complementation means that if a Boolean function $f \in C$ then the function $g(x) = 1 - f(x) \in C$.

[23] If a circuit class C does not satisfy one of these properties, it is not difficult to use the C-circuit evaluation problem to define an explicit language in the complexity class P that is not in C.

if $\mathsf{Kt}(x) \geq n - \log n$. Then $\{f_n\}_{n\geq 1} \in C$ under the assumption that $\mathsf{MKtP} \in C$, since f_n is the complement of a restriction of the language MKtP.

Next, consider the execution of $A^{\mathsf{MQ}(\cdot)}(1^n)$ on $\bar{0}$, and let $h_{\bar{0}}$ be its output hypothesis. Similarly, let h_{f_n} be the output hypothesis produced on f_n. Then

$$\mathsf{dist}(h_{\bar{0}}, \bar{0}) \;\leq\; \varepsilon \quad \text{and} \quad \mathsf{dist}(h_{f_n}, f_n) \;\leq\; \varepsilon.$$

We claim that if $T(n) = 2^{n/2}/n^{\omega(1)}$ and n is large enough, then $h_{\bar{0}} \equiv h_{f_n}$, i.e., they compute the same function. It is enough to argue that $A^{\mathsf{MQ}(\bar{0})}(1^n)$ and $A^{\mathsf{MQ}(f_n)}(1^n)$ have the same computation transcript, and therefore produce the same hypotheses. To see this, we argue that every membership query to $\mathsf{MQ}(f_n)$ produces 0 (as in $\mathsf{MQ}(\bar{0})$). Indeed, it is not hard to prove by induction on i that if $q_i \in \{0,1\}^n$ is the i-th query, where $i \leq T(n)$, then $\mathsf{Kt}(q_i) < n - \log n$. This is because we can describe q_i by knowing the code of A, n, and i, and by simulating A on input 1^n for at most $T(n)$ steps. Consequently,

$$\begin{aligned} \mathsf{Kt}(q_i) &\leq (|A| + \log n + \log i) + \log(T(n) \cdot \mathsf{poly}(n)) \\ &\leq O(\log n) + 2\log T(n) \leq n - \omega(\log n) < n - \log n, \end{aligned}$$

where we used that $T(n) = 2^{n/2}/n^{\omega(1)}$. Consequently,

$$\mathsf{dist}(h_{\bar{0}}, h_{f_n}) \;=\; 0.$$

Applying the triangle inequality, we obtain that

$$\mathsf{dist}(\bar{0}, f_n) \;\leq\; 2\varepsilon.$$

In contrast, since a random n-bit string has Kt complexity $\geq n - \log n$ with probability $1 - o(1)$, we have

$$\mathsf{dist}(\bar{0}, f_n) \;\geq\; 1 - o(1).$$

This contradicts our choice of $\varepsilon \leq 0.499$.

For the proof of the second item, an additional idea is used. Assume now that $T(n) = 2^n/n^{\omega(1)}$ and $\mathsf{MKtP} \in \mathsf{P}$. For each k, let C^k be the class of circuits of size n^k. Adapting the proof of the first item, we argue below that $\mathsf{MKtP} \notin C^k$. Since this holds for all k, it implies in particular that $\mathsf{MKtP} \notin \mathsf{P}$, which is a contradiction.

Fix $k \in \mathbb{N}$. Note that, in the argument above, we had to assume a lower running time for the learning algorithm because of the additional $\log T(n)$ contribution coming from the encoding of $i \in [T(n)]$, the index of the query. We avoid the extra term coming from the encoding of the index. We will use that $\mathsf{MKtP} \in \mathsf{P}$ to show that $\mathsf{Kt}(q_i) < n - \log n$ for every query $q_i \in \{0,1\}^n$ made by the learner. In more detail, towards a contradiction, suppose that for some i we have $\mathsf{Kt}(q_i) \geq n - \log n$, and let i be the first such index. Then, we can describe q_i as follows: q_i is the first

query made by $A(1^n)$ during the simulation such that $\mathsf{Kt}(q_i) \geq n - \log n$. Since the latter condition can be checked in polynomial time under the assumption that $\mathsf{MKtP} \in \mathsf{P}$, it is not hard to see that

$$\mathsf{Kt}(q_i) \leq O(1) + \log n + \log(T(n) \cdot \mathsf{poly}(n)) < n - \log n,$$

for large enough n. This contradiction completes the proof. □

Complementing this result, we mention that separations against C (i.e., non-uniform lower bounds) for some problem in $\mathsf{E} = \mathsf{DTIME}[2^{O(n)}]$ (instead of $\mathsf{MKtP} \in \mathsf{E}$) also follow from deterministic learners running in time $T(n) = 2^n/n^{\omega(1)}$. For more details, see the results and techniques in [KKO13].

7.2 Randomized and Quantum Learners

As we consider more powerful learning algorithms, the task of constructing a computationally hard function from them becomes more elusive. For instance, in the proof of Theorem 7.1, we used in a crucial way that the set of queries produced by the learner is uniquely determined once we fix the function $f \in C_n$ for the learning task. If the learning algorithm is *randomized*, this property no longer holds, and the strategy described above simply does not work.

Nevertheless, using more sophisticated arguments that rely on the theory of computational pseudorandomness, it is possible to extract complexity lower bounds from any non-trivial learning algorithm. Let $\mathsf{BPE} \triangleq \mathsf{BPTIME}[2^{O(n)}]$ denote randomized exponential time, i.e., the randomized analogue of E.

Theorem 7.2 (Lower Bounds from Nontrivial Randomized Learning [OS17])**.** *Let C be a typical circuit class. If polynomial-size C-circuits can be learned using membership queries with error $\varepsilon < 0.499$ under the uniform distribution in randomized time $2^n/n^{\omega(1)}$, then $\mathsf{BPE} \not\subseteq C(\mathsf{poly})$.*

It is also possible to consider a more expressive class of *quantum* learning algorithms (see [AdW17] for more on quantum learning theory). Let $\mathsf{BQE} \triangleq \mathsf{BQTIME}[2^{O(n)}]$ denote quantum exponential time.

Theorem 7.3 (Lower Bounds from Nontrivial Quantum Learning [AGG+21])**.** *Let C be a typical circuit class. If polynomial-size C-circuits can be learned using quantum membership queries with error $\varepsilon < 0.499$ under the uniform distribution in quantum time $2^n/n^{\omega(1)}$, then $\mathsf{BQE} \not\subseteq C(\mathsf{poly})$.*

In particular, it follows that if there is a randomized (or even quantum) learning algorithm for the class of depth-2 threshold circuits (TC^0_2) that runs in time $2^n/n^{\Omega(1)}$, then a new complexity lower bound would follow (see Figure 7 for a

pictorial representation and a related discussion). We refer to [APS26] and references therein for progress on the learnability of subclasses of depth-2 threshold circuits.

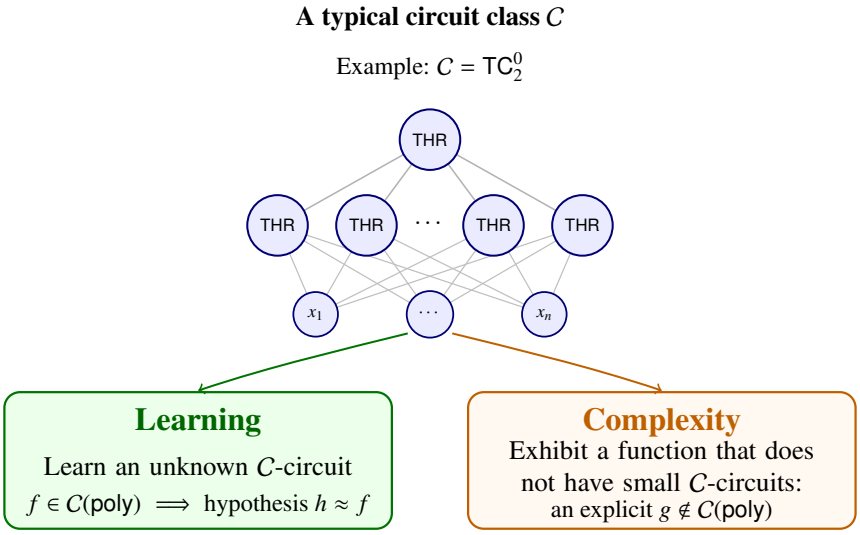


Figure 7: Two closely related questions about a typical circuit class C, illustrated by the class TC^0_2 of depth-2 threshold circuits (see Section 2.4.3). By Theorem 7.2 and Theorem 7.3, a nontrivial classical or quantum learner for C yields a new circuit size lower bound against C for an explicit function. Conversely, a combinatorial lower bound that naturalises yields a constructive property that can be converted into a learning algorithm for C, as discussed in Section 5.1. A recurring pattern across circuit classes at the frontier is that the two questions have the same status: either learners and lower bounds are known, or both remain open. In particular, for TC^0_2, neither a nontrivial learner nor a superpolynomial circuit size lower bound for an explicit function is known.

The original proof of Theorem 7.2 [OS17] makes use of the learning speedups discussed in Section 5.4, which reduces the result to a proof that *subexponential* time learning in time $2^{n^{o(1)}}$ implies complexity lower bounds. On the other hand, [AGG+21] explores a more direct perspective that can be used to derive both Theorem 7.2 and Theorem 7.3. We refer to [OS17, AGG+21] for more information about these theorems and their proofs.

While the results discussed in this section apply to a significantly larger class of learning algorithms, they come with a technical caveat: the hard function extracted from the learning assumption is hard only on infinitely many input lengths n. In contrast, it can be checked that the lower bound arguments in Theorem 7.1

establish hardness for every sufficiently large input length n. This caveat is a consequence of the more indirect nature of their proofs. It would be interesting to introduce techniques to close this gap.

Problem 7.4 (Almost-Everywhere Hardness from Learning)**.** Show that if a typical circuit class C admits nontrivial randomized learning algorithms, then there is a language in BPE that is not in $C(\mathsf{poly})$ on every large enough input length.

We refer to [Vol16] for additional settings where one can derive complexity-theoretic consequences from learning algorithms.

7.3 Unconditional Meta-Complexity Lower Bounds via Learning

Nontrivial learning algorithms have been designed for a large number of circuit classes [ST17, Kar24]. However, in all existing results, lower bounds are already known against the corresponding circuit class. A concrete class for which both lower bounds and learning remain elusive is the class of Boolean functions that can be approximated by low-degree Torus polynomials (see [BHLR19]).[24]

Problem 7.5 (Nontrivial Learnability of Torus Polynomials)**.** Is there a nontrivial randomized or quantum learning algorithm for the class of Boolean functions that can be $1/n$-approximated by $\mathsf{poly}(\log n)$-degree Torus polynomials?

Since any function in the circuit class ACC^0 admits such approximating polynomials [BHLR19], a positive answer to Problem 7.5 leads to a new lower bound against ACC^0 via Theorem 7.3.[25] The latter would be a breakthrough result in complexity theory. We refer to [Wil14] for more information on ACC^0.

A closely related problem is that of learning the class of Boolean circuits $f\colon \{0,1\}^n \to \{0,1\}$ that can be written as a disjoint union (DOR) of quasi-polynomially many MAJ-AND$_t$ circuits, where the bottom AND gate has fan-in $t = \mathsf{poly}(\log n)$.[26]

[24] Degree-d Torus polynomials and degree-d polynomial threshold functions (PTFs) can both be viewed as weighted majorities over the same feature representation. Let $\Phi_d(x)$ denote the vector of all multilinear monomials in x of degree at most d. In both cases, one first computes a weighted sum $p(x) = \langle w, \Phi_d(x)\rangle$. A PTF applies a *sign* activation $\mathbf{1}_{[p(x)\geq 0]}$, whereas a Torus polynomial uses a *periodic* activation: it reduces $p(x)$ modulo 1 and interprets values close to 0 and $1/2$ as the Boolean values 0 and 1, respectively. Thus, the two models use the same degree-bounded feature expansion but different activation functions. As discussed in Section 3, degree-d PTFs can be learned in time $\mathsf{poly}(n^d, 1/\varepsilon, \log(1/\delta))$.

[25] Recall that an ACC^0 circuit is a constant-depth polynomial-size Boolean circuit consisting of unbounded fan-in AND, OR, NOT, and MOD$_m$ gates, where m is a fixed integer and MOD$_m$ computes the sum of the input bits modulo m.

[26] Since MAJ-AND$_t$ circuits of bottom fan-in $t = \mathsf{poly}(\log n)$ can be viewed as degree-t PTFs, MAJ-AND$_t$ circuits (without a top DOR gate) can be learned in quasi-polynomial time.

Problem 7.6 (Nontrivial Learnability of DOR-MAJ-AND Circuits)**.** Is there a nontrivial randomized or quantum learning algorithm for the class of quasi-polynomial size DOR-MAJ-AND circuits of polylogarithmic bottom fan-in?

It is not hard to prove that a positive solution to this problem also implies lower bounds against ACC^0, by combining Theorem 7.3 with the simulation $\mathsf{ACC}^0 \subseteq \mathsf{SYM}^+ \subseteq \mathsf{DOR}_k\text{-}\mathsf{MAJ}\text{-}\mathsf{AND}_t$ for $k = \exp(\mathsf{poly}(\log n))$ and $t = \mathsf{poly}(\log n)$ [BT94].

Complexity Lower Bounds from Learning. While Theorem 7.2 and Theorem 7.3 have not yet led to new complexity lower bounds in a *direct* way, it is possible to use these results in an *indirect* way to establish *unconditional* complexity lower bounds. We describe next a general template:

$$\text{Easiness Assumption} \overset{(i)}{\Longrightarrow} \text{Learning Algorithm} \overset{(ii)}{\Longrightarrow} \text{Computational Hardness}$$

In more detail, we proceed as follows:

(*i*) First, from a suitable Easiness Assumption, we show that nontrivial learning *is* possible.

(*ii*) Next, we invoke a result such as Theorem 7.2 to derive some form of Computational Hardness.

To obtain an unconditional lower bound, we carefully balance the initial Easiness Assumption with the resulting Computational Hardness consequence, with the goal of achieving a contradiction between the two.

If we are able to implement this plan, it follows that the original Easiness Assumption is *false*, i.e., a corresponding complexity lower bound follows.

Hardness of Estimating Complexity. This approach has borne fruit in the form of an unconditional complexity lower bound in Meta-Complexity. In [Oli19], we considered the Minimum rKt Problem (MrKtP): given a string $w \in \{0,1\}^n$, estimate its rKt complexity, a measure of the compressibility, or regularity, of w.[27] This is a randomised analogue of a fundamental measure introduced by L. Levin in 1984 [Lev84] called Kt complexity. While the complexity of estimating the Kt complexity of an input string (MKtP) remains open, we were able to show that $\mathsf{MrKtP} \notin \mathsf{BPP}$, i.e., the rKt complexity of a string cannot be computed in probabilistic polynomial time.

[27] Random strings have rKt complexity close to n, while structured strings can be succinctly described and have much smaller rKt complexity.

This can be shown by analysing the following Easiness Assumption: MrKtP $\in$ BPP. From this and the technique of [CIKK16] described in Section 5.1, it follows that polynomial-size Boolean circuits can be learned in randomised polynomial time. This is Item (*i*) in the template presented above.

On the other hand, the existence of such a learning algorithm implies by Theorem 7.2 that some problem in BPE (randomised exponential time) cannot be computed by polynomial size Boolean circuits. This corresponds to Item (*ii*) in the aforementioned template.

Finally, since every problem in BPE can be reduced using polynomial-size Boolean circuits to MrKtP, it must be the case that MrKtP $\notin$ BPP. This contradicts the original Easiness Assumption.

In other words, it follows unconditionally that MrKtP $\notin$ BPP. Note that our original problem, MrKtP, makes no reference to learning algorithms!

Recall that it is a longstanding open problem to show the analogous result for MKtP, i.e., to prove that MKtP $\notin$ P. In light of the above discussion and Theorem 7.1, it is not hard to see that to obtain this result it would be sufficient to show that if MKtP $\in$ P then nontrivial *deterministic* learning of polynomial-size Boolean circuits is possible. In turn, this would be feasible if we had a *deterministic* analogue of the techniques from Section 5.1.

Problem 7.7 (Deterministic Learning from Meta-Complexity)**.** Is there a way to obtain deterministic learning algorithms assuming the easiness of meta-computational problems such as MKtP?

This is closely related to the problem of derandomizing learning algorithms. For more about this line of work, we refer the reader to [OS18].

To conclude this section, we note that we have only discussed lower bounds that can be extracted from PAC learning algorithms. In principle, it should be possible to obtain stronger complexity-theoretic consequences from the existence of algorithms for more expressive learning tasks (see Section 4). We leave this as an interesting direction for further research.

Problem 7.8 (Complexity-Theoretic Consequences of Learning)**.** Derive stronger complexity-theoretic consequences from the existence of algorithms for learning tasks that are more expressive than PAC learning under the uniform distribution, such as distribution learning and sequence extrapolation.

8 Related Work

Here we highlight some related lines of work that are not covered in the preceding sections, following roughly the order of the survey. We also provide pointers to some references that cover more practical aspects of learning and its connections to compression.

Search vs Decision in Meta-Complexity. The mismatch between recognising the existence of a short description and producing one is highlighted in Section 1. The learning constructions surveyed in Section 5 often circumvent this mismatch by converting a decision procedure into a predictor through a reconstruction argument. Search-to-decision reductions for time-bounded Kolmogorov complexity address the issue directly, asking when a decision procedure suffices to recover a short program. We refer to [MP24, HKLO24, HLN24] for recent work on this direction and related open problems.

Compression in Supervised Learning. The term *model compression* (Section 3) has a different meaning in modern machine learning, where one starts with a trained model and seeks a smaller approximation. Representative approaches include transferring the predictive behaviour of a large model or ensemble to a smaller network through knowledge distillation [HVD15], compressing an explicitly stored network through pruning, quantization, and entropy coding [HMD16], and training stochastic network parameters subject to an explicit information-theoretic coding budget [HPH19]. Dataset distillation moves the compression to the data side, replacing a large training set by a small collection of synthetic examples from which a prescribed training procedure recovers comparable performance [WZTE18].

Section 3 also develops the basic connection between sample compression and generalisation, mentions the qualitative equivalence with finite VC dimension, and states the sample compression conjecture. The classical binary sample compression framework was introduced by Littlestone and Warmuth [LW86] and developed further by Floyd and Warmuth [FW95]. Subsequent work extends or refines this perspective in several directions, including multiclass and general loss learning [DMY16], stable compression schemes [HK21], list compression and its limitations [HMW24], and reductions from compression problems in other learning models to classical binary compression [AHR25].

Distributional Restrictions and the Complexity of Learning. Departing from the pessimistic distribution-free definition of PAC learning, the learning paradises of Section 4.1 deliberately restrict the example distributions to feasible ones. This

choice can change the computational complexity of the learning task substantially. Blanc et al. [BKL+26] show that requiring success only on samplable distributions can enlarge the power of efficient learning, even at the level of sample complexity. Blanc, Lange, Strassle, and Tan [BLST25] instead ask when a learner for a restricted family of example distributions can be lifted to arbitrary example distributions. The latter result is governed by the complexity of expressing the new distribution as a mixture of distributions from the original family. This is distinct from the distributional inversion argument in Section 5, which moves between example distributions by inverting their samplers.

Practical Analogues of the Learning Paradises. The assumptions P.I through P.V from Section 4.1 are much stronger than anything currently achieved in practice, but the tasks they formalize have recognizable counterparts in modern machine learning. These analogies concern the form of each learning task, not the strength of its guarantee. Practical methods exploit regularities in natural data and address restricted families of instances, whereas the paradises require uniform guarantees over broad families of sources, example distributions, or circuits.

P.I is closest to predicting the continuation of a sequence from an observed context, as in models for text, video, or dynamical systems. For instance, Delétang et al. [DRD+24] study the practical connection between prediction and compression in this setting, while Grau-Moya et al. [GMGH+24] investigate neural predictors trained to approximate universal prediction.

P.II resembles generative modeling, where independent examples are used to learn a procedure for producing new samples. The denoising diffusion model of Ho, Jain, and Abbeel [HJA20] is a prominent example. Generative models can also serve as compressors. Yang, Mandt, and Theis [YMT23] survey this area, while Kingma et al. [KSPH21] turn a variational diffusion model into a lossless compressor whose rate is close to its variational coding bound. From the perspective of this survey, these systems also expose resources not captured by code length, notably encoding and decoding time and the size of the shared learned model. We note that practical systems are generally evaluated by prediction loss, sample quality, or application performance, rather than by guarantees that hold uniformly over all efficiently samplable distributions, as P.I and P.II require.

P.III is closest to learning when labels may be noisy and the target need not belong to the chosen hypothesis class. Haussler [Hau92] extends the PAC model to general loss minimization, while Kearns, Schapire, and Sellie [KSS94] formulate the agnostic model, in which performance is compared with that of the best predictor in a class. Tractable results for noisy labels generally assume additional structure. For example, Natarajan et al. [NDRT13] study loss correction when the probability of a label error depends only on the true label. This is one example

from a vast theoretical and empirical literature, which we do not attempt to survey here.

P.IV considers learning through chosen queries. Angluin [Ang88] provides a foundational framework for query learning. Practical counterparts arise when a learner can choose which inputs are labelled, as in active learning [CGJ96], or can experiment with an implemented system, as in black box checking [PVY99] and model extraction [TZJ+16]. Membership queries provide stronger access than the common pool-based model of active learning, since they allow the learner to query any input rather than only examples from a fixed unlabelled sample.

P.V has a broader practical analogue. A heuristic learning algorithm may succeed on concepts that arise naturally in an application even if it fails on adversarial or otherwise atypical concepts from the same representation class. This is precisely the relaxation captured by P.V.

The empirical success of modern machine learning remains compatible with the hardness results discussed above. As explained before, the learning paradises concern uniform guarantees over broad families of efficient sources, distributions, or circuits, which can be far more general than success on the structured instances encountered in practice. These stronger theoretical guarantees enable the connections to cryptography and average-case complexity, which in turn yield hardness of learning results.

Cryptographic Hardness of Learning. The cryptographic literature approaches learning from the opposite direction to much of this survey, using cryptographic assumptions to prove that particular concept classes are hard to learn. Classical examples include the reductions of Pitt and Valiant [PV88] and of Kearns and Valiant [KV94b], followed by other representative examples such as Klivans and Sherstov [KS09] and Daniely et al. [DLSS14] (see also [AOW15] and subsequent work [Dan16, DV21]). There is an extensive literature on the hardness of learning tasks under a variety of cryptographic conjectures, and reviewing these results is beyond the scope of this section.

A related line of work investigates whether weaker complexity-theoretic assumptions are sufficient to rule out efficient learning. Applebaum, Barak, and Xiao identify barriers to basing such results on standard worst-case assumptions [ABX08]. Goldberg and Kabanets [GK25] show that certain formulations of NP-hardness of learning characterize witness encryption, while Blanc et al. [BKST25] derive computational and statistical tradeoffs from NP hardness.

The discussion of white-box learning from Section 4.5 concentrates on one-way functions. Liu, Mazor, and Pass [LMP25] obtain a corresponding characterization of public-key encryption through a richer white-box learning problem. The existence of secure key-agreement protocols has also been characterized via

notions from Kolmogorov complexity and meta-complexity [BLMP23].

Description Length in Statistical Inference and Prediction. The treatment in Section 6 uses coding as a bridge between probabilistic prediction and compression, with particular attention to computational efficiency. Minimum Message Length (MML), originating in the work of Wallace and Boulton [WB68], and Minimum Description Length (MDL), introduced by Rissanen [Ris78], instead use description length as a criterion for statistical inference and model selection. Barron, Rissanen, and Yu [BRY98] give a standard account of the connections among MDL, universal coding, and statistical inference, while Vitányi and Li [VL00] relate MDL and MML to Bayesian inference and Kolmogorov complexity.

Dawid [Daw84] formulated the prequential ("predictive and sequential") approach, which evaluates a statistical method through the predictions it issues sequentially; under log loss, these predictions themselves define a code. Rissanen [Ris84] connects universal coding with prediction and estimation, while Feder, Merhav, and Gutman [FMG92] study universal prediction of individual sequences. Cesa-Bianchi and Lugosi [CL06] give a standard treatment of sequential prediction and regret. These traditions are close to the coding interpretation developed above, but they generally concern statistical performance rather than the efficient computational transformations that are central here.

Building on Kolmogorov's earlier ideas about separating structure from randomness, Gács, Tromp, and Vitányi [GTV01] develop algorithmic statistics, asking how much of an individual string's shortest description should be regarded as an explanatory model and how much as unexplained randomness. This is related to the use of algorithmic information theory for extrapolation in Section 5.2, but its primary object is the structure of a given string rather than prediction from a sequence.

Similarity via Compression. A different use of compression in machine learning, not covered in the survey, treats the lengths produced by a compressor as a measure of similarity between individual objects. Li et al. [LCL+04] introduce normalized information distance and demonstrate practical approximations using standard compressors, while Cilibrasi and Vitányi develop normalized compression distance into a general clustering method [CV05]. More recently, Jiang et al. [JYT+23] illustrate the same idea in text classification by combining gzip with a nearest neighbor rule. This perspective is complementary to the connections between learning and compression discussed in the survey: compression is used directly as a data analysis primitive, without learning an explicit model.

References

[AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, New York, NY, USA, 2009.

[ABK+06] Eric Allender, Harry Buhrman, Michal Koucký, Dieter van Melkebeek, and Detlef Ronneburger. Power from random strings. *SIAM J. Comput.*, 35(6):1467–1493, 2006.

[ABX08] Benny Applebaum, Boaz Barak, and David Xiao. On basing lower-bounds for learning on worst-case assumptions. In *Symposium on Foundations of Computer Science* (FOCS), pages 211–220, 2008.

[AdW17] Srinivasan Arunachalam and Ronald de Wolf. A survey of quantum learning theory. *ACM SIGACT News*, 48(2):41–67, 2017.

[AGG+21] Srinivasan Arunachalam, Alex B. Grilo, Tom Gur, Igor C. Oliveira, and Aarthi Sundaram. Quantum learning algorithms imply circuit lower bounds. In *Symposium on Foundations of Computer Science* (FOCS), pages 562–573, 2021.

[AGvM+18] Eric Allender, Joshua A. Grochow, Dieter van Melkebeek, Cristopher Moore, and Andrew Morgan. Minimum circuit size, graph isomorphism, and related problems. *SIAM J. Comput.*, 47(4):1339–1372, 2018.

[AHR25] Idan Attias, Steve Hanneke, and Arvind Ramaswami. Sample compression scheme reductions. In *International Conference on Algorithmic Learning Theory* (ALT), pages 134–162, 2025.

[All17] Eric Allender. The complexity of complexity. In *Computability and Complexity*, pages 79–94. Springer, 2017.

[All21] Eric Allender. Vaughan Jones, Kolmogorov Complexity, and the new complexity landscape around circuit minimization. *New Zealand journal of mathematics*, 52:585–604, 2021.

[Ang88] Dana Angluin. Queries and concept learning. *Machine Learning*, 2(4):319–342, 1988.

[AOW15] Sarah R. Allen, Ryan O'Donnell, and David Witmer. How to refute a random CSP. In *Symposium on Foundations of Computer Science* (FOCS), pages 689–708, 2015.

[APS26] Josh Alman, Shyamal Patel, and Rocco A. Servedio. Learning functions of halfspaces. In *Symposium on Theory of Computing* (STOC), pages 1581–1591, 2026.

[BCK+22] Eric Binnendyk, Marco Carmosino, Antonina Kolokolova, R. Ramyaa, and Manuel Sabin. Learning with distributional inverters. In *International Conference on Algorithmic Learning Theory* (ALT), pages 90–106, 2022.

[BEHW87] Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. Occam's razor. *Inf. Process. Lett.*, 24(6):377–380, 1987.

[BFKL93] Avrim Blum, Merrick L. Furst, Michael J. Kearns, and Richard J. Lipton. Cryptographic primitives based on hard learning problems. In *Advances in Cryptology* (CRYPTO), volume 773, pages 278–291, 1993.

[BFNW93] László Babai, Lance Fortnow, Noam Nisan, and Avi Wigderson. BPP has subexponential time simulations unless EXPTIME has publishable proofs. *Computational Complexity*, 3:307–318, 1993.

[BFP05] Harry Buhrman, Lance Fortnow, and Aduri Pavan. Some results on derandomization. *Theory Comput. Syst.*, 38(2):211–227, 2005.

[BHLR19] Abhishek Bhrushundi, Kaave Hosseini, Shachar Lovett, and Sankeerth Rao. Torus polynomials: An algebraic approach to ACC^0 lower bounds. In *Innovations in Theoretical Computer Science Conference* (ITCS), 2019.

[BKL+26] Guy Blanc, Caleb Koch, Jane Lange, Carmen Strassle, and Li-Yang Tan. Samplability makes learning easier. In *Innovations in Theoretical Computer Science Conference* (ITCS), volume 362, pages 20:1–20:12, 2026.

[BKST25] Guy Blanc, Caleb Koch, Carmen Strassle, and Li-Yang Tan. Computational-statistical tradeoffs from NP-hardness. In *Symposium on Foundations of Computer Science* (FOCS), pages 512–530, 2025.

[BL93] Dan Boneh and Richard J. Lipton. Amplification of weak learning under the uniform distribution. In *Conference on Computational Learning Theory* (COLT), pages 347–351. ACM, 1993.

[Ble13] Guy E. Blelloch. Introduction to data compression. *Lecture Notes – Computer Science Department, Carnegie Mellon University*, 2013.

[BLH23] Jörg Bornschein, Yazhe Li, and Marcus Hutter. Sequential learning of neural networks for prequential MDL. In *International Conference on Learning Representations* (ICLR), 2023.

[BLMP23] Marshall Ball, Yanyi Liu, Noam Mazor, and Rafael Pass. Kolmogorov comes to Cryptomania: On interactive Kolmogorov complexity and key-agreement. In *Symposium on Foundations of Computer Science* (FOCS), pages 458–483, 2023.

[BLST25] Guy Blanc, Jane Lange, Carmen Strassle, and Li-Yang Tan. A distributional-lifting theorem for PAC learning. In *Conference on Learning Theory* (COLT), pages 375–379, 2025.

[BMR+20] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Conference on Neural Information Processing Systems* (NeurIPS), 2020.

[BO18] Léonard Blier and Yann Ollivier. The description length of deep learning models. In *Conference on Neural Information Processing Systems* (NeurIPS), pages 2220–2230, 2018.

[BRY98] Andrew R. Barron, Jorma Rissanen, and Bin Yu. The minimum description length principle in coding and modeling. *IEEE Transactions on Information Theory*, 44(6):2743–2760, October 1998.

[BT94] Richard Beigel and Jun Tarui. On ACC. *Computational Complexity*, 4(4):350–366, 1994.

[BT06] Andrej Bogdanov and Luca Trevisan. Average-case complexity. *Found. Trends Theor. Comput. Sci.*, 2(1), 2006.

[CDV24] Sílvia Casacuberta, Cynthia Dwork, and Salil P. Vadhan. Complexity-theoretic implications of multicalibration. In *Symposium on Theory of Computing* (STOC), pages 1071–1082, 2024.

[CGJ96] David A. Cohn, Zoubin Ghahramani, and Michael I. Jordan. Active learning with statistical models. *Journal of Artificial Intelligence Research*, 4:129–145, 1996.

[CIKK16] Marco L. Carmosino, Russell Impagliazzo, Valentine Kabanets, and Antonina Kolokolova. Learning algorithms from natural proofs. In *Conference on Computational Complexity* (CCC), pages 10:1–10:24, 2016.

[CIKK17] Marco L. Carmosino, Russell Impagliazzo, Valentine Kabanets, and Antonina Kolokolova. Agnostic learning from tolerant natural proofs. In *International Workshop on Randomization and Approximation Techniques* (RANDOM), volume 81, pages 35:1–35:19, 2017.

[CJ26] Marco Carmosino and Mandar Juvekar. Parallel Kolmogorov complexity with connections to learning theory and cryptography. Unpublished preprint, September 2026.

[CL06] Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.

[CLO+23] Lijie Chen, Zhenjian Lu, Igor C. Oliveira, Hanlin Ren, and Rahul Santhanam. Polynomial-time pseudodeterministic construction of primes. In *Symposium on Foundations of Computer Science* (FOCS), pages 1261–1270, 2023.

[CT91] Thomas M. Cover and Joy A. Thomas. *Elements of information theory*. John Wiley & Sons, 1991.

[CTW26] Lijie Chen, Avishay Tal, and Yichuan Wang. Superquadratic lower bounds for depth-2 linear threshold circuits. In *Symposium on Theory of Computing* (STOC), pages 874–885, 2026.

[CV05] Rudi Cilibrasi and Paul M. B. Vitányi. Clustering by compression. *IEEE Trans. Inf. Theory*, 51(4):1523–1545, 2005.

[Dan16] Amit Daniely. Complexity theoretic limitations on learning halfspaces. In *Symposium on Theory of Computing* (STOC), pages 105–117, 2016.

[Dav73] L. Davisson. Universal noiseless coding. *IEEE Transactions on Information Theory*, 19(6):783–795, 1973.

[Daw84] A. P. Dawid. Present position and potential developments: Some personal views: Statistical theory: The prequential approach. *Journal of the Royal Statistical Society. Series A (General)*, 147(2):278–290, 1984.

[DLSS14] Amit Daniely, Nati Linial, and Shai Shalev-Shwartz. From average case complexity to improper learning complexity. In *Symposium on Theory of Computing* (STOC), pages 441–448, 2014.

[DMY16] Ofir David, Shay Moran, and Amir Yehudayoff. Supervised learning through the lens of compression. In *Conference on Neural Information Processing Systems* (NeurIPS), pages 2784–2792, 2016.

[DRD+24] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. In *International Conference on Learning Representations* (ICLR), 2024.

[DV21] Amit Daniely and Gal Vardi. From local pseudorandom generators to hardness of learning. In *Conference on Learning Theory* (COLT), pages 1358–1394, 2021.

[Fel10] Vitaly Feldman. Distribution-specific agnostic boosting. In *Innovations in Theoretical Computer Science* (ITCS), pages 241–250, 2010.

[Fit66] B. M. Fitingof. Optimal coding in the case of unknown and changing message statistics. *Problems of Information Transmission*, 2(2):1–7, 1966.

[FJS91] Merrick L. Furst, Jeffrey C. Jackson, and Sean W. Smith. Improved learning of AC^0 functions. In *Conference on Learning Theory* (COLT), pages 317–325, 1991.

[FK06] Lance Fortnow and Adam R. Klivans. Efficient learning algorithms yield circuit lower bounds. In *Annual Conference on Learning Theory* (COLT), pages 350–363, 2006.

[FMG92] Meir Feder, Neri Merhav, and Michael Gutman. Universal prediction of individual sequences. *IEEE Transactions on Information Theory*, 38(4):1258–1270, July 1992.

[FSS84] Merrick L. Furst, James B. Saxe, and Michael Sipser. Parity, circuits, and the polynomial-time hierarchy. *Math. Syst. Theory*, 17(1):13–27, 1984.

[FW95] Sally Floyd and Manfred K. Warmuth. Sample compression, learnability, and the Vapnik-Chervonenkis dimension. *Machine Learning*, 21(3):269–304, December 1995.

[GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *J. ACM*, 33(4):792–807, 1986.

[GHL+25] Halley Goldberg, Jinqiao Hu, Zhenjian Lu, Jingyi Lyu, and Igor C. Oliveira. Synergies between complexity theory and nondeterministic Kolmogorov complexity. *Electron. Colloquium Comput. Complex.*, TR25, 2025.

[GJK+24] Tom Gur, Mohammad Mahdi Jahanara, Mohammad Mahdi Khodabandeh, Ninad Rajgopal, Bahar Salamatian, and Igor Shinkar. On the power of interactive proofs for learning. In *Symposium on Theory of Computing* (STOC), pages 1063–1070, 2024.

[GK22] Halley Goldberg and Valentine Kabanets. A simpler proof of the worst-case to average-case reduction for polynomial hierarchy via symmetry of information. *Electron. Colloquium Comput. Complex.*, 7:1–14, 2022.

[GK23] Halley Goldberg and Valentine Kabanets. Improved learning from Kolmogorov complexity. In *Computational Complexity Conference* (CCC), pages 12:1–12:29, 2023.

[GK25] Halley Goldberg and Valentine Kabanets. Witness encryption and NP-hardness of learning. In *Computational Complexity Conference* (CCC), pages 34:1–34:43, 2025.

[GKLO22] Halley Goldberg, Valentine Kabanets, Zhenjian Lu, and Igor C. Oliveira. Probabilistic Kolmogorov complexity with applications to average-case complexity. In *Computational Complexity Conference* (CCC), pages 16:1–16:60, 2022.

[GMGH+24] Jordi Grau-Moya, Tim Genewein, Marcus Hutter, Laurent Orseau, Gregoire Deletang, Elliot Catt, Anian Ruoss, Li Kevin Wenliang, Christopher Mattern, Matthew Aitchison, and Joel Veness. Learning universal predictors. In *International Conference on Machine Learning* (ICML), pages 16178–16205, 2024.

[Gol01] Oded Goldreich. *The Foundations of Cryptography - Volume 1: Basic Tools*. Cambridge University Press, 2001.

[GS91] Andrew V. Goldberg and Michael Sipser. Compression and ranking. *SIAM J. Comput.*, 20(3):524–536, 1991.

[GTV01] Péter Gács, John T. Tromp, and Paul M. B. Vitányi. Algorithmic statistics. *IEEE Transactions on Information Theory*, 47(6):2443–2463, September 2001.

[Hau92] David Haussler. Decision theoretic generalizations of the PAC model for neural net and other learning applications. *Inf. Comput.*, 100(1):78–150, 1992.

[HIL+23] Shuichi Hirahara, Rahul Ilango, Zhenjian Lu, Mikito Nanashima, and Igor C. Oliveira. A duality between one-way functions and average-case symmetry of information. In *Symposium on Theory of Computing* (STOC), pages 1039–1050, 2023.

[HILL99] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM J. Comput.*, 28(4):1364–1396, 1999.

[Hir18] Shuichi Hirahara. Non-black-box worst-case to average-case reductions within NP. In *Symposium on Foundations of Computer Science* (FOCS), pages 247–258, 2018.

[Hir20a] Shuichi Hirahara. Characterizing average-case complexity of PH by worst-case meta-complexity. In *Symposium on Foundations of Computer Science* (FOCS), pages 50–60, 2020.

[Hir20b] Shuichi Hirahara. Non-disjoint promise problems from meta-computational view of pseudorandom generator constructions. In *Computational Complexity Conference* CCC, pages 20:1–20:47, 2020.

[Hir21] Shuichi Hirahara. Average-case hardness of NP from exponential worst-case hardness assumptions. In *Symposium on Theory of Computing* (STOC), pages 292–302, 2021.

[Hir22a] Shuichi Hirahara. Meta-computational average-case complexity: A new paradigm toward excluding Heuristica. *Bull. EATCS*, 136, 2022.

[Hir22b] Shuichi Hirahara. NP-hardness of learning programs and partial MCSP. In *Symposium on Foundations of Computer Science* (FOCS), pages 968–979, 2022.

[Hir22c] Shuichi Hirahara. Symmetry of information from meta-complexity. In *Computational Complexity Conference* (CCC), pages 26:1–26:41, 2022.

[HJA20] Jonathan Ho, Ajay N. Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Conference on Neural Information Processing Systems* (NeurIPS), pages 6840–6851, 2020.

[HK21] Steve Hanneke and Aryeh Kontorovich. Stable sample compression schemes: New applications and an optimal SVM margin bound. In *International Conference on Algorithmic Learning Theory* (ALT), pages 697–721, 2021.

[HKLO24] Shuichi Hirahara, Valentine Kabanets, Zhenjian Lu, and Igor C. Oliveira. Exact search-to-decision reductions for time-bounded Kolmogorov complexity. In *Computational Complexity Conference* (CCC), pages 29:1–29:56, 2024.

[HLN24] Shuichi Hirahara, Zhenjian Lu, and Mikito Nanashima. Optimal coding for randomized Kolmogorov complexity and its applications. In *Symposium on Foundations of Computer Science* (FOCS), pages 369–378, 2024.

[HLO24] Shuichi Hirahara, Zhenjian Lu, and Igor C. Oliveira. One-way functions and pKt complexity. In *Theory of Cryptography* (TCC), pages 253–286, 2024.

[HLO26a] Jinqiao Hu, Zhenjian Lu, and Igor C. Oliveira. Equivalence between coding and complexity lower bounds. In *International Colloquium on Automata, Languages, and Programming* (ICALP), pages 110:1–110:23, 2026.

[HLO26b] Jinqiao Hu, Zhenjian Lu, and Igor C. Oliveira. Hardness of computing nondeterministic Kolmogorov complexity. In *Computational Complexity Conference* (CCC), 2026.

[HMD16] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations* (ICLR), 2016.

[HMO26] Jinqiao Hu, Yahel Manor, and Igor C. Oliveira. Failure of symmetry of information for randomized computations. In *Symposium on Theory of Computing* (STOC), pages 1592–1603, 2026.

[HMW24] Steve Hanneke, Shay Moran, and Tom Waknine. List sample compression and uniform convergence. In *Conference on Learning Theory* (COLT), pages 2360–2388, 2024.

[HN21] Shuichi Hirahara and Mikito Nanashima. On worst-case learning in relativized Heuristica. In *Symposium on Foundations of Computer Science* (FOCS), pages 751–758, 2021.

[HN23] Shuichi Hirahara and Mikito Nanashima. Learning in Pessiland via inductive inference. In *Symposium on Foundations of Computer Science,* (FOCS), pages 447–457, 2023.

[HN26a] Shuichi Hirahara and Mikito Nanashima. Complexity-theoretic universal inductive inference. In *Symposium on Theory of Computing* (STOC), 2026.

[HN26b] Shuichi Hirahara and Mikito Nanashima. A sharp characterization of Pessiland. In *Symposium on Theory of Computing* (STOC), 2026.

[HPH19] Marton Havasi, Robert Peharz, and José Miguel Hernández-Lobato. Minimal random code learning: Getting bits back from compressed model parameters. In *International Conference on Learning Representations* (ICLR), 2019.

[HQC24] Marcus Hutter, David Quarel, and Elliot Catt. *An introduction to universal artificial intelligence*. Chapman and Hall/CRC, 2024.

[HS07] Lisa Hellerstein and Rocco A Servedio. On PAC learning algorithms for rich boolean function classes. *Theoretical Computer Science*, 384(1):66–76, 2007.

[HV25] Lunjia Hu and Salil P. Vadhan. Generalized and unified equivalences between hardness and pseudoentropy. In *Theory of Cryptography* (TCC), pages 258–288, 2025.

[HVD15] Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the knowledge in a neural network. *CoRR*, abs/1503.02531, 2015.

[IL89] Russell Impagliazzo and Michael Luby. One-way functions are essential for complexity based cryptography (extended abstract). In *Symposium on Foundations of Computer Science* (FOCS), pages 230–235, 1989.

[IL90] Russell Impagliazzo and Leonid A. Levin. No better ways to generate hard NP instances than picking uniformly at random. In *Symposium on Foundations of Computer Science* (FOCS), pages 812–821, 1990.

[ILO20] Rahul Ilango, Bruno Loff, and Igor C. Oliveira. NP-hardness of circuit minimization for multi-output functions. In *Computational Complexity Conference* (CCC), 2020.

[Imp95] Russell Impagliazzo. A personal view of average-case complexity. In *Proceedings of the Tenth Annual Structure in Complexity Theory Conference*, pages 134–147, 1995.

[IW97] Russell Impagliazzo and Avi Wigderson. P = BPP if E requires exponential circuits: Derandomizing the XOR lemma. In *Symposium on Theory of Computing* (STOC), pages 220–229. ACM, 1997.

[IW01] Russell Impagliazzo and Avi Wigderson. Randomness vs time: Derandomization under a uniform assumption. *J. Comput. Syst. Sci.*, 63(4):672–688, 2001.

[Jac97] Jeffrey C. Jackson. An efficient membership-query algorithm for learning DNF with respect to the uniform distribution. *J. Comput. Syst. Sci.*, 55(3):414–440, 1997.

[Juk12] Stasys Jukna. Boolean function complexity: advances and frontiers. *Algorithms and combinatorics*, 27, 2012.

[JYT+23] Zhiying Jiang, Matthew Y. R. Yang, Mikhail Tsirlin, Raphael Tang, Yiqin Dai, and Jimmy Lin. "low-resource" text classification: A parameter-free classification method with compressors. In *Annual Meeting of the Association for Computational Linguistics* (ACL), 2023.

[Kar24] Ari Karchmer. Agnostic membership query learning with nontrivial savings: New results and techniques. In *International Conference on Algorithmic Learning Theory* (ALT), pages 654–682, 2024.

[KC00] Valentine Kabanets and Jin-yi Cai. Circuit minimization problem. In *Symposium on Theory of Computing* (STOC), pages 73–79, 2000.

[KK09] Adam Kalai and Varun Kanade. Potential-based agnostic boosting. In *Conference on Neural Information Processing Systems* (NeurIPS), pages 880–888, 2009.

[KK25] Valentine Kabanets and Antonina Kolokolova. Chain rules for time-bounded Kolmogorov complexity. *Electron. Colloquium Comput. Complex.*, TR25-89, 2025.

[KKO13] Adam R. Klivans, Pravesh Kothari, and Igor C. Oliveira. Constructing hard functions using learning algorithms. In *Conference on Computational Complexity* (CCC), pages 86–97, 2013.

[KL07] Jonathan Katz and Yehuda Lindell. *Introduction to modern cryptography: principles and protocols*. Chapman and Hall/CRC, 2007.

[KL18] Pravesh K. Kothari and Roi Livni. Improper learning by refuting. In *Innovations in Theoretical Computer Science Conference* (ITCS), pages 55:1–55:10, 2018.

[KMR+94] Michael Kearns, Yishay Mansour, Dana Ron, Ronitt Rubinfeld, Robert E. Schapire, and Linda Sellie. On the learnability of discrete distributions. In *Symposium on Theory of Computing* (STOC), pages 273–282, 1994.

[Kol65] Andrey N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of information transmission*, 1(1):1–7, 1965.

[KS09] Adam R. Klivans and Alexander A. Sherstov. Cryptographic hardness for learning intersections of halfspaces. *J. Comput. Syst. Sci.*, 75(1):2–12, 2009.

[KSPH21] Diederik Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. In *Advances in Neural Information Processing Systems*, volume 34, pages 21696–21707, 2021.

[KSS94] Michael J. Kearns, Robert E. Schapire, and Linda Sellie. Toward efficient agnostic learning. *Mach. Learn.*, 17(2-3):115–141, 1994.

[KV94a] M. Kearns and U. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.

[KV94b] Michael J. Kearns and Leslie G. Valiant. Cryptographic limitations on learning boolean formulae and finite automata. *Journal of the ACM*, 41(1):67–95, January 1994.

[KW16] Daniel M. Kane and Ryan Williams. Super-linear gate and super-quadratic wire lower bounds for depth-two and depth-three threshold circuits. In *Symposium on Theory of Computing* (STOC), pages 633–643, 2016.

[LCL+04] Ming Li, Xin Chen, Xin Li, Bin Ma, and Paul M. B. Vitányi. The similarity metric. *IEEE Trans. Inf. Theory*, 50(12):3250–3264, 2004.

[Lev84] Leonid A. Levin. Randomness conservation inequalities; information and independence in mathematical theories. *Information and Control*, 61(1):15–37, 1984.

[Lev03] Leonid A. Levin. The tale of one-way functions. *Probl. Inf. Transm.*, 39(1):92–103, 2003.

[LM93] Luc Longpré and Sarah Mocas. Symmetry of information and one-way functions. *Inf. Process. Lett.*, 46(2):95–100, 1993.

[LMN93] Nathan Linial, Yishay Mansour, and Noam Nisan. Constant depth circuits, Fourier transform, and learnability. *J. ACM*, 40(3):607–620, 1993.

[LMP25] Yanyi Liu, Noam Mazor, and Rafael Pass. On white-box learning and public-key encryption. In *Innovations in Theoretical Computer Science Conference* (ITCS), 2025.

[LO22] Zhenjian Lu and Igor C. Oliveira. Theory and applications of probabilistic Kolmogorov complexity. *Bull. EATCS*, 137, 2022.

[LOZ22] Zhenjian Lu, Igor C. Oliveira, and Marius Zimand. Optimal coding theorems in time-bounded Kolmogorov complexity. In *International Colloquium on Automata, Languages, and Programming* (ICALP), pages 92:1–92:14, 2022.

[LP20] Yanyi Liu and Rafael Pass. On one-way functions and Kolmogorov complexity. In *Symposium on Foundations of Computer Science* (FOCS), pages 1243–1254, 2020.

[LP21] Yanyi Liu and Rafael Pass. On the possibility of basing cryptography on EXP≠BPP. In *Annual Cryptology Conference* (CRYPTO), pages 11–40, 2021.

[LP22] Yanyi Liu and Rafael Pass. On one-way functions from NP-complete problems. In *Conference on Computational Complexity* (CCC), pages 36:1–36:24, 2022.

[LR05] Troy Lee and Andrei E. Romashchenko. Resource bounded symmetry of information revisited. *Theor. Comput. Sci.*, 345(2-3):386–405, 2005.

[LS24] Zhenjian Lu and Rahul Santhanam. Impagliazzo's worlds through the lens of conditional Kolmogorov complexity. In *International Colloquium on Automata, Languages, and Programming* (ICALP), pages 110:1–110:17, 2024.

[LV19] Ming Li and Paul M. B. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications, 4th Edition*. Texts in Computer Science. Springer, 2019.

[LW86] Nick Littlestone and Manfred Warmuth. Relating data compression and learnability. Technical report, University of California Santa Cruz, 1986.

[LW95] Luc Longpré and Osamu Watanabe. On symmetry of information and polynomial time invertibility. *Inf. Comput.*, 121(1):14–22, 1995.

[McM56] B. McMillan. Two inequalities implied by unique decipherability. *IRE Transactions on Information Theory*, 2(4):115–116, 1956.

[MP24] Noam Mazor and Rafael Pass. Search-to-decision reductions for Kolmogorov complexity. *Electron. Colloquium Comput. Complex.*, 3:1–22, 2024.

[MP25] Noam Mazor and Rafael Pass. Guest column: On cryptography and metacomplexity. *SIGACT News*, 56(2):63–101, June 2025.

[MY16] Shay Moran and Amir Yehudayoff. Sample compression schemes for VC classes. *Journal of the ACM*, 63(3):1–10, 2016.

[Nan21] Mikito Nanashima. A theory of heuristic learnability. In *Annual Conference on Learning Theory* (COLT), 2021.

[NDRT13] Nagarajan Natarajan, Inderjit S. Dhillon, Pradeep K. Ravikumar, and Ambuj Tewari. Learning with noisy labels. In *Advances in Neural Information Processing Systems*, volume 26, pages 1196–1204, 2013.

[NW94] Noam Nisan and Avi Wigderson. Hardness vs randomness. *J. Comput. Syst. Sci.*, 49(2):149–167, 1994.

[Oli19] Igor C. Oliveira. Randomness and intractability in Kolmogorov complexity. In *International Colloquium on Automata, Languages, and Programming* (ICALP), pages 32:1–32:14, 2019.

[OS17] Igor C. Oliveira and Rahul Santhanam. Conspiracies between learning algorithms, circuit lower bounds, and pseudorandomness. In *Computational Complexity Conference* (CCC), pages 18:1–18:49, 2017.

[OS18] Igor C. Oliveira and Rahul Santhanam. Pseudo-derandomizing learning and approximation. In *International Workshop on Randomization and Approximation Techniques* (RANDOM), 2018.

[OW93] Rafail Ostrovsky and Avi Wigderson. One-way functions are essential for non-trivial zero-knowledge. In *Israel Symposium on Theory of Computing Systems* (ISTCS), pages 3–17, 1993.

[Pas26] Rafael Pass. The value of a prompt: An LLM-relative Kolmogorov-complexity approach, 2026.

[Pic25] Ján Pich. Learning algorithms from circuit lower bounds. *Comput. Complex.*, 34(1):6, 2025.

[PV88] Leonard Pitt and Leslie G. Valiant. Computational limitations on learning from examples. *Journal of the ACM*, 35(4):965–984, October 1988.

[PVY99] Doron Peled, Moshe Y. Vardi, and Mihalis Yannakakis. Black box checking. In *Formal Methods for Protocol Engineering and Distributed Systems* (FORTE/PSTV), pages 225–240, 1999.

[PW25] Yury Polyanskiy and Yihong Wu. *Information theory: From coding to learning*. Cambridge University Press, 2025.

[Raz87] Alexander A. Razborov. Lower bounds on the size of bounded depth circuits over a complete basis with logical addition. *Mathematical notes of the Academy of Sciences of the USSR*, 41:333–338, 1987.

[Ris78] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, September 1978.

[Ris84] Jorma Rissanen. Universal coding, information, prediction, and estimation. *IEEE Transactions on Information Theory*, 30(4):629–636, July 1984.

[RR97] Alexander A. Razborov and Steven Rudich. Natural proofs. *J. Comput. Syst. Sci.*, 55(1):24–35, 1997.

[RWC+19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI*, 2019. Accessed: 2024-11-15.

[Sch90] Robert E. Schapire. The strength of weak learnability. *Mach. Learn.*, 5:197–227, 1990.

[Ser25] Rocco A. Servedio. The probably approximately correct learning model in computational learning theory. *CoRR*, abs/2511.08791, 2025.

[SF14] Robert E. Schapire and Yoav Freund. *Boosting: Foundations and Algorithms*. MIT Press, 2014.

[SH96] Jürgen Schmidhuber and Stefan Heil. Sequential neural text compression. *IEEE Transactions on Neural Networks*, 7(1):142–146, 1996.

[Sha48] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

[Sim23] Meta-Complexity. Research Program at the Simons Institute for the Theory of Computing, UC Berkeley, Spring 2023. https://simons.berkeley.edu/programs/Meta-Complexity2023.

[Smo87] Roman Smolensky. Algebraic methods in the theory of lower bounds for boolean circuit complexity. In *Symposium on Theory of Computing* (STOC), pages 77–82, 1987.

[Sol64a] Ray J. Solomonoff. A formal theory of inductive inference. Part I. *Information and Control*, 7(1):1–22, March 1964.

[Sol64b] Ray J. Solomonoff. A formal theory of inductive inference. Part II. *Information and Control*, 7(2):224–254, June 1964.

[SSBD14] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.

[ST17] Rocco A. Servedio and Li-Yang Tan. What circuit classes can be learned with non-trivial savings? In *Innovations in Theoretical Computer Science Conference* (ITCS), pages 30:1–30:21, 2017.

[SUV17] Alexander Shen, Vladimir A. Uspensky, and Nikolay Vereshchagin. *Kolmogorov complexity and algorithmic randomness*. American Mathematical Society, 2017.

[TVZ05] Luca Trevisan, Salil P. Vadhan, and David Zuckerman. Compression of samplable sources. *Comput. Complex.*, 14(3):186–227, 2005.

[TZJ+16] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction APIs. In *USENIX Security Symposium* (USENIX Security), pages 601–618, 2016.

[Vad12] Salil P. Vadhan. Pseudorandomness. *Found. Trends Theor. Comput. Sci.*, 7(1-3):1–336, 2012.

[Vad17] Salil P. Vadhan. On learning vs. refutation. In *Conference on Learning Theory* (COLT), pages 1835–1848, 2017.

[Val84] Leslie G. Valiant. A theory of the learnable. *Commun. ACM*, 27(11):1134–1142, 1984.

[VL00] Paul M. B. Vitányi and Ming Li. Minimum description length induction, Bayesianism, and Kolmogorov complexity. *IEEE Transactions on Information Theory*, 46(2):446–464, March 2000.

[Vol16] Ilya Volkovich. A guide to learning arithmetic circuits. In *Conference on Learning Theory* (COLT), pages 1540–1561, 2016.

[VZ12] Salil P. Vadhan and Colin Jia Zheng. Characterizing pseudoentropy and simplifying pseudorandom generator constructions. In *Symposium on Theory of Computing* (STOC), pages 817–836, 2012.

[WB68] C. S. Wallace and D. M. Boulton. An information measure for classification. *The Computer Journal*, 11(2):185–194, August 1968.

[Wil14] Ryan Williams. Nonuniform ACC circuit lower bounds. *J. ACM*, 61(1):2:1–2:32, 2014.

[WZTE18] Tongzhou Wang, Jun-Yan Zhu, Antonio Torralba, and Alexei A. Efros. Dataset distillation. *CoRR*, abs/1811.10959, 2018.

[Xia09] David Xiao. *New perspectives on the complexity of computational learning, and other problems in theoretical computer science*. Phd thesis, Princeton University, 2009.

[Xia10] David Xiao. Learning to create is as hard as learning to appreciate. In *Conference on Learning Theory* (COLT), pages 516–528, 2010.

[Yao82] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In *Symposium on Foundations of Computer Science* (FOCS), pages 80–91, 1982.

[YMT23] Yibo Yang, Stephan Mandt, and Lucas Theis. An introduction to neural data compression. *Foundations and Trends in Computer Graphics and Vision*, 15(2):113–200, 2023.

[ZL70] Alexander K. Zvonkin and Leonid A. Levin. The complexity of finite objects and the algorithmic concepts of randomness and information. *UMN (Russian Math. Surveys)*, 25(6):83–124, 1970.

[ZL77] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Trans. Inf. Theory*, 23(3):337–343, 1977.

[ZL78] Jacob Ziv and Abraham Lempel. Compression of individual sequences via variable-rate coding. *IEEE Trans. Inf. Theory*, 24(5):530–536, 1978.

The Machine Learning Column

BY

Pablo Barceló

Pontificia Universidad Católica de Chile
Avda. Vicuña Mackenna 4860
Santiago, Chile
pbarcelo@uc.cl

AND

David Saulpic

Institut de Recherche en Informatique Fondamentale (IRIF)
CNRS & Université Paris Cité
8 Place Aurélie Nemour
75013 Paris, France
david.saulpic@irif.fr

The Computational Power of Transformers: Interview with Anthony Widjaja Lin

Foreword

For this edition of the Bulletin, Pablo and I interviewed Anthony Widjaja Lin back in June. The TCS world changed quite dramatically since then, and it would seem odd to publish a column speaking about TCS and ML without any mention of it.

First revolution, Pablo was appointed Director of the Chilean National Center for AI, and is therefore stepping down from the bulletin – I wish him luck in his new role, and a thousand thanks for the work we have done together.

Besides this new appointment, the theoretical research community is in crisis. The new AI models make theorem proving easier than ever, and the consequences in our community are endless: the number of papers written and submitted increases exponentially (even faster than it used to) and famous open problems are closed every day: the research time seems to contract. This in turns affects our practice. It is harder than ever to keep up-to-date with the progress made on our favorite areas, to publish papers that respects our former scientificness criterion, and to prove theorems the old way – a reason why we enjoyed our job. And those changes raise fears: of being scooped by AI-powered competitors or even by AI agents scrolling the web for open problems to be solved[1], which encourages to keep our research private; of the prospects of a mathematical practice we may not like; of our research becoming dependent on big tech companies; of the impact of this, on nature and society; of being simply not as good as AI, hence useless; or, of governments being convinced of the later, which would be a good reason for cutting theory jobs.

The mathematical community is facing similar questions, and is reacting by communicating a lot online. Members of this community write text [2], participate in blogs, engage in discussion on social media such as mathstodon. A particular

[1] see `https://yangpliu.github.io/repository.html`

[2] see `https://mathandai.org/`, which collected thousands of endorsements, or `https://webusers.imj-prg.fr/~isabelle.gallagher/AIg.pdf` (in French)

website was started, collecting and publishing opinions every day (Proofs and prompts); and a scientific organization was created, the Association for Human Mathematics.

On the other hand, the TCS community seems more silent. I am merely aware of Oded Goldreich's post[3], that of Pravesh Kothari, Raghu Meka, and Prasad Raghavendra[4], and of C. Seshadhri[5]. Nonetheless, we are affected, and there is a concerning number of people dropping theory studies, facing existential concerns, depression or anxiety – particularly people with a non-permanent job, but not only.

I am therefore calling for more discussions, more shared opinions, more debate on AI, our use of it, our vision of the job, and why humans are important (or not). We need, at least, venues for collective discussions, and to take our time thinking what's coming.

The EATCS bulletin is not the place for this, because of its periodicity. If nothing better appears in the meantime, I propose to list in the next edition all the contribution that I will be aware of – please, let me know if you read or write something! – but I would love to see nicer initiatives.

I am looking forward to reading your essays, ideas, impressions, or feelings, and I hope you will enjoy Anthony's interview.

David Saulpic

PS: AI is currently the main crisis in our small community, but the world is facing several others that will end up impacting us as well, such as wars or the climate crisis. We should also spend time and energy on anticipating, and in not accelerating them: for this too we need discussions.

[3] https://www.wisdom.weizmann.ac.il/~oded/on-proofs-t2.html
[4] https://scottaaronson.blog/?p=10116
[5] https://users.soe.ucsc.edu/~sesh/essays/ai-and-math.html

1 Introduction

Few technologies have reshaped computing as quickly as the transformer. Introduced in 2017 [6], it is the architecture beneath today's large language models, and with them the wave of systems now writing code, drafting text, and answering questions across the sciences. Yet for all this success, a basic question remains stubbornly open: what can these models actually do, and what lies beyond their reach? This is precisely where theoretical computer science enters. A transformer, stripped to its essentials, is a machine that reads strings and makes decisions—an object of exactly the kind that logicians and automata theorists have studied for decades. By asking which formal languages a transformer can recognize, and matching them against classical yardsticks like circuit complexity and first-order logic, researchers have begun to draw a rigorous map of the model's powers and its blind spots. The results are often surprising: a network that can write a sonnet may be provably unable to count whether a letter appears an even number of times. Beyond intellectual curiosity, the stakes are practical, touching trust, verification, and our hope of one day guaranteeing how these systems behave.

Few people are better placed to guide us through this territory than Anthony Widjaja Lin. Based at the University of Kaiserslautern-Landau and the Max-Planck Institute for Software Systems, Lin built his career in the classical heartland of theoretical computer science—logic, automata, automated reasoning, and the verification of software, where he is known for work on string constraint solvers and model checking. In recent years he has turned that arsenal toward machine learning, becoming one of the architects of the emerging theory of transformer expressivity: he has helped show which logics transformers can capture [1], uncovered a startling link between their counting ability and the undecidability of Diophantine equations [4], proved that they describe concepts with extraordinary compactness (at the cost of making them notoriously hard to verify) [2], and most recently established that chain-of-thought transformers with soft attention are Turing-complete [3]. Equally at home with a deep theorem and with what it means for the systems we actually deploy, he is an ideal voice for a conversation at the crossroads of theory and modern AI.

2 Technical Background

The interview turns on a single question: *what can transformers provably compute, and what lies beyond them?* The fruitful way to make this precise is through formal language theory. A transformer reads an input string and either accepts or rejects it; the set of accepted strings is its *language*. By comparing these languages against well-understood yardsticks—regular languages, circuit-complexity classes,

fragments of logic—one can pin down both the powers and the limits of the architecture. This note fixes the minimum vocabulary needed to follow that discussion.

Strings, languages, and counting Let Σ be a finite *alphabet* of tokens. A *string* is a finite sequence $w = a_1 \cdots a_n$ with $a_i \in \Sigma$, and a *language* is a set $L \subseteq \Sigma^*$ of strings. Two recurring touchstones are the *counting languages*

$$\text{PARITY} = \{w \in \{a,b\}^* : |w|_a \text{ is even}\}, \qquad \text{MAJ} = \{w \in \{a,b\}^* : |w|_a \geq |w|_b\},$$

where $|w|_a$ denotes the number of a's in w. Both are *permutation-invariant*: membership depends only on letter counts, not on order. They are deceptively simple, yet they sharply separate the models below.

Transformer encoders We describe a transformer encoder semantically. A *token embedding* $\text{em} : \Sigma \to \mathbb{R}^d$ maps each symbol to a vector, extended to strings coordinatewise. The transformer is then a composition of *length-preserving* maps $(\mathbb{R}^l)^* \to (\mathbb{R}^h)^*$ built from two ingredients: (i) *position-wise* feed-forward functions (compositions of affine maps and $\text{ReLU}(x) = \max\{0, x\}$, applied to each vector independently), and (ii) *attention layers*, which let positions exchange information. To recognize a language one fixes an output vector $\mathbf{t}$ and accepts w iff $\langle \mathbf{t}, \mathbf{v} \rangle > 0$, where $\mathbf{v}$ is the final vector at the last position.

Definition 1 (Attention layer)**.** *Given input vectors $\mathbf{x}_1, \ldots, \mathbf{x}_n$ and affine maps A, B, C, the* attention score *of query i at key j is $s_{ij} = \langle A\mathbf{x}_i,\ B\mathbf{x}_j \rangle$. A* normalizer *turns the scores $(s_{ij})_j$ into weights $w_i(j)$, and the layer outputs*

$$\mathbf{y}_i = C(\mathbf{x}_i,\ \mathbf{v}_i), \qquad \mathbf{v}_i = \sum_{j=1}^{n} w_i(j)\, \mathbf{x}_j.$$

The choice of normalizer fixes the model:

- **Softmax (SMAT).** $w_i(j) \propto e^{s_{ij}}$. This is the mechanism used in practice, but the exponentials make it hard to analyze.
- **Unique hard attention (UHAT).** All weight is placed on the single *leftmost* position j maximizing s_{ij}.
- **Average hard attention (AHAT).** The weight is spread *uniformly* over all positions j that maximize s_{ij}.

UHAT and AHAT are tractable idealizations of softmax; AHAT, in particular, can average (and hence count), which softmax approximates well.

Positional information Without positional cues, attention treats its input as a bag of tokens and can only recognize permutation-invariant languages. Order is recovered in two ways: a *positional encoding* (PE) $p : \mathbb{N} \times \mathbb{N} \to \mathbb{R}^d$ added to the embeddings, or *causal masking*, where position i attends only to $j \leq i$. The label *NoPE* marks a transformer with *no* positional encoding. As the interview stresses, this seemingly minor knob is decisive: it is exactly what lets some models recognize PARITY, and a single fixed *relative* PE is enough to push chain-of-thought transformers to Turing-completeness.

Circuit complexity. AC^0 is the class of languages decided by families of constant-depth, polynomial-size Boolean circuits with unbounded-fan-in AND/OR/NOT gates; TC^0 adds *majority* gates, which suffice to encode arithmetic and counting. A landmark fact is that PARITY $\notin AC^0$, whereas MAJ $\in TC^0 \setminus AC^0$. These classes upper-bound the models: UHAT lies (strictly) inside AC^0, and AHAT inside TC^0.

Logic on strings. A string is viewed as a structure with a linear order $<$ on positions and a unary predicate Q_a marking the positions labelled a. First-order logic (FO) over this structure defines exactly the *star-free* languages, and—by *Kamp's theorem*—is equivalent to Linear Temporal Logic (LTL), whose formulas are read as sets of positions. Enriching FO with arbitrary *unary* ("monadic") numerical predicates gives FO[Mon], a class strictly between FO and AC^0. Adding to LTL terms that *count* how often a subformula holds to the left or right yields *Counting LTL*, which can express MAJ and other counting properties.

The landscape in one breath UHAT sits strictly inside AC^0 and captures all of FO[Mon] (hence every regular language in AC^0), but cannot recognize PARITY [1]. AHAT reaches into TC^0 and captures Counting LTL, so it *can* recognize PARITY and MAJ—though, strikingly, whether it does so can hinge on the positional encoding. Finally, with *chain-of-thought* (feeding the model's own outputs back as inputs), transformers become Turing-complete [5]. These are the coordinates against which the rest of the conversation is plotted.

3 The Interview

David Saulpic: Transformers have transformed AI and our field, and they work astonishingly well, without anyone having proven a theorem about them. So, why should we care about theory? What should we care about proving what they can and cannot do?

Anthony Lin: Firstly, I should mention that this is a field I have been getting into only in the last three years, so what I say should not be taken as authoritative. For me, the most important thing is simply the joy of understanding. I already learned this many years ago with neural networks — it is very surprising to see intuitive connections between neural networks and things I had been working on, like logic, automata, and circuit complexity.

To answer your question more directly: how can TCS, logic, and automata contribute to machine learning? My general answer is that TCS provides a solid mathematical foundation where you can prove what models can or cannot do, and by what neural mechanism. This is the role TCS can play. In particular, it could be useful when you want to understand specific phenomena — for instance, why some things can or cannot be trained so easily. You may also want to develop more explainable models that are better understood by people, and once you have accurate, explainable models, you can develop ways to reason about them algorithmically at a post-learning stage.

I would add a caveat. Despite the massive progress over the last seven years or so, there is still a huge chasm between theory and practice, especially at the scale of large language models. We do not have a theory that can reliably predict why LM agents can successfully code some programs and what kinds of problems they would struggle with. Whether what we do can scale up to that level remains an open question. Terence Tao said something interesting: the mathematics behind LLMs is quite simple, but no one really understands why they work. I find this paradox quite interesting.

Pablo Barceló: I think it is not surprising, but it is very interesting to see that LLMs are models of computation. It is then very natural to try to understand what kind of computation they can pursue. Why is that important? Because then we can understand their limitations and capabilities much better. I think we currently understand the limitations somewhat better than the capabilities, and this has been instrumental in trying to design new architectures that are more capable of reasoning.

AL: And of course it is also important to go back to the history: this was already discussed in early work on neural networks as a model of computation, so it is very natural to try to understand what you can and cannot do with this model.

DS: Pablo, you said that this understanding of limitations has already led to new architectures?

PB: From a theoretical point of view, yes. They are not being implemented at the moment, or only in a proof-of-concept way. But yes, we understand what is needed for transformers to be able to reason better or to compose functions. Transformers are very poor at composing functions — they require many steps

for that. People are now trying to understand how they can be enriched so that they can compose functions more effectively, which matters because composing functions is fundamental to reasoning efficiently.

DS: So this is part of an answer to what TCS can bring to practitioners of machine learning. But when you use an LLM, it seems they are already reasoning. We understand the limitations, but it is not clear they manifest in practice. What is the gap between theory and practice? Typically, the PARITY problem is hard for a transformer in theory. Is it hard in practice as well?

AL: I think the first versions of GPT struggled with this, but it no longer happens. What I understand is that when you query a modern LLM on parity, it will write a Python program that does the calculation, using a kind of chain of thought to overcome the theoretical limitation. I would still think that if you somehow bypass this — for example by asking to count the number of balls in an image — it might actually struggle.

DS: For the record, the last time I tried, three months ago, LLMs answered correctly the number of R's in *strawberry*, but not in *raspberry*.

AL: This has to do with tokenization: the sentence is split into smaller components, so when you try to count the R's, some of them simply disappear.

PB: In a way, it is surprising that the vanilla version of transformers cannot really count, and that you need to add something. The vanilla version is surprisingly weak, but when you add some form of reasoning via chain of thought, you regain this ability — at high computational cost.

AL: There are quite a lot of things at play. Some of the challenge during inference also comes from computing the attention matrix, which is $O(n^2)$. For RNNs this is linear, which is part of why people are now talking about bringing RNNs back. But there is also the additional cost of the sequential computation steps in chain of thought.

DS: Going back to parity: it is perhaps surprising that it is a hard problem, given what transformers are doing. Can you explain why parity is so hard, and what is its particular role in the field?

AL: When people try to understand the ability of transformers, they can do so experimentally, but they can also develop theory that explains the phenomenon. On the theory side, a number of different formalizations of transformers have studied what kinds of tasks they can solve. These correspond to concepts that can be expressed in first-order logic, linear temporal logic, and related formalisms. It is a classic result that parity is not in AC^0 — and this is also connected to circuit complexity. I think one of the first papers to establish this connection for transformers was by Michael Hahn. This is basically why parity is hard: it

is connected to the kinds of concepts expressible in first-order logic and linear temporal logic, and we can tap into the classical results from theoretical computer science to infer what transformers can and cannot do.

One of the answers to dealing with the parity problem is chain of thought, which you can think of as the transformer talking to itself. When you supplement a transformer with that power, it can do more.

DS: In your recent work, positional encodings appear again and again as the decisive ingredient: without them there is no parity, and a single relative positional encoding — fixed and task-independent — is enough to reach Turing completeness. What is so special about positions? Why does this apparently minor detail change the model's power so dramatically?

AL: Let me start with a general overview. You can think of transformers as a kind of parallel machine, but without any order — they process everything in parallel, so they do not know which token comes first. Tokens are treated as a bag, without sequence information. This is why you need some kind of positional information, so that the transformer can distinguish what comes first or before. That is the first answer.

PB: And this was actually noticed as a weakness of transformers in the early practical work. It was practitioners who included positional encodings — it is not a theoretical artifact introduced to study computational power, but something that turned out to be necessary in practice.

AL: Absolutely. There are papers that have looked at what kinds of positional encodings are needed in practice, and there is not just one class. Now, without positional encoding, what can you actually do? We know you cannot do parity. It turns out you can actually capture polynomial constraints even without positional encodings — we managed to prove this even without chain of thought.

Going back to your question about Turing completeness: for a one-letter alphabet — which is of course a very artificial setting — with chain of thought, transformers are already Turing complete without any positional encoding. But if you go beyond a one-letter alphabet, the main problem is that without positional encoding you cannot refer back to the inputs, and then transformers, even with chain of thought, are not Turing complete. The main technique is related to the fact that you need to refer back to inputs in order to perform any reduction. What is nice about our result is that with a single relative positional encoding — one that we can explicitly construct and that is quite simple, even if it is not known to be learnable — the model becomes Turing complete.

DS: One of your results connects the counting power of transformers — without positional encoding — to Diophantine equations, and therefore to undecidability. How do you get from an attention mechanism to something akin to Hilbert's

tenth problem, and what does it mean that analyzing certain transformers is formally impossible?

AL: I can tell you the history. I have done quite a lot of work on logic and automata related to counting — it is one of the things I really enjoy. And if you have a hammer, everything becomes a nail; that is kind of how we got into this. One of the main questions in counting is whether you can do semi-linear counting, whether you can capture linear equations and linear constraints. We were looking at this question for transformers: what kind of counting can they do?

For the result you mentioned, we were very surprised to find that we can capture not just linear or nonlinear counting, but in fact all polynomials. This essentially allows you to encode Hilbert's tenth problem into the analysis of transformers, making their verification undecidable. The proof itself is not so complicated. The attention mechanism does averaging, so you can talk about frequencies. When you want to count, you cannot count directly, but you can simulate counting using frequencies. Using certain gadgets, we showed that we can simulate multiplication and addition this way, and then encode Hilbert's tenth problem.

PB: You also have to be careful about the type of attention you are using. With what is called unique attention — where you attend only to the position that maximizes the value of interest — the expressive power of transformers is very limited. But when you allow average attention, where you average over all positions that maximize the value, the counting power becomes much more powerful, even without positional encodings.

AL: Yes. But interestingly, even with average attention and no positional encoding, you still cannot do parity. There is a classical result showing that if you allow bidirectional attention, with both forward and backward masking, you can in fact express parity. It is expressible, though I do not know if it is trainable.

DS: That is a good distinction to draw out. Can you explain the difference between expressible and trainable, and how it shows up in your work?

AL: This is indeed a very important point. Expressibility means that there exists some transformer — perhaps one we do not have yet — that can solve a given task exactly. Trainability is about whether, given data and running stochastic gradient descent via backpropagation, we can actually converge to a model that solves the task. Of course, people have been formalizing notions of trainability for a long time — PAC learnability, exact learnability in Dana Angluin's sense, and many other notions. That is the fundamental difference.

DS: Can you explain briefly the transformer model you formalize and study?

PB: The model of transformers we use is an idealized version of what is done in practice — one that is easier to study theoretically — but we are trying to bring

it ever closer to practice. The core idea is that when you analyze a sentence, the meaning of each token depends on all the other words in the sentence and their relative positions. This is unlike an image, where a pixel is largely determined by adjacent pixels. Here the meaning of a word depends on all other words and their relative positions. The question is how you attend to other words in order to understand your own meaning. This is also what brings the quadratic complexity: for each word, you have to attend to all other words, measure their importance, and produce a summary of that attention.

AL: One thing worth remarking is that for a long time, the standard way to process text with neural networks was sequential. The problem was that long-range dependencies were always very difficult to capture using RNNs: processing from left to right, what happens at the end of the string can depend on something far back at the beginning — this is the vanishing gradient problem. What attention overcomes this by explicitly looking at all combinations: you compute the attention matrix, which is $O(n^2)$ rather than $O(n)$, but fully parallelizable.

PB: In a way, it is a brute force solution to the problem.

AL: But a parallelizable one.

DS: You proved that transformers describe certain concepts extraordinarily compactly — doubly exponentially more so than automata. The price of that compactness is that verification is EXPSPACE-complete. Are we facing some trade-off, perhaps unavoidable, between how powerful transformers are and our ability to audit or trust them?

AL: That is right, and I am still working out all the corollaries. One corollary is indeed that verification is difficult. But there are others. For example, some tasks that you want transformers to capture may require a very large amount of data to train. And there is another corollary: some concepts can be expressed more succinctly using transformers than by RNNs — exponentially more succinctly. We are running experiments to validate this theoretical claim, and we are preparing a journal paper on this. I am happy to report that after this result, quite a lot of people were excited and building upon it, including some I would not have expected.

One thing to note is that in practice you deal with finite precision — floating point numbers. This renders everything, to some extent, decidable, since transformers are no longer super-powerful under finite precision. But even then, some things can still be expressed very succinctly, which is one of the corollaries I am still working through.

DS: From a learnability perspective, is that something you have taken into account, or are the results purely about expressive power?

AL: So far only expressibility. We are running experiments to validate this, and I can report back in a few months.

DS: And a final question: one of your most recent results proves that softmax chain-of-thought transformers are Turing complete and trainable, by simulating counter machines rather than Turing machines directly. Looking ahead, where does the limit of softmax attention really lie? Which open problem would you most love to solve, and what can the whole arsenal of verification and string-solver techniques you come from bring to AI?

AL: This is difficult to answer briefly. On the softmax chain-of-thought result: our simulation uses counter machines, and counter machines are not very efficient — you would need exponential time to solve something that Turing machines can solve efficiently. So when you account for complexity, things are still quite unclear.

Beyond that, there are open questions about the cost of computing the attention matrix, which is motivating renewed interest in state-space models — essentially a return to RNNs with modifications, removing nonlinearities to simplify the architecture. These have been shown to scale and to allow training of language models, and people are also asking how they can be combined with transformers.

On the verification side, we have only recently started applying techniques from that world. Even when a problem is undecidable — as in the case where you can encode Hilbert's tenth problem — in formal verification we do not stop at undecidability. Termination is undecidable, but people still built tools like Terminator. We have been applying SMT-based techniques to analyze transformers, and this has turned out to be quite successful.

The most honest summary is that we have only scratched the surface. The field gives you many questions you would never have thought about otherwise. The classical tools from TCS never match perfectly — you never get an exact characterization, it is never exactly TC^0 or exactly any known class, always something in between. But you use those techniques to guide your understanding, and this is precisely what is exciting about the field. I hope more people from the TCS community will start working on it.

PB: There is also this very interesting and surprising intersection with the linguistics community.

AL: Absolutely — people like John Rawski and David Chiang, who are linguists but also logicians by training. I am now connected with so many different people I would not have thought to reach out to before. The field moves extremely fast, but it is also very collaborative, and there are many different perspectives to draw on.

DS: To conclude: you mentioned the field is moving extremely fast, and yet

you are using tools from the 1970s and 80s — constant-depth circuits, automata, counter machines — and they turn out to be currently the best lens for the most modern AI models. Where are they coming back to life, and what is the community forming around this?

AL: I can only philosophize here. I think we analyze things based on what we know. We want to understand things epistemologically based on the theory we have. But you see that not everything falls neatly into place: it is never exactly TC^0, never exactly $\mathsf{GAP\text{-}NC}^1$. You use these techniques to guide and understand the problems, but the classical tools never provide a perfect fit. What they do provide is new notions and exciting open questions that can inspire future people in the TCS community. I feel that not so many people from our area have been working on this yet, but more and more are starting to notice it — and I hope that continues.

References

[1] Pablo Barceló, Alexander Kozachinskiy, Anthony Widjaja Lin, and Vladimir V. Podolskii. Logical languages accepted by transformer encoders with hard attention. In *ICLR*, 2024.

[2] Pascal Bergsträßer, Ryan Cotterell, and Anthony W. Lin. Transformers are inherently succinct. *CoRR*, abs/2510.19315, 2025.

[3] Hongjian Jiang, Michael Hahn, Georg Zetzsche, and Anthony Widjaja Lin. Softmax transformers are turing-complete. *CoRR*, abs/2511.20038, 2025.

[4] Chris Köcher, Alexander Kozachinskiy, Anthony Widjaja Lin, Marco Sälzer, and Georg Zetzsche. Nope: The counting power of transformers with no positional encodings. *CoRR*, abs/2505.11199, 2025.

[5] Jorge Pérez, Pablo Barceló, and Javier Marinkovic. Attention is turing-complete. *J. Mach. Learn. Res.*, 22:75:1–75:35, 2021.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NIPS*, pages 5998–6008, 2017.

The Logic in Computer Science Column

BY

Anuj Dawar

Department of Computer Science and Technology
University of Cambridge, Cambridge, CB3 0FD, UK
anuj.dawar@cl.cam.ac.uk

In this issue, I am delighted to present a contributed column by Harry Lewis. The author needs no introduction as his contributions to logic and the theory of computation over decades will be well known to readers. I myself studied his textbook on *Elements of the Theory of Computation*, written with Christos Papadimitriou, when I was an undergraduate student, as did a generation of computer scientists. In the present contribution, he revisits a combinatorial problem which was introduced by him and Stål Aanderaa in the context of studying the classical decision problem for prefix classes of first-order logic. The article gives a clean and modern exposition and the proof of undecidability. I trust the readers of this column will find it as much of a delight to read as I did.

BEATCS no 150

LINEAR SAMPLING REVISITED

IN MEMORY OF STÅL AANDERAA (1931–2026)

Harry R. Lewis

Gordon McKay Professor of Computer Science, *emeritus*, Harvard University

1 Preface

When first Church [1936] and, soon after, Turing [1936] established the computational undecidability of the *Entscheidungsproblem*, the system to which they were referring was the first-order predicate calculus as formulated by Hilbert and Ackermann [1928]. Even before Church and Turing had formalized computability in order to resolve the main question, others had chipped away at it by identifying solvable subcases. Bernays and Schönfinkel [1928] established the decidability of satisfiability for the class of prenex formulas with quantifier prefixes of the form $\exists^*\forall^*$—it was for a variation of this problem that Frank Ramsey [1930] then introduced the combinatorial theory that bears his name. The program of classifying prefix classes was completed when Kahr et al. [1962] established that satisfiability even of the $\forall\exists\forall$ class is undecidable: Except for the $\exists^*\forall^*$, $\exists^*\forall\exists^*$, and $\exists^*\forall\forall\exists^*$ classes, all the prefix classes are undecidable. (This is for first-order logic without function signs or the identity sign; the situation changes if either is allowed.) With the $\forall\exists\forall$ case settled, the search was on to classify subclasses of the $\forall\exists\forall$ class. Suppose the prefix is $\forall x\exists u\forall y$ and only monadic and dyadic (1- and 2-place) predicate letters are allowed. If the dyadic letters are allowed to appear only with certain arrangements of x, u, and y, which combinations result in solvable classes, and are the rest all reduction classes (of maximal r.e. degree)?

More than fifty years ago, in Aanderaa and Lewis [1974], Norwegian mathematician Stål Aanderaa and I completed this classification of $\forall\exists\forall$ subclasses. The bottom line is that any combination including both Pxy and Pyu (or the equivalent pair Pyx and Puy) yields a reduction class, and all the others are solvable. (In fact, monadic letters plus only a single dyadic predicate letter P suffice for the unsolvability result.) To establish this, we introduced a combinatorial problem we dubbed Linear Sampling, proved that it was undecidable, and reduced it to the logical problem of interest.

Aanderaa and Lewis [1974] was a collaboration between a combinatorial ge-

nius and a stumbling graduate student serving as scribe. The proofs are complex, incomplete, and poorly motivated. Stål died on January 26 of this year. In a spirit of humility prompted by memories of his stunning vision and of his warm generosity, I here offer a revised and fuller account of linear sampling. In 1974 we hoped that linear sampling might prove interesting independent of our logic proof. In that spirit, I hope the community will benefit from this improved exposition.

2 Introduction

Linear sampling systems classify two-way infinite tapes, that is, mappings from $\mathbb{Z}$ to a finite alphabet Σ. A linear sampling system *accepts* or *rejects* a particular tape τ depending on whether all specified "samples" from τ meet its constraints. A *sample* is a vector of symbols drawn from cells of τ whose coordinates satisfy numerical constraints specified as part of the system.

To be precise, a *Linear Sampling System* $X = \langle \Sigma, A, B, L \rangle$ is specified by

- a finite set Σ, the tape alphabet;
- a finite subset $A \subset \Sigma^k$ for some $k > 0$;
- a finite subset $B \subset \Sigma^\ell$ for some $\ell > 0$;
- a linear equation L in ℓ variables.

For X to accept a tape, the symbols in every block of k consecutive tape cells must be in A, and every ℓ-tuple of symbols at tape coordinates satisfying L must be in B. That is,

- $\langle \tau(n), \tau(n+1), \ldots, \tau(n+k-1) \rangle \in A$ for all $n \in \mathbb{Z}$, and
- $\langle \tau(n_1), \ldots, \tau(n_\ell) \rangle \in B$ for all $n_1, \ldots, n_\ell \in \mathbb{Z}$ satisfying $L(n_1, \ldots, n_\ell)$.

We'll refer to a k-tuple satisfying the first condition as a *local sample* of the tape, and an ℓ-tuple satisfying the second condition as a *global sample*. We immediately narrow our focus to the linear equation in four variables

$$L(n_1, n_2, n_3, n_4) \quad \equiv \quad n_2 - n_1 = n_4 - n_3.$$

(We briefly consider other linear equations at the end of this paper, and there pose an open question.) So the decision problem can be stated thus: Suppose you are given the two finite sets $A \subset \Sigma^k$ and $B \subset \Sigma^4$. Is there a two-way infinite tape such that (a) the symbols in every block of k consecutive cells constitute a sequence in A and (b) the symbols in any four tape cells such that the distance between the

first two is the same as the distance between the last two comprise a sequence in B.

We refer to (a) as the *local condition* and to (b) as the *global condition*. The global condition relies on so little information about the relative positions of the cells that it may not be evident how to encode into it an undecidable word problem or automaton computation. Nonetheless, as we here show, there is no algorithm to determine from A and B whether a tape exists that satisfies both conditions.

3 The basic ideas

We're going to reduce the origin-constrained domino problem [Wang, 1961] to the linear sampling problem. A *domino system* includes a finite set of oriented, square tiles, that is quadruples whose components are drawn from a finite set of *colors*, plus a designated *origin domino*. The question is whether the first quadrant of the plane can be tiled by placing dominos at nonnegative integer coordinates in such a way that the domino at $\langle 0, 0\rangle$ is the origin domino and the colors of all abutting edges match, that is, for all $i, j \in \mathbb{N}$,

- the color of the right edge of the domino at $\langle i, j\rangle$ is the color of the left edge of the domino at $\langle i + 1, j\rangle$, and
- the color of the top edge of the domino at $\langle i, j\rangle$ is the color of the bottom edge of the domino at $\langle i, j + 1\rangle$.

It is not hard to construct from any Turing machine a set of dominoes that effectively represents in successive rows of any tiling the configuration of the Turing machine at successive instants of time. It follows that the origin-constrained domino problem is undecidable, as a tiling of the entire first quadrant would represent an unending computation by the Turing machine.

Our objective is to embed domino systems in linear sampling systems. If a domino system can tile the first quadrant, then there will be a two-way infinite tape satisfying the constraints of the linear sampling system. Each tape cell will represent the domino in a particular position in the first quadrant of the plane. However, every such domino position will be replicated infinitely often.

We will be working with two specific relatively prime positive integers p and q. Any relatively prime pair would work as long as the numbers are greater than certain thresholds that will emerge in the course of the proof. That said, it will prove convenient for expository purposes to stipulate from the beginning that $p < q$ and in fact that $q = p + 1$.

Let r be any integer ≥ 2; we use r as a variable ranging over $\{p, q\}$. Let's call the number of times r divides $n \in \mathbb{Z}$ the *r-order* of n. That is, the r-order of n is

the largest number e such that r^e divides n evenly. Every integer other than 0 has an r-order, which can also be described as the number of trailing 0s in the base-r notation for n. The r-order of 0 is infinite. Taking $r = 10$ for familiarity, nine out of every ten consecutive positive integers have 10-order 0. Of the numbers between 1 and 100 inclusive, 90 have 10-order 0, 9 have 10-order 1, and one has 10-order 2.

Our plan—which we can't carry through in the simple form we are stating here—is to represent the domino at position $\langle i, j\rangle$ of a putative tiling at all tape coordinates that have p-order i and q-order j. The local condition can be used to force the origin domino onto the tape cells that have order 0 with respect to both p and q. The hard part will be devise a global condition that can extract enough information about the relative orders of different cells to ensure that domino edge colors match up if their coordinates differ by 1 in one dimension and are equal in the other dimension.

One problem with our plan is apparent immediately. Any shift of an accepted tape (addition of a constant to all cell coordinates) will also be accepted, since if $n_2 - n_1 = n_4 - n_3$, then $(n_2 + t) - (n_1 + t) = (n_4 + t) - (n_3 + t)$ for any $t \in \mathbb{Z}$. But this is not the only problem with our plan. It's time to go a level deeper.

4 Perfect and Admissible Tapes

Domino systems will be mapped to linear sampling systems whose tapes can be regarded as having three tracks. That is, the tape alphabet is the cross product of three finite sets of symbols. The first two tracks contain "address" information, which will help the linear sampling system infer information about the relative p- and q-orders of cells' coordinates. The third track will represent the domino itself.

For the moment we'll focus just on an address channel and pick $r = p$ or $r = q$. An *r-tape* is a tape with symbols that are numbers in the range $1, \ldots, r^2 - 1$ incongruent to $0 \pmod r$. In other words, they are two-digit numerals in base r notation with the second digit nonzero. Ideally, in an address track, the two digits in the cell with coordinate n will be the rightmost nonzero digit of the base r notation for n and the immediately preceding digit. For example, taking $r = 10$ again for convenience, the address track would contain 07 if the coordinate were 60700, and would contain 89 if the coordinate were -110, since in 10's complement notation, $-110 = \ldots 999890$).

We'll need some notation for these ideas. If r is understood, we'll write $\omega(n)$ for the r-order of n, adding a subscript for clarity when needed. Similarly, with r understood, $\delta(n) = nr^{-\omega(n)}$, which can be understood as the number represented by the base-r notation for n with trailing 0s dropped.

For $n \neq 0$, $\lambda(n)$ is the remainder ($0 < \lambda(n) < r^2$) when $\delta(n)$ is divided by r^2,

that is, the last two base-r digits of $\delta(n)$. A *perfect r-tape* is then a tape in which for every $n \neq 0$, cell n contains symbol $\lambda(n)$. There are exactly $r^2 - r$ perfect r-tapes, which differ only in the cell at coordinate $n = 0$, which can contain any of the tape symbols.

Unfortunately, no linear sampling system can recognize all and only perfect r-tapes. We will define a broader class of r-tapes, the *admissible* r-tapes. Every perfect tape is admissible. A three-channel tape will be said to be *admissibly addressed* if its first channel is an admissible p-tape and its second channel is an admissible q-tape; similarly, a tape whose address channels are both perfect will be said to be *perfectly addressed*.

We'll show that it is possible to construct a LSS that recognizes all perfect tapes and only admissible tapes. and that any LSS that recognizes an admissibly addressed tape also recognizes a closely related perfectly addressed tape. The information in the address channels of such tape will provide the scaffolding for checking that the domino adjacency conditions are met by the dominos in the data track.

To define admissibility, we need the notion of i-admissibility, where i is a nonnegative integer. An admissible tape is one that is i-admissible for every i. The i-admissible tapes, for increasing values of i, are approximations of perfect tapes of increasing degrees of fidelity. An i-admissible tape can be shifted so that it agrees with a perfect tape on all but one of every r^{i+1} consecutive tape cells, and on the remaining cells, some are guaranteed to be in partial agreement.

To be precise: an r-tape is *i-admissible*, where $i \geq 0$, if it agrees with a shift of a perfect tape on cells of order less than i, and also agrees with this shifted tape on the second digits of cells of order equal to i. That is, τ is i-admissible if there is an integer $t \in \mathbb{Z}$ such that

$$\begin{aligned} &\tau(n+t) = \lambda(n) \quad \text{if } \omega(n) < i, \text{ and} \\ &\tau(n+t) \equiv \lambda(n) \pmod{r} \quad \text{if } \omega(n) = i. \end{aligned} \tag{4.1}$$

Note that if t satisfies these conditions for τ, then so does t plus any multiple of r^{i+1}, since adding r^{i+1} to a number of r-order at most i does not change its r-order.

As stated earlier, an *admissible r-tape* is one that is i-admissible for every i.

We'll define an LSS that recognizes admissible tapes, using an induction to infer $(i+1)$-admissibility from i-admissibility. First, though, let's make the leap from admissible to perfect tapes. A *central segment* of a tape is the finite sequence of symbols between coordinates $-n$ and $+n$ for some $n \geq 0$.

Theorem 1. *An LSS that accepts an admissibly addressed tape ϕ accepts a perfectly addressed tape π with the property that every central segment of π is a segment of ϕ.*

Proof. Suppose that LSS X accepts admissibly addressed tape τ. We'll define an infinite sequence of finite sequences $s_0, s_1, \ldots$, where s_k is of length $2k+1$. Each s_k is a central segment of a perfectly addressed tape, though not necessarily of the same perfectly addressed tape for different values of k. We'll then use König's lemma to construct from these segments a single perfectly addressed tape accepted by X.

Given $k \geq 0$, let i be any integer such that p^i and q^i both exceed k . Let's denote by τ and σ the two address tracks of ϕ. Because both address tracks are i-admissible, there are numbers t and u such that for every $n \in \mathbb{Z}$,

$$\tau(n+t) = \lambda_p(n) \quad \text{if } \omega_p(n) < i \tag{4.2}$$

$$\tau(n+t) \equiv \lambda_p(n) \pmod{p} \quad \text{if } \omega_p(n) = i \tag{4.3}$$

$$\sigma(n+u) = \lambda_q(n) \quad \text{if } \omega_q(n) < i \tag{4.4}$$

$$\sigma(n+u) \equiv \lambda_q(n) \pmod{q} \quad \text{if } \omega_q(n) = i. \tag{4.5}$$

Since p and q are relatively prime, so are p^{i+1} and q^{i+1}, and by the Chinese Remainder Theorem there is an integer v congruent to t (mod p^{i+1}) and to u (mod q^{i+1}). Equations (4.2)–(4.5) remain true when v is substituted for t and u, since (for $r = p$ or q) an i-admissible r-tape remains i-admissible when it is translated by a multiple of r^{i+1}. Now the sequence $s_k = \langle \phi(v-k), \ldots, \phi(v+k) \rangle$ is a central segment of a perfectly addressed tape, since τ and σ are i-admissible and numbers less than or equal to k have p- and q-order less than i.

For each $k \geq 0$, infinitely many of the s_j ($j \geq k$) must agree on a central substring from position $-k$ to $+k$. By König's Lemma there is an infinite string of which every central substring is one of these perfectly addressed substrings, and which therefore is itself perfectly addressed and is accepted by X. □

Theorem 2. *It is possible to construct an LSS that accepts all perfect tapes and only admissible tapes.*

Proof. The value of r is fixed for the remainder of this argument. The construction of the required LSS X_r is straightforward. The local and global conditions are simply the sets of local and global samples of perfect tapes. That is, $X_r = \langle \Sigma, A, B, L \rangle$, where $\Sigma = \{n : 0 < n < r^2 \text{ and } n \not\equiv 0 \pmod{r}\}$, L is, as always, the equation $n_2 - n_1 = n_4 - n_3$, and

$$A = \{\langle \pi(n), \ldots, \pi(n+r-1) \rangle : n \in \mathbb{Z} \text{ and } \pi \text{ is a perfect } r\text{-tape}$$

$$B = \{\langle \pi(n_1), \pi(n_2), \pi(n_3), \pi(n_4) \rangle : \pi \text{ is a perfect } r\text{-tape and } n_2 - n_1 = n_4 - n_3.$$

It follows immediately that X_r accepts all perfect r-tapes. The challenge will be to show that the construction of X_r from samples of perfect r-tapes can, to a

useful extent, be inverted, so that a perfect tape can be constructed from any tape accepted by X_r.

So let τ be any tape accepted by X_r. We show that τ is i-admissible by induction on i. This is a lengthy proof; the base case is Lemma 1 and the induction step is Lemma 2.

Lemma 1. *τ is 0-admissible.*

Proof. When $i = 0$, condition (4.1). that $\tau(n + t) = \lambda(n)$ if $\omega(n) < i$, is vacuously satisfied, so proving 0-admissibility amounts to showing that there is a shift of τ by some number t, $0 \le t < r$, such that

$$\tau(n + t) \equiv n \pmod{r} \text{ for every } n \in \mathbb{Z} \text{ such that } n \not\equiv 0 \pmod{r}. \tag{4.6}$$

Start by looking at $\langle \tau(0), \ldots, \tau(r-1) \rangle$; since τ is accepted, this vector must be in A, and for it to be in A there must be a perfect r-tape π and a displacement u such that $\tau(n) = \pi(n + u)$ for $0 \le n < r$. Since π is perfect, $\pi(n) \equiv n \pmod{r}$ when $0 < n < r$, so $\tau(n - u) = \pi(n) \equiv n \pmod{r}$ when $0 < n < r$ Therefore (4.6) is satisfied with $t = -u$ when $0 \le n < r$.

With (4.6) established for $0 \le n < r$, it suffices to show that if (4.6) holds for any block of r symbols on an accepted tape, then it also holds for the blocks that begin one cell earlier and one cell later. The two cases being symmetrical, we'll consider just the case of a block and the block beginning one cell later.

Consider the second digit (the remainder when divided by r) of the contents of a cell of a perfect r-tape. For any three successive cells, these digits must fit one of these patterns:

(a) $x, x + 1, x + 2$ for some x such that $1 \le x \le r - 3$

(b) $r - 2, r - 1, x$, where $1 \le x \le r - 1$

(c) $r - 1, x, 1$, where $1 \le x \le r - 1$

(d) $x, 1, 2$, where $1 \le x \le r - 1$

The r cases listed here (option (a) covers $r - 3$ different cases) are unambiguous—no triple satisfies more than one of them. For example, if the first component is $r - 1$, the triple might be one of type (c) or of type (d), but since the third component must be 1 or 2, those two cases can be distinguished by looking at the third component. This means that if a local sample from a tape matches an element of A, any three consecutive members of that sequence suffice to determine uniquely $\pmod{r}$ the amount by which that section of the tape is shifted from a perfect tape.

Now as long as $r > 3$, the last three symbols of a local sample of a tape occur one position earlier in the local sample that begins one cell later. So the shift of both from a perfect tape must be the same $\pmod r$. That is, if $\tau(n+t) \equiv \lambda(n) \equiv n \pmod r$ for all n in the first block such that $n \not\equiv 0 \pmod r$, then the same holds for all $n \not\equiv 0 \pmod r$ in the second block, including the final position as was to be shown. This completes the proof of Lemma 1, the base case. □

We note a further consequence of this argument that will prove useful later on. There is a mapping $\mu : A \to \{0,1\}^r$ from the local condition to bit vectors of length r that identifies. for any local sample of a perfect tape, which elements have coordinates of order 0. That is, if π is a perfect tape and

$$\mathbf{g} = \langle \pi(n), \ldots, \pi(n+r-1) \rangle \in A,$$

then element j $(0 \le j \le r-1)$ of $\mu(\mathbf{g})$ is 0 if and only if $\omega(n+j) = 0$, and that element is 1 if $\omega(n+j) > 0$.

Now for the induction step in the proof that X_r accepts only admissible tapes.

Lemma 2. *For each $i \ge 0$, every i-admissible tape accepted by X_r is $(i+1)$-admissible.*

Proof. We will define two kinds of global samples from a perfect tape, flat and incremental. We would like it to be the case that a flat sample indicates that the four cells have the same order and that a incremental sample indicates that the order of fourth is one greater than the order of the other three, but that is too much to ask. We can guarantee only that these properties hold under further conditions on the sample. We will then need a separate argument to guarantee that cells whose coordinates have the same p- and q-orders always have the same third track.

Flat samples first.

Let $\mathbf{g} = \langle g_1, g_2, g_3, g_4 \rangle$ be any quadruple of r-tape symbols. Then $\mathbf{g}$ is said to be *flat* provided that

(a) $g_2 - g_1 \equiv g_4 - g_3 \not\equiv 0 \pmod r$;

(b) $g_3 - g_1 \not\equiv 0 \pmod r$; and

(c) $g_1 + g_4 \not\equiv 0 \pmod r$.

Sublemma 1. *If π is a perfect tape, $n_2 - n_1 = n_4 - n_3$, and the global sample $\langle \pi(n_1), \ldots, \pi(n_4) \rangle$ is flat, then n_1, n_2, n_3, n_4, and $n_2 - n_1 = n_4 - n_3$ all have equal, finite order.*

Proof. Since r is fixed, in what follows all congruences and incongruences will be understood to be $\pmod r$. Let $g_i = \pi(n_i)$ for each $i = 1, \ldots, 4$. First note that in addition to the four quantities stipulated in (a)–(c) to be incongruent to 0, two more incongruences are implied:

(d) $g_4 - g_2 \not\equiv 0$, because

$$\begin{aligned} g_3 &\equiv g_1 - g_2 + g_4 \text{ by (a), so} \\ g_3 - g_1 &\equiv g_1 - g_2 + g_4 - g_1 = g_4 - g_2 \not\equiv 0 \text{ by (b).} \end{aligned}$$

(e) $g_2 + g_3 \not\equiv 0$, because

$$\begin{aligned} g_4 &\equiv g_2 + g_3 - g_1 \text{ by (a), so} \\ g_1 + g_4 &\equiv g_1 + g_2 + g_3 - g_1 = g_2 + g_3 \not\equiv 0 \text{ by (c).} \end{aligned}$$

Let ℓ, be an index $1 \le \ell \le 4$ such that n_ℓ has lowest order among $n_1, \ldots, n_4$. (The incongruences imply that the g_i are not all equal to each other, so at least one of the n_i must be nonzero and hence of finite order.) Let $e_i = \omega(n_i) - \omega(n_\ell)$ for $i = 1, \ldots, 4$, so that $e_\ell = 0$ and all of the e_i are nonnegative. We need to show that $e_i = 0$ for each i.

Recall that for any $n \neq 0$, $n = r^{\omega(n)}\delta(n)$. Dividing the equation $n_2 - n_1 = n_4 - n_3$ by $r^{\omega(n_\ell)}$, it follows that

$$\begin{aligned} r^{e_2}\delta(n_2) - r^{e_1}\delta(n_1) &= r^{e_4}\delta(n_4) - r^{e_3}\delta(n_3), \text{ so} \\ r^{e_2}g_2 - r^{e_1}g_1 &\equiv r^{e_4}g_4 - r^{e_3}g_3 \pmod r, \end{aligned} \tag{4.7}$$

since for each i, $g_i = \pi(n_i) = \lambda(n_i)$, the last two digits of $\delta(n_i)$. Now if $e_i > 0$ for three different i, then the fourth would also have to be > 0 for (4.7) to hold, but that would contradict the fact that $e_\ell = 0$. Nor can $e_i > 0$ for only one i, for then (4.7) would require that $g_i \equiv 0$ for that value of i, contradicting the fact that all $g_i \not\equiv 0$. Finally, each of the six cases in which two of the e_i are 0 and two are nonzero contradicts one of the six incongruences in (a)–(e):

- If $e_1 = e_2 = 0$ while $e_3 > 0$ and $e_4 > 0$, then (4.7) would imply that $g_2 - g_1 \equiv 0$, contradicting (a).
- If $e_1 = e_3 = 0$ while $e_2 > 0$ and $e_4 > 0$, then (4.7) would imply that $-g_1 \equiv -g_3$, contradicting (b).
- If $e_1 = e_4 = 0$ while $e_2 > 0$ and $e_3 > 0$, then (4.7) would imply that $-g_1 \equiv g_4$, contradicting (c).

- If $e_2 = e_3 = 0$ while $e_1 > 0$ and $e_4 > 0$, then (4.7) would imply that $g_2 \equiv -g_3$, contradicting (e).

- If $e_2 = e_4 = 0$ while $e_1 > 0$ and $e_3 > 0$, then (4.7) would imply that $g_2 \equiv g_4$, contradicting (d).

- If $e_3 = e_4 = 0$ while $e_1 > 0$ and $e_2 > 0$, then (4.7) would imply that $g_4 - g_3 \equiv 0$, contradicting (a).

This completes the proof of Sublemma 1. □

Next we define samples which will, under certain conditions, be sufficient to establish that the first three cells have order one less than the fourth.

Let $\mathbf{g} = \langle g_1, g_2, g_3, g_4 \rangle$ be a quadruple of r-tape symbols. Then $\mathbf{g}$ is said to be *incremental* provided that all of the following hold:

(A) $g_1 + g_4 \not\equiv 0 \pmod{r}$

(B) $g_1 - g_4 \not\equiv 0 \pmod{r}$

(C) $g_2 + g_4 \not\equiv 0 \pmod{r}$

(D) $g_2 - g_4 \not\equiv 0 \pmod{r}$

(E) $g_3 + g_4 \not\equiv 0 \pmod{r}$

(F) $g_3 - g_4 \not\equiv 0 \pmod{r}$

(G) $g_1 - g_2 - g_3 \equiv 0 \pmod{r}$ but $g_1 - g_2 - g_3 \not\equiv 0 \pmod{r^2}$

Sublemma 2. *If π is a perfect tape, $n_2 - n_1 = n_4 - n_3$, and the global sample $\langle \pi(n_1), \ldots, \pi(n_4) \rangle$ is incremental, then n_1, n_2, and n_3 have equal order, which is* 1 *less than the order of n_4, and $r\lambda(n_4) \equiv \lambda(n_2) + \lambda(n_3) - \lambda(n_1) \pmod{r^2}$.*

Proof. We first note that (G) implies certain additional incongruences when taken together with the fact that all tape symbols are incongruent to $0 \pmod{r}$:

(H) $g_3 - g_1 \not\equiv 0 \pmod{r}$, because $g_2 \not\equiv 0 \pmod{r}$

(I) $g_2 + g_3 \not\equiv 0 \pmod{r}$, because $g_1 \not\equiv 0 \pmod{r}$

(J) $g_2 - g_1 \not\equiv 0 \pmod{r}$, because $g_3 \not\equiv 0 \pmod{r}$.

As in the proof of Sublemma 1, let ℓ, $1 \leq \ell \leq 4$, be an index such that n_ℓ has lowest order among $n_1, \ldots, n_4$, and let $e_i = \omega(n_i) - \omega(n_\ell)$. We aim to show that $e_1 = e_2 = e_3 - 0$ and $e_4 = 1$. Equation (4.7) holds for the same reasons as in the proof of Sublemma 1.

It is impossible for all four e_i to be 0, as (4.7) would then imply that

$$g_1 - g_2 - g_3 \equiv -g_4 \not\equiv 0 \pmod{r},$$

contradicting (G). Nor can exactly three of the e_i be nonzero, since as in the proof of Sublemma 1, (4.7) would then require that the fourth be nonzero as well. Ruling out the possibility that exactly two of the e_i are 0 requires a case analysis similar to that in the proof of Sublemma 1 (all congruences being $\pmod{r}$):

- If $e_1 = e_2 = 0$ while $e_3 > 0$ and $e_4 > 0$, then (4.7) would imply that $g_2 - g_1 \equiv 0$, contradicting (J).
- If $e_1 = e_3 = 0$ while $e_2 > 0$ and $e_4 > 0$, then (4.7) would imply that $-g_1 \equiv -g_3$, contradicting (H).
- If $e_1 = e_4 = 0$ while $e_2 > 0$ and $e_3 > 0$, then (4.7) would imply that $-g_1 \equiv g_4$, contradicting (B).
- If $e_2 = e_3 = 0$ while $e_1 > 0$ and $e_4 > 0$, then (4.7) would imply that $g_2 \equiv -g_3$, contradicting (I).
- If $e_2 = e_4 = 0$ while $e_1 > 0$ and $e_3 > 0$, then (4.7) would imply that $g_2 \equiv g_4$, contradicting (D).
- If $e_3 = e_4 = 0$ while $e_1 > 0$ and $e_2 > 0$, then (4.7) would imply that $g_4 - g_3 \equiv 0$, contradicting (F).

That leaves the four cases in which exactly one of the e_i is nonzero and the others are 0.

- If $e_1 > 0$ while $e_2 = e_3 = e_4 = 0$, then (4.7) would imply that $g_2 - g_4 + g_3 \equiv 0 \pmod{r}$, which together with (G) implies that $g_1 - g_4 \equiv 0 \pmod{r}$, contradicting (B).
- If $e_2 > 0$ while $e_1 = e_3 = e_4 = 0$, then $-g_1 - g_4 + g_3 \equiv 0 \pmod{r}$, which implies $g_2 + g_4 \equiv 0 \pmod{r}$. contradicting (A).
- If $e_3 > 0$ while $e_1 = e_2 = e_4 = 0$, then $g_2 - g_1 - g_4 \equiv 0 \pmod{r}$, which implies that $g_3 + g_4 \equiv 0 \pmod{r}$, contradicting (E).

So $e_1 = e_2 = e_3 = 0$ and $e_4 > 0$. It remains to show that $e_4 \not> 1$. But if if $e_4 \geq 2$, then (4.7) would imply that $g_1 - g_2 - g_3 \equiv 0 \pmod{r^2}$, contradicting the second clause of (G).

Now substituting $e_1 = e_2 = e_3 = 0$ and $e_4 = 1$ into $r^{e_2} g_2 - r^{e_1} q_1 = r^{e_4} g_4 - r^{e_3} g_3$ yields $g_2 - g_1 = r \cdot g_4 - g_3$, which implies that $r \cdot g_4 \equiv g_2 + g_3 - g_1 \pmod{r^2}$. This completes the proof of Sublemma 2. □

We can now complete the proof of Lemma 2, that every i-admissible tape accepted by X is $(i+1)$-admissible.

Let τ be an i-admissible tape accepted by X_r, and let t an integer such that

$$\begin{aligned} \tau(n+t) &= \lambda(n) \quad \text{if } \omega(n) < i, \text{ and} \\ \tau(n+t) &\equiv \lambda(n) \pmod{r} \quad \text{if } \omega(n) = i. \end{aligned}$$

We will show that there is an integer u such that

$$\tau(N+u) = \lambda(N) \quad \text{if } \omega(N) \leq i, \text{ and} \tag{4.8}$$

$$\tau(N+u) \equiv \lambda(N) \pmod{r} \quad \text{if } \omega(N) = i+1, \tag{4.9}$$

thus establishing that τ is $(i+1)$-admissible.

We first show that if M, N, and their difference are all of order $i < \infty$, then $\tau(N+t) - \tau(M+t) \pmod{r^2}$ depends only on $N - M$, and is in fact equal to $\lambda(N - M)$.

Let k_1 and k_2 be distinct numbers of order 0 and let d be a number of order 0 such that $k_1 + d$ and $k_2 + d$ are also of order 0. Thus multiplying any of these numbers by r^i yields a number of order i. Let $N_j = k_j r^i$ for $j = 1, 2$. We plan to show that moving down the tape distance dr^i starting from either $N_1 + t$ or $N_2 + t$ results in the same change in the values in the tape cells, namely by $\lambda(d)$. To be precise, we aim to show that

$$\begin{aligned} \tau(N_1 + dr^i + t) - \tau(N_1 + t) &\equiv \tau(N_2 + dr^i + t) - \tau(N_2 + t) \\ &\equiv \lambda(d) \pmod{r^2}. \end{aligned}$$

It may not be possible to show this using Sublemma 1 directly because these tape coordinates may not satisfy the conditions of that proposition, so instead we will have to find a suitable third coordinate and compare both to that third position.

Let k be an integer in the range $0 \leq k < r$ such that each of the following is incongruent to $0 \pmod{r}$:

(i) k

(ii) $k + d$

(iii) $k - k_1$

(iv) $k - k_2$

(v) $k + k_1 + d$

(vi) $k + k_2 + d$

It is possible to find a suitable k as long as $r > 6$. Let $N = kr^i$.Then for $j = 1$ and $j = 2$, the quadruple

$$\mathbf{g}_j = \langle \tau(N_j + t), \tau(N_j + dr^i + t), \tau(N + t), \tau(N + dr^i + t) \rangle$$

is flat: condition (a) holds because τ is i-admissible and all coordinates have the same order, as does the distance separating the two members of each pair, condition (b) holds by (i) and (ii), and condition (c) holds by (iii) and (iv).

Note that the quadruples $\mathbf{g}_1$ and $\mathbf{g}_2$ have the same third and fourth components and possibly different first and second components.

Since τ is accepted by X_r, $\mathbf{g}_j \in B$, the global condition of X_r. So there are, for $j = 1$ and 2, perfect tapes π_j and nonzero integers m_j, m_j, and d_j, such that

$$\mathbf{g}_j = \langle \pi_j(m_j), \pi_j(m_j + d_j), \pi_j(n_j), \pi_j(n_j + d_j) \rangle.$$

By Sublemma 1, for $j = 1$ and 2, m_j, $m_j + d_j$, n_j, $n_j + d_j$, and d_j all have the same order. These orders being finite, we can replace π_1 and π_2 by λ throughout. So for $j = 1$ and 2,

$$\lambda(m_j + d_j) - \lambda(m_j) \equiv \lambda(n_j + d_j) - \lambda(n_j) \equiv \lambda(d_j) \pmod{r^2}$$

But

$$\langle \lambda(n_1), \lambda(n_1 + d_1) \rangle = \langle \lambda(n_2), \lambda(n_2 + d_2) \rangle,$$

since the left and right sides are equal to the last two components of $\mathbf{g}_1$ and $\mathbf{g}_2$, respectively, which are equal to each other. Because $\mathbf{g}_1$ and $\mathbf{g}_2$ are flat, it follows that

$$\begin{aligned} \tau(N_1 + dr^i + t) - \tau(N_1 + t) &\equiv \lambda(n_1 + d_1) - \lambda(n_1), \\ &= \lambda(n_2 + d_2) - \lambda(n_2) \\ &\equiv \tau(N_2 + dp^i + t) - \tau(N_2 + t) \\ &\equiv \lambda(d) \pmod{r^2}, \end{aligned}$$

as was to be shown.

It follows that there is some k_0 such that $\tau(k_0 r^i + dr^i + t) = \lambda(d)$ whenever $d \not\equiv 0 \pmod{r}$. Taking $u = k_0 r^i + t$ establishes (4.8).

It remains to establish (4.9), that $\tau(N+u) \equiv \lambda(N) \pmod{r}$ if $\omega(N) = i+1$. So let N be a number of order $i+1$ and let $n = \tau(N+u)$. To find an incremental sample, we choose two order-0 numbers, m and d, from the set $\{j : 1 \leq j \leq r-1\}$, subject to the following incongruences (mod r):

1. $m + d \not\equiv 0$
2. $n + m \not\equiv 0$
3. $n - m \not\equiv 0$
4. $n + m + d \not\equiv 0$
5. $n - d \not\equiv 0$
6. $n + d \not\equiv 0$
7. $n - m + d \not\equiv 0$

Given n, each condition rules out at most $r-1$ possible choices for the pair $\langle m, d\rangle$, so—estimating conservatively, since some pairs are ruled out by more than one incongruence—there will be at least one left provided that

$$(r-1)^2 - 7 \cdot (r-1) = (r-8) \cdot (r-1) > 0,$$

which is true provided that $r \geq 9$.

We claim that the sample

$$\mathbf{g} = \langle g_1, g_2, g_3, g_4\rangle = \langle \tau(mr^i + u), \tau(mr^i + dr^i + u), \tau(N - dr^i + u), \tau(N+u)\rangle$$

is incremental. The seven conditions are satisfied:

(A)

$$\begin{aligned} \tau(mr^i + u) + \tau(N+u) &\equiv \lambda(m) + n \text{ by (4.8) and the definition of } n \\ &\equiv m + n \\ &\not\equiv 0 \pmod{r} \text{ by (2).} \end{aligned}$$

(B) $\tau(mr^i + u) - \tau(N+u) \equiv \lambda(m) - n \equiv m - n \not\equiv 0 \pmod{r}$ by (3).

(C) $\tau(mr^i + dr^i + u) + \tau(N+u) \equiv m + d + n \not\equiv 0 \pmod{r}$ by (1) and (4).

(D) $\tau(mr^i + dr^i + u) - \tau(N+u) \equiv m + d - n \not\equiv 0 \pmod{r}$ by (1) and (7).

(E) $\tau(N - dr^i + u) + \tau(N+u) \equiv -d + n \not\equiv 0 \pmod{r}$ by (5)

(F) $\tau(N - dr^i + u) - \tau(N + u) \equiv -d - n \not\equiv 0 \pmod{r}$ by (6)

(G)

$$\begin{aligned} & \tau(mr^j + u) - \tau(mr^i + dp^i + u) - \tau(N - rr^i + u) \\ \equiv \quad & m - (m - d) - d = 0 \pmod{r} \end{aligned}$$

because all three terms have order i. And because N has order exactly $i + 1$, $0 < m < r$, and $0 < d < r$,

$$\begin{aligned} & \tau(mr^j + u) - \tau(mr^i + dr^i + u) - \tau(N - dr^i + u) \\ \equiv \quad & m - (m + d) - (r - d) \equiv r \not\equiv 0 \pmod{r^2}. \end{aligned}$$

Equation (4.7), with $e_1 = e_2 = e_3 = 0$ and $e_4 = 1$, then implies that

$$\begin{aligned} p \cdot \tau(N + u) &\equiv \tau(N - dr^i + u) + \tau(mr^i + dr^i + u) - \tau(mr^i + u) \\ &\equiv \lambda(N - dr^i) + \lambda(mr^i + dp^i) - \lambda(mr^i) \\ &\equiv \lambda(rn - d) + \lambda(m + d) - \lambda(m) \\ &\equiv r \cdot \lambda(n) \pmod{r^2}, \text{ by Sublemma 2,} \end{aligned}$$

and therefore $\tau(nr^{i+1} + u) \equiv n \pmod{r}$, as was to be shown. This completes the proof of Lemma 2 (the induction step in the proof that tapes accepted by X_r are admissible). □

Completing the proof of Lemma 2 finishes the proof of Theorem 2, establishing that X_r accepts all perfect and only admissible r-tapes. □

5 Reducing domino systems to linear sampling systems

Theorem 3 (Main Theorem)**.** *Given a domino system, it is possible to construct a linear sampling system that accepts a tape if and only if there is a tiling of the first quadrant of the plane consistent with the constraints of the domino system.*

Proof. First we need to combine the linear sampling systems guaranteed by Theorem 2 to accept all perfect and only admissible p- and q-tapes to accept all perfectly addressed and only admissibly addressed three-channel tapes. The method is basically a cross-product construction, with a small technical twist.

Suppose that for $r = p, q$, X_r is an LSS accepting all perfect r-tapes and only admissible r-tapes. Then from X_p and X_q we can construct an LSS X accepting all perfectly addressed tapes and only admissibly addressed tapes.

We construct X by taking for its global condition the cross product of the global conditions of X_p and of X_q (and all possible symbols in the data channel), but it is not immediately possible to do the same for the local conditions, which have different dimensions. So the local condition for X_p must first be extended to be of dimension q (recall that $p < q$) by including all strings of length q whose first p and last p symbols are in the local condition for X_p. It is evident that X is as required.

We'll say that the tape cell with coordinate n is of *type* $\langle i, j\rangle$ $(i, j \geq 0)$ just in case n has p-order i and q-order j.

Suppose D is the set of dominoes, $d_0 \in D$ is the domino that must be placed at the origin, and $H, V \subseteq D \times D$ are the sets of pairs of dominoes that may abut horizontally and vertically, respectively. Let $X = \langle \Sigma, A, B, L\rangle$ be an LSS that accepts all perfectly addressed tapes and only admissibly addressed tapes with members of D in the data track. The dimension of A is q. The required system X' will be $\langle \Sigma, A', B', L\rangle$, where $A' \subseteq A$ and $B' \subseteq B$. From the discussion following the proof of Lemma 1 we can find a mapping μ from A to $\{\{0, 1\} \times \{0, 1\}\}^q$ that identifies whether the cells of a sample of a perfectly addressed tape have type $\langle 0, 0\rangle$: If π is perfectly addressed tape, then component j of $\mu(\langle \pi(n), \ldots, \pi(n + q - 1)\rangle)$ is $\langle 0, 0\rangle$ just in case $n + j$ is of type $\langle 0, 0\rangle$. Then A' is formed from A by removing any member of A mapped to a bit vector for which some component j equal to $\langle 0, 0\rangle$ but d_0 is not component j of the data channel. That is, if $\mathbf{g} \in A$ and $\mu(\mathbf{g}) = \langle x_0, \ldots, x_{q-1}\rangle$, then $\mathbf{g}$ is excluded from A' unless component j the data track of $\mathbf{g}$ contains d_0 whenever $x_j = \langle 0, 0\rangle$. The effect will be that in any perfect tape accepted by X', cells of type $\langle 0, 0\rangle$ will all have d_0 in the data track.

In the same way, B' is formed by omitting certain quadruples from B. First we need to ensure that in an accepted perfectly addressed tape, all cells of the same type have the same domino in the data track. This is achieved by omitting from B' any flat quadruple in B in which the contents of the data track of the four components are not all the same.

Next we use incremental samples from the global condition to impose the adjacency constraints of the domino system. Omit any quadruple that is incremental in its first components and flat in its second components unless the data track of third and fourth components form a pair in H, and also omit any quadruple that is incremental in its second components and flat in its first components unless the data track of the third and fourth components form a pair in V.

We claim first that if there is a tiling of the first quadrant satisfying the domino constraints, then a tape is accepted by X', namely any of the perfectly addressed tapes in which the domino at position $\langle i, j\rangle$ is in the third track of all cells of type $\langle i, j\rangle$. The proof is straightforward.

The converse requires more work. Suppose that X' accepts a tape τ. That tape must be admissibly addressed, so some perfectly addressed tape π is also accepted

by X', with the further property that every central segment of π is a segment of τ. We first show that all cells of π of the same type must contain the same domino in the data track. It will suffice to show a more restricted result:

Lemma 3. *If m and n are of the same type, $\lambda_p(m) \not\equiv \lambda_p(n) \pmod{p}$, and $\lambda_q(m) \not\equiv \lambda_q(n) \pmod{q}$, then the data tracks of $\pi(m)$ and $\pi(n)$ contain the same domino.*

Given this Lemma, cells of the same type that fail the second or third premise of the lemma can be compared to any cell that satisfies all three premises (given that $p, q > 2$), so all cells of the same type will contain the same domino.

Proof. Suppose n and m are of the same type but are not equal and that $\lambda_p(m) \not\equiv \lambda_p(n) \pmod{p}$ and $\lambda_q(m) \not\equiv \lambda_q(n) \pmod{q}$. We will show that there exist N and M so that $N - M = n - m$ and both address channels of the global sample $\langle \lambda(M), \lambda(N), \lambda(m), \lambda(n) \rangle$ are flat. It will follow from Sublemma 1 that n and m (as well as N and M) have the same type.

We need to choose M and N so that $M - N = m - n$ and the flatness conditions for p and q are satisfied:

1. $\lambda_p(N) - \lambda_p(M) \equiv \lambda_p(n) - \lambda_p(m) \not\equiv 0 \pmod{p}$
2. $\lambda_p(m) - \lambda_p(M) \not\equiv 0 \pmod{p}$
3. $\lambda_p(M) + \lambda_p(n) \not\equiv 0 \pmod{p}$
4. $\lambda_q(N) - \lambda_q(M) \equiv \lambda_q(n) - \lambda_q(N) \not\equiv 0 \pmod{q}$
5. $\lambda_q(m) - \lambda_q(M) \not\equiv 0 \pmod{q}$
6. $\lambda_q(M) + \lambda_q(n) \not\equiv 0 \pmod{q}$

Since m and n are stipulated and $N = M - m + n$, we need to choose a value for M that will result in all of (1)–(6) being satisfied. Choose M so that $1 \leq M \leq p - 1$ and conditions (2), (3), (5), and (6) are met; each of these 6 conditions rules out one possible value for M in this range. We also require that

$$\lambda_p(M) \equiv \lambda_p(m) - \lambda_p(n) + \lambda_p(M + n - m) \pmod{p}$$
$$\lambda_q(M) \equiv \lambda_q(m) - \lambda_q(n) + \lambda_q(M + n - m) \pmod{q}$$

which will be true unless $M + n - m \equiv 0 \pmod{p}$ or $M + n - m \equiv 0 \pmod{q}$, ruling out two more possible values for M. With 8 of the $p - 1$ values ruled out, we can choose M to satisfy all conditions provided that $p > 9$, concluding the proof of Lemma 3. □

It remains only to show that the horizontal and vertical adjacency conditions are satisfied. Since all cells of the same type contain the same domino, it suffices to find *any* m of type $\langle i, j\rangle$ and n of type $\langle i+1, j\rangle$ (checking vertical adjacencies is analogous), and M and N such that $N - M = n - m$ and the global sample $\langle \pi(M), \pi(N), \pi(m), \pi(n)\rangle$ is incremental on the first track and flat on the second. So we want all of the following to hold ((1)–(6) are the conditions for incrementality on the first track and (9)–(11) are the conditions for flatness on the second).

1. $\lambda_p(M) + \lambda_p(n) \not\equiv 0 \pmod{p}$
2. $\lambda_p(M) - \lambda_p(n) \not\equiv 0 \pmod{p}$
3. $\lambda_p(N) + \lambda_p(n) \not\equiv 0 \pmod{p}$
4. $\lambda_p(N) - \lambda_p(n) \not\equiv 0 \pmod{p}$
5. $\lambda_p(m) + \lambda_p(n) \not\equiv 0 \pmod{p}$
6. $\lambda_p(m) - \lambda_p(n) \not\equiv 0 \pmod{p}$
7. $\lambda_p(M) - \lambda_p(N) - \lambda_p(m) \equiv 0 \pmod{p}$
8. $\lambda_p(M) - \lambda_p(N) - \lambda_p(m) \not\equiv 0 \pmod{p^2}$
9. $\lambda_q(N) - \lambda_q(M) \equiv \lambda_q(n) - \lambda_q(m) \not\equiv 0 \pmod{q}$
10. $\lambda_q(m) - \lambda_q(M) \not\equiv 0 \pmod{q}$
11. $\lambda_q(M) + \lambda_q(n) \not\equiv 0 \pmod{q}$

We have assumed already that $p > 7$ and that $q > p$. To complete the argument it will be convenient to set $q = p + 1$, so, for example, 8 and 9 would be suitable values for p and q.

To find values for m, n, M, and N, we will use parameters k and ℓ in the range $\{2, \ldots, p-2\}$ in such a way that conditions (1)–(11) are satisfied.

	m	n	M	N
Value	$p^i q^j$	$kp^{i+1}q^j$	$(p-\ell)p^i q^j$	$((k+1)p-\ell-1)p^i q^j$
ω_p	i	$i+1$	i	i
$\lambda_p \pmod{p^2}$	q^j	kq^j	$(p-\ell)q^j$	$((k+1)p-\ell-1)q^j$
ω_q	j	j	j	j
$\lambda_q \pmod{q^2}$	p^i	kp^{i+1}	$(p-\ell)p^i$	$((k+1)p-\ell-1)p^i$

First note that $N - M = n - m$ as required, and that $p - \ell$ and $(k+1)p - \ell - 1$ are both incongruent to 0 (mod p) because $2 \le \ell \le p - 2$. Also $\omega_p(n) = i + 1$ because $2 \le k \le p - 2$, and similarly $\omega_p(M) = \omega_p(N) = i$. The q-order of m, n, M, and N are equal to j because q is relatively prime to p and the coefficients k, $p - \ell$ and $(k + 1)p - \ell - 1$ are all incongruent to 0 (mod q).

Accordingly the λ_p and λ_q rows show in the column headed by variable x the quantity whose remainder (mod p^2 or q^2) is $\lambda(x)$.

We next enumerate the additional conditions on k and ℓ because of (1)–(11).

1. $\lambda_p(M) + \lambda_p(n) \equiv (p - \ell)q^j + kq^j \equiv (p + k - \ell)q^j \not\equiv 0 \pmod{p}$ provided that $k - \ell \neq 0$ (since $2 \le k, \ell \le p - 2$).

2. $\lambda_p(M) - \lambda_p(n) \equiv (p - \ell)q^j - kq^j \equiv (p - k - \ell)q^j \not\equiv 0 \pmod{p}$ provided that $k + \ell \neq p$.

3. $\lambda_p(N) + \lambda_p(n) \equiv ((k + 1)p - \ell - 1)q^j + kq^j \equiv ((k + 1)p + k - \ell - 1)q^j \not\equiv 0 \pmod{p}$ provided that $k - \ell - 1 \neq 0$.

4. $\lambda_p(N) - \lambda_p(n) \equiv ((k + 1)p - \ell - 1)q^j - kq^j \equiv ((k + 1)p - k - \ell - 1)q^j \not\equiv 0 \pmod{p}$ provided that $k + \ell + 1 \neq p$.

5. $\lambda_p(m) + \lambda_p(n) \equiv q^j + kq^j \equiv (k + 1)q^j \not\equiv 0 \pmod{p}$ since $k \le p - 2$.

6. $\lambda_p(m) - \lambda_p(n) \equiv q^j - kq^j \equiv (1 - k)q^j \not\equiv 0 \pmod{p}$ since $k \ge 2$.

7. $$\begin{aligned}\lambda_p(M) - \lambda_p(N) - \lambda_p(m) &\equiv (p - \ell)q^j - ((k + 1)p - \ell - 1)q^j - q^j \\ &\equiv (p - \ell - kp - p + \ell + 1 - 1)q^j \\ &\equiv -kp \equiv 0 \pmod{p}.\end{aligned}$$

8. The same calculation implies that $\lambda_p(M) - \lambda_p(N) - \lambda_p(m) \not\equiv 0 \pmod{p^2}$ since $2 \le k \le p - 2$.

9. $$\begin{aligned}\lambda_q(N) - \lambda_q(M) &\equiv ((k + 1)p - \ell - 1)p^i - (p - \ell)p^i \\ &\equiv kp^{i+1} - p^i \\ &\equiv \lambda_q(n) - \lambda_q(m) \\ &\not\equiv 0 \pmod{q} \text{ since } q \text{ is relatively prime to } p.\end{aligned}$$

10. $\lambda_q(m) - \lambda_q(M) \equiv p^i - (p - \ell)p^i \equiv (\ell - p + 1)p^i \not\equiv 0 \pmod{q}$ provided that $\ell + 1 \not\equiv p \pmod{q}$. But $2 \le \ell \le p - 2 < q - 2$, so $3 \le \ell + 1 \le p - 1 < q - 1$, from which it follows that $\ell + 1 \not\equiv p \pmod{q}$.

11. $\lambda_q(M) + \lambda_q(n) \equiv (p-\ell)p^i + kp^{i+1} \equiv ((k+1)p-\ell)p^i \not\equiv 0 \pmod{q}$, provided that $\ell \not\equiv p \pmod{q}$, which is guaranteed since $2 \leq \ell \leq p-2 < q-2$.

There are $(p-3)^2$ possible pairs $\langle k, \ell \rangle$ such that $2 \leq k, \ell \leq p-2$. Of these, $p-3$ are ruled out by the requirement that $k-\ell \neq 0$; fewer than $p-3$ by the requirement that $k+\ell \neq p$; fewer than $p-3$ by the requirement that $k-\ell-1 \neq 0$; and fewer than $p-3$ by the requirement that $k+\ell+1 \neq p$. So at least $(p-3) \times (p-7)$ possibilities remain, which is > 0 provided that $p \geq 8$. This completes the proof of the Main Theorem. □

Readers who wish to experiment could try these values, which satisfy all constraints: $p = 10$, $q = 11$, $k = 4$, $\ell = 2$.

6 Other linear forms

At the beginning of this exposition, we narrowed our focus to the linear equation $n_2 - n_1 = n_4 - n_3$. In Aanderaa and Lewis [1974] we showed that the linear sampling problem is also undecidable for the equation $n_2 - n_1 = n_3$. A too-casual remark on page 521 of that paper states that linear sampling systems have a solvable decision problem if the linear equation has only two variables. I am today unable either to support or to refute that claim. I have no doubt that were Stål here to help, he would see that question right through to its resolution.

References

Stål O. Aanderaa and Harry R. Lewis. Linear sampling and ∀∃∀ case of decision problem. *The Journal of Symbolic Logic*, 39(3):519–548, 1974. ISSN 0022-4812.

Paul Bernays and Moses Schönfinkel. Zum entscheidungsproblem der mathematischen logik. *Mathematische Annalen*, 99:342–272, 1928.

Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936. ISSN 00029327.

David Hilbert and Wilhelm Ackermann. *Grundzüge der theoretischen Logik / [von] D. Hilbert und W. Ackermann*. J. Springer, 1928, Germany, 1928.

A. S. Kahr, Edward F. Moore, and Hao Wang. Entscheidungsproblem reduced to the ∀∃∀ case. *Proceedings of the National Academy of Sciences*, 48:365–377, 1962.

Frank P. Ramsey. On a problem of formal logic. *Proceedings of the London Mathematical Society*, s2-30:264–286, 1930.

Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1): 230–265, 1936. URL `http://doi.org/10.1112/plms/s2-42.1.230`.

Hao Wang. Proving theorems by pattern recognition II. *Bell Systems Technical Journal*, 40:1–141, 1961.

The Concurrency Column

BY

Marino Miculan and Nobuko Yoshida

Department of Mathematics, Computer Science and Physics
University of Udine
Via delle Scienze 206, Udine, Italy
marino.miculan@uniud.it

Department of Computer Science
University of Oxford
Wolfson Building, Parks Road, Oxford, UK
nobuko.yoshida@cs.ox.ac.uk

Welcome to this installment of *The Concurrency Column*. In this issue, we are pleased to present a contribution by Luca Aceto, who collects his interviews with the recipients of the CONCUR Test-of-Time Award 2026. For the period 2006–2009, the Award Committee selected two papers: "A New Type System for Deadlock-Free Processes" by Naoki Kobayashi, presented at CONCUR 2006, and "Strategy Logic" by Krishnendu Chatterjee, Thomas A. Henzinger and Nir Piterman, presented at CONCUR 2007. From two rather different angles—type systems for the π-calculus and logics for games on graphs—both papers have durably shaped the way our community reasons about the behaviour of concurrent and interactive systems.

In the first interview, Naoki Kobayashi retraces the long lineage of type systems guaranteeing deadlock-freedom, from the early work on the linear π-calculus to the award-winning paper. The central design tension he describes is that between expressiveness and automation: "time tags" with their partial orders gave expressive power, capability and obligation levels gave automated type inference, and the CONCUR 2006 paper struck a balance between the two by combining a restricted form of the former with the latter. Kobayashi also discusses the role played by the TyPiCal implementation in validating and stress-testing the theory,

the encoding of the simply-typed λ-calculus with recursion as a yardstick for expressiveness, and the duality of obligations and capabilities that later resurfaced in the verification of security protocols and of lock-based concurrent programs. Looking ahead, he argues that the advent of GenAI-based coding agents may well overturn the design philosophy of "lightweight" formal methods: if annotations can be generated automatically, then precision and completeness may matter more than parsimony.

In the second interview, Krishnendu Chatterjee, Thomas Henzinger and Nir Piterman explain how Strategy Logic came about by treating strategies as explicit, first-class objects that the logic can quantify over. Their motivation was to express game-theoretic notions such as Nash and secure equilibria within a logical framework for games on graphs; the one-alternation fragment came first, and the full logic only afterwards. They discuss the interplay between linear- and branching-time reasoning that makes Strategy Logic and the alternating-time μ-calculus expressively incomparable, the tree-automata techniques underpinning their decidability and complexity results, the state of tool support, and the enthusiastic uptake of the logic by the multi-agent systems community. They also reflect on games on graphs as a genuinely unifying theme bridging the Volume A–Volume B divide, and on the open problems the paper left behind.

Beyond their technical content, these interviews are also research-life stories: they show how collaborations begin, how ideas mature over a decade, and how our field may reposition itself in an era in which AI-generated software needs formal checks more than ever. We hope you will enjoy reading them as much as we did.

Interviews with the Recipients of the CONCUR Test-of-Time Award 2026

Luca Aceto
ICE-TCS, Department of Computer Science,
Reykjavik University, Iceland
Gran Sasso Science Institute, L'Aquila, Italy
luca@ru.is, luca.aceto@gssi.it

The 2026 CONCUR Test-of-Time Award Committee (Anca Muscholl (chair), Javier Esparza and Prakash Panangaden) has selected the following two papers for that accolade for the period 2006–2009:

- Naoki Kobayashi, "A New Type System for Deadlock-Free Processes", which appeared at CONCUR 2006, and
- Krishnendu Chatterjee, Thomas A. Henzinger and Nir Piterman, "Strategy Logic", which appeared at CONCUR 2007.

I interviewed the awardees via email and posted their answers to my questions on my professional blog.[1]

This piece collects those interviews essentially verbatim. In order to preserve the informal nature of the email discussions I had with Naoki, Krishnendu, Nir and Tom, I only made a small number of minor edits to correct a couple of typos and to add a bibliography.

I thank Naoki, Krishnendu, Nir and Tom for their willingness to answer my half-baked questions and for sharing their thoughts with the community in such an inspiring way. I hope you will enjoy reading their answers as much as I did.

1 Interview with Naoki Kobayashi

Luca: You receive the CONCUR Test-of-Time Award 2026 for your paper "A New Type System for Deadlock-Free Processes", which appeared at CONCUR 2006 [9]. That article is the culmination of a series of contributions you gave on

[1] https://processalgebra.blogspot.com/

the development of type systems for variations on the π-calculus that guarantee deadlock-freedom. Could you briefly explain to our readers how you came to study the question addressed in your award-winning article and how the main ideas in your CONCUR 2006 paper evolved over time? Which of the ideas and results in your paper did you find most pleasing, surprising or challenging?

Naoki: If I remember correctly, the idea of using types to ensure deadlock-freedom came up during discussions on the linear π-calculus with Benjamin Pierce and David Turner [10]. Indeed, if one wants to ensure that a channel is really used exactly once, one also has to ensure that the process does not get stuck before using it. An initial result on type systems for deadlock-freedom was published in LICS 1997 [7].

After that, the type system was extended in two directions: enabling automated type inference and increasing expressiveness. These were somewhat conflicting goals, and the CONCUR 2006 paper achieved a balance between them. The core idea was as follows. To ensure deadlock-freedom, it is necessary to control dependencies among different channels. In [7], such dependencies were expressed using "time tags" and a possibly infinite partial order on them. The time tags were later replaced in [8] by natural numbers, called capability and obligation levels, to enable automated inference [8], but at the cost of expressive power. The CONCUR 2006 paper combined a restricted form of time tags with capability and obligation levels, thereby recovering much of the expressive power while retaining automated inference.

What I found most pleasing was precisely this balance: the paper showed that one could make the type system significantly more practical without completely giving up the conceptual elegance and expressiveness of the earlier formulation.

Luca: The contribution in your article achieves a good trade-off between expressiveness of the type system and ease of implementing its associated type inference algorithm. Did you or any other researcher try to extend the range of examples that could be handled by a variation on your type system? Do you think that it would be worth doing so or have we hit the point of diminishing returns, where any advance would be very technical and hard to implement?

Naoki: With Elena Giachino and Cosimo Laneve, I indeed investigated such an extension in [4]. The system developed there can deal with infinite chains of dependencies more general than those handled in the CONCUR 2006 paper. It was, however, developed for value-passing CCS rather than the π-calculus. Incorporating the idea of [4] into the π-calculus setting remains future work. Another possible direction would be to combine the approach with generic types [6], which can capture precise dependencies among a set of channels. Unfortunately, I did not have time to fully explore this direction. As for whether it is worth pushing this line further, my feeling is that further general-purpose extensions may easily

become technically involved and hard to implement. Perhaps a more promising direction would be to adapt the type system to real concurrent languages such as Go, and then see what kinds of extensions are most beneficial in practice.

Luca: Commendably, you supported the theoretical developments in your article with an implementation in TyPiCal. What role did the implementation work play in your research on type systems guaranteeing deadlock-freedom? Did the theoretical developments benefit from the implementation? Do you think that every proposal for a type system for some language ought to be accompanied by an implementation?

Naoki: TyPiCal was initially developed as a proof of concept for the type inference algorithm of [8]. At that time, the theory had already been fully developed, and the implementation was mainly used to convince readers that the algorithm indeed works in practice. In later extensions of TyPiCal, for the CONCUR 2006 paper and [11], the implementation was also useful for checking the theories.

I think it is desirable for every type-system proposal to be accompanied by an implementation, although I would not say that it is absolutely necessary. An implementation is useful not only for checking the theory, but also for identifying limitations of the theory and suggesting possible directions for improvement.

Luca: You showed, amongst other things, that the simply-typed λ-calculus with recursion could be encoded in the deadlock-free fragment of your calculus. What role did this result play in convincing you that your type system was sufficiently expressive? What were the main challenges that you had to overcome in developing that encoding and what were the main inspirations for that result?

Naoki: The encodability of the simply-typed λ-calculus with recursion in the deadlock-free fragment was a sort of minimal requirement for expressiveness. Since that property had already been shown for the original deadlock-free type system [7], I wanted to show that a similar level of expressive power was retained in the CONCUR 2006 type system.

There was also a more conceptual reason for considering such an encoding. Deadlock-freedom corresponds to the progress property of well-typed functional programs: evaluation does not get stuck. Thus, the encodability result shows that this progress property can be preserved by compilation from possibly parallel functional programs into the π-calculus, which may be viewed as a CPS-style, lower-level language with concurrency primitives.

The main challenge was how to give a finite representation, within the type system, of the infinite chains of causal dependencies induced by recursion. As mentioned in the answer to the first question, we achieved this by combining the ideas of capability/obligation levels from [8] with a partial order on time tags from [7].

Luca: Your article gave a seminal contribution to the study of type systems for the π-calculus that guarantee some desirable properties. Other examples of such contributions are, for instance, your paper with Davide Sangiorgi on a type system for lock-freedom or the paper by Yuxin Deng and Davide Sangiorgi on ensuring termination by typability amongst others. In hindsight, are there any relations amongst those contributions or ideas that underlie them and that can guide further developments in a systematic fashion?

Naoki: All three type systems you mentioned are related in that they control causal dependencies using natural numbers called "levels". The type system for deadlock-freedom forbids cyclic dependencies, while the type systems for lock-freedom and termination forbid infinite chains of dependencies. The formulation of such reasoning through types, however, suffers from inherent limitations in expressive power.

To address this limitation, my paper with Davide Sangiorgi [12] suggested a new direction: combining syntactic typing rules with semantic ones. The latter can be discharged by resorting to model checkers, interactive theorem provers, or other verification tools, thereby complementing the limitations of type systems. I think this combination of type-based reasoning and semantic verification may provide a useful guide for further developments: types can capture common structural patterns, while semantic methods can handle cases that fall outside the reach of purely syntactic typing rules.

Luca: Are there any problems that you left open in your award-winning article that you'd still love to see solved?

Naoki: I would still like to see at least two directions explored further. First, increasing the expressiveness of the type system by integrating the ideas of [4] and [6], mentioned above, would probably be necessary to make the approach practical. Second, since the type-based approach is inherently incomplete, finding a smooth integration with other methods, such as model checking and interactive theorem proving, remains an important open problem, as suggested in [11].

Luca: How did the results and the techniques you developed in your award-winning paper influence your subsequent research? Is there any result obtained by other researchers that builds on, or is related to, your work and that you like in particular?

Naoki: A key ingredient of our type systems for deadlock-freedom is the notion of obligations and capabilities. An obligation of a send or receive operation describes the need to perform that operation, while a capability describes a guarantee that the operation can succeed. For example, in a client-server model, a client has the capability to send a request successfully, while a server has an obligation to receive and handle the request. In lock-based mutual exclusion, a thread has the

capability to eventually acquire a lock, and, after acquiring the lock, it has an obligation to eventually release the lock.

These dual notions of obligations and capabilities turned out to be useful in other contexts. For example, we adapted them for automated verification of security protocols in [3]. Another line of work developed related ideas for more conventional concurrent programming languages. Leino et al. applied related techniques to deadlock-freedom of channels and locks in [13], and this line was further extended to monitors and condition variables by Hamin and Jacobs in [5]. I found this line of work particularly interesting because it shows that similar ideas arise naturally also in a more conventional programming-language setting.

Luca: What are the research topics related to type systems that you find most interesting right now? What role do you think type systems will or should have, if any, in the era of coding agents based on GenAI?

Naoki: I think GenAI may change not only type systems, but also the focus of formal methods more broadly. Traditionally, programs have been written by humans, and therefore "lightweight formal methods" have played an important role. In that setting, it was important to keep program annotations minimal and to automate verification or bug finding as much as possible, even if this sometimes came at the cost of precision or completeness.

In the era of GenAI-based coding agents, however, the situation may change. If coding agents can also generate annotations such as types and loop invariants, then the burden of writing annotations may no longer be such a serious issue. As a result, verification methods that aim at strong correctness guarantees, rather than merely finding bugs or checking lightweight properties, may become more important.

In this respect, the CONCUR 2006 type system can be seen as a product of the former design philosophy: it emphasized automation, at some cost in precision. A possible future direction would be to move beyond such lightweight type systems, by incorporating dependent types or combining type systems with other methods such as model checking, aiming at more complete verification methods.

Luca: What advice would you give to a young researcher who is keen to start working on topics related to types for programming languages?

Naoki: As discussed above, GenAI is rapidly changing the landscape of research in programming languages, and we are facing significant challenges in adapting to these changes and finding new research directions to explore. I think this is a very good opportunity for "AI-native" young researchers to play important roles. My advice would be to learn the classical foundations carefully, but at the same time not to be constrained by traditional assumptions about how programs are written and verified.

2 Interview with Krishnendu Chatterjee, Thomas Henzinger and Nir Piterman

Luca: You receive the CONCUR Test-of-Time Award 2026 for your paper "Strategy Logic", which appeared at CONCUR 2007 [1] and, in archival form, in Information and Computation [2]. In that article, you introduced a seminal logic for expressing properties of strategies over two-player games on graphs. Could you briefly explain to our readers what the main features of strategy logic are? Could you also tell us how you came to study the question addressed in your award-winning article? Which of the results in your paper did you find most surprising or challenging?

Krishnendu, Nir and Tom (henceforth abbreviated to KNT): The defining feature of Strategy Logic is that it treats strategies as explicit, first-class objects: strategies are named by variables, and the logic can quantify over them. Our main motivation was to express central concepts from game theory, such as equilibria, within a logical framework for games on graphs. As for the results, what we find most appealing is that a logic this expressive still admits decidability, and that several natural fragments come with clean and reasonable computational complexity. Establishing these decidability and complexity results for various fragments was also the most technically challenging part of the work.

Luca: With the benefit of hindsight, having a logic to describe properties of games that treats strategies as first-class objects sounds like an extremely natural idea. However, previous logics such as ATL, ATL*, the alternating-time μ-calculus and game logic followed a different path. Do you recall how you came to the realisation that treating strategies explicitly was the "way to go"?

KNT: One of the key application areas for graph games has been reactive synthesis, and until 2004 reactive synthesis was studied primarily as an adversarial game. Hence logics such as ATL, ATL*, alternating-time μ-calculus, and game logic, all focus on strictly competitive or cooperative behaviors of game theory. Around 2004–2005, we started working on connections between algorithmic game theory and graph games. As a consequence we considered aspects of Nash equilibria and other not strictly competitive notions of game theory (such as secure equilibria) in graph games. A natural question was to build a logical framework that can express these aspects of game theory, which led to Strategy Logic. In fact we first isolated the one-alternation fragment, which suffices to express these equilibria, and only afterwards arrived at the full, more expressive logic. In hindsight, the shift to treating strategies explicitly was driven by the questions we were asking rather than by a single eureka moment.

Luca: Strategy logic builds on LTL, which is a very natural choice, IMHO. Did

you consider defining a version of strategy logic basing it on the (linear-time) modal μ-calculus? Would it be worth doing so and how would such a logic relate to the alternating-time μ-calculus?

KNT: There is always a tension between LTL and stronger formalisms that can recognize all ω-regular languages. Following the tradition of ATL and ATL*, we naturally chose to go with LTL for defining the (linear-time) objectives of players. As the techniques that we developed were automata based, it was clear that extensions of LTL that can express all ω-regular languages would be handled by the same techniques. The exact choice of the linear-time formalism (for example, ETL, QLTL, LDL, or the linear-time μ-calculus) is not very important as long as it can be readily translated to automata. In the alternating-time μ-calculus, however, by carefully nesting fixpoints and coalition quantification, we can define infinitely many changes of strategic context. But only in a completely adversarial manner.

Luca: As you are hinting, in your paper, amongst other results, you showed that the alternating-time μ-calculus and strategy logic have incomparable expressive power. Would a fixed-point version of strategy logic, based on the modal μ-calculus, be a way of achieving a logic that offers the "best of both worlds" while maintaining some of the good computational properties of strategy logic? Has anyone worked on such a logic?

KNT: We did not consider such a version of Strategy Logic. Intuitively, both classical and alternating-time μ-calculi combine local, single-transition branching operators such as Pre with fixpoint operators, whereas strategies express global behavioral choices and the objectives of players are defined on outcomes that are linear paths. An important expressive difference between the alternating-time μ-calculus and Strategy Logic is therefore due to the distinction between branching and linear time.

It seems interesting to consider a logic that would combine two types of pre operators: those that continue exploring pre-defined strategies and their induced behaviors and those that allow one to change the strategic context starting a new behavioral exploration. But we believe that the techniques that handle alternating-time μ-calculus would work for such a logic and the mix of behavior and control might be very hard to understand.

Luca: Did you or anyone else ever implement the model-checking algorithms you present in your award-winning paper? If the answer is negative, do you think that there would still be interest in such a model checker and in its experimental evaluation?

KNT: As far as we know there is no full implementation of Strategy Logic. In general, we have very good implementations supporting the manipulation and analysis of automata on infinite words (such as Spot and Owl). They are also

used as a basis for creating tools that solve reactive synthesis. But we do not have good tool support for using automata on infinite trees, which would be required in order to fully support Strategy Logic. The community studying Multi-Agent Systems adopted Strategy Logic and they have some support for the analysis of some questions. There are implementations of equilibrium checking and rational synthesis in the tool Eve that is developed in the group of Michael Wooldridge in Oxford. They support the analysis of concurrent game structures for such questions. There is also a restricted version of an epistemic extension of Strategy Logic that is included in the model checker MCMAS for Multi-Agent Systems, which was developed in the group of Alessio Lomuscio in Imperial College London.

MCMAS also supports ATL model checking and, in principle, it is possible to reduce the one-alternation fragment of Strategy Logic to ATL model checking, but we are not aware of this having been implemented.

Luca: You mentioned the uptake of strategy logic by the multi-agent systems community. How is strategy logic relevant and did you think that this work would be relevant to multi-agent systems?

KNT: Strategy Logic answered a natural need in Multi-Agent Systems research. For MAS, questions about the goals of agents and hence strategies are very natural. Many questions relate to rational behavior: whether agents have an incentive to follow a protocol, stability of behavior, what can coalitions do, and whether individuals can profitably deviate. The way Strategy Logic puts strategies in the center as explicit objects makes it very natural to study these questions.

We wouldn't say that we saw it coming, but the signs of early adoption of strategic reasoning by the MAS community were already there. The uptake of ATL and ATL* started in the early 2000s and by 2004–2005 people were using it regularly. We are also very happy that some of the major developments of Strategy Logic came from this community.

Luca: Are there any problems that you left open in your award-winning article that you'd still love to see solved? Did you or any colleagues study the problem of "strategy synthesis"?

KNT: Definitely. For the full logic, our paper established only a non-elementary upper bound and left the matching lower bound open. This gap was later closed by colleagues, who proved a matching non-elementary lower bound and thereby settled the computational complexity of the full logic. We also concentrated on the case of two-player games rather than multi-player games. The interaction between the logic and the game structure means that the two-player framework, in a sense, already captures the complexity of the logic. Indeed, the same techniques based on tree automata were later used by others to extend the logic to the multi-player setting (and concurrent game structures).

Luca: I am interested in how research collaborations start, as I like to tell "research-life stories" to PhD students and young researchers of all ages. Could you tell us how you started your collaboration on the award-winning paper?

KNT: The collaboration grew naturally out of a question from Tom (Thomas Henzinger): could notions such as the equilibria we had been studying be expressed in ATL or ATL*, and if not, what would be a natural and concise logic that could express them? Pursuing this led us to the one-alternation fragment. We then realized that strategies can be viewed as trees, which meant tree automata were the right tool—and Nir (Nir Piterman) was our automata expert. So the paper really came together at the meeting point of three ingredients: the study of non-zero-sum games, the wish for a logical framework to express their concepts, and tree-automata techniques.

Luca: How did the results and the techniques you developed in your award-winning paper influence your subsequent research? Is there any result obtained by other researchers that builds on your work and that you like in particular?

KNT: The result had a lasting influence. The interplay between games and automata that we exploited in the paper fed directly into later lines of work, e.g., from the connection of games and automata the notion of good-for-games (also known as history-deterministic) automata emerged. Among the results by others that build on Strategy Logic, the matching non-elementary lower bound and the work on special classes of strategies—such as the distinction between behavioral and non-behavioral strategies—are very elegant.

Luca: To my mind, games on graphs ought to be viewed as one of the unifying themes within TCS, bridging the Volume A-Volume B divide, and I am happy to see that there is a book-length treatment[2] covering the subject now. What is your view on this matter? What impact do you think that your work has had, if any, on the community working on algorithmic game theory, broadly construed? (I am reminded of the slides[3] for a, typically thought-provoking, talk delivered by Moshe Vardi.) What has our community learnt from the work done in the field of computational game theory? And what, if anything, did they learn from the work done within the concurrency theory community?

KNT: Indeed, games on graphs is one of the unifying themes between Volume A and Volume B: it connects with logic and automata theory from Vol. B, and it connects with the notion of alternation from complexity theory and graph algorithms from Vol. A. The inflow of ideas from algorithmic game theory, or game theory in general, is a key source of ideas in this work, e.g., to express equilibria

[2] See `https://arxiv.org/abs/2305.10546`.

[3] `https://www.cs.rice.edu/~vardi/papers/sr15.pdf`

in a logical framework. Other notions of equilibria have also been studied in the context of reactive synthesis (e.g., rational synthesis), and Vol. B researchers have explored many concepts from general game theory beyond strictly competitive games. The broad area of algorithmic game theory also benefited from logical reasoning grounded in concurrency theory, e.g., Strategy Logic is the key logic for reasoning about behaviors in Multi-Agent Systems, and axiomatic characterizations for reasoning about strategy spaces have been influential in evolutionary game theory. There is also work using Strategy Logic in mechanism design.

Luca: What are the research topics related to logics for games on graphs that you find most interesting right now?

KNT: We can think of several interesting open questions: (i) the relationship between games and automata through the lens of history-determinism and similar concepts is an exciting direction; (ii) algorithmic bounds for several fundamental problems remain open, e.g., a polynomial-time algorithm for parity games; (iii) the degree to which sources of randomness can be shared between different players is an active area of research in concurrent games; and (iv) the study of computationally efficient logics which lie between ATL* and Strategy Logic is also under-explored.

Luca: What advice would you give to a young researcher who is keen to start working on topics related to logics for games and other computational problems related to games?

KNT: The rapid progress in AI makes this question a difficult one. We believe that formal methods in general will become a central part of computing, more than they have ever been, because AI-generated software needs checks even more than software written by humans, but even more so, because modern AI can, for the first time, provide or at least support such checks on a scale that was hitherto impossible. Formal checks do not necessarily have to take the form of, say, Lean proofs, but they could also include state-based reasoning involving automata and games. It is always difficult to predict the future, but finding the right place for our field in this future seems a uniquely exciting opportunity.

References

[1] Krishnendu Chatterjee, Thomas A. Henzinger, and Nir Piterman. Strategy logic. In Luís Caires and Vasco Thudichum Vasconcelos, editors, *CONCUR 2007, 18th International Conference on Concurrency Theory*, volume 4703 of *Lecture Notes in Computer Science*, pages 59–73. Springer, 2007.

[2] Krishnendu Chatterjee, Thomas A. Henzinger, and Nir Piterman. Strategy logic. *Inf. Comput.*, 208(6):677–693, 2010.

[3] Morten Dahl, Naoki Kobayashi, Yunde Sun, and Hans Hüttel. Type-based automated verification of authenticity in asymmetric cryptographic protocols. In Tevfik Bultan and Pao-Ann Hsiung, editors, *Automated Technology for Verification and Analysis, 9th International Symposium, ATVA 2011*, volume 6996 of *Lecture Notes in Computer Science*, pages 75–89. Springer, 2011.

[4] Elena Giachino, Naoki Kobayashi, and Cosimo Laneve. Deadlock analysis of unbounded process networks. In Paolo Baldan and Daniele Gorla, editors, *CONCUR 2014, 25th International Conference on Concurrency Theory*, volume 8704 of *Lecture Notes in Computer Science*, pages 63–77. Springer, 2014.

[5] Jafar Hamin and Bart Jacobs. Deadlock-free monitors. In Amal Ahmed, editor, *Programming Languages and Systems, 27th European Symposium on Programming, ESOP 2018*, volume 10801 of *Lecture Notes in Computer Science*, pages 415–441. Springer, 2018.

[6] Atsushi Igarashi and Naoki Kobayashi. A generic type system for the Pi-calculus. *Theor. Comput. Sci.*, 311(1–3):121–163, 2004.

[7] Naoki Kobayashi. A partially deadlock-free typed process calculus. In *LICS 1997, 12th Annual IEEE Symposium on Logic in Computer Science*, pages 128–139. IEEE Computer Society, 1997.

[8] Naoki Kobayashi. Type-based information flow analysis for the pi-calculus. *Acta Informatica*, 42(4–5):291–347, 2005.

[9] Naoki Kobayashi. A new type system for deadlock-free processes. In Christel Baier and Holger Hermanns, editors, *CONCUR 2006, 17th International Conference on Concurrency Theory*, volume 4137 of *Lecture Notes in Computer Science*, pages 233–247. Springer, 2006.

[10] Naoki Kobayashi, Benjamin C. Pierce, and David N. Turner. Linearity and the pi-calculus. In Hans-Juergen Boehm and Guy L. Steele Jr., editors, *Conference Record of POPL'96: The 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 358–371. ACM Press, 1996.

[11] Naoki Kobayashi and Davide Sangiorgi. A hybrid type system for lock-freedom of mobile processes. In Aarti Gupta and Sharad Malik, editors, *Computer Aided Verification, 20th International Conference, CAV 2008*, volume 5123 of *Lecture Notes in Computer Science*, pages 80–93. Springer, 2008.

[12] Naoki Kobayashi and Davide Sangiorgi. A hybrid type system for lock-freedom of mobile processes. *ACM Trans. Program. Lang. Syst.*, 32(5):16:1–16:49, 2010.

[13] K. Rustan M. Leino, Peter Müller, and Jan Smans. Deadlock-free channels and locks. In Andrew D. Gordon, editor, *Programming Languages and Systems, 19th European Symposium on Programming, ESOP 2010*, volume 6012 of *Lecture Notes in Computer Science*, pages 407–426. Springer, 2010.

Conference Reports Column

by

Laurent Feuilloley

Laboratoire d'informatique
en image et systèmes d'information (LIRIS)
CNRS and Université Lyon 1
43 Bd du 11 novembre 1918, 69622 Villeurbanne, France
laurent.feuilloley@cnrs.fr

BEATCS no 150

Conference report on STOC 2026

Matthias Bentert (Technische Universität Berlin)

The 58th Annual ACM Symposium on Theory of Computing (STOC'26), sponsored by the ACM Special Interest Group on Algorithms and Computation Theory (SIGACT), took place from June 22 to 27 in Salt Lake City, Utah. The Hilton Salt Lake City Center hosted six days of paper presentations, workshops, invited talks, social events, and—thanks to a late addition on Saturday—a full-day discussion of whether AI can do theory.

For context, I am a postdoc in Germany working on algorithms. I was at STOC to present joint work on a polynomial-time algorithm for fault-tolerant packet forwarding in communication networks. Overall, I had an amazing experience and I would like to share some of my memories with you. But first, here is a riddle to guess if you can: why did I return from a very hot week in Utah with only one regret: not having brought warmer clothes?[1]

I cannot possibly do justice to the technical program in a conference report as STOC featured 211 accepted papers often in four parallel sessions. Luckily, I do not have to. All the talks will be available on the youtube channel of the SIGACT: https://www.youtube.com/@sigactec8649. At the time of writing, only some of the talks are online, but hopefully, all talks are available at the time of reading. Let me just highlight one presentation:

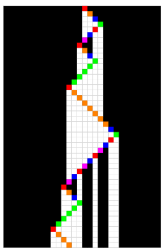
A large online collaboration set out to investigate the fifth busy beever number and, with the help of the Coq proof assistant, they proved that it is 47176870.[2] The proof classifies more than 181 million five-state Turing machines and determines whether each of them halts. It is the first new Busy Beaver value determined in more than forty years and the first one to be formally verified. Tristan Stérin presented the result on behalf of the collaboration. Besides settling a wonderfully concrete question about an uncomputable function, the project was also a strong demonstration of what a large, distributed group of contributors can accomplish.

[1] Hint: this was only the second time I visiting the United States.

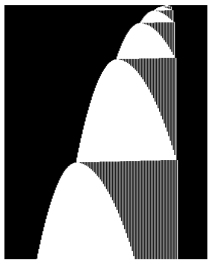
[2] Justin Blanchard, Daniel Briggs, Konrad Deka, Nathan Fenner, Yannick Forster, Georgi Georgiev (Skelet), Matthew L. House, Maja Kadziolka, Pavel Kropitz, Shawn Ligocki, mxdys, Mateusz Nasciszewski, Tristan Stérin, Chris Xu, Jason Yuen, Théo Zimmermann. *Determination of the Fifth Busy Beaver Value. STOC 2026: 565-574*

	0	1
A	1RB	1LC
B	1RC	1RB
C	1RD	0LE
D	1LA	1LD
E	—	0LA

(a) 5-state 2-symbol Busy Beaver winner. This machine was discovered by Marxen and Buntrock in 1989 [65].



(b) 45-step space-time diagram of the 5-state winner. Head position is coloured to indicate state, see (a).



(c) 20,000-step space-time diagram of the 5-state winner.

Figure 1: A figure from the Busy beaver paper (see footnote) that describes the "maximum" five-state Turing machine, and its space-time diagram at different scales.

Tuesday evening's banquet was held at the Ken Garff University Club. Unfortunately, jet lag overcome me this evening, so the following is second-hand information: both the food and the evening's Luca Trevisan Award Talk by Scott Aaronson were wonderful, so I am sad to have missed it. At least I will get the chance to listen to the talk once it is uploaded.

Thursday brought another exceptional event. Charles H. Bennett, co-recipient with Gilles Brassard of the 2025 ACM A. M. Turing Award, chose to give his award lecture at STOC. You may know Bennett for many different contributions. The first results I encountered bearing his name was the Bennett–Gill theorem: Relative to a random oracle O, it holds that $P^O \neq NP^O$ with probability one. His Turing Award lecture was titled "Quantizing Shannon, and Freeing Cosmology from the Need Define Consciousness or Count Observers". Bennett went "slightly" over time, but please, see for yourself. The recording is already up on youtube.

On Friday evening, Jane Street organized what has by now become a welcome tradition: the Estimathon. Teams had to estimate quantities that nobody could reasonably be expected to know. One memorable example question: What is the smallest integer n such that the n^{th} Fibonacci number contains the substring "2026"?

I was unsure whether to include a report on the served food or not. In the end, I decided to add some information, but please be aware that the sample size was very small. Apart from small refreshments, only three meals were provided. Monday's lunch boxes were not especially well received. The Tuesday banquet was, by all reports, excellent, and Saturday's buffet was also very good. The resulting empirical picture is therefore mixed but improves

substantially if one conditions on the food not being handed over in a box.

Let me now report some of the numbers from the business meeting: This year saw 740 submissions, continuing a the upwards trend of the last years (for reference, 721 submission in 2025, 580 in 2024, and 479 in 2023). Of these, 211 papers were accepted, giving an acceptance rate of roughly 29%. The reported acceptance rates were 28% for papers without a program-committee author and 34% for papers with at least one PC member as an author. The program committee consisted of 80 members, each assigned between 27 and 33 papers. There were 376 registered participants, most of them from North America. This was below the 490 participants in 2025, which appears to have been an outlier, and broadly in line with earlier years. Of 41 applications for student travel support, 27 were funded. Finally, STOC 2027 will take place in Atlanta.

The conference ended on Saturday with the workshop "Can AI Do Theory?". Scott Aaronson suggested that current AI systems are particularly strong at two things: trying many examples very quickly and recalling and combining facts from different areas. Prabhakar Raghavan presented compelling evidence that these abilities can contribute at the highest research level: an AI system discovered a new gadget that improves the hardness-of-approximation bound for metric TSP from $^{117}/_{116}$ to $^{111}/_{110}$. Most memorable for me was the concluding panel and I want to highlight some ideas that stuck with me. Yael Kalai observed that imperfect AI reviews should not be compared with an imaginary standard of perfect human reviewing; human reviews have serious weaknesses too. At the same time, she warned that asking AI at every opportunity is tempting but may erode our own capabilities in the long run. Jon Kleinberg emphasized that, while accelerating research is desirable, the community must preserve pathways by which young researchers can learn to do research and enter the field. Adding to Scott Aaronsons list of strong sides of AI, the panelists agreed that personalized learning is already a particularly effective application. Kleinberg also gave some encouraging words, especially for younger researchers who might feel threatened by the rapidly changing AI world. He argued that predicting the situation ten years from now is impossible, so remaining adaptive is a better strategy for the future than giving up on shaping it.

To conclude this report, I want to thank the organizers. In particular, Program Committee Chair Artur Czumaj, Local Arrangements Chair Aditya Bhaskara, and TheoryFest Chair Andrew McGregor, but also the many committee members, volunteers, and organizers who made the week possible.

Is there anything else to report? Oh yeah, the opening riddle. Salt Lake City was indeed hot, but as this was only my second trip to the United States, I was unprepared for the formidable power of American conference-venue air

conditioning. Outside, Utah supplied the summer, but inside, the Hilton felt closer to a German winter. My warmest recommendation for future STOC attendees is therefore a literal one: bring a sweater!

Conference report on PODC 2026

Dominick Banasik and Nithin Salevemula

This year's ACM Symposium on Principles of Distributed Computing (PODC) was held from July 6–10 in Egham, England, at the Royal Holloway. It was co-located with ACM Symposium on Parallelism in Algorithms and Architectures (SPAA) and EATCS International Colloquium on Automata, Languages and Programming (ICALP). Attendees of any conference were allowed to attend all three, which gave us all the opportunity to see other keynote talks and ongoing research (although the program was a lot to keep track of).

Figure 2: The Founder's Building at night.

Below are a few highlights:

- The Founder's Building, where we all enjoyed our English breakfast, is beautiful, and it glows in the dark!

- Introducing the Student Spotlight was a great idea. It gave students an opportunity to present their research interests and receive valuable suggestions and feedback from experts. It would have been even better if the poster presentations had been officially scheduled for at least 1 hour during one of the main conference days, as many participants could not attend the Student Spotlight on the first day.

- LLMs were acknowledged for their contributions to multiple papers. *Large language models* were acknowledged for developing a new lower bound technique, and *lukewarm liquid machines* were acknowledged for, presumably, helping to stay awake.

1 Business Meeting

The business meeting ran long. Below are a few key takeways:

- Co-location with SPAA and ICALP was popular, although it made organization more difficult.

- This year's Call for Papers was the first to feature an AI policy for authors. AI policies for authors, reviewers, and the program committee will be further fleshed out for next year.

- There were long discussions about having two submission windows. The idea was not so popular, but we will see how it works out for SPAA.

- Another main topic was the location of future conferences. One proposal for the next next PODC was Crete, Greece. Extended discussions followed about hosting PODC in more central locations more frequently.

2 Interesting Papers (personally)

The following are a few personally interesting talks.

- The talk *"Distributed Approximate Maximum Matching and Minimum Vertex Cover via Generalized Graph Decomposition"* by Peter Davies-Peck shed some light on the necessity of both Δ- and n-dependent terms in the well-known distributed lower bounds for these problems.

- The talk *"Ranking Opinions with Few States in Population Protocols"* by Antoine El-Hayek introduced a neat application of stable matchings for finding the plurality opinion in population protocols.

- The Dijkstra Prize Talk, *"The $\Omega(D + \sqrt{n})$ Lower Bound Story of Distributed Algorithms"* by Gopal Pandurangan, was particularly interesting. The talk provided an overview of the origins of the $\Omega(D + \sqrt{n})$ lower bound and highlighted the wide range of distributed computing problems in which this bound appears.

- The talk *"2-Coloring Cycles in One Round"* by Jukka Suomela was interesting. It presented a one-round randomized distributed algorithm that achieves an expected fraction of monochromatic edges below 0.24118, while showing that no one-round algorithm can achieve a fraction below 0.23879. It was also interesting that the proofs were largely developed using large language models and formally verified in Lean 4.

Viewpoint Column

by

Anca Muscholl and Stefan Schmid

Bordeaux University, France
and TU Berlin, Germany
anca@labri.fr and stefan.schmid@tu-berlin.de

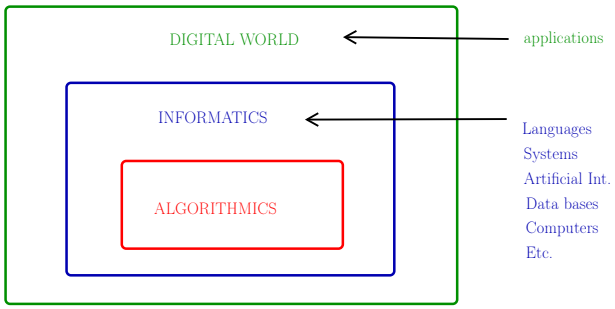
BEATCS no 150

Informatics Is a Very Old Science!

Michel Raynal

Univ Rennes (IRISA, Inria, CNRS) France

What is an algorithm? *Informatics* is the science of algorithms and their applications[1]. If we suppress algorithms, informatics disappears! Informally an algorithm is a *finite sequence of well-defined basic operations* that, under predefined assumptions (defining the context in which the algorithm has to be executed) describes what has to be done (by a human being or a machine) to obtain a result. It is defined by a specification which describes precisely what we want to obtain (e.g. bisect an angle, or sort a set of words in alphabetical order). So an algorithm manipulates symbols (not only numbers or letters), it has inputs and produces outputs. When the inputs and outputs are data, an algorithm can be seen as a data transformer producing an information which has a meaning defined by the specification of the algorithm.[2]



As a very simple example of an algorithm let us consider the multiplication of any pair of integers x and y. We assume that the only basic operations the algorithm can use are restricted to the multiplication of an integer by 2, the division of an integer by 2, and the addition of any pair of integers. To simplify the reading of the algorithm we consider there is a two-column array in which integers can be written. The algorithm is the following sequence of basic computation steps:

[1] According to Leibniz a concept or a specific domain must be identified by a single name (word) and not by a set of words (which can be its definition). That is why I use the word *Informatics* defined as being the *science of computing*. This is consistent with the words we are already using such as medicine, chemistry, biology, mathematics, geology, archeology, philosophy, literature, etc.

[2] In that sense sewing [7] and origami [17] are algorithms. A sequence of weaving cards precisely defines an algorithm (that can be executed by looms). Differently cooking recipes are not algorithms. This is because (among many other issues) the same recipe used by different cooks produces different results!

- We first write x and y at the top of the left and right columns respectively.
- Then, in the left column we write the integer part of the previous value divided by 2, and this is repeated until we obtain the value 1.
- Then, in the right column we write the previous value multiply by 2, and this is repeated until the column is filled.
- Then we cross out in the second column all the numbers associated with an even number on the left column.
- Finally we add all the numbers of the second column which have not been crossed out. This number is the result.

This multiplication algorithm is called *Russian multiplication*. It is used in some computers where, in binary notation, multiply (divide) by 2 consists in a left (right) shift. As we can see, this multiplication algorithm does not use the Pythagorean table traditionally used in many countries[3]. An example is depicted below namely the computation of 41×272. The first table corresponds the division and multiplication by 2 as previously described. The second table describes the crossing out of values. The addition of the remaining non-crossed out values provides the result, namely $272 + 2176 + 8704 = 11\,152$.

41	272
20	544
10	1088
5	2176
2	4352
1	8704

41	272
20	~~544~~
10	~~1088~~
5	2176
2	~~4352~~
1	8704

$$41 \times 272 = 272 + 2176 + 8704 = 11\,152.$$

It is worth noticing that this multiplication algorithm, which relies on base 2, was known and used in Egypt by the scribes and priests in the time of Pharaohs. With our modern notations, the proof of this algorithm is pretty easy. We give it for the specific case of 41×272, but it can be easily generalized to any pair of integers. Using decomposition of integers in base 2 we have (remember $1 = 2^0$):

- $41 = 2^5 + 2^3 + 2^0$,
- $272 = 2^8 + 2^4$,
- $(2^5 + 2^3 + 2^0) \times (2^8 + 2^4) = 2^{13} + 2^{11} + 2^9 + 2^8 + 2^7 + 2^4$,
- and finally we obtain $2^{13} + 2^{11} + 2^9 + 2^8 + 2^7 + 2^4 = 11\,152$.

[3]The Pythagorean table gives for free the product of any pair of integers in the set $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$. There are many algorithms to multiply any two numbers.

On the powers of 2: Mystery or something deeper?

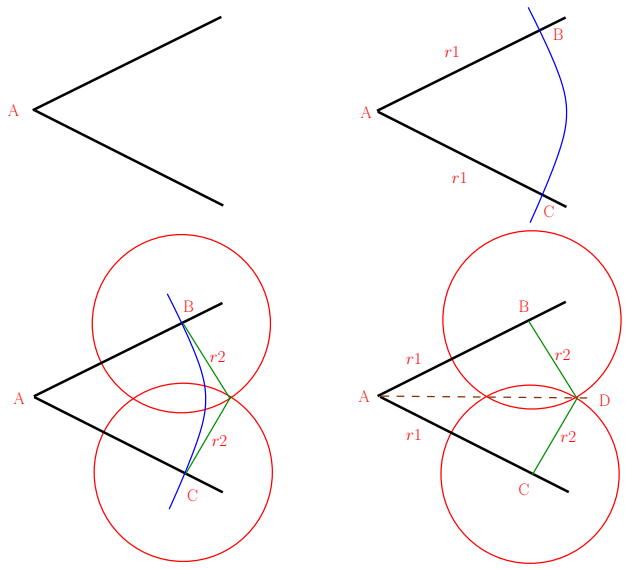
The sequence of consecutive integers from 10 to 14 has the following surprising property

$$10^2 + 11^2 + 12^2 = 13^2 + 14^2 = 365$$

which is the number of days in a calendar year! Sorcery or amusing anecdote?

Theorems (Mathematics) *vs* **Algorithms (Informatics)** There is a strong analogy between theorems and algorithms, namely each is made up of two parts: a statement and an associated correctness proof. Moreover in both cases –theorem/algorithm– it is possible to have different proofs for the same statement. A point in which they differ lies in the fact that, due to its operational nature, an algorithm assumes an underlying notion ot time (needed to execute it), while a theorem does not.

In mathematics books, algorithms devoted to geometry are called constructions. As an example let us consider the bisection of an angle with the operations provided by a ruler (draw straight line segments) and a compass (to measure segment lengths and draw circles). This is one of the geometry problems stated and solved by Ancient Greeks. It consists in dividing an angle in two equal parts, and this has to be done with a ruler and a compass (those constitute the only basic operations that can executed). Let us consider the angle (left top of the figure). The algorithm is the following sequence of operations:
1.Draw an arc of circle of center A of any radius, say $r1$. This arc cut the angle in B and C.
2. Let us now draw two two circles whose centers are B and C such that they intersect (let us note that this is always possible). Let $r2$ this radius.
3. Claim the line AD bisects the angle $\angle$BAC.



Proof of the claim. We have |AB| = |AC| = $r1$ and |BD| = |CD|= $r2$, from which it follows that the triangles ABD and ACD are equal, and consequently the the corresponding angles $\angle$BAD and $\langle$CAD are equal. QED.[4]

[4]The trisection of an angle with only a ruler and a compass is impossible. The notation π, due to William Jones

On the use of integers and letters as symbols: Any representation needs an interpretation Inherited from the Indians the ten digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) allow us to write any number. Differently, the symbols used by the Roman were a set of seven letters, namely I (1), V (5), X (10), L (50), C (100), D (500), M (1000). Then the composition of these letters, satisfying some constraints, denotes an integer. As examples, 1914 is written MCMXIV, 42 is written XLII, 914 is written MCMXIV, 42 is written XLII, 264 is written CCLXIV. It is important to see not all the conbination of letters produces an integer As an example XXVIC is meaningless. It follows from this coding system that multiply XLII (42) by CCLXXII (272) is not an easy task. Doing arithmetic with such a notation is nearly impossible.[5]. Simple notations and symbols are crucial to make life easier.

The meaning of a notation depends on the context. Given a street, the assignment of increasing integers to a sequence of houses makes easy to find the location of a given house. But house numbers are symbols that cannot be added or multiplied. They are not mathematical integers (the multiplication of two house numbers does not give a house number).

A phone number is the name of a phone, but this name has a different meaning according to the type of phone. A landline phone addresses a physical location such as a house or an office. Differently a cell phone is associated with a single person (and not a location).

It is worth noticing that (according to [5]) 30,000 years ago, the symbol used by human beings to represent small integers (1 to 5) where sequences of painted fingers as evidenced by the drawings of the Cosquer and Gargas famous caves, i.e. long before the invention of writing.

The geographic origin of algorithms According to today knowledge it seems that the very first algorithms were designed in Mesopotamia, Egypt, China, India and Peru. Their aims were to solve problems related to inheritance, law (see Hammurabi code), field area computation, economy (exchanges money/goods), and interest rates. The interested reader will consult [8, 10, 20].

The origin of the word algorithm The word *algorithm* comes from Al Khwarizmî (which means that the origin of his family was the region of Khwarem in Uzbekistan). Born in Khiva around 780, he died in Baghdad around 850. He lived during the golden period of the *House of Wisdom* (at that time the Calif of Baghdad was Hârun ar-Rachîd ben Muhammad ben al-Mansûr). It is worth noticing that the book *The Thousand and One Nights* was written in the very same period.

Al Khwarizmî was a polymath interested in many scientific and non-scientific topics: arithmetic, geometry, astronomy, geography, history, Jude calendar. He is the person who introduced Indians numbers in Baghdad and Arabic countries. He was also interested in astrolabes and created a "start-up" to build and sell astrolabes. As a mathematician and astronomer he computed a lot of astronomical tables. He wrote books on many other domains. His main book

in 1706, was made popular Euler. The transcendence of π was proved by Ferdinand von Lindemann in 1882 by showing that it is impossible to build a square that as the same area as a circle of a given radius using only a ruler and a compass.

[5]It is interesting to notice that the number obtained by writing all the symbols (but M) in their decreasing order, namely DCLXVI, is the the number 666 attributed to the *Beast*: hazard!

was the Al Kitab Al-Jabr wa'al-Muqabala. This is the book that gave rise to the word *algebra*. It is in this book that are presented algorithms to solve what we call today quadratic equations.

To attain this goal he identified six patterns of quadratic equations produced six algorithms, one for each pattern. Such a pattern was a construction of a square from rectangles and triangles obtained from the explicit numbers appearing in the equation. Then he tried to solve cubic equations, without success. This was successfully done by Omar Khayyam (1048-1131), who was also the great poet who wrote the *Rubaiyat*. [6]

Mathematics *vs* **physics** The goal of physics is to understand, model and explain the phenomena of the universe. For example, the formulas $F = G\frac{m \times m'}{d^2}$ and $E = mc^2$ are not mathematics but physics. The first captures the law of universal gravitation between two bodies; m and m' are the masses of the two bodies, G is the gravitational constant, and d is the distance between the two bodies. The second captures the equivalence between the mass m of a body and the energy E associated with it (c is the speed of light). In both cases, these are physics formulas and not mathematical formulas, namely they are a conceptual tool that has made it possible to arrive at these physics formulas that enrich our knowledge of the world in which we live.

In the same way, since Turing, thanks to their power, the algorithms have become inescapable and are no longer restricted to their uses in mathematics. As already said, they constitute the heart of *Informatics*, which, in addition to algorithms, includes all its applications).

When a new scientific domain acquires a name A new scientific domains first appear informally and then become autonomous when great results are produced. More precisely, each scientific domain has its

geniuses who produced fundamental results that provide us with deep foundations such that, for each domain, there is the period before and the period after. A few examples appear in the table below.

Physics	Newton (1642-1727)
Chemistry	Lavoisier (1643-1794)
Informatics	Turing (1912-1954)

Mathematicians *vs* **informaticians** While one of the aims of science was to "put the world into equations", we know today that everything cannot be captured by a formula. So, an aim of today science is to put the world into equations and algorithms. Said in another way, Galileo Galileo said "the great book of nature is written in mathematical language". This can be true for physics, not for other sciences such as life sciences. Today, the pair made up of mathematics + informatics seems to be the language of all sciences. As said by Claude Berrou (co-inventor of Turbo-codes) *Informatics is to science what language is to intelligence*.

[6] While Al Khwarizmî had the intuition that solving quadratic equations was related to a two-dimensional space puzzle (to solve with squares, rectangles and triangles), he did not have the intuition that solving cubic equations could be related to a tri-dimensional space.

Among others, informatics and mathematics present an important difference, namely mathematics does not have the touch "run" which transforms an algorithm (written in some programming language) into a finite execution! This has a fundamental consequence, namely informatics evolves in the field of finite time and finite memory. As an example, while mathematics (correctly!) considers π (the Archimedes number) as having an infinite number of digits, any algorithm using π is restricted to consider a finite approximation of it. [7]

To illustrate the previous claim let us consider the following two objects: the object denoted "1+2" and the object denoted "3" with their usual meaning, and let us ask the question: "are these two objects the same object or are they different objects?" Nearly all mathematicians answer "it is the same object", while nearly all informaticians answer "they are different objects". This is due to the simple observation that informaticians see "1+2" as an algorithm which needs to be executed to obtain a result, namely "3". These answers show that answering a question has a bias depending on the cultural and scientific contexts of the person who has to answer. Algorithms are the roots which gave rise to informatics, and this has a long history that started a very long time ago.

ps1. The references not cited in the text are for the readers interested in more developments on the nature and the history of algorithms. The readers interested in the history of *computers* (machines that can execute algorithms) is invited to read the very interesting book [9].

ps2. Computing is not restricted to sequential computing. Parallel computing consists in identifying independent data so that they can be processed simultaneously on a multiprocessor. So the aim of parallel computing is efficiency[8]. Distributed computing (DC) is different: it is the science of *formalized cooperation* among a set of distinct computing entities that need to cooperate to achieve a comon goal. DC was introduced by E. W. Dijkstra in 1965 with the definition of the *mutual exclusion* problem. Since then, thanks to its theoretical foundations DC has become an important domain of informatics. Its most famous problem is *consensus* in the presence of failures. It is shown in [16] that consensus and mutual exclusion can be seen as two sides of the same coin : one suited to physical objects, the other suited to logical objects (objects that are implemented by sequences of bits).

ps3.Informatics is made up of theory and technology. Theory is what allows us to understand the deep nature what we are doing. Technology is what should make our lifes easier.

ps4. From Paul Valéry (1871-1945):
Modern life tends to spare us intellectual effort just as it does for physical effort. For example it replaces imagination with images, reasoning with symbols, writing with mechanics, and often with nothing at all. It offers us every facility and every shortcut to reach our goal without having to travel the whole way. And that is great, but quite dangerous...

[7] As a simple example it is possible to use a finite prefix of the nice Gregory-Leibniz formula which, as it is infinite, is not an algorithm, namely $\pi = \frac{4}{1} - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9}$....

As humorously said in a popular joke, infinity/eternity is very long, mainly at the end!

[8] Any parallel algorithm could be executed on a mono-processor with the help of an appropriate scheduler.

References

[1] Ausiello G., Algoritmi, monaci e mercanti: il calcolo nella vita quotidiana del Medioevo. *Codice Edizioni*, 240 pages (2022)

[2] Behan T.S., A modern introduction to ancient Indian mathematics. *Wiley Eastern Limited*, 214 pages (1995)

[3] Chart J.L. (Editor), A history of algorithms (From the pebble to microchip). Translated from French: Histoire d'algorithmes (608 pages) *Springer* (1999)

[4] Cohen A. and Haberman B., Computer science: a language of technology, *SICS Bulletin* 39(4), 6 pages (2007)

[5] Ferrari S., Il salton. sgni, figure, parole: viaggio all'origine del l'immaginazione. Feltrinelli Editore, 320 pages (2022)

[6] Harel D. and Fieldsman Y., Algorithmic: the spirit of computing. *Springer*, 572 pages (2004)

[7] Harlots-Luck E., Weaving as binary art and the algebra of patterns. *Textile*, 15(2):176-197 (2017)

[8] Ifrah G., Histoire universelle des chiffres, Part one: 1024 pages, Part two:1056 pages. (1981). Translated in English: From one to zero: a universal history of numbers. 503 pages, (1994)

[9] Isaacson W., The innovators, 488 pages, Simon & Schussed (2014)

[10] Knuth D. E., Ancient Babylonian algorithms. *Communications of the ACM*, 15(7) pp. 671–677 (1972)

[11] Knuth D. E., Computer science and its relation to mathematics. *American Mathematical Monthly*, 81:323-343 (1974)

[12] Knuth D. E., Algorithmic thinking and mathematical thinking. *American Mathematical Monthly*, 92:170-181 (1985)

[13] Kumar A., Ancient Indy science: its transmission and impact on world cultures. *Morgan & Clay pool*, 197 pages (2019)

[14] Lamport L., The future of computing: logic or biology. *Text of a talk given at Christian Brecht's University, Kiel* (2003)

[15] Neugebauer O., The exact sciences in antiquity. *Dower Publications*, 240 pages (1957)

[16] Raynal M., Mutual exclusion vs consensus: both sides of the same coin? *Bulletin of the European Association of Theoretical Computer Science (EATCS)*, Vol.140, 14 pages (2023)

[17] Robinson N., The encyclopedia of origami techniques: the complete, fully illustrated guide to the folded paper arts. *Search Press*, 160 pages (2016)

[18] Vardi M. Y., What is theoretical CS? *Communications of ACM* 67-11) page 4 (2024)

[19] Van der Waerden B.L., Geometry and algebra in ancient civilizations. 223 pages, *Springer* (1983)

[20] Zhu Shijie, The jade mirror of the four unknowns. (1303)

[21] Zemanek H., Dixit Algorizmi: His background, his personality, his work and his influence. *Springer LNCS* vol. 122, 1-81 (1981)

The Education Column

BY

Dennis Komm and Thomas Zeume

ETH Zurich, Switzerland and Ruhr University Bochum, Germany
dennis.komm@inf.ethz.ch and thomas.zeume@rub.de

BEATCS no 150

Informatics Education in the Age of AI
Lessons from Austria's New Upper-Secondary Curriculum

Andreas Bollin
Institute of Informatics Didactics
University of Klagenfurt, Austria
Andreas.Bollin@aau.at

Roderick Bloem
Graz University of Technology, Austria
Roderick.Bloem@tugraz.at

Abstract

The rapid rise of artificial intelligence has renewed the question of what students need to understand about computational systems and what role informatics should play in general education. Austria's response is a new upper-secondary subject, "Informatics and AI," extending the existing compulsory informatics provision by one weekly lesson and distributing instruction across grades 9–11. This outcome followed a contested process in which informatics came close to being merged into a broader subject combining digital and media education. The resulting standalone subject combines broad informatics foundations with explicit attention to AI, treating AI as an integral subfield of informatics rather than as a digital application skill. The reform nevertheless remains incomplete: compulsory informatics still ends before the final school year and is not an independent subject in the final school-leaving examination. Drawing on the Austrian experience, we discuss lessons for establishing informatics as a visible discipline, designing curricula, and shaping educational policy in the age of AI.

1 Introduction

The rapid diffusion of artificial intelligence (AI), and generative AI in particular, has renewed educational attention to computational systems. AI literacy has become an important educational goal, encompassing not only AI use, but also an

understanding of its foundations and reflection on its societal and ethical implications [22, 23]. At the European policy level, *DigComp 3.0* similarly integrates AI competence throughout the *European Digital Competence Framework* [6].

Yet, using computational systems is not the same as understanding them. In public debate, informatics is still often equated with the use of digital tools; yet using office software or a chatbot has about as much to do with informatics as touch-typing has with literature. Informatics is part of general education because it provides concepts and methods for representing information, abstracting problems, designing algorithms, constructing computational systems, and reasoning about their capabilities and limitations [2, 12, 21]. Its contribution is not familiarity with technology, but a discipline of thought: students learn to formulate procedures precisely, construct formal systems, and reason about their properties. AI makes this distinction visible: the educational challenge is not merely to use AI-enabled tools, but to understand their capabilities, limitations, and outputs.

The rise of AI, therefore, creates both an opportunity and a challenge for informatics education. It brings renewed attention to the discipline while raising a fundamental curricular question: does general education in a world in which AI increasingly performs computational tasks still require informatics as a visible field, or can this role be assumed by broader approaches to digital and media education? In Austria, this was not hypothetical: until late 2025, the plan was to merge informatics with media education into a joint subject.

We argue that informatics should be a compulsory, independent subject, beginning in lower-secondary education, continuing throughout upper-secondary education, and being available for the final school-leaving examination. This requires sustained instructional time rather than isolated exposure; as a curricular orientation, we consider at least two weekly lessons necessary to address the breadth of the discipline and develop its concepts progressively. The *Informatics Reference Framework for School* similarly calls for informatics to have a status and standing comparable to language and mathematics [5]. Our central thesis is:

The rise of AI does not reduce the need for informatics; it makes that need more visible.

In Austria, however, AI was also used to argue for the opposite conclusion. Which of the two positions prevails is decided not only by the strength of the arguments but also by advocacy.

Austria's recent upper-secondary curriculum reform provides an opportunity to examine this tension. We place the reform in its European context, examine the debate about the role of informatics in schools, and show how the resulting standalone subject, "Informatics and AI," combines broad informatics foundations with explicit attention to AI. Measured against the objectives above, the Austrian reform represents a partial success.

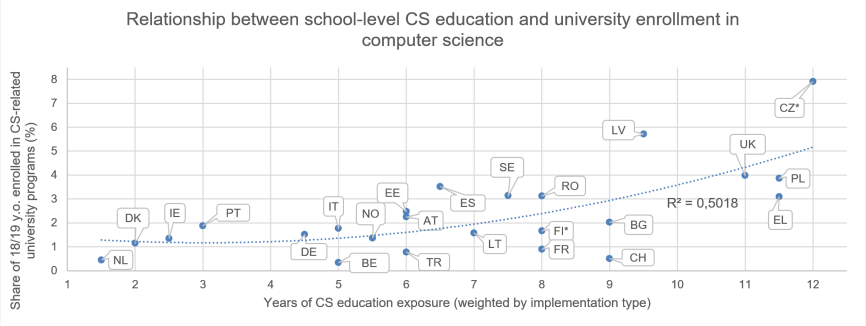


Figure 1: Results from Bollin et al. [3], relating weighted informatics exposure to enrollment in informatics-related university programs across 24 European countries. The fitted trend ($R^2 = 0.5018$) accounts for about half of the variation in enrollment. Exposure is weighted by implementation type, with integrated instruction counting less than standalone compulsory instruction. Asterisks mark countries where mixed forms required exposure to be estimated.

2 Background

2.1 Informatics Education in Europe

Informatics education in Europe has developed substantially over the past decade, but implementation still differs across countries and educational levels. Informatics may be taught separately, integrated into other subjects, or addressed cross-curricularly [9]. European policy increasingly recognizes informatics as a distinct field of education rather than merely the use of digital technologies. The *Council Recommendation* of 2023 calls for high-quality informatics or computational thinking education from the beginning of compulsory education, with clear learning objectives, dedicated time, assessment, and appropriately qualified teachers [7]. More recently, the European Commission's *Guidelines for Teaching Informatics* distinguish informatics from the use of digital technology, and emphasize concepts and practices for understanding how digital technology works [8].

The form and extent of informatics education matter. Across 24 European countries, mandatory informatics was positively associated with enrollment in informatics-related university programs, while no significant association was found for integrative or optional models [3]. Greater compulsory exposure across grades was also associated with higher enrollment (Figure 1). While these observational findings do not establish causality, they support the case for informatics education that is compulsory, sustained across grades, and allocated sufficient curricular time.

Switzerland illustrates a possible time lag: despite substantial school-level exposure, informatics enrollment remains modest, as affected cohorts may not yet have reached higher education. In 2024, informatics was further upgraded from an obligatory to a foundational subject (*Grundlagenfach*) in upper-secondary academic education [20]. Such time lags also argue against delaying curricular reform if future cohorts are to benefit from it.

This European context frames the Austrian reform: the question was not whether informatics should be taught, but whether it receives a distinct curricular place, sufficient instructional time, and continuity across grades.

2.2 The Austrian Curriculum Reform

The upper-secondary reform followed substantial changes in lower-secondary education. In 2022, Austria introduced "Digital Education" (*Digitale Grundbildung*) as a compulsory subject in grades 5–8 (ISCED 2), following its introduction as a mandatory curriculum in 2018 [1, 10]. The subject combines informatics-related content, digital application skills, and media education, reflecting the technological, societal, and interaction-oriented perspectives of the *Frankfurt Triangle* [4]. Its rapid introduction posed challenges for teacher preparation, as many teachers had limited prior training [10].

At upper-secondary level (ISCED 3), compulsory informatics was limited to two weekly lessons in grade 9, with none in grades 10–12 [16]. Informatics could continue as an elective, but did not provide a common education for all students.

The reform process changed direction several times. In 2024, plans initially envisaged six to eight weekly lessons in total across grades 9–12. A complete curriculum based on the *Informatics for All* reference framework had been developed before this allocation was reduced to four weekly lessons. In 2025, the direction changed more fundamentally: informatics and media education were to be combined into a joint subject, discussed under titles such as "Media Education and Informatics" and "Digital Education," with the *Frankfurt Triangle* as a common framework. Disagreement over the role and scope of informatics eventually led to the development group's dissolution in late 2025.

In early 2026, the process restarted with a new group for the standalone subject "Informatics and AI." Its politically chosen title remained contested within the group and the informatics community, who regarded AI as part of informatics rather than a separate discipline. The reform makes the subject compulsory in grades 9–11, with one weekly lesson per grade instead of two in grade 9 only. Media education received its own curricular space through the compulsory subject "Media and Democracy." The changes take effect in September 2027, with "Informatics and AI" introduced progressively by grade [17]. Reaching this outcome required more than curriculum development.

2.3 Advocating for Informatics

Throughout the reform process, representatives of the Austrian informatics community, coordinated by *Informatik Austria*, engaged with the Ministry of Education, political parties, chambers of labor and commerce, trade unions, teacher organizations, and other stakeholders. Beyond contributing disciplinary expertise, they collected evidence and argued that informatics is part of general education and therefore needs a visible place in the curriculum.

Stakeholders did not share a common understanding of informatics. The arguments raised against a visible, standalone informatics subject fell into four recurring categories:

- **Understanding of the discipline.** Informatics concerns ICT use and digital application skills; it is not a scientific discipline.
- **Need for disciplinary visibility.** Informatics can be integrated with other subjects and does not require curricular space of its own.
- **AI as a substitute for informatics.** AI systems perform computational tasks, so students no longer need informatics.
- **Perceptions of the subject.** Informatics is difficult, boring, and unattractive, especially to girls.

We responded with five main arguments:

1. **General education.** Understanding and shaping a world organized through computational systems requires informatics concepts and methods [2]. This was our primary argument, because it derives the need for informatics from the purpose of school education itself.
2. **Autonomy and democratic agency (*Mündigkeit*).** Computational systems allocate credit, rank public debate, sort applicants, and support administrative decisions. They are not natural phenomena: they are designed and encode choices open to political contest. A democracy in which citizens cannot form a judgment about such systems cannot hold those choices to account and is left to defer to whoever built them.
3. **Participation.** Compulsory, early instruction reaches students who otherwise would not choose the subject.
4. **Digital sovereignty.** The systems that run public administration, health care, energy, and finance are largely built and operated elsewhere. A country that cannot build, audit, and maintain such systems cannot choose its

own terms of use and must accept those of its suppliers. That capacity rests on a supply of people who understand the technology, which in turn rests on school education. Sovereignty is therefore an educational question before it is a procurement question.

5. **Skills and economic value.** A study by the *Austrian Economic Chamber* projects a shortfall of some 39 000 IT specialists by 2030 and an annual loss in value added of about seven billion euros.

The economic argument is often reached first. While effective with industry, it invites the objection that general schools educate for life, not jobs. Our response is that informatics is not primarily justified by labor-market demand; understanding the computational systems that shape everyday life is itself part of general education.

A second argument concerned gender and accessibility. We were told that informatics sounds forbidding, particularly to girls, and should therefore be folded into a broader subject, such as media education, so that they encounter it without noticing.

This argument assumes that the difficulty lies with the students and their gender. It withholds what enables students to act on their aptitude: knowing what they have learned and what it is called. A student who does not know she has done informatics cannot choose it, claim it, or build on it. Stereotypes are dismantled by teaching the subject early and visibly, not by removing it from the timetable.

Empirical evidence challenges the underlying assumption. In Germany, compulsory informatics is associated with smaller gender differences in competence and higher participation by girls in upper-secondary informatics [18]. A similar pattern is reported for socioeconomic background. International work on inclusive informatics education finds that participation depends on access to the subject [14], with Northern Ireland as an example [13]. Low participation is therefore not evidence of low interest; it may reflect the conditions under which girls encounter the subject. The remedy is better teaching and a visible subject, not concealment.

AI posed a communication problem. From a disciplinary perspective, AI has long been part of informatics and builds on its foundations. Its public visibility created expectations of a visible curricular response while also feeding the argument that AI reduces the need for informatics. We therefore had to communicate continuity and change at once: *the foundations remain, but their relevance and contexts of application are changing* [2].

A separate argument for integration concerned the proposed merger of informatics and media education itself. The two fields differ fundamentally in method: informatics formalizes, constructs, and tests artifacts, while media education interprets and criticizes texts and social practices. Neither is a special case of the

other. They differ in teacher training, and a merged subject may be taught by whoever is available, causing its content to drift towards the teacher's home discipline. Shared subject matter is not an argument for a shared subject: physics and ethics overlap, as do chemistry and environmental policy, yet no one proposes merging them. This was not a partnership of equals. The underlying conflation of informatics with computer use is illustrated by the position that computational thinking is a synonym for digital competency and a subfield of media competency [19].

The decisive opposition was not about informatics at all. Additional periods had to come from somewhere, and the ministry proposed reducing curricular space for Latin or other languages. The teachers' union objected vehemently, calling it an assault on the "Gymnasium." A petition against reductions in Latin received over 40 000 signatures, including three Austrian Nobel Prize winners. This created an unfortunate and inaccurate perception of informatics as an adversary of classical education.

Our response was not to argue against Latin. The case for Latin rests on sound educational grounds: structure, precision, and understanding of formal systems. Those grounds support informatics equally. In informatics, the formal system is also executable, so imprecision produces immediate and non-negotiable feedback, and students build systems instead of only analyzing inherited ones.

An important argument by the unions was university readiness, which we could answer directly. Informatics is among the largest fields of study, and computational methods are used across the natural sciences, engineering, economics, medicine, and increasingly the humanities. Students enter these programs without systematic preparation or a way to certify it. University readiness thus supports the inclusion of informatics in the final school-leaving examination.

Timetables are zero-sum, so a new subject creates opponents who have no quarrel with its content. The outcome, therefore, cannot be attributed to the strength of our arguments alone.

The process showed that sound arguments do not automatically make good policy arguments. Different concerns required different answers, and the order mattered. Disciplinary visibility cannot be assumed; it has to be argued for in terms the addressees already care about.

3 What "Informatics and AI" Means

Establishing a visible place for informatics was only one part of the challenge. The stakeholder debate also revealed different understandings of what informatics encompasses. The curriculum, therefore, had to answer a second question: what constitutes a broad and contemporary informatics education, and how should AI

fit within it? Rather than defining this scope from scratch, the Austrian curriculum builds on international reference frameworks and explicitly incorporates AI.

3.1 A Broad Informatics Foundation

The curriculum was deliberately designed as a broad informatics curriculum rather than one focused on programming, digital skills, or AI tools. It treats informatics as a scientific discipline with its own concepts, methods, and models, spanning theoretical, practical, technical, and applied informatics. Its disciplinary scope builds on the *Informatics Reference Framework for School* developed by the *Informatics for All* coalition [5]. Its eleven core topic areas provide an internationally developed reference for the breadth of school informatics, extending well beyond programming and digital technology use.

The Austrian curriculum complements these areas with *eight foundational concepts*: abstraction and modeling; data and information; algorithmization and automation; structuring and decomposition; formalization and language; systems and interactions; networking and communication; and uncertainty, data-driven modeling, and responsibility. They recur across topic areas at increasing levels of abstraction and differentiation in grades 9–11. Topic areas provide breadth, while foundational concepts support connections and cumulative progression.

This structure prioritizes systematic concepts and methods over specific software products. While technologies change, abstraction, representation, algorithms, modeling, and systems provide more enduring structures for understanding them.

3.2 Extending the Framework Towards AI

The international reference framework provided a broad disciplinary basis, but did not include AI as a core topic area [5]. The Austrian development group therefore complemented it with the UNESCO *AI Competency Framework for Students* [11] and its experience with school-based AI projects in Germany, linking AI competencies to informatics concepts and methods.

The curriculum introduces *Artificial Intelligence* as one of eleven central topic areas. To retain the reference framework's compact structure, its separate *Algorithms* and *Programming* areas were combined into *Algorithms and Data Structures, and Programming*. AI thus gains explicit curricular visibility without displacing another major area of informatics.

The AI competencies follow a deliberate progression rather than focusing on currently prominent tools. In the compulsory subject, students move from decision trees, classification, training, validation, and test data in grade 9 to supervised and unsupervised learning in grade 10. In grade 11, they examine neural networks and generative AI, compare AI approaches, and justify their suitability

Table 1: AI competency progression in Austrian informatics (ISCED 3).

Grade	Curriculum	Competency (Students can…)	Application Areas
9	Compulsory	Create, apply, and evaluate simple AI models using an algorithm and assess AI systems with regard to their suitability as well as ethical and inclusive aspects.	Decision trees; training, validation, and test data; classification.
10	Compulsory	Carry out basic machine-learning procedures step by step using appropriate algorithms and explain how they work.	Supervised learning (perceptron as a linear classifier); unsupervised learning (clustering algorithms).
	Elective	Model problems using facts and rules, implement them as a knowledge base, and derive solutions from it.	Knowledge-based systems.
11	Compulsory	Explain how neural networks and generative AI work, analyze their outputs, including errors and biases, and reflect on impacts on the world of work. They can compare AI applications and justify suitable approaches for given problem types.	Neural networks; forward propagation; generative AI; AI in the workplace.
	Elective	Implement selected machine-learning algorithms, apply them to data, and explain how they work.	Implementation of machine-learning methods; reinforcement learning.
12	Elective	Investigate the influence of training data and selected hyperparameters on learned models and compare supervised classification methods based on their decision boundaries.	Advanced supervised classification methods.

for different problems. The elective curriculum extends this progression towards implementation and deeper investigation, including knowledge-based systems, machine-learning algorithms, reinforcement learning, and the effects of training data and model parameters. The complete progression is shown in Table 1.

This visibility does not imply treating AI as a separate discipline. The curriculum defines AI as an integral field of informatics, with models and methods that build on foundational informatics principles. AI is therefore distinct enough

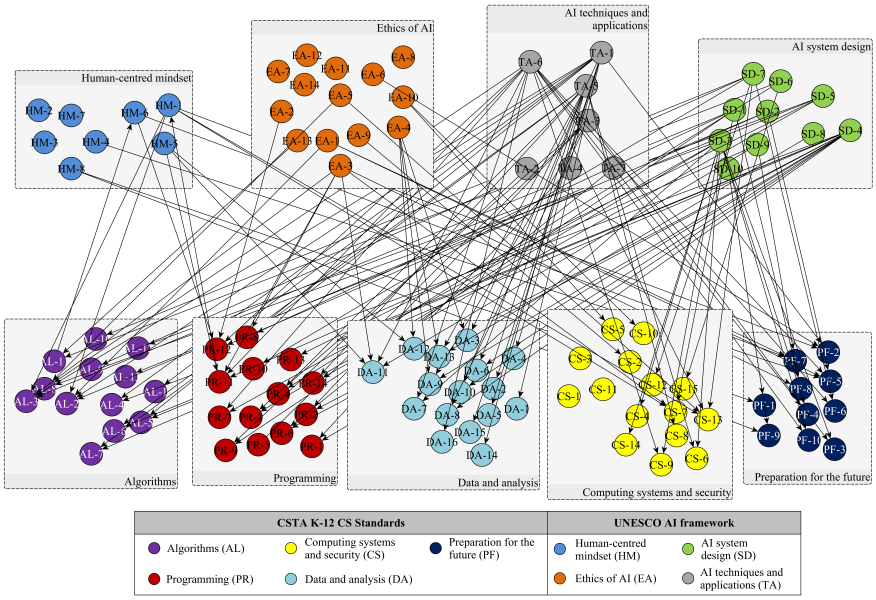


Figure 2: Dependencies between foundational informatics and AI competencies identified by Pasterk et al. [15].

to require explicit curricular visibility, but not independent enough to be taught without its informatics foundations.

The aim is not proficiency with a particular AI tool, but an understanding of different AI approaches that enables students to make informed choices about their suitability for particular problems.

3.3 AI as Part of Informatics

AI as an integral part of informatics is further supported by a systematic comparison of informatics and AI competency frameworks. Pasterk et al. compared the UNESCO *AI Competency Framework for Students* with the CSTA *Reimagining CS Pathways* framework and identified 130 direct dependencies, 127 connecting a foundational informatics competence to an AI competence [15]. Figure 2 shows that these dependencies span informatics areas including data analysis, algorithms, programming, modeling and abstraction, and computer systems.

These dependencies have a direct educational implication. Foundational informatics knowledge enables students to move beyond using AI systems to understanding and evaluating them. Concepts such as data representation, algorithms,

modeling, uncertainty, testing, and evaluation enable reasoning about their capabilities and limitations, and link societal and ethical questions (bias, fairness, transparency, and responsibility) to technical issues of data quality, model assumptions, algorithms, and system design.

AI also provides contemporary contexts for applying established informatics concepts: recommendation systems connect data, algorithms, and modeling; generative AI makes uncertainty and evaluation visible; and AI-generated programs raise questions of testing, debugging, and correctness. The curricular response is therefore neither to add AI tool use to an unchanged curriculum nor to replace informatics foundations with general AI literacy. AI becomes both an object of study and a context for applying foundational informatics concepts.

4 Conclusion

Austria's new subject "Informatics and AI" emerged from a contested process concerning both the curricular position of informatics and the educational response to AI. The resulting curriculum combines broad and enduring informatics foundations with explicit visibility for AI. Three lessons may be relevant for countries in which similar debates are only beginning.

4.1 Lessons Learned

Make informatics visible, and learn to advocate for it. European comparative evidence shows that the curricular form, continuity, and instructional time of informatics education matter [3]. The Austrian experience nevertheless showed that evidence alone does not make an effective policy argument. Disciplinary visibility cannot be taken for granted; it has to be argued for.

Curricular reform also negotiates scarce instructional time. Strengthening informatics may require reallocating lessons from established subjects, turning disciplinary arguments into concrete negotiations over the timetable.

Effective advocacy therefore required communication beyond academic discourse. Arguments were condensed into simple cards, supported by concrete examples, and adapted to stakeholders, emphasizing educational opportunities, economic consequences, girls' participation, technological dependence, or the ability to understand and shape computational systems. This addressed not only evidence, but also concerns, aspirations, and perceived risks.

It also required persistence: stakeholders were approached repeatedly to establish a shared understanding of informatics. For researchers and informatics experts, this may require unfamiliar engagement: scientifically sound arguments

must remain the foundation while becoming concrete, memorable, and relevant to educational decision-makers.

Use AI to strengthen informatics foundations, not to replace them. The rapid development of AI can itself provide an argument for informatics education. The educational choice is not merely whether students should use AI systems, but whether they should also understand, evaluate, and eventually help shape them. Contemporary AI illustrates this need: outputs require evaluation, errors need to be detected, and capabilities and limitations cannot be understood from application skills alone.

The Austrian curriculum, therefore, treats AI as both an object of study and a contemporary context for established informatics concepts. This shifts the focus from particular AI tools towards the knowledge needed to understand, evaluate, and shape computational systems. AI thus provides a reason to reinforce and extend informatics foundations rather than replace them.

Build interdisciplinarity on disciplinary foundations and capacity. Digitalization and AI raise questions that extend beyond informatics and benefit from perspectives from media education, civic education, mathematics, economics, and other disciplines. Interdisciplinary learning is valuable, but presupposes disciplinary foundations. Students need concepts, methods, and sufficient curricular time before they can meaningfully connect them across disciplines.

The same applies to implementation. Interdisciplinary approaches depend on teachers with the necessary disciplinary expertise. Where this is not available, substantial teacher education and professional development are required. Otherwise, the depth and emphasis of integrative instruction may depend too strongly on individual teacher backgrounds. Interdisciplinarity should therefore build on disciplinary competence, not substitute for it.

4.2 A Final Remark

Austria should not be understood as a blueprint for other educational systems. Important gaps also remain: lower-secondary education still does not provide a standalone informatics subject, and informatics is not an independent subject in the final school-leaving examination. Curriculum reforms depend on national school structures, institutional responsibilities, instructional time, teacher education, and political priorities. The Austrian experience nevertheless suggests a broader strategy: establish what informatics contributes to general education, communicate this in terms that different stakeholders can relate to, and respond to AI by making both the technology and its underlying informatics foundations visible. Indeed, the rise of AI does not reduce the need for foundational informatics education; it makes that need more visible.

References

[1] Austrian Federal Ministry of Education. Digitale Grundbildung, 2022. Compulsory subject in lower-secondary education, introduced from the 2022/23 school year. URL: https://www.bmb.gv.at/Themen/schule/zrp/dibi/dgb/.

[2] R. Bloem and J. Hromkovič. Warum Informatik zur Allgemeinbildung gehört. Frankfurter Allgemeine Zeitung, 2026. 2 September 2026, p. 32.

[3] A. Bollin, N. A. Lobnig, S. Pasterk, and M. Wieser. Making Computer Science Count: Curriculum Implementation and University Enrolment across Europe. In D. Passey, M. Leahy, and T. Keane, editors, *Rethinking Education through the Lens of AI and Other Breakthrough Technologies*, volume 785 of *IFIP Advances in Information and Communication Technology*. Springer, 2027.

[4] T. Brinda, N. Brüggen, I. Diethelm, T. Knaus, S. Kommer, C. Kopf, P. Missomelius, R. Leschke, F. Tilemann, and A. Weich. Frankfurt-Dreieck zur Bildung in der digital vernetzten Welt: Ein interdisziplinäres Modell. In T. Knaus and O. Merz, editors, *Schnittstellen und Interfaces: Digitaler Wandel in Bildungseinrichtungen*, volume 7 of *FraMediale*, pages 157–167. kopaed, München, 2020.

[5] M. E. Caspersen, I. Diethelm, J. Gal-Ezer, A. McGettrick, E. Nardelli, D. Passey, B. Rovan, and M. Webb. Informatics reference framework for school. Informatics for All Coalition, Feb. 2022. Published by Informatics Europe. doi:10.1145/3592625.

[6] J. Cosgrove and R. Cachia. DigComp 3.0: European digital competence framework – fifth edition. Technical Report JRC144121, Publications Office of the European Union, Luxembourg, 2025. doi:10.2760/0001149.

[7] Council of the European Union. Council recommendation of 23 november 2023 on improving the provision of digital skills and competences in education and training. Official Journal of the European Union, 2023. C/2024/1030, published 23 January 2024.

[8] European Commission, Directorate-General for Education, Youth, Sport and Culture. *Guidelines for Teaching Informatics: Practical Strategies for European Classrooms*. Publications Office of the European Union, Luxembourg, 2026. doi:10.2766/3365988.

[9] European Commission/EACEA/Eurydice. *Informatics Education at School in Europe: Eurydice Report*. Publications Office of the European Union, Luxembourg, 2022. doi:10.2797/268406.

[10] C. Hörmann, L. Kuka, E. Schmidthaler, and B. Sabitzer. Digital Education Training for Teachers – Learnings from Austria. *Frontiers in Education*, 9:1490123, 2024. doi:10.3389/feduc.2024.1490123.

[11] F. Miao, K. Shiohira, and N. Lao. *AI Competency Framework for Students*. UNESCO, Paris, 2024. doi:10.54675/JKJB9835.

[12] S. Papert. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, New York, 1980.

[13] D. Passey. IFIP Task Force on 'Sustaining Relevant Digital Inclusive Education for Young People (5–18 Years of Age)': Northern Ireland Case Study. Technical report, International Federation for Information Processing, 2025. doi:10.52545/2-9.

[14] D. Passey, L. Soares Barbosa, J. Dalvean, A. Gwanzura, G. Lane, D. Hettihewa, A. Sodiya, and A. Min Tjoa. Sustaining relevant digital inclusive education for young people (5–18 years of age): Final report. Technical report, International Federation for Information Processing, 2024. URL: https://inria.hal.science/hal-04736825v1.

[15] S. Pasterk, C. Mößlacher, and A. Bollin. Aligning computer science curricula with ai competency frameworks: Overlaps, gaps, and future priorities. In *Proceedings of the 2026 IEEE Frontiers in Education Conference (FIE)*, 2026. Forthcoming.

[16] Republic of Austria. Academic Upper-Secondary Curriculum, 2026. URL: https://www.ris.bka.gv.at/NormDokument.wxe?Abfrage=Bundesnormen&Anlage=1&Gesetzesnummer=10008568&Uebergangsrecht=.

[17] Republic of Austria. BGBl. I No. 76/2026, 2026. URL: https://www.ris.bka.gv.at/Dokument.wxe?Abfrage=BgblAuth&Dokumentnummer=BGBLA_2026_I_76.

[18] E. Schröder, F. Suessenbach, and M. Winde. Informatik für alle! Informatikunterricht zur gesellschaftlichen Teilhabe und Chancengleichheit. Policy Paper 05, Stifterverband, Sept. 2022. URL: https://www.stifterverband.org/sites/default/files/informatik_fuer_alle.pdf.

[19] C. Swertz. Medienkompetenz und digitale Bildung aus medienpädagogischer Perspektive. Bericht für das Grünbuch "Digitalisierung und Politik" des Zukunfts- und Verfassungsausschuss des österreichischen Bundesrates. *Medienimpulse*, 1, 2017. doi:10.25656/01:15549.

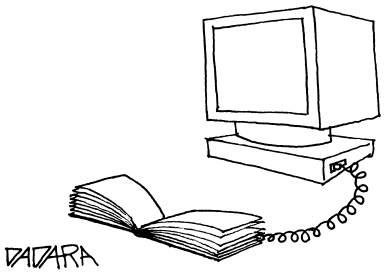
[20] Swiss Federal Council. Revised Regulations for the Swiss Matura, 2023. URL: https://www.sbfi.admin.ch/de/rechtliche-grundlagen.

[21] J. M. Wing. Computational thinking. *Communications of the ACM*, 49(3):33–35, 2006. doi:10.1145/1118178.1118215.

[22] I. H. Y. Yim. A critical review of teaching and learning artificial intelligence (ai) literacy: Developing an intelligence-based ai literacy framework for primary school education. *Computers and Education: Artificial Intelligence*, 7:100319, 2024. doi:10.1016/j.caeai.2024.100319.

[23] X. Zhou, Y. Li, C. S. Chai, and T. K. F. Chiu. Defining, enhancing, and assessing artificial intelligence literacy and competency in k–12 education from a systematic review. *Interactive Learning Environments*, 33(10):5766–5788, 2025. doi:10.1080/10494820.2025.2487538.

News and Conference Reports



Report from the Italian Chapter

L. Geatti and C. Piazza (University of Udine)

ICTCS 2026

The 27th Italian Conference on Theoretical Computer Science (ICTCS 2026) was held in Udine, Italy, from the 7th to the 9th of September 2026 (`https://ictcs2026.uniud.it/`).

Participants at ICTCS 2026 in Udine.

ICTCS is the conference of the Italian Chapter of the European Association for Theoretical Computer Science and, besides being a forum for the exchange of ideas among researchers working in different areas of theoretical computer science, it provides an ideal environment where junior researchers and PhD students can meet senior researchers. ICTCS is open to researchers from all countries, who are welcome to submit papers and attend the conference.

This year we had 71 submissions. Of these, 44 papers were accepted, including 27 regular papers and 17 short papers. Each submission entering the peer-review process was assigned to at least three reviewers, and the review process resulted in a total of 209 reviews, 51 of which were provided by sub-reviewers.

Among the highest-weight relevant keyphrases identified by EasyChair were polynomial time, state preparation, block encoding, quantum state preparation, modal logic, reactive synthesis, probabilistic programs, regular languages, non-interference, directed edge-disjoint paths, fixed-length window, non-idempotent intersection types, Wheeler transducers, complexity classes, and temporal logic.

Most of the authors were affiliated with Italian universities. Authors affiliated with institutions outside Italy came from a wide range of countries, including

Austria, Belgium, Brazil, Canada, France, Germany, India, Israel, Japan, Kuwait, Poland, Portugal, Tunisia, the United Kingdom, and the United States of America, among others.

As a new feature of this edition, ICTCS 2026 introduced three special tracks devoted to significant application domains of theoretical computer science: Cyber-Physical Systems, Quantum Computing, and Bioinformatics. The aim was to attract contributions that, while motivated by applied research problems, address substantial theoretical questions.

The programme included the invited lectures of Parosh Aziz Abdulla (Uppsala University), who gave the talk "Quantum Circuit Verification: Challenges and Opportunities", and Nadia Pisanti (University of Pisa), who gave the talk "Alignments to and among Degenerate Strings".

The conference also hosted the annual meeting of the Italian Chapter of the European Association for Theoretical Computer Science. During the conference, the President of the Italian Chapter of EATCS, Blerina Sinaimeri, presented the IC-EATCS awards.

IC-EATCS awards

The IC-EATCS Best Young Researcher Award was assigned to Giuseppe Perelli (Sapienza University of Rome), who gave the talk "Mechanism Redesign: Repairing Techniques for Synthesis and Rational Synthesis".

The IC-EATCS Best PhD Dissertation Award was assigned to Matteo Russo (EPFL), who gave the talk "Adaptive and scalable algorithms for little, uncertain and strategic data".

The IC-EATCS Best MSc Thesis Award was assigned to Beniamino Todone (University of Udine), who gave the talk "QAOA and Approximability".

We wish to thank the Program Committee members, all the sub-reviewers, the invited speakers, the authors of the papers, and all the participants. We also gratefully acknowledge the support of Danieli Automation, beanTech, and the Department of Mathematics, Computer Science and Physics (DMIF) of the University of Udine.

The next edition of ICTCS is planned to be held at the University of Camerino. We look forward to welcoming the community to ICTCS 2027 in Camerino!

Report on CIAA 2026

30th Conference on Implementation and Application of Automata
Kingston, ON, Canada, August 5–8, 2026

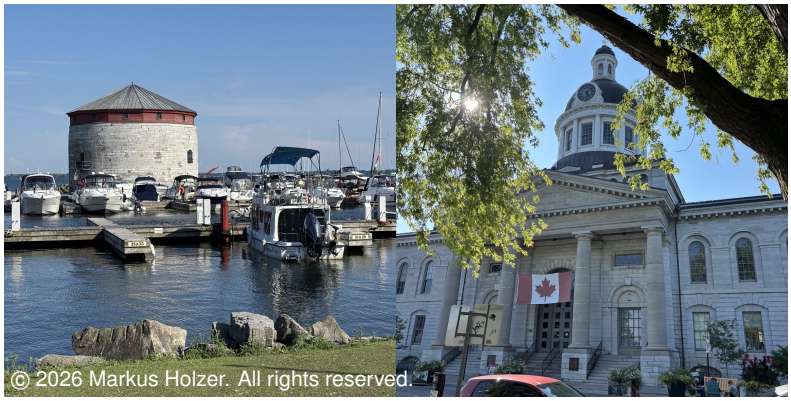
Markus Holzer

The 30th International Conference on Implementation and Application of Automata (CIAA) was held at Queen's University on 5–8 August 2026, back to back with the 27th International Conference on Descriptional Complexity of Formal Systems (DCFS), held at the same venue immediately afterwards. CIAA drew 32 participants from 15 countries.

CIAA 2026 was organized by Queen's University's School of Computing under the direction of Taylor J. Smith and Salimur Choudhury. Of the submissions received, 14 were accepted, and the scientific programme was rounded out by three invited talks: Anthony Bonato on *Burning, Cooling, and Liminal Burning Graphs*, Nataša Jonoska on *Traversing Crystals by Automata*, and Ian Mertz on *Computing with a Full Hard Drive*. The proceedings appeared as volume 16695 of Lecture Notes in Computer Science (LNCS) from Springer-Verlag, and a special issue of selected contributions is planned for Theoretical Computer Science (TCS). The invitation to contribute has already been sent out. The best paper

award went to *Bandwidth of nondeterministic finite automata* by Da-Jung Cho, Szilárd Zsolt Fazekas, Daihei Ise, Shinnosuke Seki, Wataru Tamehira, and Max Wiedenhöft—congratulations to the authors.

Fittingly for a conference celebrating its 30th edition, this year's CIAA also gave participants a taste of its host city's much longer history. Queen's University itself, founded in 1841 by royal charter of Queen Victoria, is one of Canada's oldest universities and predates the country by 26 years; its campus sits on the shore of Lake Ontario, right at the point where the lake narrows into the St. Lawrence River. Kingston is older still: settled by United Empire Loyalists in the 1780s on the site of the former French trading post Fort Frontenac, it briefly served as the first capital of the Province of Canada (1841–1844) before the seat of government moved on, and it later became home to the Royal Military College of Canada and to Canada's first prime minister, Sir John A. Macdonald, who practised law in the city. Its downtown core of nineteenth-century limestone buildings has earned Kingston the nickname the "Limestone City," and the city today serves as the gateway to the Thousand Islands, the archipelago of roughly 1,800 islands scattered along the St. Lawrence between Lake Ontario and Brockville.

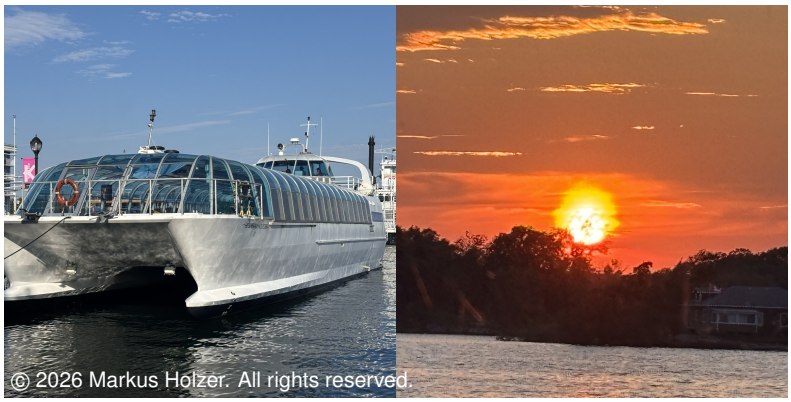


That sense of history was matched by a rather informal atmosphere inside the conference room itself. Rather than the usual lecture-hall layout of rows facing a single screen, the contributed talks were held in a room set up with round tables seating six to eight participants each, with the slides projected simultaneously onto four screens around the walls so that every table had a clear line of sight to one of them. Sitting face to face rather than in rows made it easy to lean over and discuss a point with one's neighbours during and between talks, and the arrangement noticeably encouraged interaction throughout the scientific programme, not just during the breaks. Complimentary coffee breaks and lunches kept everyone together and free to focus on the science rather than on finding a place to eat,

the weather cooperated throughout, and the overall atmosphere was warm and friendly.

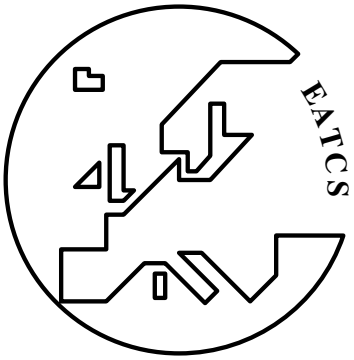
One of the highlights of the week was a special session held in honour of Kai Salomaa, celebrating his career and his many contributions to automata theory and formal language theory over the years—not least his long service to the CIAA community itself. Colleagues, collaborators, and former students gathered to mark the occasion, and a Festschrift, *From Possibility to Reality* (LNCS volume 16865), was presented to him during the session. The warmth and genuine affection in the room made clear just how deeply Kai's generosity, mentorship, and scientific insight have shaped both the field and the people in it.

The conference dinner offered a fitting close to the programme: a Thousand Islands dinner cruise departing from downtown Kingston, just a short walk from the conference venue. As the boat pulled away from the harbour, the Kingston skyline receded behind it and the St. Lawrence widened out into the maze of granite islets and summer cottages that gives the Thousand Islands their name. Participants sailed past the small cottage islands of the Admiralty Group as well as some of the grander turn-of-the-century summer estates further along the river, while dinner was served over multiple courses at tables around the deck—a relaxed setting to continue the day's conversations, far removed from a conventional banquet hall. As the meal went on, the sun set over the water, turning the channels between the islands orange and then pink before the boat turned back toward Kingston in the dark: a scenic and memorable close to the conference.



We thank everyone who made CIAA 2026 such a success. The conference was sponsored by Queen's University's School of Computing, Queen's University, and Springer-Verlag. During the business meeting it was announced that next year's CIAA will take place in Stellenbosch, South Africa, from September 20 to 23, 2027.

European Association for Theoretical Computer Science



E A T C S

EATCS

HISTORY AND ORGANIZATION

EATCS is an international organization founded in 1972. Its aim is to facilitate the exchange of ideas and results among theoretical computer scientists as well as to stimulate cooperation between the theoretical and the practical community in computer science.

Its activities are coordinated by the Council of EATCS, which elects a President, Vice Presidents, and a Treasurer. Policy guidelines are determined by the Council and the General Assembly of EATCS. This assembly is scheduled to take place during the annual **I**nternational **C**olloquium on **A**utomata, **L**anguages and **P**rogramming (ICALP), the conference of EATCS.

MAJOR ACTIVITIES OF EATCS

- Organization of ICALP;
- Publication of the "Bulletin of the EATCS;"
- Award of research and academic career prizes, including the EATCS Award, the Gödel Prize (with SIGACT), the Presburger Award, the EATCS Distinguished Dissertation Award, the Nerode Prize (joint with IPEC) and best papers awards at several top conferences;
- Active involvement in publications generally within theoretical computer science.

Other activities of EATCS include the sponsorship or the cooperation in the organization of various more specialized meetings in theoretical computer science. Among such meetings are: CIAC (Conference of Algorithms and Complexity), CiE (Conference of Computer Science Models of Computation in Context), DISC (International Symposium on Distributed Computing), DLT (International Conference on Developments in Language Theory), ESA (European Symposium on Algorithms), ETAPS (The European Joint Conferences on Theory and Practice of Software), LICS (Logic in Computer Science), MFCS (Mathematical Foundations of Computer Science), WADS (Algorithms and Data Structures Symposium), WoLLIC (Workshop on Logic, Language, Information and Computation), WORDS (International Conference on Words).

Benefits offered by EATCS include:

- Subscription to the "Bulletin of the EATCS;"
- Access to the Springer Reading Room;
- Reduced registration fees at various conferences;
- Reciprocity agreements with other organizations;
- 25% discount when purchasing ICALP proceedings;
- 25% discount in purchasing books from "EATCS Monographs" and "EATCS Texts;"
- Discount (about 70%) per individual annual subscription to "Theoretical Computer Science;"
- Discount (about 70%) per individual annual subscription to "Fundamenta Informaticae."

Benefits offered by EATCS to Young Researchers also include:

- Database for Phd/MSc thesis
- Job search/announcements at Young Researchers area

(1) THE ICALP CONFERENCE

ICALP is an international conference covering all aspects of theoretical computer science and now customarily taking place during the second or third week of July. Typical topics discussed during recent ICALP conferences are: computability, automata theory, formal language theory, analysis of algorithms, computational complexity, mathematical aspects of programming language definition, logic and semantics of programming languages, foundations of logic programming, theorem proving, software specification, computational geometry, data types and data structures, theory of data bases and knowledge based systems, data security, cryptography, VLSI structures, parallel and distributed computing, models of concurrency and robotics.

SITES OF ICALP MEETINGS:

- Paris, France 1972
- Saarbrücken, Germany 1974
- Edinburgh, UK 1976
- Turku, Finland 1977
- Udine, Italy 1978
- Graz, Austria 1979
- Noordwijkerhout, The Netherlands 1980
- Haifa, Israel 1981
- Aarhus, Denmark 1982
- Barcelona, Spain 1983
- Antwerp, Belgium 1984
- Nafplion, Greece 1985
- Rennes, France 1986
- Karlsruhe, Germany 1987
- Tampere, Finland 1988
- Stresa, Italy 1989
- Warwick, UK 1990
- Madrid, Spain 1991
- Wien, Austria 1992
- Lund, Sweden 1993
- Jerusalem, Israel 1994
- Szeged, Hungary 1995
- Paderborn, Germany 1996
- Bologne, Italy 1997
- Aalborg, Denmark 1998
- Prague, Czech Republic 1999
- Genève, Switzerland 2000
- Heraklion, Greece 2001
- Malaga, Spain 2002
- Eindhoven, The Netherlands 2003
- Turku, Finland 2004
- Lisabon, Portugal 2005
- Venezia, Italy 2006
- Wrocław, Poland 2007
- Reykjavik, Iceland 2008
- Rhodes, Greece 2009
- Bordeaux, France 2010
- Zürich, Switzerland 2011
- Warwick, UK 2012
- Riga, Latvia 2013
- Copenhagen, Denmark 2014
- Kyoto, Japan 2015
- Rome, Italy 2016
- Warsaw, Poland 2017
- Prague, Czech Republic 2018
- Patras, Greece 2019
- Saarbrücken, Germany (virtual conference) 2020
- Glasgow, UK (virtual conference) 2021
- Paris, France 2022
- Paderborn, Germany 2023
- Tallinn, Estonia 2024
- Aarhus, Denmark 2025
- Royal Holloway, University of London 2026

(2) THE BULLETIN OF THE EATCS

Three issues of the Bulletin are published annually, in February, June and October respectively. The Bulletin is a medium for *rapid* publication and wide distribution of material such as:

- EATCS matters;
- Technical contributions;
- Columns;
- Surveys and tutorials;
- Reports on conferences;
- Information about the current ICALP;
- Reports on computer science departments and institutes;
- Open problems and solutions;
- Abstracts of Ph.D. theses;
- Entertainments and pictures related to computer science.

Contributions to any of the above areas are solicited, in electronic form only according to formats, deadlines and submissions procedures illustrated at `http://www.eatcs.org/bulletin`. Questions and proposals can be addressed to the Editor by email at `bulletin@eatcs.org`.

(3) OTHER PUBLICATIONS

EATCS has played a major role in establishing what today are some of the most prestigious publication within theoretical computer science.

These include the *EATCS Texts* and the *EATCS Monographs* published by Springer-Verlag and launched during ICALP in 1984. The Springer series include *monographs* covering all areas of theoretical computer science, and aimed at the research community and graduate students, as well as *texts* intended mostly for the graduate level, where an undergraduate background in computer science is typically assumed.

Updated information about the series can be obtained from the publisher.

The editors of the EATCS Monographs and Texts are now M. Henzinger (Vienna), J. Hromkovič (Zürich), M. Nielsen (Aarhus), G. Rozenberg (Leiden), A. Salomaa (Turku). Potential authors should contact one of the editors.

EATCS members can purchase books from the series with 25% discount. Order should be sent to:

Prof.Dr. G. Rozenberg, LIACS, University of Leiden,
P.O. Box 9512, 2300 RA Leiden, The Netherlands

who acknowledges EATCS membership and forwards the order to Springer-Verlag.

The journal *Theoretical Computer Science*, founded in 1975 on the initiative of EATCS, is published by Elsevier Science Publishers. Its contents are mathematical and abstract in spirit, but it derives its motivation from practical and everyday computation. Its aim is to understand the nature of computation and, as a consequence of this understanding, provide more efficient methodologies.

The Editor-in-Chief of the journal currently are D. Sannella (Edinburgh), L. Kari and P.G. Spirakis (Patras).

ADDITIONAL EATCS INFORMATION

For further information please visit `http://www.eatcs.org`, or contact the President of EATCS:

Prof. Giuseppe F. Italiano,
Email: `president@eatcs.org`

EATCS MEMBERSHIP

DUES

The dues are € 40 for a period of one year (two years for students / Young Researchers). Young Researchers, after paying, have to contact `secretary@eatcs.org`, in order to get additional years. A new membership starts upon registration of the payment. Memberships can always be prolonged for one or more years.

In order to encourage double registration, we are offering a discount for SIGACT members, who can join EATCS for € 35 per year. We also offer a five-euro discount on the EATCS membership fee to those who register both to the EATCS and to one of its chapters. Additional € 35 fee is required for ensuring the *air mail* delivery of the EATCS Bulletin outside Europe.

HOW TO JOIN EATCS

You are strongly encouraged to join (or prolong your membership) directly from the EATCS website `www.eatcs.org`, where you will find an online registration form and the possibility of secure online payment. Alternatively, contact the Secretary Office of EATCS:

Mrs. Efi Chita,
Computer Technology Institute & Press (CTI)
1 N. Kazantzaki Str., University of Patras campus,
26504, Rio, Greece
Email: `secretary@eatcs.org`,
Tel: +30 2610 960333, Fax: +30 2610 960490

If you are an EATCS member and you wish to prolong your membership or renew the subscription you have to use the Renew Subscription form. The dues can be paid via paypal and all major credit cards are accepted.

For adittional information please contact the Secretary of EATCS:

Dmitry Chistikov
Computer Science
University of Warwick
Coventry
CV4 7AL
United Kingdom
Email: `secretary@eatcs.org`,